TESTES DE SOFTWARE

Do teste manual à verificação formal

Mini E-book

Testes manuais • pytest • ferramentas • IA • testes matemáticos • verificação formal • laboratório de erros

Um único projeto do começo ao fim: a Todo List

Sumário

Parte 1 — Fundamentos

- 1. Introdução: por que testar
- 2. O projeto Todo List
- 3. Erro, defeito e falha
- 4. Verificação e validação
- 5. Níveis de teste e a pirâmide
- 6. Tipos de teste

Parte 2 — Testes manuais

- 7. O caso de teste
- 8. Técnicas para escolher bons casos de teste
- 9. Teste exploratório
- 10. O relatório de defeito

Parte 3 — Testes automatizados com pytest

- 11. Do assert ao pytest
- 12. Primeiro teste automatizado: o defeito do prazo
- 13. A entidade Tarefa: fixtures e o arquivo conftest.py
- 14. Parametrização e testes de exceção
- 15. Isolamento: o erro do estado compartilhado
- 16. Dublês de teste: fake, stub e mock
- 17. monkeypatch: testando configurações
- 18. Testando a API com TestClient
- 19. TDD: criando a pontuação de urgência
- 20. Cobertura de código
- 21. A suíte completa da Todo List

Parte 4 — Ferramentas de teste

- 22. Panorama das ferramentas

Parte 5 — Testes com inteligência artificial

- 23. Como a IA pode ajudar
- 24. O risco: testes que protegem o defeito
- 25. Como usar a IA com segurança

Parte 6 — Testes matemáticos

- 26. Oráculos, propriedades e invariantes
- 27. Testes baseados em propriedades (Hypothesis)
- 28. Testes metamórficos e números decimais
- 29. Explosão combinatória e testes em pares

Parte 7 — Verificação formal

- 30. Testar não é provar
- 31. Especificação: pré-condições, pós-condições e invariantes
- 32. Provadores automáticos: o Z3
- 33. Ferramentas e usos reais

Parte 8 — Laboratório: identificação de erros

- 34. Como usar o laboratório (exercícios L1 a L8)
- 35. Resumo
- 36. Glossário

Anexo

- Gabarito do laboratório

PARTE 1

Fundamentos de testes de software

Nesta parte você conhece o vocabulário dos testes e o projeto que vai acompanhar todo o e-book: a Todo List. Nenhum código ainda — só os conceitos e as regras que vamos testar.

1. Introdução: por que testar

Todo programa tem defeitos. Não porque quem programa seja descuidado, mas porque software é complexo: mesmo um sistema simples como a Todo List combina regras de negócio, validações, datas, banco de dados e comunicação pela internet. Cada combinação é uma oportunidade para algo dar errado.

Testar é executar o software de forma planejada para descobrir esses defeitos antes que o usuário os encontre. E quanto mais cedo um defeito é encontrado, mais barato é corrigi-lo — o mesmo princípio que você já viu na aula de briefing.

> **Um caso real**
> Em 1996, o foguete europeu Ariane 5 explodiu cerca de 40 segundos após o lançamento. A causa foi um erro de conversão: um número grande demais para o tipo de dado de destino, em um código reaproveitado do Ariane 4. Um único defeito de software custou a missão inteira — um lembrete de que testes não são burocracia.

Este e-book cobre os testes de ponta a ponta usando **um único projeto do começo ao fim: a Todo List e após a leitura desse primeiro ebook siga para a sua continuação, Ebook - construindo teste na pratica com pyteste –** desenvolvimento de teste de uma api real e uma exploração guiada para encontrar e resolver erros com a ferramenta pytest. Cada parte aproveita o que a anterior construiu, e o mesmo conjunto de regras de negócio aparece em todas elas — primeiro testado à mão, depois automatizado, depois verificado com matemática:

Parte	O que você faz com a Todo List
1. Fundamentos	Conhece o projeto, suas regras e o vocabulário de testes
2. Testes manuais	Escreve casos de teste, escolhe bons valores e relata um defeito real
3. pytest	Automatiza esses mesmos casos, subindo do domínio até a API
4. Ferramentas	Vê onde cada ferramenta de mercado entraria no projeto
5. Inteligência artificial	Pede testes a uma IA e aprende a revisá-los
6. Testes matemáticos	Testa propriedades das funções com centenas de entradas geradas
7. Verificação formal	Prova regras do projeto para todos os valores possíveis
8. Laboratório	Encontra defeitos escondidos em funções da própria Todo List

2. O projeto Todo List

A Todo List é uma aplicação web para organizar tarefas. Cada pessoa usuária se cadastra, faz login e gerencia a própria lista: cria tarefas com título, prioridade e, se quiser, um prazo; marca tarefas como concluídas (ou as reabre); edita e remove. Ao lado de cada tarefa, a lista mostra um selo com a situação do prazo, e as tarefas pendentes aparecem ordenadas pela urgência.

O back-end é escrito em Python com FastAPI, e os dados ficam em um banco SQL (SQLite no desenvolvimento, PostgreSQL em produção). Neste e-book usamos versões didáticas e simplificadas de cada peça, para que o foco fique nos testes.

As regras de negócio que vamos testar

Regra	O que diz	Aparece a partir da
RN-01	Título obrigatório: o título não pode ser vazio nem conter só espaços.	Seção 13
RN-02	Toda tarefa nasce pendente, sem data de conclusão.	Seção 7
RN-03	Data de conclusão: (1) concluir registra a data, e concluir de novo uma tarefa já concluída preserva a data original; (2) reabrir apaga a data; (3) editar não altera a data.	Seção 7
RN-04	Prioridade: baixa, media, alta ou urgente (padrão: media).	Seção 13
RN-05	Situação do prazo, calculada na data de hoje: tarefa concluída → "concluida"; pendente sem prazo → "sem prazo"; prazo antes de hoje → "atrasada"; prazo de hoje até 2 dias depois → "vence em breve"; prazo mais de 2 dias depois → "no prazo".	Seção 8
RN-06	Pontuação de urgência, usada para ordenar a lista: peso da prioridade × 10 (baixa = 1, media = 2, alta = 3, urgente = 4), mais 20 pontos se a tarefa está atrasada ou 10 se vence em breve. Tarefas concluídas valem 0.	Seção 19

Os casos de teste também citam os requisitos funcionais, como o RF-08 (concluir e reabrir tarefas).

> ⚠ **Guarde a RN-05**
>
> A RN-05 é a regra mais traiçoeira do e-book: ela tem fronteiras (ontem, hoje, daqui a 2 e daqui a 3 dias) e combina duas informações (o status e o prazo). Ela vai acompanhar você das técnicas de teste manual até a verificação formal.

Os arquivos que vamos construir

A cada seção da Parte 3 em diante, o projeto ganha uma peça nova. Cada arquivo de código tem um arquivo de teste com o mesmo nome e o prefixo test_ (por exemplo, prazo.py e test_prazo.py):

Arquivo	O que contém	Seção
prazo.py	A função da RN-05 (situação do prazo)	12
tarefa.py	A entidade Tarefa (RN-01 a RN-04)	13
conftest.py	Fixtures compartilhadas pelos testes	13
repositorio.py	Guarda as tarefas e gera os ids	15
notificacao.py	Avisa o dono quando uma tarefa é concluída	16
config.py	Lê as configurações do ambiente	17
api_tarefas.py	A API HTTP, com FastAPI	18
urgencia.py	A pontuação da RN-06, criada com TDD	19
exportacao.py, titulos.py	Exportar e importar tarefas; remover títulos repetidos	27
estatisticas.py	Percentual de tarefas concluídas	28
verificacao_z3.py	Provas formais sobre as regras do projeto	32

3. Erro, defeito e falha

Três palavras parecidas descrevem momentos diferentes do mesmo problema. Veja o caso que vai nos acompanhar nas próximas seções, envolvendo a RN-05:

Termo	O que é	Exemplo na Todo List
Erro (engano)	Uma ação humana equivocada	A pessoa programadora entendeu que uma tarefa que vence hoje já está atrasada
Defeito (bug)	O resultado do engano, gravado no código	`dias <= 0` em vez de `dias < 0` na verificação de atraso
Falha	O comportamento errado observado ao executar	Uma tarefa com prazo para hoje aparece com o selo "atrasada"

Um defeito pode ficar escondido por muito tempo, sem causar falha, até que alguém use exatamente o valor que o revela — aqui, uma tarefa que vence justamente no dia de hoje. Por isso testes bem planejados procuram esses valores, como veremos na seção 8.

4. Verificação e validação

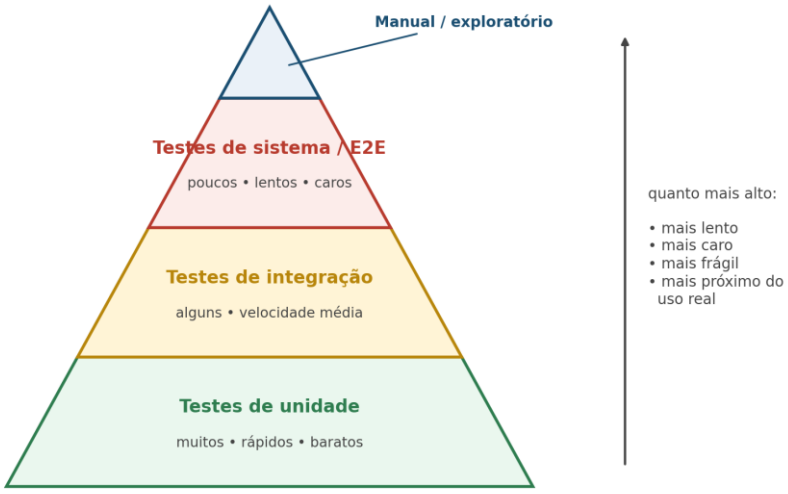
	Verificação	Validação
Pergunta	Estamos construindo o produto corretamente?	Estamos construindo o produto certo?
Compara com	A especificação (requisitos, regras de negócio)	A necessidade real da cliente
Exemplo	A situação do prazo obedece à RN-05, que diz "de hoje até 2 dias depois"?	Avisar com 2 dias de antecedência é o que a cliente precisa?

A diferença importa: um sistema pode passar em todos os testes de verificação e, ainda assim, falhar na validação — se a especificação estiver errada. Na seção 32 você verá um caso exatamente assim, na regra de ordenação da Todo List, encontrado por verificação formal.

5. Níveis de teste e a pirâmide

Nível	O que testa	Exemplo na Todo List
Unidade	Uma peça isolada: uma função, um método, uma classe	A função da RN-05 classifica corretamente uma tarefa que vence hoje
Integração	Peças funcionando juntas	A API grava a tarefa no repositório e a devolve pelo id
Sistema (ponta a ponta)	O sistema inteiro, pelo lado de fora	Cadastrar, autenticar, criar e concluir uma tarefa pela API
Aceitação	Se o sistema atende à cliente	A cliente confirma que o fluxo resolve o problema do briefing

A pirâmide de testes resume como distribuir o esforço: muitos testes de unidade na base (rápidos e baratos), alguns de integração no meio e poucos testes de ponta a ponta no topo (lentos, caros e mais frágeis). Os testes manuais exploratórios ficam na ponta, complementando os automatizados. A Parte 3 segue exatamente essa ordem: começa pelas unidades e só no fim chega à API.



A pirâmide de testes

6. Tipos de teste

Tipo	Objetivo
Funcional	Verificar se o sistema faz o que deveria (as regras RN-01 a RN-06 e os requisitos funcionais)
Não funcional	Verificar como o sistema se comporta: desempenho, segurança, usabilidade, compatibilidade
Regressão	Garantir que algo que funcionava continua funcionando depois de uma mudança
Fumaça (smoke)	Checagem rápida de que o essencial funciona antes de testes mais longos ("a aplicação sobe? a lista abre?")
Caixa-preta	Testar pelas entradas e saídas, sem olhar o código
Caixa-branca	Testar com base na estrutura interna do código (caminhos, condições)

> **Dica**
>
> Testes automatizados são, acima de tudo, a melhor proteção contra regressão: a cada mudança no código, a suíte inteira roda de novo em segundos e avisa se algo que funcionava quebrou.

PARTE 2

Testes manuais

Antes de escrever código de teste, vamos testar a Todo List como uma pessoa testadora: escrever casos de teste, escolher bons valores para a RN-05 e relatar o defeito encontrado. Os casos criados aqui serão automatizados na Parte 3.

7. O caso de teste

Um teste manual é executado por uma pessoa, seguindo um roteiro. A unidade desse roteiro é o caso de teste: a descrição de uma situação, dos passos e do resultado esperado. Um bom caso de teste é tão preciso que duas pessoas diferentes, executando-o, chegam à mesma conclusão.

Campo	CT-01 — Concluir uma tarefa	CT-02 — Reabrir uma tarefa concluída
Rastreabilidade	RF-08, RN-03 (caso 1)	RF-08, RN-03 (caso 2)
Pré-condição	Usuário autenticado com uma tarefa pendente	Usuário autenticado com uma tarefa concluída
Passos	1. Abrir a lista de tarefas 2. Clicar em "Concluir" na tarefa	1. Abrir a lista de tarefas 2. Clicar em "Reabrir" na tarefa
Resultado esperado	Status "concluída" e data de conclusão preenchida	Status "pendente" e data de conclusão vazia
Resultado obtido		
Situação	( ) Passou ( ) Falhou	( ) Passou ( ) Falhou

> ⚠ **Atenção**
> Repare na linha de rastreabilidade: cada caso de teste aponta para o requisito e a regra de negócio que verifica. É assim que a matriz de rastreabilidade da Aula 11 vira prática — cada RN dá origem a pelo menos um caso de teste, e cada fluxo alternativo de um caso de uso também.

8. Técnicas para escolher bons casos de teste

Não é possível testar todos os valores. Uma tarefa pode ter qualquer prazo, em qualquer dia do calendário — testar todos levaria uma eternidade. As técnicas abaixo escolhem poucos casos com grande chance de revelar defeitos. Vamos aplicá-las à RN-05, considerando que **hoje é 10/03/2026**:

```
RN-05 — situação do prazo
Tarefa concluída                    -> "concluida"
Pendente sem prazo                  -> "sem prazo"
Prazo antes de hoje                 -> "atrasada"
Prazo de hoje até 2 dias depois     -> "vence em breve"
Prazo mais de 2 dias depois         -> "no prazo"
```

Partição de equivalência

Divida as entradas em grupos que o sistema deveria tratar da mesma forma e teste um representante de cada grupo:

Partição	Representante	Resultado esperado
Pendente, prazo já passou	Prazo 05/03	atrasada
Pendente, prazo entre hoje e daqui a 2 dias	Prazo 11/03	vence em breve
Pendente, prazo depois de 2 dias	Prazo 20/03	no prazo
Pendente, sem prazo	Prazo vazio	sem prazo
Concluída	Prazo 20/03	concluida

Análise de valor-limite

Defeitos adoram as fronteiras entre partições: é ali que se troca `<` por `<=`. A RN-05 tem duas fronteiras — entre ontem e hoje, e entre daqui a 2 e daqui a 3 dias. Teste exatamente no limite e logo ao lado dele:

Limite	Casos de teste (tarefa pendente)	Esperado
Início de "vence em breve" (hoje)	Prazo 09/03 (ontem) e 10/03 (hoje)	atrasada / vence em breve
Fim de "vence em breve" (hoje + 2)	Prazo 12/03 e 13/03	vence em breve / no prazo

Tabela de decisão

Quando várias condições se combinam, liste todas as combinações. A RN-05 combina o status com o prazo, e a tabela revela a pergunta que mais se esquece: e uma tarefa **concluída** cujo prazo já passou?

Regra	Status	Prazo	Situação	Caso de teste
1	Concluída	Já passou	concluida (e não "atrasada")	Concluída, prazo 09/03
2	Concluída	Não passou	concluida	Concluída, prazo 15/03
3	Pendente	Já passou	atrasada	Pendente, prazo 09/03
4	Pendente	De hoje até +2	vence em breve	Pendente, prazo 10/03
5	Pendente	Depois de +2	no prazo	Pendente, prazo 13/03
6	Pendente	Sem prazo	sem prazo	Pendente, sem prazo

Esses casos serão automatizados na seção 12 — e dois deles vão encontrar defeitos reais.

9. Teste exploratório

No teste exploratório, a pessoa testadora aprende sobre o sistema enquanto testa, criando os próximos testes com base no que acabou de observar. Ele não substitui os casos roteirizados: complementa, encontrando o que ninguém pensou em escrever. Para não virar "clicar aleatoriamente", ele é organizado em sessões curtas, cada uma com uma missão:

Elemento da sessão	Exemplo
Missão	Explorar a criação de tarefas com títulos incomuns
Duração	30 minutos
Ideias a explorar	Título só com espaços, com 500 caracteres, com emojis, com acentos, com aspas, com ponto e vírgula
Anotações	O que foi testado, o que foi observado, dúvidas e defeitos encontrados

Guarde a ideia do título com ponto e vírgula: na seção 27, uma ferramenta vai encontrar sozinha um defeito exatamente com esse tipo de título.

10. O relatório de defeito

Encontrar um defeito é metade do trabalho: a outra metade é descrevê-lo de forma que qualquer pessoa consiga reproduzi-lo. Durante os testes da RN-05, uma testadora encontrou a falha da seção 3. Veja como ela a relatou — com título claro, passos exatos e a diferença entre o esperado e o obtido:

Campo	Conteúdo
ID e título	BUG-07 — Tarefa que vence hoje aparece como "atrasada"
Severidade	Média (assusta o usuário e põe a tarefa no lugar errado da lista; nenhum dado é perdido)
Ambiente	Todo List versão 1.2, Google Chrome, Windows 11
Passos para reproduzir	1. Fazer login com o usuário de teste 2. Criar a tarefa "Entregar relatório" com prazo para hoje (10/03/2026) 3. Abrir a lista de tarefas
Resultado esperado	Selo "vence em breve", conforme a RN-05
Resultado obtido	Selo vermelho "atrasada"
Observação	Com prazo para amanhã (11/03), o selo aparece certo: o problema parece estar exatamente no limite

> **Erro comum**
>
> Relatórios como "o selo está errado" ou "deu erro na lista" obrigam quem vai corrigir a adivinhar o que aconteceu. Sem passos exatos e sem a data usada, o defeito pode nunca ser reproduzido — principalmente um defeito que depende do dia de hoje.

PARTE 3

Testes automatizados com pytest

Agora a Todo List ganha testes automatizados, seguindo a pirâmide de baixo para cima: primeiro a função da RN-05, depois a entidade Tarefa, o repositório, os serviços e, por fim, a API. Em cada seção aparece um recurso novo do pytest — sempre aplicado a uma peça nova do projeto.

11. Do assert ao pytest

Um teste automatizado é um programa que verifica outro programa. A peça mais básica é o `assert`: ele não faz nada se a condição for verdadeira e interrompe o programa com AssertionError se for falsa.

```
assert 2 + 2 == 4    # verdadeiro: nada acontece
assert 2 + 2 == 5    # falso: AssertionError
```

O pytest é a ferramenta mais usada em Python para organizar e executar testes. Ele encontra os testes sozinho, executa todos, mostra quais passaram e, quando algo falha, explica exatamente o que era esperado e o que foi obtido. As convenções são simples:

Convenção	Exemplo
Instalação	`pip install pytest`
Arquivos de teste começam com test_	`test_prazo.py`, `test_tarefa.py`
Funções de teste começam com test_	`def test_tarefa_nasce_pendente():`
Cada teste usa assert para verificar	`assert tarefa.status == "pendente"`
Execução (na pasta do projeto)	`python -m pytest` `-v` detalha cada teste, `-q` resume, `--tb=short` encurta as mensagens de falha

Símbolo na saída	Significado
. ou PASSED	O teste passou
F ou FAILED	Um assert falhou: o código não fez o esperado
E ou ERROR	Algo quebrou antes ou fora do assert (por exemplo, um import que falhou)

12. Primeiro teste automatizado: o defeito do prazo

Vamos automatizar os casos da seção 8. A função abaixo deveria implementar a RN-05 — mas contém defeitos. Tente encontrá-los antes de ver o resultado dos testes.

```
# arquivo: prazo.py (versão com defeito)
def situacao_do_prazo(status, prazo, hoje):
    """RN-05: situação do prazo de uma tarefa na data 'hoje'."""
    if prazo is not None and (prazo - hoje).days <= 0:
        return "atrasada"
    if status == "concluida":
        return "concluida"
    if prazo is None:
        return "sem prazo"
    if (prazo - hoje).days <= 2:
        return "vence em breve"
    return "no prazo"
```

Repare que a função recebe a data de hoje como parâmetro (`hoje`), em vez de consultar o relógio do computador. Assim o teste pode fixar o dia, e o resultado não muda conforme a data em que for executado.

Os testes: valores-limite e tabela de decisão

```
# arquivo: test_prazo.py
from datetime import date, timedelta

import pytest

from prazo import situacao_do_prazo

HOJE = date(2026, 3, 10)  # data fixa: o teste não depende do relógio


@pytest.mark.parametrize("status, dias, esperado", [
    ("pendente", -1, "atrasada"),         # ontem: logo antes do limite
    ("pendente", 0, "vence em breve"),    # hoje: exatamente no limite
    ("pendente", 2, "vence em breve"),    # último dia de "vence em breve"
    ("pendente", 3, "no prazo"),          # primeiro dia de "no prazo"
    ("pendente", None, "sem prazo"),      # tarefa sem prazo
    ("concluida", 5, "concluida"),        # concluída dentro do prazo
    ("concluida", -1, "concluida"),       # concluída E vencida: a linha esquecida
])
def test_situacao_do_prazo(status, dias, esperado):
    prazo = None if dias is None else HOJE + timedelta(days=dias)
    assert situacao_do_prazo(status, prazo, HOJE) == esperado
```

O decorador `@pytest.mark.parametrize` executa a mesma função de teste uma vez para cada linha da lista — sete testes escritos uma única vez. Cada linha informa o status, quantos dias faltam para o prazo (`None` = sem prazo) e a situação esperada; o teste calcula a data do prazo a partir de `HOJE`.

Resultado: python -m pytest test_prazo.py -q --tb=short

```
.F....F                                                                  [100%]
================================== FAILURES ===================================
_____________ test_situacao_do_prazo[pendente-0-vence em breve] _____________
test_prazo.py:21: in test_situacao_do_prazo
    assert situacao_do_prazo(status, prazo, HOJE) == esperado
E   AssertionError: assert 'atrasada' == 'vence em breve'
E
E     - vence em breve
E     + atrasada
_____________ test_situacao_do_prazo[concluida--1-concluida] ______________
test_prazo.py:21: in test_situacao_do_prazo
    assert situacao_do_prazo(status, prazo, HOJE) == esperado
E   AssertionError: assert 'atrasada' == 'concluida'
E
E     - concluida
E     + atrasada
========================== short test summary info ============================
FAILED test_prazo.py::test_situacao_do_prazo[pendente-0-vence em breve] - Ass...
FAILED test_prazo.py::test_situacao_do_prazo[concluida--1-concluida] - Assert...
2 failed, 5 passed in 0.21s
```

Lendo a falha

A saída diz exatamente o que aconteceu. No caso [pendente-0-vence em breve], uma tarefa que vence hoje foi classificada como "atrasada": o `<= 0` deveria ser `< 0` — é o BUG-07 da seção 10. No caso [concluida--1-concluida], uma tarefa já concluída, mas com prazo vencido, também virou "atrasada": a função verifica o atraso antes de olhar o status.

Os outros cinco casos passaram. Um teste "comum", com prazo para daqui a dez dias, também passaria. Só o valor-limite e a linha esquecida da tabela de decisão revelaram os defeitos — cada técnica encontrou um. A versão corrigida verifica o status primeiro e usa `<` na fronteira de hoje:

```
# arquivo: prazo.py (versão corrigida)
def situacao_do_prazo(status, prazo, hoje):
    """RN-05: situação do prazo de uma tarefa na data 'hoje'."""
    if status == "concluida":
        return "concluida"
    if prazo is None:
        return "sem prazo"
    dias = (prazo - hoje).days
    if dias < 0:
        return "atrasada"
    if dias <= 2:
        return "vence em breve"
    return "no prazo"
```

Depois da correção: python -m pytest test_prazo.py -v

```
============================= test session starts ==============================
collecting ... collected 7 items

test_prazo.py::test_situacao_do_prazo[pendente--1-atrasada] PASSED       [ 14%]
test_prazo.py::test_situacao_do_prazo[pendente-0-vence em breve] PASSED  [ 28%]
test_prazo.py::test_situacao_do_prazo[pendente-2-vence em breve] PASSED  [ 42%]
test_prazo.py::test_situacao_do_prazo[pendente-3-no prazo] PASSED        [ 57%]
test_prazo.py::test_situacao_do_prazo[pendente-None-sem prazo] PASSED    [ 71%]
test_prazo.py::test_situacao_do_prazo[concluida-5-concluida] PASSED      [ 85%]
test_prazo.py::test_situacao_do_prazo[concluida--1-concluida] PASSED     [100%]

============================== 7 passed in 0.12s ===============================
```

> **Dica**
>
> Um teste que nunca falhou não prova muita coisa. Quando escrever um teste novo, confira se ele é capaz de falhar — por exemplo, quebrando de propósito o código por um instante. Se o teste continuar verde com o código errado, ele não está testando nada.

13. A entidade Tarefa: fixtures e o arquivo conftest.py

A função de prazo era simples: dados de entrada, um resultado de saída. A próxima peça da Todo List é a entidade Tarefa, que guarda estado e muda com o tempo. Muitos testes precisam do mesmo ponto de partida: uma tarefa nova, uma tarefa já concluída. Repetir essa preparação em cada teste deixa o código longo e frágil. Uma **fixture** é uma função que prepara esse ponto de partida e o entrega a cada teste que pedir por ele, pelo nome do parâmetro.

O código abaixo é uma versão didática e simplificada da entidade Tarefa, com as regras RN-01 a RN-04:

```
# arquivo: tarefa.py
from datetime import datetime
PRIORIDADES = ("baixa", "media", "alta", "urgente")
class ErroDeDominio(Exception):
    """Base de todos os erros de regra de negócio."""
class TituloInvalido(ErroDeDominio):
    pass
class PrioridadeInvalida(ErroDeDominio):
    pass
class Tarefa:
    def __init__(self, titulo, prioridade="media"):
        self.titulo = self._validar_titulo(titulo)          # RN-01
        self.prioridade = self._validar_prioridade(prioridade)  # RN-04
        self._status = "pendente"  # RN-02: toda tarefa nasce pendente
        self.concluido_em = None
    @property
    def status(self):  # sem setter: não dá para alterar direto
        return self._status
    def concluir(self):
        if self._status != "concluida":  # RN-03: concluir de novo preserva a data
            self._status = "concluida"
            self.concluido_em = datetime.now()
    def reabrir(self):
        self._status = "pendente"  # RN-03: reabrir remove a data
        self.concluido_em = None
    def editar(self, titulo=None, prioridade=None):
        if titulo is not None:  # RN-03: editar não mexe na data
            self.titulo = self._validar_titulo(titulo)
        if prioridade is not None:
            self.prioridade = self._validar_prioridade(prioridade)

    @staticmethod
    def _validar_titulo(titulo):
        titulo = titulo.strip()
        if titulo == "":
            raise TituloInvalido("O título não pode ser vazio.")
        return titulo

    @staticmethod
    def _validar_prioridade(prioridade):
        if prioridade not in PRIORIDADES:
            raise PrioridadeInvalida(f"Prioridade inválida: {prioridade}")
        return prioridade
```

As fixtures ficam no arquivo conftest.py. O pytest o carrega automaticamente: tudo o que estiver nele fica disponível para os testes da pasta, sem import.

```
# arquivo: conftest.py
import pytest

from tarefa import Tarefa


@pytest.fixture
def tarefa():
    """Uma tarefa nova e pendente, criada do zero para cada teste."""
    return Tarefa("Estudar pytest", prioridade="alta")


@pytest.fixture
def tarefa_concluida(tarefa):
    """Reaproveita a fixture 'tarefa' e já a entrega concluída."""
    tarefa.concluir()
    return tarefa
```

Os testes do domínio — cada um aponta a regra que verifica

```
# arquivo: test_tarefa.py
import pytest
from tarefa import Tarefa, TituloInvalido, PrioridadeInvalida
def test_tarefa_nasce_pendente(tarefa):  # RN-02
    assert tarefa.status == "pendente"
    assert tarefa.concluido_em is None
def test_concluir_registra_a_data(tarefa):  # RN-03, caso 1
    tarefa.concluir()
    assert tarefa.status == "concluida"
    assert tarefa.concluido_em is not None
def test_concluir_de_novo_preserva_a_data(tarefa_concluida):  # RN-03, caso 1
    data_original = tarefa_concluida.concluido_em
    tarefa_concluida.concluir()
    assert tarefa_concluida.concluido_em == data_original
def test_reabrir_remove_a_data(tarefa_concluida):  # RN-03, caso 2
    tarefa_concluida.reabrir()
    assert tarefa_concluida.status == "pendente"
    assert tarefa_concluida.concluido_em is None
def test_editar_titulo_nao_altera_a_data(tarefa_concluida):  # RN-03, caso 3
    data_original = tarefa_concluida.concluido_em
    tarefa_concluida.editar(titulo="Estudar pytest e fixtures")
    assert tarefa_concluida.concluido_em == data_original
def test_status_nao_pode_ser_alterado_diretamente(tarefa):
    with pytest.raises(AttributeError):
        tarefa.status = "concluida"
@pytest.mark.parametrize("titulo", ["", "   ", "\t"])
def test_titulo_vazio_e_recusado(titulo):  # RN-01
    with pytest.raises(TituloInvalido):
        Tarefa(titulo)
@pytest.mark.parametrize("prioridade", ["baixa", "media", "alta", "urgente"])
def test_prioridades_validas_sao_aceitas(prioridade):  # RN-04
    assert Tarefa("Ler", prioridade=prioridade).prioridade == prioridade
@pytest.mark.parametrize("prioridade", ["Alta", "critica", ""])
def test_prioridade_invalida_e_recusada(prioridade):  # RN-04
    with pytest.raises(PrioridadeInvalida, match="Prioridade inválida"):
        Tarefa("Ler", prioridade=prioridade)
```

Resultado: python -m pytest test_tarefa.py -v

```
============================= test session starts ==============================
collecting ... collected 16 items

test_tarefa.py::test_tarefa_nasce_pendente PASSED                        [  6%]
test_tarefa.py::test_concluir_registra_a_data PASSED                     [ 12%]
test_tarefa.py::test_concluir_de_novo_preserva_a_data PASSED             [ 18%]
test_tarefa.py::test_reabrir_remove_a_data PASSED                        [ 25%]
test_tarefa.py::test_editar_titulo_nao_altera_a_data PASSED              [ 31%]
test_tarefa.py::test_status_nao_pode_ser_alterado_diretamente PASSED     [ 37%]
test_tarefa.py::test_titulo_vazio_e_recusado[] PASSED                    [ 43%]
test_tarefa.py::test_titulo_vazio_e_recusado[   ] PASSED                 [ 50%]
test_tarefa.py::test_titulo_vazio_e_recusado[\t] PASSED                  [ 56%]
test_tarefa.py::test_prioridades_validas_sao_aceitas[baixa] PASSED       [ 62%]
test_tarefa.py::test_prioridades_validas_sao_aceitas[media] PASSED       [ 68%]
test_tarefa.py::test_prioridades_validas_sao_aceitas[alta] PASSED        [ 75%]
test_tarefa.py::test_prioridades_validas_sao_aceitas[urgente] PASSED     [ 81%]
test_tarefa.py::test_prioridade_invalida_e_recusada[Alta] PASSED         [ 87%]
test_tarefa.py::test_prioridade_invalida_e_recusada[critica] PASSED      [ 93%]
test_tarefa.py::test_prioridade_invalida_e_recusada[] PASSED             [100%]

============================== 16 passed in 0.17s ==============================
```

14. Parametrização e testes de exceção

As duas últimas seções usaram os recursos mais importantes do pytest para testar regras de negócio. Vale reuni-los em um só lugar:

Recurso	Para que serve	Exemplo na Todo List
`@pytest.mark.parametrize`	Rodar o mesmo teste com várias entradas	Os 7 casos da RN-05; os 3 títulos vazios e as 4 prioridades válidas
`pytest.raises(Erro)`	Verificar que um erro é levantado	Título vazio levanta TituloInvalido
`pytest.raises(Erro, match="texto")`	Verificar também a mensagem do erro	`match="Prioridade inválida"`
`pytest.approx(valor)`	Comparar números decimais com tolerância	Percentual de tarefas concluídas (seção 28)

> **Erro comum**
>
> Testar só o "caminho feliz". Metade das regras de negócio fala do que o sistema deve RECUSAR: título vazio, prioridade inválida, e-mail repetido. Cada recusa merece seu próprio teste com pytest.raises — senão, um sistema que aceita tudo passaria em todos os testes.

15. Isolamento: o erro do estado compartilhado

Cada teste precisa funcionar sozinho, em qualquer ordem. Para ver o que acontece quando isso não é respeitado, vamos apresentar a próxima peça da Todo List: o repositório, que guarda as tarefas e dá a cada uma um id sequencial.

```
# arquivo: repositorio.py
class TarefaNaoEncontrada(Exception):
    pass


class RepositorioEmMemoria:
    """Guarda as tarefas em um dicionário. Cada tarefa recebe
    um id sequencial, começando em 1."""

    def __init__(self):
        self._tarefas = {}
        self._proximo_id = 1

    def adicionar(self, tarefa):
        tarefa_id = self._proximo_id
        self._tarefas[tarefa_id] = tarefa
        self._proximo_id += 1
        return tarefa_id

    def obter(self, tarefa_id):
        if tarefa_id not in self._tarefas:
            raise TarefaNaoEncontrada(f"Tarefa {tarefa_id} não encontrada.")
        return self._tarefas[tarefa_id]

    def remover(self, tarefa_id):
        self.obter(tarefa_id)  # levanta TarefaNaoEncontrada se não existir
        del self._tarefas[tarefa_id]

    def listar(self):
        return list(self._tarefas.values())
```

Agora, dois testes que compartilham o mesmo repositório:

```
# arquivo: test_repositorio_compartilhado.py
from repositorio import RepositorioEmMemoria
from tarefa import Tarefa

repo = RepositorioEmMemoria()  # ERRO: o mesmo repositório para todos os testes


def test_primeira_tarefa_recebe_id_1():
    assert repo.adicionar(Tarefa("Primeira")) == 1


def test_adicionar_e_buscar_pelo_id_1():
    repo.adicionar(Tarefa("Outra"))
    assert repo.obter(1).titulo == "Outra"
```

Resultado: python -m pytest test_repositorio_compartilhado.py -q --tb=short

```
.F                                                                       [100%]
================================== FAILURES ===================================
_____________________ test_adicionar_e_buscar_pelo_id_1 _____________________
test_repositorio_compartilhado.py:13: in test_adicionar_e_buscar_pelo_id_1
    assert repo.obter(1).titulo == "Outra"
E   AssertionError: assert 'Primeira' == 'Outra'
E
E     - Outra
E     + Primeira
========================== short test summary info ============================
FAILED test_repositorio_compartilhado.py::test_adicionar_e_buscar_pelo_id_1
1 failed, 1 passed in 0.12s
```

O segundo teste criou a tarefa "Outra", mas o id 1 já pertencia à "Primeira", criada pelo teste anterior. Rodado sozinho, o segundo teste passa; rodado depois do primeiro, falha. É um defeito do teste, e não do sistema. A solução é uma fixture que entrega um repositório novo a cada teste. Acrescente ao conftest.py:

```
# arquivo: conftest.py (acréscimos)
from repositorio import RepositorioEmMemoria   # junto dos outros imports


@pytest.fixture
def repo():
    """Um repositório novo e vazio para cada teste."""
    return RepositorioEmMemoria()
```

```
# arquivo: test_repositorio.py
import pytest

from repositorio import TarefaNaoEncontrada
from tarefa import Tarefa


def test_primeira_tarefa_recebe_id_1(repo):
    assert repo.adicionar(Tarefa("Primeira")) == 1


def test_adicionar_e_buscar_pelo_id_1(repo):
    repo.adicionar(Tarefa("Outra"))
    assert repo.obter(1).titulo == "Outra"


def test_ids_sao_sequenciais(repo):
    ids = [repo.adicionar(Tarefa(t)) for t in ("A", "B", "C")]
    assert ids == [1, 2, 3]


def test_buscar_id_inexistente_levanta_erro(repo):
    with pytest.raises(TarefaNaoEncontrada):
        repo.obter(999)


def test_remover_tira_a_tarefa_da_lista(repo):
    tarefa_id = repo.adicionar(Tarefa("Temporária"))
    repo.remover(tarefa_id)
    assert repo.listar() == []
```

Resultado: python -m pytest test_repositorio.py -v

```
============================= test session starts ==============================
collecting ... collected 5 items

test_repositorio.py::test_primeira_tarefa_recebe_id_1 PASSED             [ 20%]
test_repositorio.py::test_adicionar_e_buscar_pelo_id_1 PASSED            [ 40%]
test_repositorio.py::test_ids_sao_sequenciais PASSED                     [ 60%]
test_repositorio.py::test_buscar_id_inexistente_levanta_erro PASSED      [ 80%]
test_repositorio.py::test_remover_tira_a_tarefa_da_lista PASSED          [100%]

============================== 5 passed in 0.11s ===============================
```

> ⚠ **Atenção**
>
> Na Todo List completa, o mesmo cuidado aparece no banco de dados: os testes usam um SQLite em memória, recriado a cada teste. Assim nenhum teste depende de dados deixados por outro, e nenhum dado real é tocado.

16. Dublês de teste: fake, stub e mock

Algumas partes do sistema são lentas, caras ou têm efeitos colaterais: enviar e-mail, cobrar um cartão, consultar uma API externa, calcular um hash de senha seguro. Nos testes, elas são substituídas por dublês — como um dublê de cinema, que substitui o ator nas cenas arriscadas.

Dublê	O que faz	Exemplo na Todo List
Fake	Implementação simples, mas funcional	Um enviador de e-mail que só guarda as mensagens em uma lista; o HashFake, que evita o bcrypt lento nos testes
Stub	Devolve respostas prontas, sem lógica	Um serviço de feriados que sempre responde "não é feriado"
Mock	Registra como foi chamado, para o teste conferir	Verificar que enviar() foi chamado uma vez, com o e-mail certo

A Todo List avisa o dono por e-mail quando uma tarefa é concluída. O envio fica em um objeto separado, recebido como parâmetro — e é isso que permite trocá-lo por um dublê:

```
# arquivo: notificacao.py
def concluir_e_notificar(tarefa, email_dono, enviador):
    """Conclui a tarefa e avisa o dono por e-mail.
    O envio em si é feito pelo objeto 'enviador'."""
    tarefa.concluir()
    enviador.enviar(email_dono, f"Tarefa concluída: {tarefa.titulo}")
```

```
# arquivo: test_notificacao.py
from unittest.mock import Mock

from tarefa import Tarefa
from notificacao import concluir_e_notificar


class EnviadorFake:
    """Fake: implementação simples que só guarda as mensagens em uma lista."""

    def __init__(self):
        self.enviadas = []

    def enviar(self, destino, mensagem):
        self.enviadas.append((destino, mensagem))


def test_com_fake_guarda_a_mensagem():
    enviador = EnviadorFake()
    concluir_e_notificar(Tarefa("Pagar boleto"), "ana@email.com", enviador)
    assert enviador.enviadas == [("ana@email.com", "Tarefa concluída: Pagar boleto")]


def test_com_mock_verifica_a_chamada():
    enviador = Mock()  # Mock: registra como foi chamado
    concluir_e_notificar(Tarefa("Pagar boleto"), "ana@email.com", enviador)
    enviador.enviar.assert_called_once_with(
        "ana@email.com", "Tarefa concluída: Pagar boleto"
    )
```

Os dois testes verificam a mesma coisa de formas diferentes: com o fake, olhamos o estado que ele guardou; com o mock (da biblioteca padrão unittest.mock), perguntamos a ele como foi chamado. Nenhum e-mail de verdade é enviado.

17. monkeypatch: testando configurações

Configurações costumam vir de variáveis de ambiente — como a DATABASE_URL da Todo List, que aponta para o SQLite no desenvolvimento e para o PostgreSQL em produção. Para testar diferentes configurações sem alterar o computador de verdade, o pytest oferece a fixture `monkeypatch`, que muda o ambiente só durante o teste e desfaz tudo ao final.

```
# arquivo: config.py
import os


def ler_configuracao():
    """Lê as configurações das variáveis de ambiente, com valores padrão."""
    url = os.environ.get("DATABASE_URL", "sqlite:///./todo_list.db")
    echo = os.environ.get("ECHO_SQL", "false").strip().lower() in ("true", "1", "sim")
    if url.startswith("postgres://"):  # normaliza o prefixo antigo
        url = "postgresql://" + url[len("postgres://"):]
    return {"database_url": url, "echo_sql": echo}
```

```
# arquivo: test_config.py
from config import ler_configuracao


def test_sem_variaveis_usa_sqlite_local(monkeypatch):
    monkeypatch.delenv("DATABASE_URL", raising=False)
    monkeypatch.delenv("ECHO_SQL", raising=False)
    assert ler_configuracao() == {"database_url": "sqlite:///./todo_list.db",
                                  "echo_sql": False}


def test_prefixo_postgres_antigo_e_normalizado(monkeypatch):
    monkeypatch.setenv("DATABASE_URL", "postgres://usuario:senha@servidor/banco")
    url = ler_configuracao()["database_url"]
    assert url == "postgresql://usuario:senha@servidor/banco"


def test_echo_sql_aceita_true_em_qualquer_caixa(monkeypatch):
    monkeypatch.setenv("ECHO_SQL", "TRUE")
    assert ler_configuracao()["echo_sql"] is True
```

> ⚠ **Armadilha do cache**
>
> Se a função de configuração usar cache (guardar o resultado na primeira chamada), o monkeypatch não terá efeito nas chamadas seguintes: o valor antigo continua guardado. Por isso, na Todo List, os testes de configuração chamam a função que lê o ambiente diretamente, sem passar pela versão com cache.

18. Testando a API com TestClient

O FastAPI traz o TestClient, que faz requisições HTTP para a aplicação sem precisar subir um servidor. A API abaixo junta as peças das seções anteriores: usa a entidade Tarefa e o RepositorioEmMemoria. Os erros de regra de negócio viram respostas 422, e uma tarefa inexistente vira 404. A aplicação é criada por uma função — assim cada teste pode ter a sua, isolada, exatamente como fizemos com o repositório na seção 15.

```
# arquivo: api_tarefas.py
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from pydantic import BaseModel

from repositorio import RepositorioEmMemoria, TarefaNaoEncontrada
from tarefa import Tarefa, ErroDeDominio


class TarefaEntrada(BaseModel):
    titulo: str
    prioridade: str = "media"


def criar_app():
    """Cria uma aplicação NOVA, com repositório próprio, a cada chamada."""
    app = FastAPI()
    repo = RepositorioEmMemoria()

    @app.exception_handler(ErroDeDominio)
    def tratar_erro_de_dominio(request: Request, erro: ErroDeDominio):
        return JSONResponse(status_code=422, content={"detail": str(erro)})

    @app.exception_handler(TarefaNaoEncontrada)
    def tratar_nao_encontrada(request: Request, erro: TarefaNaoEncontrada):
        return JSONResponse(status_code=404, content={"detail": str(erro)})

    def como_json(tarefa_id, t):
        return {"id": tarefa_id, "titulo": t.titulo, "prioridade": t.prioridade,
                "status": t.status, "concluidoEm": t.concluido_em}

    @app.post("/tarefas", status_code=201)
    def criar(dados: TarefaEntrada):
        tarefa = Tarefa(dados.titulo, dados.prioridade)
        return como_json(repo.adicionar(tarefa), tarefa)

    @app.get("/tarefas/{tarefa_id}")
    def obter(tarefa_id: int):
        return como_json(tarefa_id, repo.obter(tarefa_id))

    @app.post("/tarefas/{tarefa_id}/concluir")
    def concluir(tarefa_id: int):
        tarefa = repo.obter(tarefa_id)
        tarefa.concluir()
        return como_json(tarefa_id, tarefa)

    @app.delete("/tarefas/{tarefa_id}", status_code=204)
    def remover(tarefa_id: int):
        repo.remover(tarefa_id)

    return app
```

```python
# arquivo: test_api.py
import pytest
from fastapi.testclient import TestClient

from api_tarefas import criar_app


@pytest.fixture
def cliente():
    # Uma aplicação nova por teste: nenhum teste enxerga dados de outro
    with TestClient(criar_app()) as c:
        yield c


def test_criar_tarefa_devolve_201_e_status_pendente(cliente):
    resposta = cliente.post("/tarefas", json={"titulo": "Estudar API"})
    assert resposta.status_code == 201
    corpo = resposta.json()
    assert corpo["status"] == "pendente"
    assert corpo["concluidoEm"] is None


def test_titulo_vazio_devolve_422(cliente):
    resposta = cliente.post("/tarefas", json={"titulo": "   "})
    assert resposta.status_code == 422
    assert "título" in resposta.json()["detail"]


def test_concluir_preenche_concluido_em(cliente):
    cliente.post("/tarefas", json={"titulo": "Estudar API"})
    resposta = cliente.post("/tarefas/1/concluir")
    assert resposta.status_code == 200
    assert resposta.json()["concluidoEm"] is not None


def test_tarefa_inexistente_devolve_404(cliente):
    assert cliente.get("/tarefas/999").status_code == 404


def test_remover_duas_vezes_devolve_204_e_depois_404(cliente):
    cliente.post("/tarefas", json={"titulo": "Temporária"})
    assert cliente.delete("/tarefas/1").status_code == 204
    assert cliente.delete("/tarefas/1").status_code == 404
```

O `with TestClient(...)` é importante: além de isolar, ele executa os eventos de início e fim da aplicação (o lifespan), que na Todo List completa criam as tabelas do banco. Sem o `with`, esses eventos não rodam.

Resultado (API, configuração e dublês): python -m pytest test_api.py test_config.py test_notificacao.py -v

```
============================= test session starts ==============================
collecting ... collected 10 items

test_api.py::test_criar_tarefa_devolve_201_e_status_pendente PASSED      [ 10%]
test_api.py::test_titulo_vazio_devolve_422 PASSED                        [ 20%]
test_api.py::test_concluir_preenche_concluido_em PASSED                  [ 30%]
test_api.py::test_tarefa_inexistente_devolve_404 PASSED                  [ 40%]
test_api.py::test_remover_duas_vezes_devolve_204_e_depois_404 PASSED     [ 50%]
test_config.py::test_sem_variaveis_usa_sqlite_local PASSED               [ 60%]
test_config.py::test_prefixo_postgres_antigo_e_normalizado PASSED        [ 70%]
test_config.py::test_echo_sql_aceita_true_em_qualquer_caixa PASSED       [ 80%]
test_notificacao.py::test_com_fake_guarda_a_mensagem PASSED              [ 90%]
test_notificacao.py::test_com_mock_verifica_a_chamada PASSED             [100%]

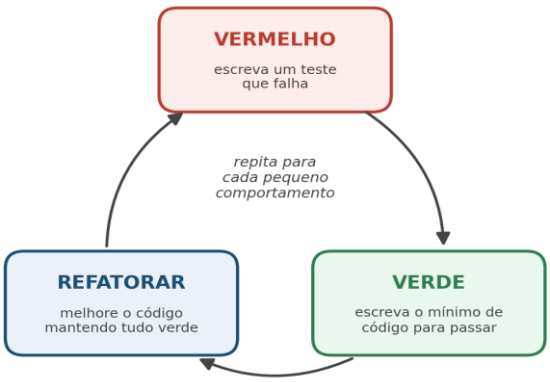
============================== warnings summary ================================
.../site-packages/fastapi/testclient.py:1
  .../site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 10 passed, 1 warning in 0.67s =========================
```

Repare no aviso (warning) no final da saída: ele vem de uma biblioteca instalada, e não do código da Todo List. Os 10 testes passaram — veja o quadro "Avisos não são falhas" na seção 21.

19. TDD: criando a pontuação de urgência

Até aqui, o código sempre existia antes do teste. No TDD (Test-Driven Development), a ordem se inverte: o teste é escrito antes do código. O ciclo tem três passos curtos, repetidos para cada pequeno comportamento:



O ciclo do TDD

Vermelho: o teste primeiro

A Todo List precisa ordenar as tarefas pendentes pela urgência (RN-06). Antes de escrever a função, escrevemos o teste — com a menor e a maior pontuação possíveis, o caso sem bônus, a tarefa concluída e as prioridades inválidas:

```
# arquivo: test_urgencia.py
import pytest

from urgencia import pontuacao_urgencia


@pytest.mark.parametrize("prioridade, situacao, esperado", [
    ("baixa", "no prazo", 10),         # menor pontuação de uma pendente
    ("media", "sem prazo", 20),        # sem prazo não ganha bônus
    ("alta", "vence em breve", 40),    # 30 + 10
    ("urgente", "atrasada", 60),       # maior pontuação possível: 40 + 20
    ("urgente", "concluida", 0),       # concluída sai da fila
])
def test_pontuacao_por_prioridade_e_situacao(prioridade, situacao, esperado):
    assert pontuacao_urgencia(prioridade, situacao) == esperado


@pytest.mark.parametrize("prioridade", ["Alta", "critica"])
def test_prioridade_invalida_e_recusada(prioridade):
    with pytest.raises(ValueError):
        pontuacao_urgencia(prioridade, "no prazo")
```

Resultado: o teste falha, porque a função ainda nem existe (trecho)

```
=================================== ERRORS ====================================
____________________ ERROR collecting test_urgencia.py _____________________
test_urgencia.py:3: in <module>
    from urgencia import pontuacao_urgencia
E   ModuleNotFoundError: No module named 'urgencia'
=========================== short test summary info ===========================
ERROR test_urgencia.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!
1 error in 0.18s
```

Verde: o mínimo para passar

```
# arquivo: urgencia.py
PESOS = {"baixa": 1, "media": 2, "alta": 3, "urgente": 4}
BONUS = {"atrasada": 20, "vence em breve": 10}


def pontuacao_urgencia(prioridade, situacao):
    """RN-06: peso da prioridade x 10, mais o bônus da situação do prazo.
    Tarefas concluídas valem 0."""
    if prioridade not in PESOS:
        raise ValueError(f"Prioridade inválida: {prioridade}")
    if situacao == "concluida":
        return 0
    return PESOS[prioridade] * 10 + BONUS.get(situacao, 0)
```

Resultado: python -m pytest test_urgencia.py -q

```
.......                                                                  [100%]
7 passed in 0.11s
```

Refatorar

Com os testes verdes, o código pode ser melhorado com segurança — renomear variáveis, extrair constantes, simplificar condições. Por exemplo: os pesos poderiam ser derivados da tupla PRIORIDADES de tarefa.py, para que uma prioridade nova não precise ser cadastrada em dois lugares. Se algo quebrar, os testes avisam na hora. A vantagem do TDD não é só ter testes: é pensar no comportamento esperado antes de pensar na implementação.

20. Cobertura de código

A cobertura mede quais linhas do código foram executadas pelos testes. A ferramenta pytest-cov (`pip install pytest-cov`) gera o relatório. Veja a cobertura de tarefa.py quando só um teste é executado:

python -m pytest test_tarefa.py::test_tarefa_nasce_pendente --cov=tarefa --cov-report=term-missing -q

```
.                                                                        [100%]
================================ tests coverage ================================
_____________ coverage: platform linux, python 3.12.3-final-0 _____________

Name        Stmts   Miss  Cover   Missing
-----------------------------------------
tarefa.py      39     11    72%   30-32, 35-36, 39-42, 48, 54
-----------------------------------------
TOTAL          39     11    72%
1 passed in 0.16s
```

A coluna Missing lista as linhas nunca executadas. Agora, com todos os testes do domínio construídos até aqui — prazo, tarefa, repositório e urgência:

python -m pytest test_prazo.py test_tarefa.py test_repositorio.py test_urgencia.py --cov=prazo --cov=tarefa --cov=repositorio --cov=urgencia --cov-report=term-missing -q

```
...................................                                      [100%]
================================ tests coverage ================================
_____________ coverage: platform linux, python 3.12.3-final-0 _____________

Name             Stmts   Miss  Cover   Missing
------------------------------------------------
prazo.py            11      0   100%
repositorio.py      20      0   100%
tarefa.py           39      1    97%   42
urgencia.py          8      0   100%
------------------------------------------------
TOTAL               78      1    99%
35 passed in 0.25s
```

Mesmo com 35 testes passando, a linha 42 de tarefa.py nunca rodou. Ela é a edição da prioridade dentro de `editar()`: nenhum teste edita a prioridade. A cobertura não diz se o código está certo — mas aponta exatamente onde ninguém olhou. Dois testes novos fecham a lacuna:

```python
# arquivo: test_tarefa_edicao.py
import pytest

from tarefa import PrioridadeInvalida


def test_editar_prioridade_valida(tarefa):
    tarefa.editar(prioridade="urgente")
    assert tarefa.prioridade == "urgente"


def test_editar_prioridade_invalida_mantem_a_anterior(tarefa):
    with pytest.raises(PrioridadeInvalida):
        tarefa.editar(prioridade="altíssima")
    assert tarefa.prioridade == "alta"  # o valor antigo continua lá
```

python -m pytest test_tarefa.py test_tarefa_edicao.py --cov=tarefa --cov-report=term-missing -q

```
..................                                                       [100%]
================================ tests coverage ================================
_____________ coverage: platform linux, python 3.12.3-final-0 _____________

Name        Stmts   Miss  Cover   Missing
-----------------------------------------
tarefa.py      39      0   100%
-----------------------------------------
TOTAL          39      0   100%
18 passed in 0.18s
```

> **Erro comum**
>
> Tratar 100% de cobertura como garantia de qualidade. Cobertura só diz que a linha RODOU, não que o resultado foi conferido. Um teste que chama a função e não faz nenhum assert aumenta a cobertura sem testar nada. Use a cobertura para achar lacunas, não como meta.

21. A suíte completa da Todo List

Tudo o que vimos forma a suíte de testes da Todo List. Rodando tudo de uma vez, na pasta do projeto:

Resultado: python -m pytest -q

```
.....................................                                    [100%]
============================== warnings summary ===============================
.../site-packages/fastapi/testclient.py:1
  .../site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
47 passed, 1 warning in 0.49s
```

A tabela resume como cada conceito da Parte 3 aparece no projeto completo:

Decisão do projeto	Conceito	Por quê
Cada RN gera ao menos um caso de teste	Rastreabilidade	A matriz RF × RN da Aula 11 diz o que precisa ser testado
Cada fluxo alternativo de caso de uso vira um teste	Rastreabilidade	Ex.: e-mail repetido no cadastro → teste do 409
A data de "hoje" é um parâmetro, não o relógio	Teste determinístico	O teste da RN-05 dá o mesmo resultado em qualquer dia
Testes de domínio sem banco e sem HTTP	Teste de unidade	São os mais rápidos e protegem as regras de negócio
Testes de API com TestClient dentro de with	Teste de integração	Executam o lifespan e exercitam a borda HTTP
Repositório (ou SQLite em memória) novo a cada teste	Isolamento	Nenhum teste depende de outro nem toca dados reais
HashFake no lugar do bcrypt	Dublê (fake)	O bcrypt é lento de propósito; nos testes, isso só atrapalha
monkeypatch na configuração, sem a função com cache	Isolamento do ambiente	Evita que o valor guardado em cache ignore o teste
Teste de chave estrangeira obrigatório	Teste de regressão crítico	Protege contra gravar dados inválidos em silêncio

> ⚠ **Avisos não são falhas**
>
> Ao rodar a suíte, podem aparecer avisos (warnings) vindos das bibliotecas instaladas, e não do seu código — como o aviso do TestClient na saída acima. Leia-os, mas não os confunda com falhas: um teste só falhou se aparecer F ou E no resultado.

PARTE 4

Ferramentas de teste

O pytest não é a única ferramenta. Nesta parte você vê o panorama do mercado e onde cada ferramenta entraria na Todo List — inclusive para rodar a suíte sozinha a cada envio de código.

22. Panorama das ferramentas

Cada tipo de teste tem suas ferramentas. Conhecer o panorama ajuda a escolher a certa para cada situação:

Ferramenta	Tipo de teste	Para que serve
unittest	Unidade (Python)	Já vem com o Python; estilo baseado em classes
pytest	Unidade e integração (Python)	O mais usado em Python; testes curtos, fixtures e parametrização (Parte 3)
pytest-cov / coverage.py	Cobertura	Mostra quais linhas os testes executaram
Hypothesis	Baseado em propriedades	Gera centenas de entradas automaticamente (Parte 6)
mutmut	Teste de mutação	Altera o código de propósito (troca < por <=, por exemplo) e confere se os testes percebem
TestClient (FastAPI)	Integração de API	Faz requisições à aplicação sem subir servidor
Postman, Insomnia	API (manual e automatizado)	Montar e repetir requisições HTTP com interface gráfica
Selenium, Playwright	Ponta a ponta (navegador)	Controlam um navegador real, clicando e preenchendo como um usuário
Locust, JMeter	Desempenho e carga	Simulam muitos usuários ao mesmo tempo
GitHub Actions	Integração contínua (CI)	Roda a suíte de testes automaticamente a cada envio de código
Z3, Dafny, TLA+	Verificação formal	Provam propriedades para todos os valores (Parte 7)

Repare no mutmut: ele faz automaticamente o que a dica da seção 12 sugere fazer à mão. Se ele trocar o `<` da RN-05 por `<=` e nenhum teste falhar, a suíte não está protegendo aquela fronteira.

Testes em integração contínua

A integração contínua (CI) faz os testes rodarem sozinhos em um servidor a cada push no GitHub. Se algum teste falhar, o commit fica marcado com um X vermelho, e o problema é descoberto em minutos. Um exemplo de configuração para a Todo List, no arquivo .github/workflows/testes.yml:

```yaml
# arquivo: .github/workflows/testes.yml
name: Testes
on: [push, pull_request]
jobs:
  testes:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: python -m pytest -q
```

PARTE 5

Testes com inteligência artificial

Assistentes de IA escrevem testes em segundos — inclusive testes que protegem defeitos. Nesta parte, a IA recebe a função da RN-05 com defeito, e você aprende a pedir e a revisar testes do jeito certo.

23. Como a IA pode ajudar

Assistentes de IA (como GitHub Copilot, Claude, ChatGPT e outros) já fazem parte do dia a dia de muitas equipes. Nos testes, eles podem acelerar bastante o trabalho:

Uso	Exemplo de pedido
Sugerir casos de teste	"Liste casos de teste, incluindo valores-limite, para a regra de situação do prazo (RN-05)."
Escrever o código dos testes	"Transforme estes 7 casos em um teste pytest parametrizado."
Gerar dados de teste	"Crie 20 títulos de tarefa incomuns para testar a validação."
Explicar uma falha	"Esta é a saída do pytest. O que ela significa?"
Revisar uma suíte	"Que cenários importantes os testes de test_tarefa.py não cobrem?"
Apoiar o teste exploratório	"Que entradas poderiam quebrar o formulário de criação de tarefas?"

24. O risco: testes que protegem o defeito

O maior risco de pedir testes a uma IA é entregar a ela apenas o código. Sem a regra de negócio, ela só consegue descrever o que o código faz hoje — inclusive os defeitos. O exemplo abaixo mostra o tipo de teste que costuma surgir quando um assistente recebe somente a função da RN-05 com defeito, da seção 12:

```
# arquivo: test_prazo_gerado_por_ia.py
from datetime import date, timedelta

from prazo import situacao_do_prazo

HOJE = date(2026, 3, 10)

# Testes no estilo que um assistente costuma gerar quando recebe SÓ o código
# da função (sem a regra de negócio): ele descreve o que o código FAZ hoje.


def test_prazo_daqui_a_10_dias_esta_no_prazo():
    assert situacao_do_prazo("pendente", HOJE + timedelta(days=10), HOJE) == "no prazo"


def test_prazo_hoje_esta_atrasada():
    assert situacao_do_prazo("pendente", HOJE, HOJE) == "atrasada"  # "congela" o erro


def test_prazo_amanha_vence_em_breve():
    amanha = HOJE + timedelta(days=1)
    assert situacao_do_prazo("pendente", amanha, HOJE) == "vence em breve"
```

Contra o código COM defeito: python -m pytest test_prazo_gerado_por_ia.py -q

```
...                                                                      [100%]
3 passed in 0.11s
```

Contra o código CORRIGIDO: python -m pytest test_prazo_gerado_por_ia.py -q --tb=line

```
.F.                                                                      [100%]
================================== FAILURES ===================================
E   AssertionError: assert 'vence em breve' == 'atrasada'

      - atrasada
      + vence em breve
test_prazo_gerado_por_ia.py:16: AssertionError: assert 'vence em breve' == 'atrasada'
=========================== short test summary info ===========================
FAILED test_prazo_gerado_por_ia.py::test_prazo_hoje_esta_atrasada - Assertion...
1 failed, 2 passed in 0.13s
```

O resultado é o oposto do que um teste deveria fazer: tudo passa enquanto o defeito existe, e um teste falha quando alguém o corrige. O teste "congelou" o BUG-07. Quem lê só a saída verde acredita que está tudo certo — e quem corrigir o defeito vai achar que estragou alguma coisa.

> **Erro comum**
>
> Aceitar testes gerados por IA sem ler. A IA também pode inventar funções que não existem na biblioteca, escrever asserts que nunca falham ou criar testes que dependem uns dos outros. Teste gerado é rascunho: precisa de revisão como qualquer código.

25. Como usar a IA com segurança

Um pedido bem feito

A regra de ouro é dar à IA a especificação, e não só o código. Compare:

```
PEDIDO FRACO:
Escreva testes para esta função. [cola o código]

PEDIDO BOM:
Regra de negócio (RN-05): uma tarefa concluída tem situação "concluida".
Uma pendente sem prazo é "sem prazo". Com prazo antes de hoje, "atrasada";
de hoje até 2 dias depois, "vence em breve"; mais de 2 dias depois, "no prazo".
1. Liste os casos de teste usando partição de equivalência,
   valores-limite e tabela de decisão, explicando cada um.
2. Depois, escreva um teste pytest parametrizado com esses casos,
   usando uma data fixa como "hoje".
3. Use a REGRA como referência, não o código.
```

Checklist para revisar testes gerados por IA

- Cada valor esperado vem da regra de negócio, e não do comportamento atual do código?
- Os valores-limite estão presentes (exatamente no limite e logo ao lado)?
- As combinações da tabela de decisão estão presentes (como a tarefa concluída com prazo vencido)?
- Existem testes para o que o sistema deve recusar (pytest.raises)?
- Todas as funções e parâmetros usados existem de verdade na biblioteca?
- Cada teste é capaz de falhar? (Quebre o código de propósito e confira.)
- Os testes funcionam sozinhos, em qualquer ordem e em qualquer dia?

> ⚠ **Privacidade**
>
> Nunca cole em um assistente de IA senhas, tokens, chaves de API, dados pessoais de clientes ou código que a empresa não autorizou compartilhar. O conteúdo do arquivo .env, por exemplo, nunca deve sair da sua máquina.

PARTE 6

Testes matemáticos

Até aqui, cada teste conferia um exemplo escolhido por uma pessoa. Nesta parte, a Todo List é testada com propriedades: frases que valem para TODAS as entradas, conferidas com centenas de exemplos gerados automaticamente.

26. Oráculos, propriedades e invariantes

Todo teste precisa de um oráculo: algo que diga qual é o resultado correto. Nos testes que vimos até aqui, o oráculo é a pessoa que escreveu o valor esperado ("uma tarefa pendente que vence hoje está 'vence em breve'"). Mas e quando há infinitas entradas possíveis, ou quando calcular o resultado correto à mão é difícil? A matemática oferece outras formas de oráculo:

Conceito	O que é	Exemplo na Todo List
Propriedade	Algo que vale para TODAS as entradas	Uma lista sem títulos repetidos não tem nenhum título repetido — e não perdeu nenhum
Invariante	Algo que nunca muda durante a execução	Uma tarefa está concluída se, e somente se, tem data de conclusão
Relação metamórfica	Como a saída deve mudar quando a entrada muda de um jeito conhecido	Duplicar a lista de tarefas não muda o percentual de concluídas
Operação inversa	Uma operação que desfaz a outra	Exportar uma tarefa para texto e importar de volta devolve a mesma tarefa

27. Testes baseados em propriedades (Hypothesis)

Em vez de escrever os exemplos à mão, descrevemos uma propriedade e deixamos a ferramenta gerar as entradas. A biblioteca Hypothesis (`pip install hypothesis`) gera, por padrão, 100 entradas diferentes por teste — e, quando encontra uma falha, procura automaticamente o menor exemplo que ainda falha. A ideia nasceu na ferramenta QuickCheck, da linguagem Haskell.

Vamos testar duas funções novas da Todo List, ambas com defeito. A primeira exporta uma tarefa para uma linha de texto (para gerar um arquivo de backup) e a importa de volta — um par de operações inversas:

```
# arquivo: exportacao.py (versão com defeito)
def exportar(titulo, prioridade):
    """Transforma uma tarefa em uma linha de texto: "titulo;prioridade"."""
    return f"{titulo};{prioridade}"


def importar(linha):
    """Lê de volta uma linha gerada por exportar()."""
    titulo, prioridade = linha.split(";")
    return titulo, prioridade
```

A segunda remove títulos repetidos de uma lista importada — mas da forma errada, removendo itens da lista enquanto a percorre:

```
# arquivo: titulos.py (versão com defeito)
def remover_titulos_repetidos(titulos):
    """Devolve a lista de títulos sem repetições."""
    titulos = titulos.copy()
    for titulo in titulos:  # ERRO: remove da lista que está percorrendo
        if titulos.count(titulo) > 1:
            titulos.remove(titulo)
    return titulos
```

As propriedades

```
# arquivo: test_propriedades.py
from hypothesis import given, strategies as st

from exportacao import exportar, importar
from titulos import remover_titulos_repetidos

PRIORIDADES = ["baixa", "media", "alta", "urgente"]


@given(titulo=st.text(min_size=1), prioridade=st.sampled_from(PRIORIDADES))
def test_importar_desfaz_exportar(titulo, prioridade):
    # Propriedade: para QUALQUER título, importar devolve o que foi exportado
    assert importar(exportar(titulo, prioridade)) == (titulo, prioridade)


@given(st.lists(st.sampled_from(["Estudar", "Ler", "Pagar boleto"])))
def test_resultado_nao_tem_repetidos(titulos):
    resultado = remover_titulos_repetidos(titulos)
    assert len(resultado) == len(set(resultado))  # propriedade 1: nada se repete
    assert set(resultado) == set(titulos)  # propriedade 2: nada se perde
```

`st.text(min_size=1)` gera títulos quaisquer, com pelo menos um caractere — letras, números, acentos, símbolos, emojis. `st.sampled_from(...)` sorteia valores de uma lista, e `st.lists(...)` gera listas desses valores, de tamanhos variados.

Resultado: python -m pytest test_propriedades.py -q --tb=short

```
FF                                                                       [100%]
================================== FAILURES ===================================
___________________________ test_importar_desfaz_exportar ____________________________
test_propriedades.py:10: in test_importar_desfaz_exportar
    def test_importar_desfaz_exportar(titulo, prioridade):
test_propriedades.py:12: in test_importar_desfaz_exportar
    assert importar(exportar(titulo, prioridade)) == (titulo, prioridade)
exportacao.py:8: in importar
    titulo, prioridade = linha.split(";")
E   ValueError: too many values to unpack (expected 2)
E   Failing test case: test_importar_desfaz_exportar(
E       titulo=';',
E       prioridade='baixa',  # or any other generated value
E   )
_____________________ test_resultado_nao_tem_repetidos ______________________
test_propriedades.py:16: in test_resultado_nao_tem_repetidos
    def test_resultado_nao_tem_repetidos(titulos):
test_propriedades.py:18: in test_resultado_nao_tem_repetidos
    assert len(resultado) == len(set(resultado))  # propriedade 1: nada se repete
E   AssertionError: assert 2 == 1
E    +  where 2 = len(['Estudar', 'Estudar'])
E    +  and   1 = len({'Estudar'})
E    +    where {'Estudar'} = set(['Estudar', 'Estudar'])
E   Failing test case: test_resultado_nao_tem_repetidos(
E       titulos=['Estudar', 'Estudar', 'Estudar', 'Estudar'],
E   )
========================== short test summary info ===========================
FAILED test_propriedades.py::test_importar_desfaz_exportar - ValueError: too ...
FAILED test_propriedades.py::test_resultado_nao_tem_repetidos - AssertionErro...
2 failed in 2.61s
```

O que o Hypothesis encontrou

Na exportação, o menor contraexemplo é o título ";". A linha exportada vira ";;baixa", e o `split` encontra três pedaços em vez de dois. Um título real como "Reunião; levar notebook" quebraria a importação de um backup inteiro — a mesma ideia do título com ponto e vírgula da seção 9, agora encontrada sem ninguém pensar nela.

Na remoção de títulos, a lista com quatro "Estudar" devolve dois: ao remover um item, os seguintes andam uma posição para trás, e o for pula elementos.

Ninguém precisou imaginar esses casos: bastou descrever o que deveria ser sempre verdade. As correções:

```
# arquivo: exportacao.py (versão corrigida)
def exportar(titulo, prioridade):
    """Transforma uma tarefa em uma linha de texto: "titulo;prioridade"."""
    return f"{titulo};{prioridade}"


def importar(linha):
    """Lê de volta uma linha gerada por exportar()."""
    titulo, prioridade = linha.rsplit(";", 1)  # só o ÚLTIMO ";" separa
    return titulo, prioridade
```

```
# arquivo: titulos.py (versão corrigida)
def remover_titulos_repetidos(titulos):
    """Devolve a lista de títulos sem repetições."""
    resultado = []
    for titulo in titulos:
        if titulo not in resultado:
            resultado.append(titulo)
    return resultado
```

Depois das correções: python -m pytest test_propriedades.py -q

```
..                                                                       [100%]
2 passed in 1.90s
```

28. Testes metamórficos e números decimais

O problema do ponto flutuante

No computador, muitos números decimais não têm representação exata. Em Python, `0.1 + 0.2` resulta em 0.30000000000000004. Comparar decimais com `==` é uma armadilha clássica — a solução é comparar com uma tolerância, usando `pytest.approx` (ou `math.isclose`).

A Todo List mostra, no topo da lista, o percentual de tarefas concluídas. Com 7 de 25 tarefas concluídas, a conta é 7 ÷ 25 × 100 = 28. No computador, porém, o resultado é 28.000000000000004.

Testes metamórficos

Para listas aleatórias, nem sempre sabemos o percentual exato. Mas sabemos como ele deve mudar quando a lista muda: duplicar a lista não muda o percentual; reordenar também não; acrescentar uma tarefa concluída nunca faz o percentual cair. Essas relações viram testes:

```
# arquivo: estatisticas.py
def percentual_concluido(status):
    """Percentual de tarefas concluídas, dada a lista com o status de cada uma."""
    if len(status) == 0:
        raise ValueError("Não há tarefas na lista.")
    return status.count("concluida") / len(status) * 100
```

```
# arquivo: test_estatisticas.py
import pytest
from hypothesis import given, strategies as st
from estatisticas import percentual_concluido
listas = st.lists(st.sampled_from(["pendente", "concluida"]), min_size=1, max_size=50)
SETE_DE_25 = ["concluida"] * 7 + ["pendente"] * 18  # 7 de 25 tarefas = 28%
def test_percentual_comparado_com_igual():
    assert percentual_concluido(SETE_DE_25) == 28.0  # falha: ponto flutuante

def test_percentual_comparado_com_approx():
    assert percentual_concluido(SETE_DE_25) == pytest.approx(28.0)  # tolerância

# Testes metamórficos: não sabemos o percentual exato de cada lista aleatória,
# mas sabemos como ele deve MUDAR (ou não mudar) quando a lista muda.
@given(listas)
def test_duplicar_a_lista_nao_muda_o_percentual(status):
    esperado = percentual_concluido(status)
    assert percentual_concluido(status * 2) == pytest.approx(esperado)

@given(listas)
def test_reordenar_nao_muda_o_percentual(status):
    esperado = percentual_concluido(status)
    assert percentual_concluido(sorted(status)) == pytest.approx(esperado)

@given(listas)
def test_mais_uma_concluida_nunca_diminui_o_percentual(status):
    assert percentual_concluido(status + ["concluida"]) >= percentual_concluido(status)
```

Resultado: python -m pytest test_estatisticas.py -q --tb=short

```
F....                                                                    [100%]
================================== FAILURES ===================================
_____________________ test_percentual_comparado_com_igual _____________________
test_estatisticas.py:11: in test_percentual_comparado_com_igual
    assert percentual_concluido(SETE_DE_25) == 28.0  # falha: ponto flutuante
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E   AssertionError: assert 28.000000000000004 == 28.0
E    +  where 28.000000000000004 = percentual_concluido(['concluida', 'concluida', 'concluida',
'concluida', 'concluida', 'concluida', ...])
=========================== short test summary info ============================
FAILED test_estatisticas.py::test_percentual_comparado_com_igual - AssertionE...
1 failed, 4 passed in 0.46s
```

Só o primeiro teste falhou — e pelo motivo esperado: 7 ÷ 25 × 100 dá 28.000000000000004, e não exatamente 28. Os quatro testes que usam a tolerância certa passaram, incluindo os três testes metamórficos, cada um com 100 listas geradas pelo Hypothesis.

29. Explosão combinatória e testes em pares

A tela da Todo List tem um formulário de busca com 5 filtros (status, prioridade, ordenação, período e situação do prazo), cada um com 4 opções. Testar todas as combinações exige $4 \times 4 \times 4 \times 4 \times 4 = 4^5 = 1.024$ casos de teste. Com 10 filtros, seriam mais de um milhão. É a explosão combinatória.

Estudos práticos mostram que a maioria dos defeitos de combinação é provocada pela interação de apenas um ou dois parâmetros. O teste em pares (pairwise) garante que todo par de valores apareça junto em pelo menos um caso de teste. Para 5 parâmetros com 4 valores, a matemática (os chamados arranjos ortogonais) mostra que 16 casos bastam para cobrir todos os pares — em vez de 1.024.

Estratégia	Casos para 5 filtros com 4 opções	O que garante
Todas as combinações	1.024	Toda combinação possível
Testes em pares (pairwise)	16	Todo par de valores aparece junto ao menos uma vez
Um caso por valor	4	Cada valor aparece ao menos uma vez (cobertura fraca)

PARTE 7

Verificação formal

Testes conferem exemplos; provas conferem todos os casos. Nesta parte, as regras da Todo List são provadas matematicamente — e a prova encontra uma brecha nos requisitos e um conflito na regra de ordenação que nenhum teste tinha visto.

30. Testar não é provar

Edsger Dijkstra, um dos pioneiros da computação, observou que testes podem mostrar a presença de defeitos, mas nunca a sua ausência. Os 7 testes da RN-05 conferem 7 situações; o Hypothesis confere 100 exemplos por propriedade. Mas existem infinitas datas possíveis. Como ter certeza de que a regra vale para todas?

A verificação formal usa matemática e lógica para provar que um programa (ou um modelo dele) obedece a uma especificação para todas as entradas possíveis. Quando a prova falha, a ferramenta costuma devolver um contraexemplo concreto.

	Testes	Verificação formal
O que verifica	Os casos escolhidos	Todos os casos possíveis
Resultado	"Não encontrei defeitos nestes casos"	"Está provado" ou "aqui está um contraexemplo"
Custo	Baixo a médio	Alto: exige especificação precisa e ferramentas especializadas
Uso típico	Qualquer software	Sistemas críticos: aviação, criptografia, sistemas operacionais, infraestrutura de nuvem

31. Especificação: pré-condições, pós-condições e invariantes

Para provar algo, primeiro é preciso dizer com precisão o que o código deve fazer. Isso é feito com três ferramentas da lógica. Veja-as aplicadas ao método `concluir()` da entidade Tarefa:

Conceito	Significado	Exemplo na Todo List
Pré-condição	O que precisa ser verdade antes da execução	A tarefa existe (foi encontrada pelo id)
Pós-condição	O que será verdade depois, se a pré-condição valia	status = "concluida" e concluido_em preenchido; se a tarefa já estava concluída, concluido_em continua igual ao de antes
Invariante de classe	O que vale para todo objeto, antes e depois de qualquer método	status = "concluida" se, e somente se, concluido_em estiver preenchido
Invariante de laço	O que é verdade no início de cada volta de um laço	Ver o exemplo abaixo

Essa ideia é formalizada pela tripla de Hoare, escrita {P} C {Q}: se a pré-condição P vale antes de executar o comando C, então a pós-condição Q vale depois.

Exemplo: provando um laço à mão

A Todo List conta quantas tarefas da lista estão concluídas:

```
def contar_concluidas(tarefas):           # pré-condição: tarefas é uma lista
    total = 0
    i = 0
    while i < len(tarefas):
        if tarefas[i].status == "concluida":
            total += 1
        i += 1
    return total   # pós-condição: total = nº de concluídas na lista inteira
```

A invariante do laço é: **total = número de tarefas concluídas entre as i primeiras** (de tarefas[0] até tarefas[i − 1]). A prova tem três passos:

Passo	O que mostrar	Por quê
1. Início	A invariante vale antes do laço	i = 0 e total = 0: entre as "zero primeiras" tarefas não há nenhuma concluída
2. Manutenção	Se vale no início de uma volta, vale no fim dela	A volta olha tarefas[i]: se está concluída, total ganha 1; depois i ganha 1. Assim, total passa a contar as concluídas entre as i + 1 primeiras
3. Término	Quando o laço para, a invariante dá a pós-condição	O laço para com i = len(tarefas): "as i primeiras" são a lista inteira

O passo 3 também exige mostrar que o laço termina: i aumenta 1 a cada volta, o tamanho da lista não muda, e o laço para quando i chega ao tamanho.

32. Provadores automáticos: o Z3

Provas à mão são trabalhosas e também podem ter erros. Provadores automáticos fazem esse trabalho. O Z3, desenvolvido pela Microsoft Research, é um dos mais usados e pode ser chamado pelo Python (`pip install z3-solver`). A estratégia é sempre a mesma: pedir ao Z3 que procure um contraexemplo; se não existir nenhum, a propriedade está provada.

O Z3 não lê o código Python da Todo List: escrevemos um **modelo** das regras em lógica. Na RN-05, por exemplo, d representa quantos dias faltam até o prazo (prazo − hoje); d negativo significa que o prazo já passou.

```
# arquivo: verificacao_z3.py
from z3 import Int, BitVec, And, Or, Not, Implies, Solver, unsat


def verificar(nome, propriedade):
    """Procura um contraexemplo. Se não existir nenhum,
    a propriedade está provada."""
    solver = Solver()
    solver.add(Not(propriedade))
    if solver.check() == unsat:
        print(f"[{nome}] PROVADO: vale para todos os valores possíveis.")
    else:
        print(f"[{nome}] CONTRAEXEMPLO ENCONTRADO:")
        print(f"   {solver.model()}")


d = Int("d")  # dias até o prazo (prazo - hoje); negativo = o prazo já passou

# 1) RN-05 como estava no primeiro rascunho dos requisitos:
#    "atrasada: prazo antes de hoje"; "vence em breve: nos próximos 2 dias"
breve_rascunho = And(d >= 1, d <= 2)  # "próximos 2 dias" = amanhã e depois
verificar("rascunho: toda tarefa com prazo tem uma situação",
          Or(d < 0, breve_rascunho, d > 2))

# 2) RN-05 na versão final: "vence em breve: de hoje até 2 dias depois"
atrasada, breve, no_prazo = d < 0, And(d >= 0, d <= 2), d > 2
exatamente_uma = Or(And(atrasada, Not(breve), Not(no_prazo)),
                    And(Not(atrasada), breve, Not(no_prazo)),
                    And(Not(atrasada), Not(breve), no_prazo))
verificar("final: toda tarefa tem exatamente uma situação", exatamente_uma)

# 3) Amanhã falta um dia a menos: quem está atrasada hoje continua atrasada
verificar("atrasada hoje continua atrasada amanhã", Implies(d < 0, d - 1 < 0))

# 4) O id das tarefas guardado em uma coluna de inteiros de 32 bits
ultimo = BitVec("ultimo", 32)
verificar("o próximo id é sempre maior que o último",
          Implies(ultimo >= 1, ultimo + 1 > ultimo))

# 5) RN-06: pontuação = peso da prioridade x 10 + bônus da situação
p1, p2, bonus2 = Int("p1"), Int("p2"), Int("bonus2")
valido = And(p1 >= 1, p1 <= 4, p2 >= 1, p2 <= 4,  # pesos: baixa=1 ... urgente=4
             Or(bonus2 == 0, bonus2 == 10))       # tarefa 2 NÃO está atrasada
atrasada_pontos = p1 * 10 + 20                    # tarefa 1 está atrasada
outra_pontos = p2 * 10 + bonus2
verificar("mesma prioridade: a atrasada fica à frente",
          Implies(And(valido, p1 == p2), atrasada_pontos > outra_pontos))
verificar("toda atrasada fica à frente das não atrasadas",
          Implies(valido, atrasada_pontos > outra_pontos))
```

Resultado: python verificacao_z3.py

```
[rascunho: toda tarefa com prazo tem uma situação] CONTRAEXEMPLO ENCONTRADO:
   [d = 0]
[final: toda tarefa tem exatamente uma situação] PROVADO: vale para todos os valores possíveis.
[atrasada hoje continua atrasada amanhã] PROVADO: vale para todos os valores possíveis.
[o próximo id é sempre maior que o último] CONTRAEXEMPLO ENCONTRADO:
   [ultimo = 2147483647]
[mesma prioridade: a atrasada fica à frente] PROVADO: vale para todos os valores possíveis.
[toda atrasada fica à frente das não atrasadas] CONTRAEXEMPLO ENCONTRADO:
   [p2 = 3, bonus2 = 0, p1 = 1]
```

Caso 1 — a brecha no rascunho dos requisitos

No primeiro rascunho, a RN-05 dizia "vence em breve: nos próximos 2 dias", e quem leu entendeu "amanhã e depois de amanhã". O Z3 procurou uma tarefa que não caísse em nenhuma situação e encontrou d = 0: o dia de hoje não estava coberto por nenhuma frase. Cada pessoa preencheu a lacuna do seu jeito — e foi assim que nasceu o engano da seção 3, em que a tarefa que vence hoje virou "atrasada".

Na versão final, com "de hoje até 2 dias depois", o Z3 prova que toda tarefa cai em exatamente uma situação — para todos os inteiros, e não para 7 exemplos.

Caso 2 — atrasada hoje, atrasada amanhã

A propriedade parece óbvia, mas é o tipo de garantia que um usuário espera sem pensar: uma tarefa atrasada nunca "desatrasa" sozinha com a passagem do tempo. O Z3 a prova para qualquer número de dias.

Caso 3 — o id que estoura

Os ids das tarefas são sequenciais, e parece óbvio que o próximo id é sempre maior que o último. Mas, se o id for guardado como inteiro de 32 bits — como o tipo INTEGER de muitos bancos de dados, ou o int de Java e C —, o maior valor possível é 2.147.483.647. O Z3 encontrou exatamente esse valor: somando 1, em 32 bits, o número "dá a volta" e vira negativo. No banco de dados, a próxima inserção falharia. Por isso tabelas que podem crescer muito usam inteiros de 64 bits (BIGINT).

> **Um caso real**
>
> O mesmo tipo de estouro de 32 bits existiu por anos na busca binária da biblioteca padrão do Java: o cálculo do meio, (inicio + fim) / 2, estourava com listas gigantescas. O defeito foi divulgado por Joshua Bloch em 2006. E o Ariane 5, da seção 1, também caiu por um número grande demais para o seu tipo de dado.

> **⚠ E em Python?**
>
> Em Python, os inteiros não têm limite de tamanho, então o RepositorioEmMemoria nunca estoura. O modelo usa inteiros de 32 bits para mostrar o problema que existe em bancos de dados e em outras linguagens.

Caso 4 — a regra de negócio que tinha uma falha

O Z3 provou que, entre duas tarefas de mesma prioridade, a atrasada sempre fica à frente. Mas, no briefing, a cliente disse que tarefas atrasadas devem aparecer sempre no topo da lista — e essa propriedade falhou. O contraexemplo: p1 = 1 (baixa e atrasada: 1 × 10 + 20 = 30 pontos) e p2 = 3 com bônus 0 (alta e no prazo: 3 × 10 = 30 pontos). As duas empatam, e uma tarefa urgente no prazo (40 pontos) passaria à frente da atrasada.

Repare: o código está correto — ele faz exatamente o que a RN-06 manda, e todos os testes da seção 19 passam. O que está em conflito é a própria regra com a expectativa da cliente. É a diferença entre verificação e validação da seção 4, e um ótimo assunto para levar de volta à cliente: ou o bônus de atraso sobe (com 50 pontos, o Z3 prova a propriedade na hora), ou a expectativa é ajustada.

33. Ferramentas e usos reais

Ferramenta	O que é
Z3	Provador automático (resolvedor SMT) da Microsoft Research, usado por muitas outras ferramentas
Dafny	Linguagem de programação com verificação embutida: o código só compila se as especificações forem provadas
TLA+	Linguagem para especificar e verificar sistemas concorrentes e distribuídos, criada por Leslie Lamport
Coq (hoje Rocq) e Lean	Assistentes de prova: a pessoa constrói a prova e a ferramenta confere cada passo
Frama-C	Plataforma de análise e verificação de programas em C
CrossHair	Procura contraexemplos para contratos (pré e pós-condições) em funções Python
mypy	Verificador de tipos para Python: uma forma leve de verificação, que pega erros antes de executar

Onde a verificação formal é usada

- A Amazon Web Services publicou, em 2015, como usa TLA+ para encontrar defeitos sutis no projeto de sistemas de armazenamento e bancos de dados distribuídos.
- O núcleo de sistema operacional seL4 foi provado formalmente correto em relação à sua especificação, um marco publicado em 2009.
- O CompCert é um compilador de C cuja tradução foi provada correta, usado em contextos em que um erro do compilador seria inaceitável.

Limitações

- A prova vale para a especificação: se a especificação estiver errada, o programa será "provadamente" errado — como no caso 4 da seção 32.
- Prova-se um modelo do sistema; detalhes fora do modelo (hardware, rede, bibliotecas, o próprio código Python) ficam de fora.
- O custo é alto. Por isso ela é reservada às partes mais críticas, e os testes continuam sendo a base do dia a dia.

PARTE 8

Laboratório: identificação de erros

Hora de praticar tudo de uma vez. Cada exercício traz uma função da Todo List com um defeito escondido. Use as técnicas da Parte 2 para escolher os casos e o pytest da Parte 3 para encontrá-lo.

34. Como usar o laboratório (exercícios L1 a L8)

Cada exercício traz uma função com a especificação (na docstring) e um defeito escondido. Os exercícios estão em ordem crescente de dificuldade. Para cada um:

1. Leia a especificação e liste os casos de teste usando as técnicas da seção 8 (partições, limites, tabela de decisão).
2. Escreva os testes em pytest — a partir da especificação, não do código.
3. Rode os testes. Algum precisa falhar: é ele que identifica o defeito.
4. Explique o defeito em uma frase, corrija o código e rode os testes de novo até tudo ficar verde.

> **Dica**
>
> Um teste que só usa valores "comuns" provavelmente vai passar. Os defeitos deste laboratório estão nas fronteiras, nas listas sem nenhuma tarefa atrasada, nas maiúsculas, nos espaços repetidos e nas combinações de status com prazo.
>
> O gabarito está no anexo, no final do e-book.

L1 — Buscar tarefas por palavra

```
# arquivo: buscar.py
def buscar_por_palavra(titulos, palavra):
    """Devolve os títulos que contêm a palavra, sem diferenciar
    maiúsculas de minúsculas."""
    return [t for t in titulos if palavra in t]
```

L2 — Menor número de dias restantes

```
# arquivo: dias_restantes.py
def menos_dias_restantes(dias):
    """Recebe uma lista NÃO vazia com os dias que faltam até o prazo de
    cada tarefa (negativo = atrasada) e devolve o menor valor."""
    menor = 0
    for d in dias:
        if d < menor:
            menor = d
    return menor
```

L3 — Normalizar o título

```
# arquivo: normalizar.py
def normalizar_titulo(titulo):
    """Remove os espaços das pontas e troca qualquer sequência de
    espaços no meio do título por um único espaço."""
    return titulo.strip().replace("  ", " ")
```

L4 — Paginação da lista de tarefas

```
# arquivo: paginacao.py
def paginar(tarefas, pagina, por_pagina):
    """Devolve as tarefas da página informada. As páginas começam em 1."""
    inicio = pagina * por_pagina
    return tarefas[inicio:inicio + por_pagina]
```

L5 — Contar tarefas atrasadas

```
# arquivo: atrasadas.py
def contar_atrasadas(tarefas):
    """tarefas é uma lista de pares (status, dias_restantes).
    Atrasada = tarefa PENDENTE com dias_restantes menor que zero."""
    total = 0
    for status, dias in tarefas:
        if dias < 0:
            total += 1
    return total
```

L6 — Remover uma tarefa

```
# arquivo: remocao.py
class TarefaNaoEncontrada(Exception):
    pass


def remover_tarefa(tarefas, tarefa_id):
    """Remove do dicionário 'tarefas' a tarefa com o id informado.
    Se o id não existir, levanta TarefaNaoEncontrada."""
    tarefas.pop(tarefa_id, None)
```

L7 — Filtrar por prioridade mínima

```
# arquivo: filtro.py
PRIORIDADES = ["baixa", "media", "alta", "urgente"]


def filtrar_por_prioridade(tarefas, minima):
    """tarefas é uma lista de pares (titulo, prioridade). Devolve os títulos
    das tarefas com prioridade IGUAL OU MAIOR que 'minima'."""
    limite = PRIORIDADES.index(minima)
    return [titulo for titulo, prioridade in tarefas
            if PRIORIDADES.index(prioridade) > limite]
```

L8 — Tarefa e a RN-03

```
# arquivo: tarefa_lab.py
from datetime import datetime


class Tarefa:
    """RN-03: concluir registra a data de conclusão; concluir de novo uma
    tarefa já concluída preserva a data original; reabrir apaga a data."""

    def __init__(self, titulo):
        self.titulo = titulo
        self.status = "pendente"
        self.concluido_em = None

    def concluir(self):
        self.status = "concluida"
        self.concluido_em = datetime.now()

    def reabrir(self):
        self.status = "pendente"
```

35. Resumo

- Testar é executar o software de forma planejada para encontrar defeitos antes do usuário; quanto mais cedo, mais barato.
- Erro é o engano humano, defeito é o que ficou no código, falha é o comportamento errado observado — como a tarefa que vence hoje marcada como "atrasada".
- A pirâmide recomenda muitos testes de unidade, alguns de integração e poucos de ponta a ponta.
- Partição de equivalência, valor-limite e tabela de decisão escolhem poucos casos com grande chance de achar defeitos; na RN-05, o valor-limite e a tabela encontraram um defeito cada.
- Receber a data de "hoje" como parâmetro, em vez de ler o relógio, deixa os testes com datas previsíveis.
- O pytest organiza testes com assert, fixtures, parametrização, pytest.raises e pytest.approx.
- Testes precisam ser isolados; dublês (fake, stub, mock) substituem partes lentas ou com efeitos colaterais.
- No TDD, o teste vem antes do código e guia a implementação.
- Cobertura mostra onde ninguém testou, mas não garante que o resultado foi conferido.
- A IA acelera a escrita de testes, mas precisa receber a especificação e ter cada teste revisado.
- Testes baseados em propriedades e metamórficos verificam relações que valem para muitas entradas geradas automaticamente.
- A verificação formal prova propriedades para todos os valores e pode encontrar falhas até na própria especificação.

36. Glossário

Termo	Significado
Asserção (assert)	Comando que verifica uma condição e falha se ela for falsa
Caso de teste	Situação, passos e resultado esperado de um teste
CI (integração contínua)	Execução automática dos testes a cada envio de código
Cobertura	Proporção do código executada pelos testes

Termo	Significado
Contraexemplo	Entrada concreta que mostra que uma propriedade é falsa
Defeito (bug)	Problema no código que pode causar uma falha
Dublê de teste	Substituto de uma parte do sistema durante o teste (fake, stub, mock)
Erro (engano)	Ação humana equivocada que dá origem a um defeito
Falha	Comportamento errado observado durante a execução
Fixture	Função que prepara o ponto de partida de um teste
Invariante	Condição que permanece verdadeira durante toda a execução
Oráculo	Aquilo que define o resultado correto de um teste
Pairwise	Técnica que cobre todos os pares de valores com poucos casos
Partição de equivalência	Grupo de entradas que o sistema deveria tratar da mesma forma
Regressão	Defeito que reaparece ou surge em algo que antes funcionava
Tabela de decisão	Lista de todas as combinações de condições e do resultado esperado para cada uma
TDD	Desenvolvimento guiado por testes: o teste é escrito antes do código
Teste baseado em propriedades	Teste que verifica uma propriedade com entradas geradas automaticamente
Teste instável (flaky)	Teste que às vezes passa e às vezes falha sem mudança no código (por depender do relógio, da ordem ou da sorte)
Teste metamórfico	Teste que verifica como a saída muda quando a entrada muda de forma conhecida
Valor-limite	Valor na fronteira entre duas partições de entrada
Verificação formal	Uso de lógica e matemática para provar propriedades de um programa

ANEXO

Gabarito do laboratório

Para cada exercício: os testes que revelam o defeito, a mensagem de falha obtida ao rodá-los contra o código com defeito e a correção. Todos os testes foram executados contra as duas versões: contra o código com defeito, 11 testes falham; contra o código corrigido, os 19 testes passam.

L1 — Buscar tarefas por palavra

```
# arquivo: test_l1_buscar.py
from buscar import buscar_por_palavra

TITULOS = ["Estudar Pytest", "Ler livro", "Revisar testes do pytest"]


def test_palavra_em_minusculas():
    assert buscar_por_palavra(TITULOS, "livro") == ["Ler livro"]


def test_maiusculas_e_minusculas_nao_importam():
    assert buscar_por_palavra(TITULOS, "pytest") == ["Estudar Pytest",
                                                     "Revisar testes do pytest"]
```

Falha obtida contra o código com defeito

```
assert ['Revisar testes do pytest'] == ['Estudar Pyt...es do pytest']
```

Defeito e correção

A comparação diferencia maiúsculas de minúsculas: "Estudar Pytest" não contém "pytest". Correção: comparar `palavra.lower()` com `t.lower()`.

L2 — Menor número de dias restantes

```
# arquivo: test_l2_dias_restantes.py
from dias_restantes import menos_dias_restantes


def test_com_tarefas_atrasadas():
    assert menos_dias_restantes([-3, 5, -1]) == -3


def test_sem_nenhuma_tarefa_atrasada():
    assert menos_dias_restantes([4, 2, 9]) == 2
```

Falha obtida contra o código com defeito

```
assert 0 == 2
```

Defeito e correção

`menor` começa em 0, valor que nunca é superado quando nenhuma tarefa está atrasada (todos os números positivos). Correção: `menor = dias[0]`.

L3 — Normalizar o título

```
# arquivo: test_l3_normalizar.py
import pytest

from normalizar import normalizar_titulo


@pytest.mark.parametrize("entrada, esperado", [
    ("Estudar pytest", "Estudar pytest"),       # nada a fazer
    ("  Estudar pytest  ", "Estudar pytest"),   # espaços nas pontas
    ("Estudar  pytest", "Estudar pytest"),      # 2 espaços no meio
    ("Estudar   pytest", "Estudar pytest"),     # 3 espaços no meio
])
def test_normalizar_titulo(entrada, esperado):
    assert normalizar_titulo(entrada) == esperado
```

Falha obtida contra o código com defeito

```
assert 'Estudar  pytest' == 'Estudar pytest'
```

Defeito e correção

`replace` troca cada par de espaços por um só, em uma única passada: com três espaços seguidos, sobram dois. Com exatamente dois, o defeito fica escondido. Correção: `" ".join(titulo.split())`.

L4 — Paginação da lista de tarefas

```
# arquivo: test_l4_paginacao.py
from paginacao import paginar

TAREFAS = ["a", "b", "c", "d", "e"]


def test_primeira_pagina():
    assert paginar(TAREFAS, 1, 2) == ["a", "b"]


def test_ultima_pagina_incompleta():
    assert paginar(TAREFAS, 3, 2) == ["e"]
```

Falha obtida contra o código com defeito

```
assert ['c', 'd'] == ['a', 'b']
assert [] == ['e']
```

Defeito e correção

As páginas começam em 1, mas o cálculo trata a primeira como 0 (erro de "um a mais"). Correção: `inicio = (pagina - 1) * por_pagina`.

L5 — Contar tarefas atrasadas

```
# arquivo: test_l5_atrasadas.py
from atrasadas import contar_atrasadas


def test_so_pendentes():
    assert contar_atrasadas([("pendente", -2), ("pendente", 3)]) == 1


def test_concluida_vencida_nao_conta():
    assert contar_atrasadas([("concluida", -2), ("pendente", -1)]) == 1
```

Falha obtida contra o código com defeito

```
assert 2 == 1
```

Defeito e correção

A função ignora o status: uma tarefa concluída com prazo vencido é contada como atrasada — a mesma linha esquecida da tabela de decisão da seção 8. Correção: `if status == "pendente" and dias < 0:`.

L6 — Remover uma tarefa

```
# arquivo: test_l6_remocao.py
import pytest

from remocao import remover_tarefa, TarefaNaoEncontrada


def test_remover_tarefa_existente():
    tarefas = {1: "Estudar", 2: "Ler"}
    remover_tarefa(tarefas, 1)
    assert tarefas == {2: "Ler"}


def test_id_inexistente_e_recusado():
    with pytest.raises(TarefaNaoEncontrada):
        remover_tarefa({1: "Estudar"}, 99)
```

Falha obtida contra o código com defeito

```
Failed: DID NOT RAISE TarefaNaoEncontrada
```

Defeito e correção

A especificação manda recusar ids inexistentes, mas `pop(tarefa_id, None)` os ignora em silêncio. Correção: `if tarefa_id not in tarefas: raise TarefaNaoEncontrada(tarefa_id)`, e só então remover.

L7 — Filtrar por prioridade mínima

```
# arquivo: test_l7_filtro.py
import pytest

from filtro import filtrar_por_prioridade

TAREFAS = [("Ler", "baixa"), ("Estudar", "alta"), ("Pagar boleto", "urgente")]


@pytest.mark.parametrize("minima, esperado", [
    ("media", ["Estudar", "Pagar boleto"]),   # nenhuma tarefa é "media"
    ("alta", ["Estudar", "Pagar boleto"]),    # limite: "alta" também entra
    ("baixa", ["Ler", "Estudar", "Pagar boleto"]),
])
def test_filtrar_por_prioridade(minima, esperado):
    assert filtrar_por_prioridade(TAREFAS, minima) == esperado
```

Falha obtida contra o código com defeito

```
assert ['Pagar boleto'] == ['Estudar', 'Pagar boleto']
assert ['Estudar', 'Pagar boleto'] == ['Ler', 'Estu...Pagar boleto']
```

Defeito e correção

"Igual ou maior" foi escrito como `>`: as tarefas com exatamente a prioridade mínima ficam de fora. O caso "media" passa porque nenhuma tarefa da lista é "media". Correção: `>=`.

L8 — Tarefa e a RN-03

```
# arquivo: test_l8_tarefa.py
from datetime import datetime

from tarefa_lab import Tarefa


def test_reabrir_remove_a_data():
    t = Tarefa("Estudar")
    t.concluir()
    t.reabrir()
    assert t.concluido_em is None


def test_concluir_duas_vezes_preserva_a_data():
    t = Tarefa("Estudar")
    t.concluir()
    data_conhecida = datetime(2026, 1, 10, 8, 0)  # não depende do relógio
    t.concluido_em = data_conhecida
    t.concluir()  # concluir de novo não troca a data
    assert t.concluido_em == data_conhecida
```

Falha obtida contra o código com defeito

```
assert datetime.datetime(...) is None
assert datetime.datetime(...) == datetime.datetime(2026, 1, 10, 8, 0)
```

Defeito e correção

Dois defeitos: `reabrir()` não apaga a data, e `concluir()` sobrescreve a data de uma tarefa já concluída. Correção: `concluido_em = None` em `reabrir()` e um `if` em `concluir()`.

Uma observação sobre o teste da L8

Repare que o teste de "concluir duas vezes" grava uma data fixa antes da segunda chamada, em vez de comparar duas leituras do relógio. Comparar duas chamadas a `datetime.now()` parece natural, mas em alguns sistemas o relógio tem resolução grosseira: as duas leituras podem devolver o mesmo instante, e o teste passaria mesmo com o defeito. É a mesma lição da RN-05, que recebe a data de hoje como parâmetro. Testes que dependem do relógio, da ordem de execução ou de sorte são chamados de testes instáveis (flaky) — e devem ser evitados.

Fim do e-book. Bons testes!

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 1 — Anatomia de um teste automatizado

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	raises	parametrize	fixture	banco	monkeypatch	HTTP	sessão	duas contas

> **Hoje:** instalar o pytest, configurá-lo e escrever os **9 primeiros testes**.
>
> **Ao final:** `pytest` responde **9 passed** em menos de um segundo.
>
> **Defeito a encontrar hoje:** nenhum — hoje você aprende a ferramenta.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

O sistema que você recebeu

A **Todo List API** é o backend de uma lista de tarefas. Com ela, uma pessoa cria uma conta, entra com e-mail e senha e gerencia as próprias tarefas: cada tarefa tem um **título**, uma **prioridade** (baixa, média, alta ou urgente), um **prazo** opcional e um **status** (pendente ou concluída).

O backend chega **pronto, funcionando e sem nenhum teste**. O trabalho dos próximos nove dias é construir a suíte de testes que o verifica — e, no caminho, encontrar os defeitos que foram colocados nele de propósito.

Onde o teste de hoje entra

Quando o aplicativo pede algo à API, a requisição atravessa várias camadas até chegar ao banco de dados. Hoje vamos testar a camada do meio — as **entidades** — diretamente, sem passar por nenhuma das outras:

```
Cliente  -->  rota      -->  serviço      -->  [ ENTIDADE ]  -->  banco
(app)         routers/       services.py        models.py         SQLite
                                                    ^
                                                    |
                                        os testes de hoje entram AQUI
```

As entidades são as classes `Tarefa` e `Usuario`, em `app/models.py`. É nelas que as **regras de negócio** estão escritas: como uma tarefa nasce, quando ela está atrasada, como o e-mail é guardado. Testar ali tem duas vantagens:

- **É rápido.** Não há servidor, banco nem rede: o teste cria o objeto na memória e verifica o resultado. Nove testes rodam em menos de um segundo.
- **É preciso.** Quando um teste desses falha, ele aponta exatamente a regra quebrada — não "algo deu errado em algum lugar do caminho".

As duas entidades

Entidade	O que guarda	O que sabe fazer
`Tarefa`	`usuario_id`, `titulo`, `prioridade`, `prazo`, `status`, `concluido_em`	`esta_atrasada()`, `pertence_a(id)`, `concluir()`, `reabrir()`
`Usuario`	`nome`, `email`, o hash da senha	`definir_senha()`, `senha_confere()`

As regras que vamos provar hoje

Nenhum teste é inventado. Cada um sai de uma regra documentada no manual técnico (`docs/manual_tecnico.md`). A tabela abaixo é o **plano do dia**: o que o sistema promete, e como o teste vai provar.

#	O que o sistema promete	Origem	Como o teste prova
1	Toda tarefa nasce pendente e sem data de conclusão	RN-02	cria uma tarefa e confere `status` e `concluido_em`
2	O título é gravado sem espaços nas pontas	validação	cria com `"  Revisar  "` e espera `"Revisar"`
3	A prioridade aceita o enum e o texto	validação	cria com `PrioridadeTarefa.ALTA` e com `"urgente"`
4	Tarefa sem prazo nunca está atrasada	comportamento	`prazo=None` → `esta_atrasada()` é `False`
5	Tarefa pendente com prazo vencido está atrasada	comportamento	prazo = ontem → `esta_atrasada()` é `True`
6	`pertence_a` reconhece o dono	comportamento	dono 7: `pertence_a(7)` é `True`, `pertence_a(8)` é `False`
7	As prioridades vão da mais urgente para a menos	comportamento	os pesos de urgente, alta, média e baixa estão em ordem
8	O e-mail é normalizado na criação	RN-01	cria com `"  ANA@Exemplo.COM "` e espera `"ana@exemplo.com"`
9	O nome é gravado sem espaços nas pontas	validação	cria com `"  Ana Souza  "` e espera `"Ana Souza"`

O que você vai aprender hoje

Ferramenta nova	`pytest`, o arquivo `pytest.ini` e a palavra `assert`
Arquivos que nascem	`pytest.ini`, `tests/__init__.py`, `tests/test_dominio.py`
A suíte passa a saber	comparar um valor com o esperado — e mostrar os dois lados quando eles não batem
Ainda não sabe	verificar que uma operação é **recusada** — é o Dia 2
Testes ao final	9
Commits	0 a 3

As três regras da trilha

> **1. Este código contém defeitos introduzidos de propósito.** Não é um projeto ruim: é um projeto que funciona, com erros plantados para que você os encontre. Quando um teste falhar, a primeira hipótese é que ele achou um defeito de verdade.

> **2. Encontrar e corrigir o defeito é a atividade.** O ciclo é sempre o mesmo: teste vermelho → diagnóstico → correção em `app/` → teste verde. Você só altera `app/` quando um teste vermelho justificar.

3. A documentação é a especificação. Os testes saem do manual técnico, e não da leitura do código. Se você testar lendo o código, o teste concorda com o erro.

Hoje você tem uma garantia: nenhum defeito plantado afeta os nove testes do Dia 1. Se algo falhar hoje, o problema está no seu teste ou no seu ambiente.

2. Como usar este roteiro

Cada passo segue sempre a mesma sequência. Conhecendo os blocos, você sabe o que fazer em cada um sem precisar reler as instruções:

Bloco	O que você faz
Código para digitar	Digite o código no VS Code. **Digite, não copie e cole**: é digitando que você percebe cada detalhe.
Entendendo o código	Leia a explicação de cada trecho antes de rodar. Se uma linha não fizer sentido, pergunte antes de seguir.
Execute no terminal	Rode o comando indicado, na raiz do projeto, com o `(venv)` ativo.

3. Preparação

Passo 1 — O ponto de partida

Arquivo: todo o projeto recebido (nenhuma alteração) · Commit 0

O backend chega **sem histórico de versão**. A primeira coisa a fazer é criar esse histórico: a trilha inteira é guiada por commits, e vários exercícios dos próximos dias dependem de voltar a um commit anterior e ver um teste falhar.

Execute no terminal

```
Git clone https://github.com/JeremiasONunes/back-end-todo-list-software-teste
python -m venv venv
venv\Scripts\Activate.ps1     # Windows (PowerShell)
git status
```

Entendendo os comandos

`python -m venv venv`	Cria um **ambiente virtual**: uma instalação de Python só deste projeto, na pasta `venv/`. O que você instalar aqui não afeta outros projetos.
`venv\Scripts\Activate.ps1`	Ativa o ambiente. A partir daqui o terminal mostra `(venv)` no começo da linha.
`git clone`	Faz um clone do projeto do github em seu computador.
`git status`	Lista o que o git está vendo. **Leia a lista antes de qualquer commit.**

Confira no seu print

- A lista mostra `app/`, `docs/`, `requirements.txt`, `README.md`, `cliente_teste.py`, `.env.example`, `.gitignore` e `.gitattributes`.
- A lista **não** mostra `venv/`. Se mostrar, pare: o `.gitignore` não está sendo lido.

> `git add .` é seguro **só aqui**, porque nesta pasta ainda não há nada que não deva ser versionado. Do próximo commit em diante, os arquivos são adicionados **pelo nome**.

Agora sim, crie o `.env` a partir do exemplo e confira que o git **não** o enxerga:

```
Copy-Item .env.example .env
git status
```

O que deve aparecer

```
On branch master
nothing to commit, working tree clean
```

> Se o `.env` aparecer no `git status`, pare e confira o `.gitignore`. Senha versionada não se resolve apagando o arquivo depois: ela fica para sempre no histórico.

4. A ferramenta de teste

Passo 2 — Instalar o pytest

Arquivo: requirements.txt (alterado) · Commit 1

O `requirements.txt` lista tudo de que o projeto precisa. Repare: o **pytest não está lá**. Ele precisa ser acrescentado — e numa seção separada, porque ferramenta de teste não vai para o servidor de produção junto com a aplicação.

Código para digitar — no FINAL do requirements.txt

```
# ------------------------------------------------------------------------
# Somente para desenvolvimento e testes
# ------------------------------------------------------------------------
# Uma dependência de produção vai para o servidor junto com a aplicação; uma
# dependência de desenvolvimento fica na máquina de quem programa.
#
# O `pytest` entra no Dia 1. O `httpx` só será usado no Dia 7, quando os
# testes passarem a fazer requisições HTTP — mas entra junto, para que o
# arquivo não precise ser alterado no meio do caminho.
pytest
httpx
```

Entendendo o código

`# Somente para desenvolvimento...`	O comentário separa os dois grupos e diz **por que** eles existem. Sem ele, o próximo a mexer junta tudo.
`pytest`	A ferramenta de teste. Ela encontra os testes, executa e mostra o resultado.
`httpx`	Biblioteca de requisições HTTP que o pytest vai usar no Dia 7. Entra hoje para que o arquivo não mude no meio da trilha.

Execute no terminal

```
pip install -r requirements.txt
pytest --version
```

O que deve aparecer (o número da versão pode ser outro)

```
pytest 9.2.0
```

Feche o commit 1

```
git add requirements.txt
git commit -m "Adiciona o pytest como dependencia de desenvolvimento"
```

Passo 3 — Configurar o pytest para este projeto

Arquivo: pytest.ini e tests/__init__.py (novos) · Commit 2

Dois arquivos pequenos. O primeiro diz ao pytest **onde** procurar os testes; o segundo transforma a pasta `tests/` num pacote Python.

Código para digitar — pytest.ini (novo, na RAIZ do projeto)

```
[pytest]
# Onde o pytest deve procurar os testes.
#
# Sem esta linha, a ferramenta varre o projeto inteiro em busca de arquivos que
# comecem com `test_`. Funciona, mas é mais lento e pode encontrar coisas que
# não são teste — a pasta `venv/`, por exemplo, tem centenas de arquivos de
# teste das bibliotecas instaladas.
testpaths = tests
```

Código para digitar — tests/__init__.py (crie a pasta tests/ e o arquivo)

```
# Este arquivo, mesmo vazio, é o que transforma a pasta `tests/` em um pacote
# Python. Sem ele, `from app.models import Tarefa` dentro de um teste pode não
# encontrar o pacote `app`.
```

Entendendo o código

`[pytest]`	Abre a seção de configuração do pytest. O arquivo é lido sozinho, sempre que você roda `pytest` nesta pasta.
`testpaths = tests`	Restringe a busca à pasta `tests/`. Sem isso, o pytest entraria na `venv/` e acharia centenas de testes que não são seus.
`tests/__init__.py`	Mesmo só com comentários, a presença dele faz de `tests` um **pacote**. É o que permite importar `app` de dentro de um teste.

Execute no terminal

```
pytest
```

O que deve aparecer

```
collected 0 items

============================ no tests ran in 0.01s ============================
```

Confira no seu print

- A saída diz `configfile: pytest.ini` e `testpaths: tests` — o arquivo foi lido.
- `no tests ran`: a ferramenta está pronta e ainda não tem trabalho. É o estado esperado.

Feche o commit 2

```
git add pytest.ini tests/__init__.py
git commit -m "Configura a suite de testes"
```

5. Os primeiros testes

Passo 4 — A anatomia de um teste

Arquivo: tests/test_dominio.py (novo) · ainda sem commit

Um teste é uma **função** cujo nome começa com `test_` e que usa `assert` para afirmar algo. Não existe lista de testes para manter: o pytest procura sozinho, em `tests/`, os arquivos e as funções que começam com `test_`.

O primeiro teste é a **RN-02** do manual técnico: *toda tarefa nasce pendente e sem data de conclusão*.

Código para digitar — tests/test_dominio.py

```
from app.enums import PrioridadeTarefa, StatusTarefa
from app.models import Tarefa, Usuario


def test_tarefa_nasce_pendente_e_sem_data_de_conclusao():
    """RN-02: toda tarefa nasce pendente."""
    tarefa = Tarefa(usuario_id=1, titulo="Estudar orientação a objetos")
    assert tarefa.status is StatusTarefa.PENDENTE
    assert tarefa.concluido_em is None
```

Entendendo o código

`from app.enums import ...` `from app.models import ...`	Traz para o teste as classes do sistema que ele vai usar. `PrioridadeTarefa` e `Usuario` só serão usados nos próximos passos, mas já ficam importados.
`def test_tarefa_nasce_...():`	O nome começa com `test_` — é assim que o pytest o encontra. E o resto do nome **descreve a regra**: quando o teste falhar, é esse nome que aparece no relatório.
`"""RN-02: ..."""`	A docstring diz de onde a regra veio. Todo teste de hoje tem origem no manual técnico.
`tarefa = Tarefa(...)`	**Prepara** o cenário: cria uma tarefa em memória, com o mínimo obrigatório — dono e título.
`assert tarefa.status is` `StatusTarefa.PENDENTE`	**Afirma** o resultado. Se a expressão for verdadeira, nada acontece; se for falsa, o teste falha. Usamos `is` porque o status é um valor único do enum.
`assert tarefa.concluido_em` `is None`	Segunda afirmação sobre o **mesmo** fato — como a tarefa nasce. Um teste pode ter mais de um `assert`, desde que verifique um único comportamento.

Os três tempos de todo teste: prepara (cria o objeto), **executa** (aqui, a própria criação) e **verifica** (o `assert`). Repare neles em todos os testes da trilha.

Execute no terminal

```
pytest
```

O que deve aparecer

```
collected 1 item

tests\test_dominio.py .                                                  [100%]

============================== 1 passed in 0.31s ==============================
```

Confira no seu print

- `collected 1 item`: o pytest encontrou o teste sozinho.
- Cada **ponto** (`.`) depois do nome do arquivo é um teste que passou.

Agora faça o teste falhar — de propósito

Antes de escrever os outros, é importante ver como o pytest **avisa** quando algo dá errado. Troque `PENDENTE` por `CONCLUIDA` na primeira linha de `assert`:

```
    assert tarefa.status is StatusTarefa.CONCLUIDA     # errado de propósito
```

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
tests\test_dominio.py F                                                  [100%]

================================== FAILURES ===================================
_____________ test_tarefa_nasce_pendente_e_sem_data_de_conclusao ______________

    def test_tarefa_nasce_pendente_e_sem_data_de_conclusao():
        """RN-02: toda tarefa nasce pendente."""
        tarefa = Tarefa(usuario_id=1, titulo="Estudar orientação a objetos")
>       assert tarefa.status is StatusTarefa.CONCLUIDA
E       AssertionError: assert <StatusTarefa.PENDENTE: 'pendente'> is <StatusTarefa.CONCLUIDA:
'concluida'>

tests\test_dominio.py:8: AssertionError
=========================== short test summary info ===========================
FAILED tests/test_dominio.py::test_tarefa_nasce_pendente_e_sem_data_de_conclusao
============================== 1 failed in 0.31s ==============================
```

Como ler uma falha do pytest

`F`	No lugar do ponto, um `F`: um teste falhou.
`>  assert ...`	A seta aponta **a linha exata** que falhou.
`E  AssertionError: assert <PENDENTE> is <CONCLUIDA>`	As linhas com `E` mostram **os dois lados** da comparação: o valor que o sistema produziu (`PENDENTE`) e o que o teste esperava (`CONCLUIDA`).
`tests\test_dominio.py:8`	O arquivo e o número da linha da falha.
`FAILED tests/...::test_...`	O resumo final: qual teste falhou, pelo nome. É por isso que o nome precisa descrever a regra.

> Sem o pytest, você descobriria isso espalhando `print()` pelo código e procurando a linha certa no terminal. O `assert` faz a comparação **e** o relatório. **Agora desfaça a alteração** (volte para `PENDENTE`) e rode `pytest` de novo: precisa voltar a `1 passed`.

Passo 5 — As fábricas: criar objetos sem repetição

Arquivo: tests/test_dominio.py (alterado) · ainda sem commit

Quase todo teste de hoje precisa de uma tarefa ou de um usuário. Se cada teste escrever `Tarefa(usuario_id=1, titulo="...")`, o arquivo fica repetitivo — e, se um dia a tarefa ganhar um campo obrigatório novo, será preciso alterar todos os testes.

A solução são duas **fábricas**: funções que criam um objeto válido e deixam o teste trocar só o campo que ele quer investigar. Organize o arquivo assim — a docstring no topo, os imports, as fábricas, o cabeçalho da seção do dia e o primeiro teste já usando a fábrica:

Código para digitar — tests/test_dominio.py, reorganizado

```
"""Testes do domínio: as regras de negócio verificadas direto nos objetos.

Sem banco, sem HTTP e sem servidor. São os testes mais rápidos e os mais
valiosos da suíte: quando falham, apontam exatamente o método em que a regra
está escrita.

O arquivo cresce ao longo da trilha, sempre **acrescentando** uma seção ao
final. Nenhum teste escrito em um dia é alterado nos dias seguintes.
"""

from app.enums import PrioridadeTarefa, StatusTarefa
from app.models import Tarefa, Usuario


def nova_tarefa(**ajustes) -> Tarefa:
    """Cria uma tarefa válida, permitindo sobrescrever campos pontualmente.

    Esta função não é um teste — é uma **fábrica**.
    """
    dados = {"usuario_id": 1, "titulo": "Estudar orientação a objetos"}
    dados.update(ajustes)
    return Tarefa(**dados)


def novo_usuario(**ajustes) -> Usuario:
    """Cria um usuário válido, permitindo sobrescrever campos pontualmente."""
    dados = {"nome": "Ana Souza", "email": "ana@exemplo.com"}
    dados.update(ajustes)
    return Usuario(**dados)


# ============================================================================
# Dia 1 — `assert`: o que o objeto faz
# ============================================================================

def test_tarefa_nasce_pendente_e_sem_data_de_conclusao():
    """RN-02: toda tarefa nasce pendente."""
    tarefa = nova_tarefa()
    assert tarefa.status is StatusTarefa.PENDENTE
    assert tarefa.concluido_em is None
```

Entendendo o código

`"""Testes do domínio..."""`	A docstring do arquivo explica a quem chegar depois o que há aqui e por que esses testes são rápidos.
`def nova_tarefa(**ajustes)`	`**ajustes` recebe **qualquer** argumento nomeado e o entrega como dicionário. `nova_tarefa(prazo=ontem)` → `ajustes == {"prazo": ontem}`.
`-> Tarefa`	Indica que a função devolve uma `Tarefa`. Não muda a execução; ajuda quem lê e o VS Code.
`dados = {...}`	Os valores **padrão**: o mínimo para criar uma tarefa válida.
`dados.update(ajustes)`	Sobrescreve os padrões com o que o teste pediu. Se o teste não pediu nada, os padrões ficam.
`return Tarefa(**dados)`	Desempacota o dicionário como argumentos nomeados: é o mesmo que `Tarefa(usuario_id=1, titulo="...")`.
`# ====== Dia 1 ======`	O cabeçalho da seção do dia. Nos próximos dias, cada bateria nova entra **no final** do arquivo, sob o seu próprio cabeçalho.
`tarefa = nova_tarefa()`	O primeiro teste agora usa a fábrica. Ele verifica a mesma coisa de antes, com menos código.

Execute no terminal

```
pytest
```

Confira no seu print

- Continua `1 passed`. Você mudou a **forma** do teste, não o que ele verifica — e o pytest confirma que nada quebrou.

Passo 6 — Os dados da tarefa entram limpos

Arquivo: tests/test_dominio.py (acrescentar ao final) · ainda sem commit

Dois testes sobre **validação**: o sistema precisa aceitar o que o usuário digita e guardar numa forma padronizada. Acrescente-os logo depois do primeiro teste:

Código para digitar

```
def test_titulo_e_gravado_sem_espacos_nas_pontas():
    assert nova_tarefa(titulo="  Revisar  ").titulo == "Revisar"


def test_prioridade_aceita_enum_e_texto():
    assert nova_tarefa(prioridade=PrioridadeTarefa.ALTA).prioridade == "alta"
    assert nova_tarefa(prioridade="urgente").prioridade == "urgente"
```

Entendendo o código

`nova_tarefa(titulo="  Revisar  ")`	A fábrica em ação: troca **só** o título, com espaços sobrando de propósito — como alguém que digitou com pressa.
`.titulo == "Revisar"`	Lê o título guardado e compara com a forma limpa. O teste cabe em uma linha: prepara, executa e verifica juntos.
`prioridade=PrioridadeTarefa.ALTA`	O código Python costuma usar o **enum**…
`prioridade="urgente"`	…e o que chega da API é **texto**. A regra é aceitar os dois e guardar sempre o texto. Por isso há dois `assert`: um por forma de entrada.
`==`	Compara **valores**. Aqui usamos `==`, e não `is`, porque estamos comparando textos.

Execute no terminal

```
pytest
```

Confira no seu print

- `3 passed` — três pontos depois do nome do arquivo.

Passo 7 — O comportamento da tarefa

Arquivo: tests/test_dominio.py (acrescentar ao final) · ainda sem commit

Agora quatro testes sobre o que a tarefa **sabe fazer**. Dois deles trabalham com datas, então primeiro acrescente este import **no topo do arquivo**, antes dos outros:

Código para digitar — no topo do arquivo

```
from datetime import date, timedelta
```

E os quatro testes, depois do teste da prioridade:

Código para digitar

```
def test_tarefa_sem_prazo_nunca_esta_atrasada():
    assert nova_tarefa(prazo=None).esta_atrasada() is False


def test_tarefa_com_prazo_vencido_e_pendente_esta_atrasada():
    ontem = date.today() - timedelta(days=1)
    assert nova_tarefa(prazo=ontem).esta_atrasada() is True


def test_pertence_a():
    tarefa = nova_tarefa(usuario_id=7)
    assert tarefa.pertence_a(7) is True
    assert tarefa.pertence_a(8) is False


def test_ordem_das_prioridades_vai_da_mais_urgente_para_a_menos():
    """Ordenar pelo texto daria "alta, baixa, media, urgente".

    O peso existe para que a ordenação siga o que o usuário espera, e não o
    alfabeto. Este teste trava essa expectativa.
    """
    pesos = [p.peso for p in (
        PrioridadeTarefa.URGENTE,
        PrioridadeTarefa.ALTA,
        PrioridadeTarefa.MEDIA,
        PrioridadeTarefa.BAIXA,
    )]
    assert pesos == sorted(pesos)
```

Entendendo o código

`esta_atrasada() is False`	Chama o método e afirma que a resposta é exatamente `False`. Uma tarefa sem prazo não tem como atrasar.
`date.today() - timedelta(days=1)`	A data de **ontem**, calculada a partir de hoje. Nunca escreva uma data fixa: um teste com `2026-12-20` mudaria de resultado sozinho quando o calendário passasse.
`is True`	Tarefa pendente com prazo vencido **está** atrasada.
`nova_tarefa(usuario_id=7)`	Uma tarefa cujo dono é o usuário 7.
`pertence_a(7) is True` `pertence_a(8) is False`	Os dois lados da regra: o dono é reconhecido **e** um estranho não é. Testar só o primeiro deixaria passar um método que respondesse `True` para todos.
`[p.peso for p in (...)]`	Uma *list comprehension*: monta a lista com o peso de cada prioridade, na ordem urgente → alta → média → baixa.
`pesos == sorted(pesos)`	Se a lista já está igual à versão ordenada dela, os pesos estão em ordem crescente: a mais urgente tem o menor peso.

Execute no terminal

```
pytest
```

Confira no seu print

- `7 passed`.

Passo 8 — Os dados do usuário entram limpos

Arquivo: tests/test_dominio.py (acrescentar ao final) · ainda sem commit

Os dois últimos testes usam a outra fábrica, `novo_usuario`. O primeiro é a **RN-01**: o e-mail não pode ser usado por duas pessoas — e isso só funciona se ele for guardado sempre da mesma forma.

Código para digitar

```
def test_email_e_normalizado_na_criacao():
    """RN-01: o e-mail é guardado sempre na mesma forma."""
    assert novo_usuario(email="  ANA@Exemplo.COM ").email == "ana@exemplo.com"


def test_nome_e_gravado_sem_espacos_nas_pontas():
    assert novo_usuario(nome="  Ana Souza  ").nome == "Ana Souza"
```

Entendendo o código

`email="  ANA@Exemplo.COM "`	Espaços nas pontas **e** letras maiúsculas misturadas — os dois erros de digitação mais comuns num e-mail.
`== "ana@exemplo.com"`	A forma normalizada: sem espaços e em minúsculas. Sem isso, `ANA@...` e `ana@...` virariam duas contas diferentes, quebrando a RN-01.
`nome="  Ana Souza  "`	Os espaços das **pontas** saem; o espaço do **meio**, entre nome e sobrenome, fica.

Execute no terminal

```
pytest
```

O que deve aparecer

```
collected 9 items

tests\test_dominio.py .........                                        [100%]

============================== 9 passed in 0.27s ==============================
```

Confira no seu print

- `9 passed` — nove pontos.
- O tempo fica **abaixo de um segundo**. Se passar disso, algum teste está fazendo mais do que deveria.

Passo 9 — Ler a suíte como um relatório

Arquivo: nenhum arquivo novo · Commit 3

Os pontos dizem **quantos** testes passaram. A opção `-v` (*verbose*, detalhado) mostra **quais** — pelo nome. É agora que o cuidado com os nomes se paga.

Execute no terminal

```
pytest -v
```

O que deve aparecer

```
tests/test_dominio.py::test_tarefa_nasce_pendente_e_sem_data_de_conclusao PASSED [ 11%]
tests/test_dominio.py::test_titulo_e_gravado_sem_espacos_nas_pontas PASSED [ 22%]
tests/test_dominio.py::test_prioridade_aceita_enum_e_texto PASSED        [ 33%]
tests/test_dominio.py::test_tarefa_sem_prazo_nunca_esta_atrasada PASSED  [ 44%]
tests/test_dominio.py::test_tarefa_com_prazo_vencido_e_pendente_esta_atrasada PASSED [ 55%]
tests/test_dominio.py::test_pertence_a PASSED                            [ 66%]
tests/test_dominio.py::test_ordem_das_prioridades_vai_da_mais_urgente_para_a_menos PASSED [ 77%]
tests/test_dominio.py::test_email_e_normalizado_na_criacao PASSED        [ 88%]
tests/test_dominio.py::test_nome_e_gravado_sem_espacos_nas_pontas PASSED [100%]

============================== 9 passed in 0.25s ===============================
```

Confira no seu print

- Leia só os nomes, de cima para baixo: dá para entender **o que o sistema promete** sem abrir o código. Esse é o teste de que você nomeou bem.

Feche o commit 3

```
git add tests/test_dominio.py
git commit -m "Adiciona os primeiros testes de dominio"
```

6. Fechamento do dia

Confira o que o dia entregou

Verificação	Esperado
`pytest` encontra os testes sozinho	9 coletados
Nenhum teste toca banco, rede ou servidor	o tempo total fica abaixo de 1 s
Os nomes descrevem regras	a lista do `pytest -v` se lê como um relatório
`git status` está limpo	sem `.env`, `.pytest_cache/` nem `__pycache__/`
Nada foi alterado em `app/`	obrigatório hoje: nenhum teste revelou defeito

O histórico do dia

Execute no terminal

```
git log --oneline
```

O que deve aparecer (os códigos dos commits serão outros)

```
3  Adiciona os primeiros testes de dominio
2  Configura a suite de testes
1  Adiciona o pytest como dependencia de desenvolvimento
0  Adiciona o backend recebido, sem testes
```

Se algo der errado

Sintoma	Causa provável	O que fazer
`ModuleNotFoundError: No module named 'app'`	`pytest` rodado de dentro de `tests/`, ou falta o `__init__.py`	Rode da raiz do projeto; confira se `tests/__init__.py` existe
`no tests ran` com o arquivo escrito	O nome do arquivo ou da função não começa com `test_`	A descoberta é por nome: renomeie
`RuntimeError: A variável de ambiente ... não foi definida`	Falta o `.env`	`Copy-Item .env.example .env`
`NameError: name 'date' is not defined`	Faltou o import do Passo 7	`from datetime import date, timedelta` no topo
O pytest demora e lista testes da `venv/`	Falta o `testpaths` no `pytest.ini`	Crie o arquivo na raiz

Amanhã

> **Dia 2 — o teste do comportamento proibido.** Até hoje você verificou o que o sistema **faz**. Amanhã entra o `pytest.raises`, para verificar o que ele **recusa**. E a garantia de hoje expira: a partir de amanhã, um teste vermelho pode ser um **defeito real** — e encontrá-lo e corrigi-lo será tarefa sua.

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 2 — O teste do comportamento proibido

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	**raises**	parametrize	fixture	banco	monkeypatch	HTTP	sessão	duas contas

> **Hoje:** aprender o `pytest.raises` e escrever **13 testes** sobre o que a tarefa recusa e sobre o seu ciclo de vida.
>
> **Ao final:** `pytest` responde **22 passed**.
>
> **Defeito a encontrar hoje: SIM.** Um dos testes vai falhar — e diagnosticar e corrigir o problema é tarefa sua.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

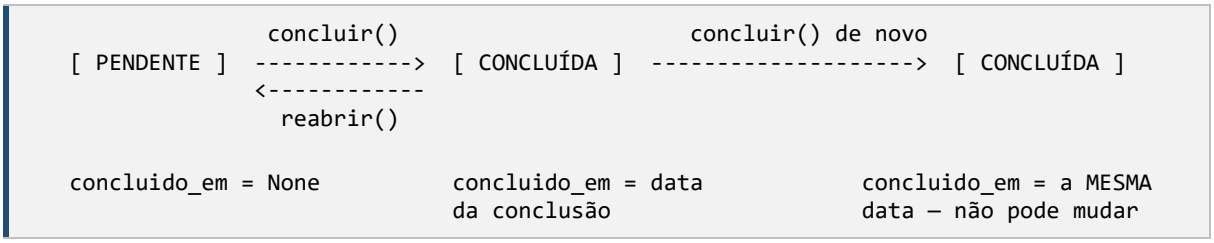
De onde paramos

Ontem você escreveu 9 testes que verificam o que as entidades **fazem**: a tarefa nasce pendente, o título é limpo, o e-mail é normalizado. Todos usam a mesma ferramenta — o `assert` — e todos passaram.

Mas isso é só metade do trabalho. A outra metade é verificar o que o sistema **não deve fazer**. Um sistema que aceita uma prioridade inventada não "para de funcionar": ele funciona, gravando lixo. E nenhum dos testes de ontem perceberia.

O ciclo de vida de uma tarefa

O assunto de hoje é a **tarefa ao longo do tempo**. Ela nasce pendente, pode ser concluída, pode ser reaberta — e cada mudança mexe na data de conclusão (`concluido_em`):



A regra que governa essas transições é a **RN-03**. Leia-a no manual técnico (`docs/manual_tecnico.md`, seção **RN-03 em detalhe**) — em especial a tabela de transições:

Transição	O que acontece com concluido_em
pendente → concluída	recebe a data e hora da conclusão
concluída → pendente	volta a ser nula — a tarefa foi reaberta
concluída → concluída	**não muda** — preserva a data da primeira conclusão
alteração que não envolve o status	não muda

O que é proibido

Para que a RN-03 funcione, algumas portas precisam estar **fechadas**. Se fosse possível escrever `tarefa.status = CONCLUIDA` direto, ninguém passaria pelo `concluir()` — e a data de conclusão ficaria errada. Os testes de hoje verificam que essas portas estão trancadas:

Porta que deve estar fechada	O que o sistema faz ao tentar
mudar o `status` direto, sem `concluir()`/`reabrir()`	recusa com `AttributeError`
criar uma tarefa já concluída (RN-02)	recusa com `TypeError`
criar um usuário sem dizer qual texto é o nome e qual é o e-mail	recusa com `TypeError`
passar a senha direto no construtor do usuário	recusa com `TypeError`
usar uma prioridade que não existe	recusa com `ValorInvalido`
trocar o nome por um texto em branco	recusa com `ValorInvalido`

As regras que vamos provar hoje

#	O que o sistema promete	Ferramenta
1	O `status` é somente leitura	`pytest.raises`
2	O construtor de `Tarefa` não aceita `status` (RN-02)	`pytest.raises`
3	O construtor de `Usuario` exige argumentos nomeados	`pytest.raises`
4	O construtor de `Usuario` não aceita `senha`	`pytest.raises`
5	Prioridade inexistente é recusada	`pytest.raises`
6	Nome em branco é recusado também depois da criação	`pytest.raises`
7	Concluir registra a data	`assert`
8	Reabrir apaga a data	`assert`
9	Editar outro campo não mexe na data de conclusão	`assert`
10	`definir_status` encaminha para `concluir` ou `reabrir`	`assert`
11	Tarefa concluída não está atrasada, mesmo com prazo vencido	`assert`
12	`tarefas_pendentes` deixa de fora as concluídas	`assert`
13	**Concluir uma tarefa já concluída preserva a data original**	`assert`

O que você vai aprender hoje

Ferramenta nova	`pytest.raises` — o teste que passa quando a operação é recusada
Arquivo que cresce	`tests/test_dominio.py` — uma seção nova no final
A suíte passa a saber	exigir que uma operação inválida levante exceção
Ainda não sabe	repetir o mesmo teste com vários valores — é o Dia 3
Testes ao final	22 (9 de ontem + 13 de hoje)
Commits	4 a 7

A partir de hoje, um teste vermelho é uma hipótese

O procedimento do teste vermelho

Ontem havia uma garantia: nenhum defeito afetava os testes do Dia 1. **Essa garantia acabou.** Quando um teste falhar, siga sempre estes quatro passos, nesta ordem:

1. Leia a saída do pytest até o fim — ela mostra o valor obtido e o esperado.

2. Confira o teste contra a **especificação** (o manual técnico), não contra o código.

3. Se o teste está certo, abra o código da aplicação e encontre a linha.

4. Corrija, rode de novo, e confira que tudo ficou verde.

Pular o passo 2 é o erro mais comum: lendo primeiro o código, você tende a achar que o código está certo — ele *parece* certo — e que o teste está errado.

2. Preparando o arquivo

Passo 1 — Os imports e a seção do dia

Arquivo: tests/test_dominio.py (alterado) · ainda sem commit

Hoje o arquivo precisa de três coisas novas no topo. Altere a linha do `datetime` e acrescente as outras duas:

Código para digitar — o topo do arquivo, depois da docstring

```
from datetime import date, datetime, timedelta

import pytest

from app.enums import PrioridadeTarefa, StatusTarefa
from app.exceptions import ValorInvalido
from app.models import Tarefa, Usuario
```

E, **no final do arquivo**, depois do último teste de ontem, abra a seção do dia:

Código para digitar — no final do arquivo

```
# ============================================================================
# Dia 2 — `pytest.raises`: o que o objeto recusa
# ============================================================================
```

Entendendo o código

`datetime`	Entra ao lado de `date` e `timedelta`: hoje os testes vão fixar data **e hora** exatas de conclusão.
`import pytest`	Até ontem o `assert` bastava, e ele é do próprio Python. O `pytest.raises` é da ferramenta — por isso, pela primeira vez, o teste importa o pytest.
`from app.exceptions import ValorInvalido`	A exceção que o sistema levanta quando recebe um valor que não aceita. O teste vai dizer: "espero exatamente esta".
`# ==== Dia 2 ====`	Os 9 testes de ontem ficam como estão. Tudo de hoje entra abaixo deste cabeçalho.

Execute no terminal

```
pytest
```

Confira no seu print

- Continua `9 passed`: você mexeu nos imports, e os testes de ontem seguem verdes.

3. O que deve ser recusado

Passo 2 — O primeiro pytest.raises

Arquivo: tests/test_dominio.py (acrescentar ao final) · ainda sem commit

O primeiro teste fecha a porta mais importante: o `status` não pode ser alterado direto. A única forma de mudá-lo é chamar `concluir()` ou `reabrir()` — que é onde a data é tratada.

Código para digitar

```
def test_status_e_somente_leitura():
    """A única porta para mudar o status são `concluir` e `reabrir`."""
    tarefa = nova_tarefa()
    with pytest.raises(AttributeError):
        tarefa.status = StatusTarefa.CONCLUIDA
```

Entendendo o código

`tarefa = nova_tarefa()`	**Prepara**: uma tarefa pendente, criada pela fábrica de ontem.
`with pytest.raises(AttributeError):`	Abre um bloco que **espera** uma exceção. O teste só passa se o código dentro do bloco levantar exatamente um `AttributeError`.
`tarefa.status = ...`	**Executa** a operação proibida. Ela fica **dentro** do `with` (repare no recuo): é ela que deve ser recusada.

Como o pytest.raises decide:

- a operação levantou `AttributeError` → a porta está fechada → **teste passa**;
- a operação rodou sem erro → a porta está aberta → **teste falha** com `DID NOT RAISE`;
- a operação levantou **outra** exceção → algo diferente aconteceu → **teste falha**.

Execute no terminal

```
pytest
```

Confira no seu print

- `10 passed`.

Experimento 1 — tire o pytest.raises

Para entender o que o `"with pytest.raises(AttributeError):"` faz, tire-o. Deixe só a atribuição, sem o recuo extra:

```
    tarefa = nova_tarefa()
    tarefa.status = StatusTarefa.CONCLUIDA     # sem o with, de propósito
```

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
>       tarefa.status = StatusTarefa.CONCLUIDA
...
E           AttributeError: can't set attribute
...
FAILED tests/test_dominio.py::test_status_e_somente_leitura - AttributeError:...
========================= 1 failed, 9 passed in 0.37s =========================
```

É a **mesma** exceção — `AttributeError` — com o significado oposto. Com o `with`, ela era a prova de que a porta está fechada. Sem o `with`, ela quebra o teste. **Desfaça o experimento** antes de seguir.

Experimento 2 — ponha uma operação permitida dentro do with

Agora o contrário: troque a operação proibida por uma **permitida** — mudar o título:

```
    with pytest.raises(AttributeError):
        tarefa.titulo = "Outro"                # permitido, de propósito
```

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
>       with pytest.raises(AttributeError):
E       Failed: DID NOT RAISE AttributeError

tests\test_dominio.py:106: Failed
FAILED tests/test_dominio.py::test_status_e_somente_leitura - Failed: DID NOT...
========================= 1 failed, 9 passed in 0.31s =========================
```

> `DID NOT RAISE` quer dizer: **"esperei uma recusa e a operação foi aceita"**. Guarde esta mensagem — quando ela aparecer num teste que você não alterou, é sinal de uma porta que devia estar fechada e está aberta. **Desfaça o experimento** e confirme `10 passed`.

Passo 3 — Os construtores que recusam

Arquivo: tests/test_dominio.py (acrescentar ao final) · ainda sem commit

Três testes sobre o **construtor** — o momento em que o objeto é criado. Todos esperam `TypeError`, a exceção do Python para "você chamou esta função do jeito errado".

Código para digitar

```
def test_construtor_nao_aceita_status_como_parametro():
    """RN-02: não existe forma de criar uma tarefa já concluída."""
    with pytest.raises(TypeError):
        Tarefa(usuario_id=1, titulo="X", status=StatusTarefa.CONCLUIDA)


def test_construtor_exige_argumentos_nomeados():
    """`Usuario("Ana", "ana@exemplo.com")` é erro, e de propósito: nome e
    e-mail são os dois texto, e uma troca entre eles não seria pega por
    nenhuma verificação de tipo."""
    with pytest.raises(TypeError):
        Usuario("Ana Souza", "ana@exemplo.com")


def test_construtor_nao_aceita_senha():
    """A senha não tem como entrar sem passar pelo hash.

    Este teste protege uma decisão de projeto, não um cálculo. Se alguém
    acrescentar `senha` ao construtor por conveniência, a suíte avisa.
    """
    with pytest.raises(TypeError):
        Usuario(nome="Ana", email="ana@exemplo.com", senha="senha123")
```

Entendendo o código

`Tarefa(..., status=StatusTarefa.CONCLUIDA)`	Tenta criar uma tarefa **já concluída**. A RN-02 diz que toda tarefa nasce pendente — então o construtor nem aceita o parâmetro.
`Usuario("Ana Souza", "ana@exemplo.com")`	Argumentos **sem nome**. Nome e e-mail são os dois texto: se alguém trocasse a ordem, nada perceberia. Por isso o construtor exige `nome=` e `email=`.
`Usuario(..., senha="senha123")`	A senha só pode entrar por `definir_senha()`, que gera o hash. Se o construtor a aceitasse, ela poderia ser guardada em texto puro.

> Os dois últimos testes não verificam um **cálculo**: verificam uma **decisão de projeto**. Se alguém, um dia, acrescentar `senha` ao construtor "por conveniência", a suíte avisa na hora.

Execute no terminal

```
pytest
```

Confira no seu print

- `13 passed`.

Passo 4 — Valores que não podem entrar

Arquivo: tests/test_dominio.py (acrescentar ao final) · Commit 4

Os dois últimos testes de recusa esperam a exceção **do próprio sistema**: `ValorInvalido`.

Código para digitar

```
def test_prioridade_invalida_e_recusada():
    with pytest.raises(ValorInvalido):
        nova_tarefa(prioridade="altissima")


def test_nome_invalido_e_recusado_tambem_depois_da_criacao():
    """A validação não vale só no construtor.

    É o que diferencia `@validates` de uma checagem feita dentro do `__init__`:
    a regra passa a valer em qualquer atribuição, em qualquer lugar do sistema.
    """
    usuario = novo_usuario()
    with pytest.raises(ValorInvalido):
        usuario.nome = "   "
```

Entendendo o código

`prioridade="altissima"`	Uma prioridade que **não existe**. As válidas são baixa, media, alta e urgente. Ontem você provou que o sistema aceita as certas; hoje, que recusa as erradas.
`ValorInvalido`	Uma exceção **do sistema**, e não do Python. Esperar a exceção certa faz parte da regra: `pytest.raises(Exception)` aceitaria qualquer erro, até um defeito.
`usuario = novo_usuario()`	Primeiro cria um usuário **válido**…
`usuario.nome = "   "`	…e só depois tenta estragar o nome. A validação precisa valer em **qualquer** momento, não só na criação.

Execute no terminal

```
pytest
```

Confira no seu print

- `15 passed`. Os seis testes de recusa estão prontos.

Feche o commit 4

```
git add tests/test_dominio.py
git commit -m "Acrescenta os testes de operacao proibida"
```

4. O ciclo de vida da tarefa (RN-03)

Passo 5 — Concluir, reabrir e editar

Arquivo: tests/test_dominio.py (acrescentar ao final) · ainda sem commit

Agora o `assert` de ontem volta, para percorrer as transições da tabela da RN-03. Comece com um subtítulo, para separar o bloco dentro da seção do dia:

Código para digitar

```
# ------------------------------------ RN-03: o ciclo de vida da data de conclusão

def test_concluir_registra_a_data():
    tarefa = nova_tarefa()
    tarefa.concluir()
    assert tarefa.status is StatusTarefa.CONCLUIDA
    assert tarefa.concluido_em is not None


def test_reabrir_apaga_a_data():
    tarefa = nova_tarefa()
    tarefa.concluir()
    tarefa.reabrir()
    assert tarefa.status is StatusTarefa.PENDENTE
    assert tarefa.concluido_em is None


def test_editar_outro_campo_nao_mexe_na_data_de_conclusao():
    tarefa = nova_tarefa()
    tarefa.concluir(agora=datetime(2026, 1, 10, 9, 0, 0))
    antes = tarefa.concluido_em
    tarefa.titulo = "Título novo"
    assert tarefa.concluido_em == antes


def test_definir_status_encaminha_para_a_operacao_correta():
    tarefa = nova_tarefa()
    tarefa.definir_status("concluida")
    assert tarefa.concluido_em is not None
    tarefa.definir_status(StatusTarefa.PENDENTE)
    assert tarefa.concluido_em is None
```

Entendendo o código

`tarefa.concluir()`	**Executa** a transição pendente → concluída.
`is not None`	A data foi **preenchida** — não importa qual, só que existe. É a primeira linha da tabela da RN-03.
`concluir()` e depois `reabrir()`	Ida e volta: concluída → pendente. A data precisa voltar a ser `None` (segunda linha da tabela).
`concluir(agora=datetime(2026, 1, 10, 9, 0, 0))`	O parâmetro `agora` **fixa** o instante da conclusão. Com uma data conhecida, o teste pode provar que ela não mudou.
`antes = tarefa.concluido_em`	Guarda a data **antes** da edição…
`tarefa.titulo = "Título novo"` `assert ... == antes`	…edita outro campo e confere que a data continua a mesma (última linha da tabela).
`definir_status("concluida")`	É o que a API usa quando recebe `"status": "concluida"`. O teste prova que ele **passa por** `concluir()` e `reabrir()`, e não altera o status por fora.

Execute no terminal

```
pytest
```

Confira no seu print

- `19 passed`.

Passo 6 — O ciclo de vida, visto de outros ângulos

Arquivo: tests/test_dominio.py (acrescentar ao final) · Commit 5

Dois testes que dependem de a tarefa saber se está concluída:

Código para digitar

```
def test_tarefa_concluida_nao_esta_atrasada_mesmo_com_prazo_vencido():
    tarefa = nova_tarefa(prazo=date.today() - timedelta(days=5))
    tarefa.concluir()
    assert tarefa.esta_atrasada() is False


def test_tarefas_pendentes_filtra_as_concluidas():
    usuario = novo_usuario()
    pendente = nova_tarefa(titulo="Pendente")
    concluida = nova_tarefa(titulo="Concluída")
    concluida.concluir()
    usuario.tarefas = [pendente, concluida]
    assert usuario.tarefas_pendentes() == [pendente]
```

Entendendo o código

`prazo=date.today() - timedelta(days=5)`	Prazo vencido há cinco dias. Ontem você provou que uma tarefa **pendente** assim está atrasada…
`concluir() → esta_atrasada() is False`	…hoje, que uma **concluída** não está: o que já foi feito não pode estar atrasado.
`usuario.tarefas = [pendente, concluida]`	Monta, em memória, um usuário com duas tarefas — uma de cada tipo.
`tarefas_pendentes() == [pendente]`	Compara a **lista inteira**: só a pendente pode estar lá, e nada mais.

Execute no terminal

```
pytest
```

Confira no seu print

- `21 passed`. Falta só uma linha da tabela da RN-03.

Feche o commit 5

```
git add tests/test_dominio.py
git commit -m "Acrescenta os testes do ciclo de vida da tarefa"
```

5. O caso crítico — e o defeito

Passo 7 — Concluir duas vezes

Arquivo: tests/test_dominio.py (acrescentar ao final) · Commit 6 — vermelho, de propósito

Falta a linha da tabela que o manual chama de "a mais fácil de errar": **concluída → concluída**. Concluir de novo uma tarefa que já está concluída **não pode** mudar a data da primeira conclusão.

Código para digitar

```
def test_concluir_tarefa_ja_concluida_preserva_a_data_original():
    """O caso que mais reprova implementações ingênuas da RN-03.

    Concluir de novo uma tarefa já concluída não pode reescrever a data em
    que ela foi concluída da primeira vez.
    """
    original = datetime(2026, 1, 10, 9, 0, 0)
    tarefa = nova_tarefa()
    tarefa.concluir(agora=original)
    tarefa.concluir(agora=datetime(2026, 3, 1, 15, 30, 0))
    assert tarefa.concluido_em == original
```

Entendendo o código

`original = datetime(2026, 1, 10, 9, 0, 0)`	A data da **primeira** conclusão: 10 de janeiro, 9h. Guardada numa variável para comparar no fim.
`concluir(agora=original)`	Primeira conclusão: pendente → concluída.
`concluir(agora=datetime(2026, 3, 1, 15, 30, 0))`	Segunda conclusão, com uma data **bem diferente** — 1º de março, 15h30. Se o código errar, o erro fica visível.
`concluido_em == original`	A data guardada tem de continuar sendo a de janeiro.

> Por que datas fixas, e não `datetime.now()`? Duas chamadas seguidas a `now()` dariam horários separados por microssegundos, e a falha ficaria quase ilegível. Com janeiro e março, quem lê a saída entende o problema na hora.

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
tests\test_dominio.py .....................F                            [100%]

================================= FAILURES =================================
_________ test_concluir_tarefa_ja_concluida_preserva_a_data_original _________
...
        original = datetime(2026, 1, 10, 9, 0, 0)
        tarefa = nova_tarefa()
        tarefa.concluir(agora=original)
        tarefa.concluir(agora=datetime(2026, 3, 1, 15, 30, 0))
>       assert tarefa.concluido_em == original
E       AssertionError: assert datetime.datetime(2026, 3, 1, 15, 30) == datetime.datetime(2026, 1,
10, 9, 0)

tests\test_dominio.py:208: AssertionError
========================== short test summary info ==========================
FAILED tests/test_dominio.py::test_concluir_tarefa_ja_concluida_preserva_a_data_original
======================== 1 failed, 21 passed in 0.33s ========================
```

Confira no seu print

- **1 failed, 21 passed** — e o que falhou é **exatamente** este teste.
- Se falhar mais de um, pare: algo além do esperado foi afetado. Confira se você não mexeu em `app/` sem querer.

Feche o commit 6 — vermelho, de propósito

```
git add tests/test_dominio.py
git commit -m "Acrescenta o teste da RN-03 que preserva a data - FALHA"
```

> **Não corrija nada ainda.** O commit do teste vermelho vem **antes** da correção: é ele que registra no histórico que o defeito existia, e que transforma a sua correção em consequência de uma evidência. O `- FALHA` na mensagem avisa a quem encontrar este commit que o vermelho é intencional.

Passo 8 — O diagnóstico

Arquivo: leitura: a saída do pytest, o manual técnico e app/models.py · sem commit

Agora você aplica o procedimento do teste vermelho. Este passo **não tem código para digitar**: tem perguntas para guiar a investigação.

1. O que a saída diz?

Lendo a falha

`>  assert tarefa.concluido_em == original`	A linha que falhou: a comparação final da data.
`datetime(2026, 3, 1, 15, 30)`	O lado **esquerdo** é o que o sistema guardou: a data da **segunda** chamada.
`datetime(2026, 1, 10, 9, 0)`	O lado **direito** é o que o teste esperava: a data da **primeira** chamada.

Em português: **concluir de novo sobrescreveu a data original.**

2. O teste está certo?

Volte à tabela de transições da RN-03 no manual técnico. A linha **concluída → concluída** diz que a data **não muda**. O teste cobra exatamente isso — então o teste está certo, e o problema está na aplicação.

3. Onde está o defeito?

O teste chama um único método da tarefa duas vezes. Abra `app/models.py`, encontre esse método e leia **primeiro a docstring, depois o corpo**.

> **Pistas**
>
> **Perguntas para guiar a leitura:**
>
> • O que a docstring promete que acontece quando a tarefa **já está** concluída?
>
> • No corpo do método, existe alguma linha que **verifica** o status atual antes de gravar?
>
> • O que acontece, linha a linha, na **segunda** chamada?

Você vai encontrar uma docstring que promete exatamente o comportamento que o teste cobra, e um corpo que não o cumpre. Essa contradição — a documentação diz uma coisa, o código faz outra — é a assinatura mais comum de um defeito real.

Passo 9 — A correção

Arquivo: app/models.py (alterado) · Commit 7

Agora você tem o que a regra da trilha exige: um teste vermelho que justifica alterar `app/`. **A correção é sua.** Ela é pequena — e só precisa fazer o método respeitar o que a própria docstring promete.

> **Duas orientações:**
>
> • Altere **só** o método que você identificou. Nenhum outro arquivo de `app/` precisa mudar.
>
> • A ordem das linhas importa: uma verificação posta **depois** de gravar a data não impede nada. Se o teste continuar vermelho depois da sua correção, confira **onde** ela está antes de concluir que o diagnóstico estava errado.

Execute no terminal

```
pytest
```

O que deve aparecer

```
collected 22 items

tests\test_dominio.py ......................                            [100%]

============================== 22 passed in 0.32s ==============================
```

Confira no seu print

- `22 passed` — o teste que estava vermelho agora passa, e **nenhum** outro quebrou.

Feche o commit 7

```
git add app/models.py
git commit -m "Corrige concluir() para nao sobrescrever a data original"
```

A prova de ida e volta

Os dois commits separados permitem uma coisa que um commit único não permitiria: **voltar ao passado e ver o defeito de novo**.

Execute no terminal

```
git checkout HEAD~1       # volta ao commit do teste, antes da correção
pytest                    # 1 failed, 21 passed
git checkout -            # volta ao presente
pytest                    # 22 passed
```

6. Fechamento do dia

Erro, defeito e falha

O que você acabou de percorrer tem nome, e os três estão no vocabulário desta UC:

Conceito	O que foi, aqui
Erro	Alguém escreveu o método sem a verificação.
Defeito	A verificação ausente estava no código, parada, desde então.
Falha	O teste executou e a data saiu errada.

Repare: ontem, nove testes passaram sobre este mesmo código com defeito. **Defeito não é falha — é a possibilidade dela.** Ele só apareceu quando um teste percorreu o caminho em que ele estava.

> **Por que isso importa?** Com o defeito, concluir de novo uma tarefa antiga **falsificaria a data** em que ela foi concluída — sem erro, sem aviso, sem nada quebrar. Em um sistema com auditoria, é adulteração de registro.

Confira o que o dia entregou

Verificação	Esperado
Os 9 testes de ontem continuam idênticos	nenhuma linha deles mudou
A seção do Dia 2 está no final do arquivo	sob o cabeçalho "Dia 2"
`pytest`	22 passed
A única alteração em `app/` é a correção	`git diff HEAD~4 --stat` lista só `app/models.py` dentro de `app/`

O histórico do dia

Execute no terminal

```
git log --oneline -4
```

O que deve aparecer (os códigos dos commits serão outros)

```
7  Corrige concluir() para nao sobrescrever a data original
6  Acrescenta o teste da RN-03 que preserva a data - FALHA
5  Acrescenta os testes do ciclo de vida da tarefa
4  Acrescenta os testes de operacao proibida
```

Leia essa sequência como alguém que chega ao projeto depois: ela conta uma história completa — a suíte cresceu, um teste encontrou um defeito, o defeito foi corrigido.

Se algo der errado

Sintoma	Causa provável	O que fazer
`NameError: name 'pytest' is not defined`	Faltou o `import pytest`	Passo 1
`NameError: name 'datetime' is not defined`	A linha do `datetime` não foi alterada	`from datetime import date, datetime, timedelta`
`DID NOT RAISE`	O `pytest.raises` esperava uma recusa e a operação foi aceita	Confira se a operação está **dentro** do `with`; se estiver, pode ser defeito real
O teste passa com qualquer erro	Foi usado `pytest.raises(Exception)`	Use a exceção exata: ela faz parte da regra
O teste 13 continua vermelho depois da correção	A verificação foi posta depois da gravação	Ela precisa vir **antes**
Mais de um teste falha no Passo 7	Algo foi alterado em `app/` sem querer	`git diff app/` e desfaça

Amanhã

> **Dia 3 — um teste, várias entradas.** Entra o `@pytest.mark.parametrize`: escrever um teste uma vez e rodá-lo com uma lista de valores. E com ele a técnica que diz **quais** valores escolher — a classe de equivalência. Amanhã não há defeito: é um dia para praticar a ferramenta com calma.

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 3 — Um teste, várias entradas

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	raises	**parametrize**	fixture	banco	monkeypatch	HTTP	sessão	duas contas

Hoje: aprender o `@pytest.mark.parametrize` e a técnica que escolhe os valores de teste — a **classe de equivalência**.

Ao final: `pytest` responde **37 passed** (22 de antes + 15 de hoje).

Defeito a encontrar hoje: nenhum — é um dia para praticar a ferramenta com calma.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

De onde paramos

Nos dois primeiros dias você escreveu 22 testes, cada um verificando **um** valor: um título com espaços, uma prioridade inexistente, uma data de conclusão. Ontem, um deles encontrou um defeito, e você o corrigiu.

Mas muitas regras do sistema não dependem de um valor só. "Nome em branco é recusado" — mas o que é um nome em branco? Um texto vazio? Só espaços? Um TAB? Uma quebra de linha? Para ter certeza, o teste precisaria rodar com **cada um** deles.

O problema da cópia

A primeira ideia é copiar o teste quatro vezes, trocando só o valor:

Como NÃO fazer

```
def test_nome_vazio_e_recusado():
    with pytest.raises(ValorInvalido):
        novo_usuario(nome="")

def test_nome_so_espacos_e_recusado():
    with pytest.raises(ValorInvalido):
        novo_usuario(nome="   ")

def test_nome_tab_e_recusado():          # ... e assim por diante
    ...
```

Funciona, mas é o mesmo código repetido, e qualquer mudança na regra obriga a alterar todas as cópias. A segunda ideia — um `for` dentro de um teste — tem um problema pior, que você vai ver no Passo 2. A ferramenta de hoje resolve os dois: **escrever o teste uma vez e rodá-lo com uma lista de valores**.

Quais valores escolher: a classe de equivalência

Se o teste vai rodar com uma lista, a próxima pergunta é: **quais valores entram nela?** Testar todos os textos possíveis é impossível. A técnica de **classe de equivalência** responde assim:

- Divida as entradas possíveis em **classes**: grupos em que todos os valores devem ter o **mesmo** comportamento.
- Se um valor da classe falha, os outros falhariam também — então basta testar **representantes** de cada classe.
- Escolha representantes **diferentes entre si**: valores que uma implementação descuidada poderia tratar de formas diferentes.

Para o nome do usuário, por exemplo:

Classe	Representantes escolhidos	Comportamento esperado
texto sem nenhum caractere útil	`""`, `"  "`, `"\t"` (TAB), `"\n"` (quebra de linha)	recusado com `ValorInvalido`
texto com caracteres úteis	`"Ana Souza"` — já testado no Dia 1	aceito

> Por que esses quatro, e não quarenta? Porque cada um testa uma forma **diferente** de "estar em branco": um código que só verificasse `nome == ""` passaria no primeiro e falharia nos outros três. Quarenta valores parecidos não pegariam nada a mais.

O que vamos testar hoje

#	Regra	Casos	Arquivo
1	Título vazio é recusado	2	`test_dominio.py`
2	Nome vazio é recusado	4	`test_dominio.py`
3	O e-mail é normalizado (espaços e maiúsculas)	4	`test_dominio.py`
4	A URL de PostgreSQL recebe o driver do projeto	2	`test_configuracao.py` (novo)
5	Driver escrito na URL é respeitado	1	`test_configuracao.py`
6	URL de SQLite não é alterada	1	`test_configuracao.py`
7	Parâmetros da URL são preservados	1	`test_configuracao.py`

Repare: são **7 funções de teste**, mas **15 testes**. Cada caso de um `parametrize` conta como um teste separado.

O que você vai aprender hoje

Ferramenta nova	`@pytest.mark.parametrize` — um teste, uma lista de entradas
Técnica nova	classe de equivalência — como escolher as entradas
Arquivos	`tests/test_dominio.py` cresce; `tests/test_configuracao.py` nasce
Ainda não sabe	preparar um objeto uma vez e reaproveitá-lo em vários testes — é o Dia 4
Testes ao final	37 (22 de antes + 15 de hoje)
Commits	8 e 9

> **Hoje não há defeito a encontrar.** Se algum teste falhar, o problema está no teste ou no ambiente. Aproveite para entender bem a ferramenta: ela vai aparecer em quase todos os dias daqui para frente.

2. A ferramenta: parametrize

Passo 1 — O import e a seção do dia

Arquivo: tests/test_dominio.py (alterado) · ainda sem commit

O primeiro teste de hoje espera uma exceção nova, `TituloObrigatorio`. Acrescente-a à linha que já importa `ValorInvalido`:

Código para digitar — altere a linha de app.exceptions, no topo

```
from app.exceptions import TituloObrigatorio, ValorInvalido
```

E, no final do arquivo, depois do último teste de ontem, abra a seção do dia:

Código para digitar — no final do arquivo

```
# ============================================================================
# Dia 3 — `@pytest.mark.parametrize`: a mesma regra, várias entradas
# ============================================================================
```

Entendendo o código

`TituloObrigatorio`	A exceção que a tarefa levanta quando o título está em branco. É mais específica que `ValorInvalido`: o título é o único campo obrigatório que o usuário digita.
`# ==== Dia 3 ====`	Como ontem: os testes anteriores ficam intactos, e tudo de hoje entra abaixo do cabeçalho.

Execute no terminal

```
pytest
```

Confira no seu print

- Continua `22 passed`.

Passo 2 — O primeiro parametrize

Arquivo: tests/test_dominio.py (acrescentar ao final) · ainda sem commit

O título de uma tarefa é obrigatório. Dois representantes da classe "título sem conteúdo": o texto vazio e o texto só com espaços.

Código para digitar

```
@pytest.mark.parametrize("titulo_invalido", ["", "   "])
def test_titulo_vazio_e_recusado(titulo_invalido):
    with pytest.raises(TituloObrigatorio):
        nova_tarefa(titulo=titulo_invalido)
```

Entendendo o código

`@pytest.mark.parametrize(`	Um **decorador**: a linha com `@` logo acima do `def` muda o comportamento da função. Este diz ao pytest para rodar o teste uma vez para cada valor da lista.
`"titulo_invalido"`	O **nome** do parâmetro, entre aspas. Precisa ser exatamente igual ao nome que aparece entre os parênteses do `def`.
`["", "   "]`	A **lista de valores**: os representantes da classe. O teste vai rodar duas vezes.
`def test_...(titulo_invalido):`	A função recebe o valor da vez nesse parâmetro. Na primeira execução ele vale `""`; na segunda, `"   "`.
`nova_tarefa(titulo=titulo_invalido)`	O corpo é igual a um `pytest.raises` de ontem — só que o valor vem da lista, e não escrito direto.

> O `parametrize` não substitui nada do que você já sabe: o `pytest.raises` continua igual. Ele só **multiplica** um teste que você já sabe escrever.

Rode com `-v`, para ver os nomes:

Execute no terminal

```
pytest -v
```

O que deve aparecer (final)

```
...
tests/test_dominio.py::test_titulo_vazio_e_recusado[] PASSED            [ 95%]
tests/test_dominio.py::test_titulo_vazio_e_recusado[   ] PASSED         [100%]

============================== 24 passed in 0.28s ==============================
```

Confira no seu print

- Uma função, **duas linhas** no relatório.
- Entre colchetes, o valor de cada caso: `[]` é o texto vazio, `[   ]` são os espaços. Esse é o **ID do caso de teste**.
- `24 passed` — 22 de antes + 2.

Experimento — ponha um intruso na lista

Por que não um `for`? Porque o `for` para no primeiro valor que falha e esconde **qual** foi. Veja o que o `parametrize` faz no lugar. Acrescente à lista um título **válido**, que não deveria ser recusado:

```
@pytest.mark.parametrize("titulo_invalido", ["", "   ", "Revisar"])   # intruso!
```

Execute no terminal

```
pytest -v
```

O que deve aparecer (trecho)

```
tests/test_dominio.py::test_titulo_vazio_e_recusado[] PASSED             [ 92%]
tests/test_dominio.py::test_titulo_vazio_e_recusado[   ] PASSED          [ 96%]
tests/test_dominio.py::test_titulo_vazio_e_recusado[Revisar] FAILED      [100%]
...
>       with pytest.raises(TituloObrigatorio):
E       Failed: DID NOT RAISE TituloObrigatorio
...
FAILED tests/test_dominio.py::test_titulo_vazio_e_recusado[Revisar] - Failed:...
======================== 1 failed, 24 passed in 0.33s ========================
```

Repare em três coisas:

- os dois casos certos **continuam passando** — um caso que falha não interrompe os outros;
- o relatório diz **exatamente** qual valor falhou: `[Revisar]`;
- a mensagem é a de ontem: `DID NOT RAISE` — esperava recusa, e `"Revisar"` foi aceito (corretamente).

Tire o intruso da lista e confirme que voltou a `24 passed` antes de seguir.

Passo 3 — Quatro representantes da mesma classe

Arquivo: tests/test_dominio.py (acrescentar ao final) · ainda sem commit

Agora o nome do usuário, com os quatro representantes da tabela do cenário:

Código para digitar

```
@pytest.mark.parametrize("nome_invalido", ["", "   ", "\t", "\n"])
def test_nome_vazio_e_recusado(nome_invalido):
    """Quatro representantes da mesma classe: texto sem caractere útil."""
    with pytest.raises(ValorInvalido):
        novo_usuario(nome=nome_invalido)
```

Entendendo o código

`""`	Texto vazio: o caso mais óbvio.
`"   "`	Só espaços: parece preenchido, mas não tem conteúdo.
`"\t"`	Um **TAB**. No Python, `\t` dentro de uma string é o caractere de tabulação — quem cola um texto copiado de uma planilha costuma trazer um.
`"\n"`	Uma **quebra de linha**. Aparece quando alguém aperta Enter num campo de formulário.
`ValorInvalido`	O nome não tem exceção própria: é recusado com a exceção genérica de valor inválido.

Execute no terminal

```
pytest -v
```

O que deve aparecer (final)

```
tests/test_dominio.py::test_nome_vazio_e_recusado[] PASSED              [ 89%]
tests/test_dominio.py::test_nome_vazio_e_recusado[   ] PASSED           [ 92%]
tests/test_dominio.py::test_nome_vazio_e_recusado[\t] PASSED            [ 96%]
tests/test_dominio.py::test_nome_vazio_e_recusado[\n] PASSED            [100%]

============================== 28 passed in 0.28s ==============================
```

Confira no seu print

- Quatro linhas, uma por representante, com os IDs `[]`, `[ ]`, `[\t]` e `[\n]`.
- `28 passed`.

Passo 4 — Entrada e resultado esperado na mesma lista

Arquivo: tests/test_dominio.py (acrescentar ao final) · Commit 8

Até aqui todos os casos esperavam o **mesmo** resultado: uma recusa. Quando cada caso tem um resultado diferente, a lista passa a ter **pares**: o valor de entrada e o valor esperado.

O assunto é a **RN-01**: um e-mail não pode ser usado por duas pessoas. Isso só funciona se `ANA@EXEMPLO.COM` e `ana@exemplo.com` forem reconhecidos como o mesmo endereço — ou seja, se o e-mail for **normalizado** antes de comparar.

Código para digitar

```
@pytest.mark.parametrize(
    "informado, esperado",
    [
        ("ana@exemplo.com", "ana@exemplo.com"),
        ("ANA@EXEMPLO.COM", "ana@exemplo.com"),
        ("  ana@exemplo.com  ", "ana@exemplo.com"),
        ("Ana@Exemplo.Com\n", "ana@exemplo.com"),
    ],
)
def test_normalizar_email_padroniza_espacos_e_maiusculas(informado, esperado):
    """A RN-01 só se sustenta se a comparação for feita sobre a forma
    normalizada: sem isso, `ANA@...` e `ana@...` seriam dois cadastros."""
    assert Usuario.normalizar_email(informado) == esperado
```

Entendendo o código

`"informado, esperado"`	**Dois** nomes de parâmetro, separados por vírgula, numa única string.
`("ANA@EXEMPLO.COM", "ana@exemplo.com")`	Cada item da lista é uma **tupla** com dois valores, na mesma ordem dos nomes: o primeiro vai para `informado`, o segundo para `esperado`.
`("ana@exemplo.com", ...)`	O primeiro caso já está normalizado: prova que a função **não estraga** o que já está certo.
`os outros três`	Uma classe de erro por caso: maiúsculas, espaços nas pontas, e uma mistura de maiúsculas com quebra de linha.
`Usuario.normalizar_email(informado)`	Chama a função **direto na classe**, sem criar um usuário: ela é a regra de normalização isolada.
`== esperado`	O resultado é comparado com o valor esperado **daquele** caso.

> No Dia 1 você testou a normalização com **um** e-mail, pelo construtor. Hoje testa a função que faz o trabalho, com quatro entradas de classes diferentes. Os dois testes convivem: um prova que o construtor usa a regra; o outro, que a regra está certa.

Execute no terminal

```
pytest -v
```

O que deve aparecer (final, com os nomes encurtados)

```
...normalizar_email_padroniza_espacos_e_maiusculas[ana@exemplo.com-ana@exemplo.com] PASSED
...normalizar_email_padroniza_espacos_e_maiusculas[ANA@EXEMPLO.COM-ana@exemplo.com] PASSED
...normalizar_email_padroniza_espacos_e_maiusculas[  ana@exemplo.com  -ana@exemplo.com] PASSED
...normalizar_email_padroniza_espacos_e_maiusculas[Ana@Exemplo.Com\n-ana@exemplo.com] PASSED

============================== 32 passed in 0.29s ==============================
```

Confira no seu print

- Com dois parâmetros, o ID junta os dois valores com um hífen: `[entrada-esperado]`.
- `32 passed`. A seção do Dia 3 no `test_dominio.py` está completa.

Feche o commit 8

```
git add tests/test_dominio.py
git commit -m "Acrescenta os testes parametrizados de validacao"
```

3. Um arquivo novo: a configuração

Passo 5 — A normalização da URL do banco

Arquivo: tests/test_configuracao.py (novo) · Commit 9

O cenário

Em produção, o banco de dados é um **PostgreSQL** contratado numa plataforma de hospedagem. A plataforma entrega um endereço de conexão — a URL — e a pessoa cola esse endereço na configuração da aplicação. O problema: cada plataforma escreve a URL de um jeito, e nem todos funcionam com a biblioteca de banco deste projeto.

A plataforma entrega	O problema	A aplicação precisa de
`postgres://...`	a versão atual do SQLAlchemy não aceita mais	`postgresql+psycopg://...`
`postgresql://...`	escolhe um driver antigo, que não está instalado	`postgresql+psycopg://...`
`postgresql+psycopg2://...`	nenhum: quem escreveu escolheu o driver	deixar como está
`sqlite:///./todo_list.db`	nenhum: é o banco de desenvolvimento	deixar como está

Quem resolve isso é a função `normalizar_url`, em `app/config.py`. Leia a **docstring** dela — é a especificação dos testes de hoje. Ela é uma **função pura**: recebe um texto e devolve outro, sem consultar arquivo, banco ou variável de ambiente. Mesma entrada, mesma saída, sempre — o terreno ideal para o `parametrize`.

O código

Crie o arquivo `tests/test_configuracao.py`:

Código para digitar — tests/test_configuracao.py

```
"""Testes da configuração e da escolha do banco de dados.

Estes testes protegem a troca de ambiente: garantem que a URL que as
plataformas de hospedagem entregam é aceita como vem, que configuração ausente
é erro e não padrão silencioso, e que o segredo do token é forte o bastante.

O arquivo nasce com uma função pura (`normalizar_url`), que só precisa de
`parametrize`. A leitura do ambiente entra depois, junto com o `monkeypatch`.
"""

import pytest

from app.config import normalizar_url


# ============================================================================
# Dia 3 — `parametrize` numa função pura: a normalização da URL
# ============================================================================

@pytest.mark.parametrize(
    "url_recebida",
    [
        "postgres://usuario:senha@host:5432/banco",
        "postgresql://usuario:senha@host:5432/banco",
    ],
)
def test_url_de_postgres_recebe_o_driver_do_projeto(url_recebida):
    """A URL entregue pela hospedagem deve funcionar colada como veio."""
    normalizada = normalizar_url(url_recebida)
    assert normalizada.startswith("postgresql+psycopg://")
    assert normalizada.endswith("usuario:senha@host:5432/banco")


def test_driver_declarado_explicitamente_e_respeitado():
    """Quem escreveu o driver na URL sabe o que quer — não sobrescrever."""
    url = "postgresql+psycopg2://usuario:senha@host:5432/banco"
    assert normalizar_url(url) == url


def test_url_de_sqlite_nao_e_alterada():
    assert normalizar_url("sqlite:///./todo_list.db") == "sqlite:///./todo_list.db"


def test_parametros_da_url_sao_preservados():
    """Banco gerenciado costuma exigir SSL, que vem como parâmetro na URL."""
    url = "postgres://u:s@host:5432/banco?sslmode=require"
    assert normalizar_url(url).endswith("?sslmode=require")
```

Entendendo o código

`"""Testes da configuração..."""`	A docstring já anuncia o futuro: hoje o arquivo só tem a função pura; no Dia 6 ganha os testes que mexem no ambiente.
`from app.config import normalizar_url`	Importa **só** a função. Ela não depende de nada mais da aplicação.
`"postgres://..."` `"postgresql://..."`	Os dois formatos problemáticos são **uma classe**: "URL de PostgreSQL sem driver". Por isso dividem o mesmo `parametrize`.
`.startswith( "postgresql+psycopg://")`	O começo da URL precisa ser o driver do projeto…
`.endswith( "usuario:senha@host:5432/banco")`	…e o resto — usuário, senha, servidor, porta, banco — tem de chegar **intacto**. Trocar o começo não pode estragar o fim.
`test_driver_declarado_...`	Outra classe, outro teste: URL que **já tem** driver sai igual. Resultado diferente → não cabe no mesmo `parametrize`.
`test_url_de_sqlite_...`	A URL de desenvolvimento também sai igual. É o caso que mais importa no dia a dia: se a função estragasse esta, nada rodaria na sua máquina.
`?sslmode=require`	Bancos na nuvem costumam exigir conexão criptografada, pedida por um parâmetro no fim da URL. Ele não pode se perder na normalização.

> **Quando usar parametrize e quando escrever outro teste?** Use o mesmo `parametrize` quando os valores são da **mesma classe** e esperam o **mesmo resultado**. Quando o resultado esperado muda — "vira psycopg" versus "sai igual" —, é outra regra, e merece outro teste, com um nome que diga o que ele promete.

Execute no terminal

```
pytest -v
```

O que deve aparecer (início e fim)

```
tests/test_configuracao.py::test_url_de_postgres_recebe_o_driver_do_projeto[postgres://usuario:senh
a@host:5432/banco] PASSED
tests/test_configuracao.py::test_url_de_postgres_recebe_o_driver_do_projeto[postgresql://usuario:se
nha@host:5432/banco] PASSED
tests/test_configuracao.py::test_driver_declarado_explicitamente_e_respeitado PASSED
tests/test_configuracao.py::test_url_de_sqlite_nao_e_alterada PASSED
tests/test_configuracao.py::test_parametros_da_url_sao_preservados PASSED
tests/test_dominio.py::test_tarefa_nasce_pendente_e_sem_data_de_conclusao PASSED
...
============================== 37 passed in 0.32s ==============================
```

Confira no seu print

- O `test_configuracao.py` aparece **antes** do `test_dominio.py`: o pytest roda os arquivos em ordem alfabética. Nenhum teste depende do outro, então a ordem não importa.
- `37 passed`.

Feche o commit 9

```
git add tests/test_configuracao.py
git commit -m "Adiciona os testes da normalizacao da URL"
```

4. Fechamento do dia

A derivação dos casos de hoje

Esta é a tabela que mostra que os casos foram escolhidos **por método**, e não por tentativa. Cada linha é um caso de teste, com o ID que aparece no `pytest -v`:

Teste	Classe	Representante	Esperado
título vazio	título sem caractere útil	`""`, `"  "`	`TituloObrigatorio`
nome vazio	nome sem caractere útil	`""`, `"  "`, `"\t"`, `"\n"`	`ValorInvalido`
normalizar e-mail	já normalizado	`ana@exemplo.com`	igual
	com maiúsculas	`ANA@EXEMPLO.COM`	minúsculas
	com espaços nas pontas	`  ana@exemplo.com`	sem espaços
	maiúsculas + quebra de linha	`Ana@Exemplo.Com\n`	normalizado
URL de PostgreSQL	PostgreSQL sem driver	`postgres://`, `postgresql://`	`postgresql+psycopg://`
driver explícito	PostgreSQL com driver	`postgresql+psycopg2://`	igual
URL de SQLite	banco de desenvolvimento	`sqlite:///./todo_list.db`	igual
parâmetros	URL com parâmetro	`...?sslmode=require`	parâmetro preservado

Confira o que o dia entregou

Verificação	Esperado
`pytest`	37 passed
Os parametrizados aparecem um por caso	o `-v` mostra `[...]` em cada linha
Os testes dos dias anteriores continuam idênticos	nenhuma linha deles mudou
Nada mudou em `app/`	obrigatório hoje: não houve defeito

O histórico do dia

Execute no terminal

```
git log --oneline -2
git diff HEAD~2 --stat
```

O que deve aparecer no git log (os códigos serão outros)

```
9  Adiciona os testes da normalizacao da URL
8  Acrescenta os testes parametrizados de validacao
```

Confira no seu print

- O `git diff --stat` lista só arquivos de `tests/`. **Nada em** `app/`.

Se algo der errado

Sintoma	Causa provável	O que fazer
O teste parametrizado aparece como **um** só	O decorador não está imediatamente acima do `def`	Nada entre o `@` e o `def`
`fixture 'nome_invalido' not found`	O nome na string do `parametrize` é diferente do parâmetro da função	Os dois nomes precisam ser idênticos
`wrong number of values`	Uma tupla tem um valor a mais ou a menos	Cada tupla tem um valor por nome declarado
`NameError: name 'TituloObrigatorio'`	Faltou o import do Passo 1	`from app.exceptions import TituloObrigatorio, ValorInvalido`
`ModuleNotFoundError` no `test_configuracao.py`	O arquivo foi criado fora de `tests/`	Mova para `tests/`

Amanhã

> **Dia 4 — fixtures.** Hoje você multiplicou um teste por vários valores. Amanhã, o contrário: preparar um objeto **uma vez** e entregá-lo a todo teste que pedir, só pelo nome. Entram o `@pytest.fixture` e o `conftest.py`, e com eles os testes da senha e do token de acesso. Sem defeito, de novo: é a última ferramenta antes do banco de dados.

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 4 — Fixtures: preparar uma vez, entregar a quem pedir

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	raises	parametrize	**fixture**	banco	monkeypatch	HTTP	sessão	duas contas

Hoje: aprender o `@pytest.fixture` e o `conftest.py`, e testar as duas peças de segurança do sistema — **a senha** e **o token de acesso**.

Ao final: `pytest` responde **67 passed** (37 de antes + 30 de hoje).

Defeito a encontrar hoje: nenhum — é a última ferramenta antes do banco de dados.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

De onde paramos

Até aqui, cada teste criou sozinho tudo de que precisava: uma tarefa com `nova_tarefa()`, um usuário com `novo_usuario()`. Funcionou porque esses objetos são simples. Hoje entram objetos que precisam ser **configurados** antes do uso — e que vários testes, em vários arquivos, vão querer.

A primeira peça: a senha (RN-05)

A **RN-05** do manual técnico diz: *a senha nunca é armazenada nem exposta em texto puro*. O sistema guarda um **hash** da senha — o resultado de um cálculo que não pode ser desfeito:

```
 cadastro:   "senha123"  --[ bcrypt ]-->  "$2b$04$Kx9...q2"   <- é isto que vai para o
banco
 login:      "senha123"  --[ bcrypt ]-->  confere com o guardado?  sim / não
```

O algoritmo usado é o **bcrypt**, em `app/security.py`. Três propriedades dele importam para os testes de hoje:

Propriedade	O que significa
Não volta	Do hash não se recupera a senha. Se o banco vazar, as senhas continuam protegidas.
Tem salt	Cada hash leva um valor aleatório embutido: a mesma senha gera hashes **diferentes**. Dois usuários com a mesma senha não podem ser identificados comparando a coluna.
É lento de propósito	Cada hash custa centenas de milissegundos. Isso encarece um ataque que tente milhões de senhas — e é ajustável pelo **custo**.

A segunda peça: o token de acesso

Quando alguém faz login, a API devolve um **token**: um texto assinado que identifica o usuário. Nas requisições seguintes, o aplicativo envia o token em vez da senha. Quem emite e lê o token é a classe `TokenDeAcesso`, em `app/tokens.py`:

```
login ok  -->  tokens.emitir(42)  -->  "eyJhbGciOi...x7Q"   (o usuário 42, assinado)
depois    -->  tokens.ler("eyJhbGciOi...x7Q")  -->  42       (se a assinatura conferir)
               tokens.ler("token falsificado")  -->  recusa  (NaoAutenticado)
```

A assinatura é feita com um **segredo** que só o servidor conhece. Sem o segredo, ninguém consegue fabricar um token válido — e é exatamente isso que vamos testar.

A ferramenta: fixture

Todos os testes de senha precisam de um objeto que gere e confira hashes. Todos os testes de token precisam de um emissor configurado com segredo e validade. Em vez de cada teste construir o seu, a construção é escrita **uma vez**, numa **fixture**, e cada teste a **pede pelo nome**.

> No vocabulário de casos de teste, a fixture é a **pré-condição** — "dado um gerador de hash", "dado um emissor de tokens" — escrita como código que o pytest executa antes do teste, em vez de um parágrafo para quem executa à mão.

O que vamos testar hoje

Bloco	Arquivo	Testes	Fixture usada
O contrato do hash	`test_seguranca.py` (novo)	3	`hasher` (conftest)
O bcrypt de verdade	`test_seguranca.py`	5	`hasher`
O limite de 72 bytes	`test_seguranca.py`	4	`hasher`
A senha vista pelo usuário	`test_dominio.py`	5	`hasher`
A mecânica do token	`test_autenticacao.py` (novo)	13	`tokens` (local)

O que você vai aprender hoje

Ferramenta nova	`@pytest.fixture` e o arquivo `conftest.py`
Arquivos que nascem	`tests/conftest.py`, `tests/test_seguranca.py`, `tests/test_autenticacao.py`
A suíte passa a saber	receber um objeto pronto só por pedir pelo nome; compartilhar a preparação entre arquivos; rodar sem o `.env`
Ainda não sabe	preparar **e desfazer** — o banco limpo a cada teste é o Dia 5
Testes ao final	67 (37 de antes + 30 de hoje)
Commits	10, 11 e 12

> São 30 testes, mas em **7 passos pequenos**, de 3 a 10 testes cada — e quase todos com o mesmo formato. Faça um passo por vez, sempre conferindo o terminal antes de seguir.

2. O conftest.py

Passo 1 — O ambiente de teste

Arquivo: tests/conftest.py (novo) · ainda sem commit

Antes da primeira fixture, um problema que você talvez ainda não tenha notado: **a suíte depende do `.env`**. Veja acontecer. Renomeie o arquivo e rode:

Execute no terminal

```
Rename-Item .env .env.bak
pytest
```

O que deve aparecer (trecho)

```
collected 5 items / 1 error
=================================== ERRORS ====================================
____________________ ERROR collecting tests/test_dominio.py ____________________
...
E   RuntimeError: A variável de ambiente JWT_SECRET não foi definida.
E   Esperado um arquivo .env em: C:\projetos\todo-list-backend\.env
...
============================== 1 error in 0.88s ===============================
```

Os testes de domínio não usam o segredo do token — mas importar `app.models` carrega a configuração da aplicação inteira, e ela exige as variáveis. Os testes estão presos a um arquivo que existe na **sua** máquina. A solução é a suíte definir o **próprio** ambiente.

Crie o arquivo `tests/conftest.py`:

Código para digitar — tests/conftest.py (novo)

```
"""Configuração comum dos testes.

O pytest encontra este arquivo sozinho e aplica o que há nele a **todos** os
testes da pasta — não é preciso importá-lo em lugar nenhum. É isso que faz dele
o lugar das fixtures compartilhadas.
"""

import os

# As variáveis de ambiente são definidas **antes** de importar a aplicação, e a
# ordem é o que importa aqui: a configuração é carregada na primeira importação
# e fica em cache. Definir depois do import não teria efeito nenhum.
#
# `sqlite://` sem caminho é o banco **em memória**: ele existe enquanto a
# conexão existir e desaparece ao final. Nenhum arquivo é criado, e o banco de
# trabalho em `todo_list.db` não é tocado pela suíte.
os.environ["DATABASE_URL"] = "sqlite://"

# A aplicação exige um segredo de token para subir. Valor fixo e previsível,
# para que as falhas sejam reproduzíveis. Não é o segredo da aplicação — a
# configuração real vem do `.env`, que a suíte deixa de precisar.
os.environ["JWT_SECRET"] = "segredo-de-teste-nao-usar-em-producao"

import pytest  # noqa: E402
```

Entendendo o código

`conftest.py`	Um nome **especial**: o pytest procura este arquivo sozinho, antes de qualquer teste, e o aplica a todos os testes da pasta. **Nunca** se importa o `conftest.py`.
`import os`	O módulo do Python que dá acesso às variáveis de ambiente.
`os.environ["DATABASE_URL"] = "sqlite://"`	Define a variável **dentro do processo do teste**. `sqlite://` sem caminho é um banco em memória: nada é gravado em disco.
`os.environ["JWT_SECRET"] = ...`	Um segredo fixo, só para teste. Fixo de propósito: se um teste falhar, a falha se repete igual.
`import pytest  # noqa: E402`	O import vem **depois** das variáveis. O `# noqa: E402` avisa ao verificador de estilo que essa ordem é intencional.

> **A ordem é tudo aqui.** A configuração da aplicação é lida na primeira importação e guardada. Se as linhas de `os.environ` viessem depois de algum `from app...`, seria tarde demais.

Agora rode de novo — **ainda sem o `.env`**:

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
============================== 37 passed in 0.42s ==============================
```

Confira no seu print

- `37 passed` **sem o `.env`**: a suíte agora é independente da máquina.

Devolva o nome do arquivo — o `.env` continua necessário para rodar a **aplicação**, só não para rodar os **testes**:

```
Rename-Item .env.bak .env
```

Passo 2 — A primeira fixture

Arquivo: tests/conftest.py (acrescentar) e tests/test_seguranca.py (novo) · ainda sem commit

No **final** do `conftest.py`, acrescente o import do hash e a primeira fixture:

Código para digitar — no final do tests/conftest.py

```
from app.security import HashBcrypt  # noqa: E402


# ============================================================================
# Dia 4 — a primeira fixture compartilhada
# ============================================================================

@pytest.fixture
def hasher() -> HashBcrypt:
    """O hash de senha **de verdade**, com o custo no mínimo.

    Está aqui, e não dentro de um arquivo de teste, porque dois arquivos a
    usam: `test_seguranca.py` e `test_dominio.py`. Fixture que serve a mais de
    um arquivo sobe para o `conftest.py`.

    O `custo=4` é o mínimo aceito pelo bcrypt e deixa cada hash em poucos
    milissegundos. Nenhum comportamento verificado depende do custo; em
    produção vale o padrão de 12, definido em `HashBcrypt.__init__`.
    """
    return HashBcrypt(custo=4)
```

Entendendo o código

`from app.security import HashBcrypt`	A classe que implementa o bcrypt. Também vem depois das variáveis de ambiente.
`@pytest.fixture`	Marca a função como **fixture**: ela não é um teste, é uma preparação que o pytest entrega a quem pedir.
`def hasher()`	O **nome da função é o nome da fixture**. Um teste que tiver um parâmetro chamado `hasher` recebe o que esta função devolve.
`return HashBcrypt(custo=4)`	Constrói e devolve o objeto. O `custo=4` é o mínimo do bcrypt: rápido o bastante para a suíte, e o comportamento verificado é o mesmo.

Agora o primeiro arquivo que usa a fixture. Crie `tests/test_seguranca.py` com os três testes do **contrato** do hash:

Código para digitar — tests/test_seguranca.py (novo)

```
"""Testes do hash de senha — com o bcrypt de verdade.

Este arquivo verifica a implementação real do algoritmo: o salt aleatório, a
conferência e o limite de bytes. Nada disso pode ser verificado sobre um
substituto.

O `hasher` usado aqui não é criado em nenhum teste: ele vem da fixture de mesmo
nome, no `conftest.py`. Ela fica lá, e não neste arquivo, porque
`test_dominio.py` também precisa dela — e fixture que serve a mais de um
arquivo sobe para o `conftest.py`.
"""

import pytest

from app.exceptions import ValorInvalido
from app.security import LIMITE_DE_BYTES_DO_BCRYPT, HashDeSenha


# ---------------------------------------------------------------- o contrato

def test_hash_bcrypt_cumpre_o_contrato(hasher):
    assert isinstance(hasher, HashDeSenha)


def test_contrato_nao_pode_ser_instanciado_direto():
    with pytest.raises(TypeError):
        HashDeSenha()


def test_implementacao_incompleta_nao_pode_ser_instanciada():
    """Esquecer um método do contrato falha na criação, não no login."""

    class HashPelaMetade(HashDeSenha):
        def gerar(self, senha: str) -> str:
            return senha

        # `verificar` ficou de fora de propósito

    with pytest.raises(TypeError):
        HashPelaMetade()
```

Entendendo o código

`def test_...(hasher):`	**O momento-chave do dia.** O teste não cria o hasher: só declara um parâmetro chamado `hasher`. O pytest procura uma fixture com esse nome, executa-a e entrega o resultado.
`isinstance(hasher, HashDeSenha)`	`HashDeSenha` é o **contrato**: uma classe abstrata que diz que todo hash precisa ter `gerar()` e `verificar()`. O teste confirma que o bcrypt cumpre esse contrato.
`HashDeSenha()`	O contrato **não pode** ser usado direto — ele só descreve. Tentar criar um objeto dele dá `TypeError`.
`class HashPelaMetade(...)`	Uma classe criada **dentro do teste**, de propósito incompleta: tem `gerar` e esqueceu `verificar`.
`HashPelaMetade()`	O Python recusa na **criação** do objeto. Sem isso, o esquecimento só apareceria no primeiro login — em produção.

> Repare que **nenhum teste importa** o `conftest.py` nem a fixture. Pedir pelo nome é tudo — o pytest faz a ligação.

Execute no terminal

```
pytest
```

Confira no seu print

- `40 passed`.

Veja o pytest montando a fixture

A opção `--setup-show` mostra o que o pytest prepara antes de cada teste:

Execute no terminal

```
pytest --setup-show tests/test_seguranca.py::test_hash_bcrypt_cumpre_o_contrato
```

O que deve aparecer (trecho)

```
        SETUP    F hasher
        tests/test_seguranca.py::test_hash_bcrypt_cumpre_o_contrato (fixtures used: hasher) .
        TEARDOWN F hasher
============================== 1 passed in 0.02s ===============================
```

Lendo a saída

`SETUP F hasher`	Antes do teste, o pytest **executou** a fixture `hasher`. O `F` quer dizer que ela vale para uma **função** de teste: cada teste recebe um hasher novo.
`(fixtures used: hasher)`	O teste rodou com a fixture que pediu.
`TEARDOWN F hasher`	Depois do teste, a fixture foi **desmontada**. Hoje não há nada a desfazer — amanhã haverá.

Experimento — erre o nome da fixture

Troque `hasher` por `hashr` no parâmetro do primeiro teste e rode `pytest`:

O que deve aparecer (trecho)

```
____________ ERROR at setup of test_hash_bcrypt_cumpre_o_contrato _____________
  def test_hash_bcrypt_cumpre_o_contrato(hashr):
E       fixture 'hashr' not found
>       available fixtures: ..., hasher, monkeypatch, ..., tmp_path, ...
>       use 'pytest --fixtures [testpath]' for help on them.
========================= 39 passed, 1 error in 0.54s ==========================
```

> Repare em duas coisas:
>
> • o resultado é **error**, e não **failed**: o teste nem chegou a rodar, porque a preparação falhou;
>
> • a lista de fixtures disponíveis traz a sua `hasher` — e outras que já vêm com o pytest, como `monkeypatch` e `tmp_path`, que você vai usar no Dia 6.
>
> **Desfaça o erro** e confirme `40 passed`.

3. A senha

Passo 3 — O bcrypt de verdade

Arquivo: tests/test_seguranca.py (acrescentar ao final) · ainda sem commit

Cinco testes sobre as propriedades do bcrypt que o cenário apresentou. Todos pedem o `hasher`:

Código para digitar

```
# ---------------------------------------------------------------- RN-05: a senha

def test_hash_nao_e_a_senha_em_texto_puro(hasher):
    assert hasher.gerar("senha123") != "senha123"


def test_mesma_senha_gera_hashes_diferentes(hasher):
    """O salt é aleatório por senha: dois usuários com a mesma senha não têm
    o mesmo hash, o que impede identificá-los comparando a coluna."""
    assert hasher.gerar("senha123") != hasher.gerar("senha123")


def test_hash_gerado_confere_com_a_senha(hasher):
    assert hasher.verificar("senha123", hasher.gerar("senha123")) is True


def test_senha_errada_nao_confere(hasher):
    assert hasher.verificar("errada", hasher.gerar("senha123")) is False


def test_hash_cabe_na_coluna_do_banco(hasher):
    """O bcrypt produz sempre 60 caracteres. A coluna tem 255: folga para
    trocar de algoritmo sem migração."""
    assert len(hasher.gerar("senha123")) == 60
```

Entendendo o código

`gerar("senha123") != "senha123"`	O hash não é a senha. Parece óbvio — e é o teste que falharia se alguém, um dia, trocasse o algoritmo por um que "não faz nada".
`gerar(...) != gerar(...)`	A **mesma** senha, duas vezes, dá dois hashes **diferentes**: é o salt aleatório funcionando.
`verificar("senha123", gerar("senha123"))`	Se os hashes são sempre diferentes, como conferir? O `verificar` lê o salt de dentro do hash guardado e refaz a conta.
`verificar("errada", ...) is False`	O outro lado da regra: senha errada não confere.
`len(...) == 60`	O bcrypt sempre gera 60 caracteres. A coluna do banco tem 255 — o teste garante que cabe.

Execute no terminal

```
pytest
```

Confira no seu print

- `45 passed`.

Passo 4 — O limite de 72 bytes

Arquivo: tests/test_seguranca.py (acrescentar ao final) · Commit 10

O bcrypt só considera os primeiros **72 bytes** da senha. Uma senha maior seria cortada em silêncio — então o sistema prefere recusá-la. E o limite é de **bytes**, não de caracteres: uma letra acentuada ocupa 2 bytes em UTF-8.

Código para digitar

```
# ------------------------------------------------------ o limite de 72 *bytes*

def test_senha_no_limite_de_bytes_e_aceita(hasher):
    senha = "a" * LIMITE_DE_BYTES_DO_BCRYPT
    assert hasher.verificar(senha, hasher.gerar(senha)) is True


def test_senha_acima_do_limite_de_bytes_e_recusada(hasher):
    with pytest.raises(ValorInvalido):
        hasher.gerar("a" * (LIMITE_DE_BYTES_DO_BCRYPT + 1))


def test_o_limite_conta_bytes_e_nao_caracteres(hasher):
    """40 caracteres acentuados são 80 bytes em UTF-8 — acima do limite, ainda
    que o campo da API aceite até 72 caracteres."""
    senha = "ã" * 40
    assert len(senha) < LIMITE_DE_BYTES_DO_BCRYPT
    assert len(senha.encode("utf-8")) > LIMITE_DE_BYTES_DO_BCRYPT
    with pytest.raises(ValorInvalido):
        hasher.gerar(senha)


# ------------------------------------------------------------ entrada malformada

def test_hash_invalido_nao_derruba_a_verificacao(hasher):
    """Uma coluna corrompida devolve "senha não confere", não erro 500."""
    assert hasher.verificar("senha123", "isto-nao-e-um-hash") is False
```

Entendendo o código

`LIMITE_DE_BYTES_DO_BCRYPT`	A constante vale 72 e vem de `app/security.py`. Usar a constante, e não escrever `72`, mantém o teste certo se o limite mudar.
`"a" * LIMITE...`	Exatamente 72 bytes: o **último valor aceito**. É uma fronteira, e fronteiras são onde os defeitos se escondem.
`"a" * (LIMITE... + 1)`	73 bytes: o **primeiro valor recusado**. Os dois testes juntos provam que o limite está exatamente no lugar certo.
`senha = "ã" * 40`	40 caracteres — menos que 72 —, mas 80 bytes, porque cada `ã` ocupa 2 bytes.
`len(senha) < ...` `len(senha.encode()) > ...`	Dois `assert` que verificam o **próprio cenário** antes de testar: garantem que o teste está mesmo no caso "poucos caracteres, muitos bytes".
`"isto-nao-e-um-hash"`	Se a coluna do banco for corrompida, conferir a senha deve responder `False` — e não derrubar o servidor.

> **Experimente (opcional):** troque `HashBcrypt(custo=4)` por `HashBcrypt()` na fixture e rode `pytest`. Compare o tempo com o anterior: é o custo 12 de produção. Depois volte para `custo=4`.

Execute no terminal

```
pytest
```

Confira no seu print

- `49 passed`. O `test_seguranca.py` está completo: 12 testes.

Feche o commit 10

```
git add tests/conftest.py tests/test_seguranca.py
git commit -m "Cria o conftest.py com a fixture hasher e os testes do bcrypt"
```

Passo 5 — A mesma fixture, em outro arquivo

Arquivo: tests/test_dominio.py (acrescentar ao final) · Commit 11

O `test_seguranca.py` testou o **algoritmo**. Agora a RN-05 vista pelo **usuário**: o que acontece com a senha quando ela passa pelo objeto `Usuario`. Estes testes também precisam de um hasher — e pedem **a mesma fixture**, de outro arquivo. Acrescente ao final do `test_dominio.py`:

Código para digitar — no final do tests/test_dominio.py

```
# ============================================================================
# Dia 4 — fixtures: a senha (RN-05), com o `hasher` do conftest.py
# ============================================================================

def test_a_senha_nao_fica_guardada_em_atributo_algum(hasher):
    """O texto puro é usado dentro de `definir_senha` e descartado.

    O que se verifica aqui é que ele não sobrou em nenhum atributo do objeto.
    Que o *hash* não revela a senha é outra garantia, e está em
    `test_seguranca.py`.
    """
    usuario = novo_usuario()
    usuario.definir_senha("senha123", hasher)
    assert not hasattr(usuario, "senha")
    assert usuario._senha_hash != "senha123"


def test_senha_correta_confere(hasher):
    usuario = novo_usuario()
    usuario.definir_senha("senha123", hasher)
    assert usuario.senha_confere("senha123", hasher) is True


def test_senha_errada_nao_confere(hasher):
    usuario = novo_usuario()
    usuario.definir_senha("senha123", hasher)
    assert usuario.senha_confere("errada", hasher) is False


def test_definir_senha_de_novo_substitui_a_anterior(hasher):
    usuario = novo_usuario()
    usuario.definir_senha("antiga", hasher)
    usuario.definir_senha("nova", hasher)
    assert usuario.senha_confere("nova", hasher) is True
    assert usuario.senha_confere("antiga", hasher) is False


def test_repr_nao_expoe_o_hash_da_senha(hasher):
    """Um `print` de objeto não deve despejar credencial em log."""
    usuario = novo_usuario()
    usuario.definir_senha("senha123", hasher)
    assert "senha123" not in repr(usuario)
    assert usuario._senha_hash not in repr(usuario)
```

Entendendo o código

`definir_senha("senha123", hasher)`	O usuário recebe o hasher como argumento: ele não sabe qual algoritmo está usando, só que cumpre o contrato.
`not hasattr(usuario, "senha")`	Confere que **não existe** um atributo `senha` no objeto: o texto puro foi usado e descartado.
`_senha_hash != "senha123"`	O que ficou guardado não é a senha.
`senha_confere(...) is True ... is False`	Os dois lados da conferência, agora pelo objeto do usuário.
`definir "antiga", depois "nova"`	Trocar a senha substitui de verdade: a nova confere, a antiga **deixa** de conferir.
`repr(usuario)`	É o texto que aparece num `print(usuario)` ou num log. Nem a senha nem o hash podem estar ali.

> **Por que a `hasher` mora no `conftest.py`?** Porque dois arquivos — `test_seguranca.py` e `test_dominio.py` — precisam dela. A regra prática: a fixture começa no arquivo que a usa, e **sobe para o `conftest.py` quando um segundo arquivo precisar dela**.

Execute no terminal

```
pytest
pytest --setup-show tests/test_dominio.py::test_senha_correta_confere
```

O que deve aparecer (trechos)

```
============================== 54 passed in 0.54s ==============================
...
        SETUP    F hasher
        tests/test_dominio.py::test_senha_correta_confere (fixtures used: hasher) .
        TEARDOWN F hasher
```

Confira no seu print

- `54 passed`.
- O `--setup-show` mostra a **mesma** fixture `hasher` agora servindo a um teste do `test_dominio.py`.

Feche o commit 11

```
git add tests/test_dominio.py
git commit -m "Acrescenta os testes de senha do dominio"
```

4. O token de acesso

Passo 6 — A fixture local

Arquivo: tests/test_autenticacao.py (novo) · ainda sem commit

Os testes do token precisam de um emissor configurado. Mas, diferente da `hasher`, **só este arquivo** vai usá-lo. Por isso a fixture fica **dentro do próprio arquivo**, e não no `conftest.py`.

Código para digitar — tests/test_autenticacao.py (novo)

```
"""Testes da autenticação: do token emitido ao isolamento entre contas.
O arquivo cresce em três etapas, cada uma com uma seção nova no final:
1. a mecânica do token, sem HTTP — chamando `TokenDeAcesso` diretamente;
2. o cabeçalho `Authorization`, pela API;
3. o isolamento entre contas — uma conta não alcança os dados da outra.
"""
from datetime import timedelta
import jwt
import pytes
from app.exceptions import NaoAutenticado, SessaoExpirada
from app.tokens import ALGORITMO, TokenDeAcesso
#: Segredo próprio deste arquivo, com mais de 32 bytes.
SEGREDO = "segredo-de-teste-com-mais-de-trinta-e-dois-bytes"
@pytest.fixture
def tokens() -> TokenDeAcesso:
    """Fixture **local**: só este arquivo precisa de um emissor de tokens.
    Compare com a `hasher`, que mora no `conftest.py` porque dois arquivos a
    usam. O lugar de uma fixture é decidido pelo **alcance** de quem precisa
    dela.
    """
    return TokenDeAcesso(segredo=SEGREDO, validade=timedelta(hours=1))
# ============================================================================
# Dia 4 — a mecânica do token, sem HTTP
# ============================================================================
def test_token_emitido_devolve_o_mesmo_usuario(tokens: TokenDeAcesso):
    """O caminho normal, em uma linha: o que entra é o que sai."""
    assert tokens.ler(tokens.emitir(42)) == 42

def test_token_nao_e_o_id_em_texto_puro(tokens: TokenDeAcesso):
    """O token é assinado: não dá para fabricar um escrevendo o id."""
    assert tokens.emitir(42) != "42"
    assert len(tokens.emitir(42)) > 40

def test_token_assinado_com_outro_segredo_e_recusado(tokens: TokenDeAcesso):
    """O caso que a assinatura existe para impedir."""
    intruso = TokenDeAcesso(
        segredo="outro-segredo-tambem-com-trinta-e-dois-bytes",
        validade=timedelta(hours=1),
    )
    with pytest.raises(NaoAutenticado):
        tokens.ler(intruso.emitir(42))

def test_token_alterado_e_recusado(tokens: TokenDeAcesso):
    """Trocar um caractere invalida a assinatura."""
    token = tokens.emitir(42)
    adulterado = token[:-2] + ("ab" if not token.endswith("ab") else "cd")
    with pytest.raises(NaoAutenticado):
        tokens.ler(adulterado)
```

Entendendo o código

`import jwt`	A biblioteca do formato **JWT**, o padrão do token. Vai ser usada no Passo 7 para olhar dentro de um token.
`SEGREDO = ...`	Um segredo **deste arquivo**, com mais de 32 bytes, o mínimo para a assinatura ser forte.
`@pytest.fixture` `def tokens():`	A fixture **local**. Funciona igual à `hasher`: o teste pede pelo nome. A diferença é o alcance — só os testes deste arquivo a enxergam.
`TokenDeAcesso(segredo=...,` `validade=...)`	O emissor recebe segredo e validade **pelo construtor**. É o que permite, no próximo passo, criar um emissor com validade artificial.
`tokens: TokenDeAcesso`	A anotação de tipo depois do `:` é opcional. Ela ajuda o VS Code a sugerir os métodos de `tokens` enquanto você digita.
`tokens.ler(tokens.emitir(42))` `== 42`	Emite um token para o usuário 42 e lê de volta: o que entra é o que sai.
`intruso = TokenDeAcesso(` `segredo="outro...")`	O teste **constrói o ataque**: um segundo emissor, com outro segredo, fabrica um token perfeito na forma. O nosso emissor recusa, porque a assinatura não confere.
`token[:-2] + ("ab" if ...` `else "cd")`	Troca os dois últimos caracteres. O `if` garante que a troca muda de verdade: se o token já terminasse em `ab`, o "adulterado" seria igual ao original.

Execute no terminal

```
pytest
```

Confira no seu print

- `58 passed`.

Passo 7 — O resto da mecânica do token

Arquivo: tests/test_autenticacao.py (acrescentar ao final) · Commit 12

Os últimos nove testes do dia. Repare que eles reúnem **tudo** o que você aprendeu até aqui: `assert`, `pytest.raises`, `parametrize` e fixture.

Código para digitar

```
@pytest.mark.parametrize(
    "texto_invalido", ["", "   ", "nao-e-um-token", "a.b.c", "Bearer abc"]
)
def test_texto_que_nao_e_token_e_recusado(
    tokens: TokenDeAcesso, texto_invalido: str
):
    """Cinco representantes de "isto não é um token" — o `parametrize` do Dia 3."""
    with pytest.raises(NaoAutenticado):
        tokens.ler(texto_invalido)


def test_token_expirado_e_recusado_com_excecao_propria(tokens: TokenDeAcesso):
    """A validade negativa é o truque que torna o teste instantâneo: o token
    nasce vencido há dez segundos."""
    vencido = TokenDeAcesso(segredo=SEGREDO, validade=timedelta(seconds=-10))
    with pytest.raises(SessaoExpirada):
        tokens.ler(vencido.emitir(42))


def test_sessao_expirada_tambem_e_nao_autenticado():
    """A herança é o que faz as duas responderem 401 com um só tratador."""
    assert issubclass(SessaoExpirada, NaoAutenticado)


def test_token_sem_sub_utilizavel_e_recusado(tokens: TokenDeAcesso):
    """Token assinado por nós, mas com conteúdo inesperado."""
    sem_sub = jwt.encode({"outro": "campo"}, SEGREDO, algorithm=ALGORITMO)
    with pytest.raises(NaoAutenticado):
        tokens.ler(sem_sub)


def test_token_nao_guarda_dados_pessoais(tokens: TokenDeAcesso):
    """O conteúdo de um JWT é legível por qualquer um que tenha o token."""
    conteudo = jwt.decode(tokens.emitir(42), SEGREDO, algorithms=[ALGORITMO])
    assert set(conteudo) == {"sub", "iat", "exp"}
```

Entendendo o código

`def test_...(tokens, texto_invalido)`	Um teste pode receber uma **fixture** e um **parâmetro do parametrize** ao mesmo tempo. O pytest distingue os dois pelo nome.
`"", "   "`	Classe "sem conteúdo".
`"nao-e-um-token"`	Classe "sem a estrutura": um JWT tem três partes separadas por ponto.
`"a.b.c"`	Classe "estrutura certa, conteúdo inválido".
`"Bearer abc"`	O erro mais comum de quem integra: mandar o cabeçalho inteiro em vez de só o token.
`validade=timedelta(seconds=-10)`	**O truque do dia.** Um emissor com validade **negativa** gera um token que nasce vencido. Sem esperar uma semana, sem `sleep`.
`SessaoExpirada`	Uma exceção **própria** para token vencido: a mensagem "sua sessão expirou" é diferente de "token inválido".
`issubclass(SessaoExpirada, NaoAutenticado)`	Um teste sobre o **desenho**: a exceção de expiração é um tipo de não autenticado. Se alguém separasse as duas, o token vencido viraria um erro 500.
`jwt.encode({"outro": "campo"}, ...)`	Um token assinado com o segredo **certo**, mas sem o campo `sub` (o usuário). Forma certa, conteúdo errado: também é recusado.
`jwt.decode(...)`	Abre o token e lê o conteúdo. Qualquer um com o token consegue fazer isso — ele é assinado, **não** criptografado.
`set(conteudo) == {"sub", "iat", "exp"}`	Exige que o conteúdo tenha **exatamente** estes campos: usuário, data de emissão e validade. Nome ou e-mail ali seriam dados pessoais expostos.

Execute no terminal

```
pytest -v tests/test_autenticacao.py
```

O que deve aparecer

```
tests/test_autenticacao.py::test_token_emitido_devolve_o_mesmo_usuario PASSED [  7%]
tests/test_autenticacao.py::test_token_nao_e_o_id_em_texto_puro PASSED   [ 15%]
tests/test_autenticacao.py::test_token_assinado_com_outro_segredo_e_recusado PASSED [ 23%]
tests/test_autenticacao.py::test_token_alterado_e_recusado PASSED        [ 30%]
tests/test_autenticacao.py::test_texto_que_nao_e_token_e_recusado[] PASSED [ 38%]
tests/test_autenticacao.py::test_texto_que_nao_e_token_e_recusado[   ] PASSED [ 46%]
tests/test_autenticacao.py::test_texto_que_nao_e_token_e_recusado[nao-e-um-token] PASSED [ 53%]
tests/test_autenticacao.py::test_texto_que_nao_e_token_e_recusado[a.b.c] PASSED [ 61%]
tests/test_autenticacao.py::test_texto_que_nao_e_token_e_recusado[Bearer abc] PASSED [ 69%]
tests/test_autenticacao.py::test_token_expirado_e_recusado_com_excecao_propria PASSED [ 76%]
tests/test_autenticacao.py::test_sessao_expirada_tambem_e_nao_autenticado PASSED [ 84%]
tests/test_autenticacao.py::test_token_sem_sub_utilizavel_e_recusado PASSED [ 92%]
tests/test_autenticacao.py::test_token_nao_guarda_dados_pessoais PASSED  [100%]

============================== 13 passed in 0.37s ==============================
```

Execute no terminal

```
pytest
```

Confira no seu print

- `67 passed` na suíte inteira.

Feche o commit 12

```
git add tests/test_autenticacao.py
git commit -m "Adiciona os testes do token com fixture local"
```

5. Fechamento do dia

Onde cada fixture mora

O pytest lista todas as fixtures disponíveis para um arquivo — inclusive de onde cada uma vem:

Execute no terminal

```
pytest --fixtures tests/test_autenticacao.py
```

O que deve aparecer (final; os números de linha podem ser outros)

```
-------------------- fixtures defined from tests.conftest --------------------
hasher -- tests\conftest.py:42
    O hash de senha **de verdade**, com o custo no mínimo.

--------------- fixtures defined from tests.test_autenticacao ---------------
tokens -- tests\test_autenticacao.py:29
    Fixture **local**: só este arquivo precisa de um emissor de tokens.
```

Fixture	Onde mora	Quem enxerga	Por quê
`hasher`	`tests/conftest.py`	todos os testes da pasta	dois arquivos precisam dela
`tokens`	`tests/test_autenticacao.py`	só os testes desse arquivo	nenhum outro arquivo precisa

> A primeira linha da saída que você **não** colou mostra dezenas de fixtures que já vêm com o pytest — `tmp_path`, `monkeypatch`, `capsys`… Uma delas é a ferramenta do Dia 6.

Confira o que o dia entregou

Verificação	Esperado
`pytest`	67 passed
A suíte roda sem o `.env`	sim — o `conftest.py` define o ambiente
Nenhum teste cria um `HashBcrypt` sozinho	todos recebem a fixture `hasher`
A `tokens` não está no `conftest.py`	só um arquivo a usa
Nada mudou em `app/`	obrigatório hoje: não houve defeito

O histórico do dia

Execute no terminal

```
git log --oneline -3
```

O que deve aparecer (os códigos dos commits serão outros)

```
12  Adiciona os testes do token com fixture local
11  Acrescenta os testes de senha do dominio
10  Cria o conftest.py com a fixture hasher e os testes do bcrypt
```

Se algo der errado

Sintoma	Causa provável	O que fazer
`fixture 'hasher' not found`	O `conftest.py` está fora de `tests/`, ou o nome do parâmetro está diferente	O arquivo vai em `tests/`; o nome é o da função
`RuntimeError` sobre variável de ambiente	As linhas de `os.environ` vieram depois de um `from app...`	Elas vêm **antes** de qualquer import da aplicação
`ModuleNotFoundError: tests.conftest`	Um teste tentou importar o `conftest.py`	Não se importa: o pytest aplica sozinho
Os testes de senha ficaram lentos	Faltou o `custo=4` na fixture	`HashBcrypt(custo=4)`
`fixture 'tokens' not found` em outro arquivo	A `tokens` é local	Se outro arquivo precisar, ela sobe para o `conftest.py`

Amanhã

> **Dia 5 — o banco de dados.** Hoje a fixture só **preparou**. Amanhã ela vai preparar **e desfazer** — o `yield` — e vai se aplicar sozinha a todos os testes — o `autouse`. Com elas, o banco entra na suíte, limpo a cada teste. **E volta a haver um defeito para encontrar.**

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 5 — Isolamento: o banco limpo a cada teste

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	raises	parametrize	fixture	**banco**	monkeypatch	HTTP	sessão	duas contas

Hoje: a fixture que **prepara e desfaz** (`yield`), a que se aplica sozinha (`autouse`), e os primeiros testes que gravam no **banco de dados**.

Ao final: `pytest` responde **74 passed** (67 de antes + 7 de hoje).

Defeito a encontrar hoje: SIM. Dois testes vão falhar — e diagnosticar e corrigir o problema é tarefa sua.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

De onde paramos

Os 67 testes que você escreveu até aqui rodam **só em memória**: criam um objeto, chamam um método, conferem o resultado. Nenhum deles grava nada. Mas as regras de um sistema não vivem só nos objetos — algumas só existem no **banco de dados**.

O que só o banco pode garantir

O banco tem regras próprias, escritas no **esquema** — a definição das tabelas. Duas delas protegem regras de negócio deste sistema:

Regra de negócio	Como o banco a garante	Por que o banco, e não só o código
RN-01 — um e-mail não pode ser usado por dois usuários	a coluna `email` é **única** (`unique`)	entre a aplicação consultar "esse e-mail existe?" e gravar, outra requisição pode cadastrar o mesmo e-mail. Só o banco fecha essa janela.
RN-04 — toda tarefa pertence a um usuário existente	a coluna `usuario_id` é uma **chave estrangeira** que aponta para `usuarios`	se alguém gravar direto, passando por cima da aplicação, o banco recusa uma tarefa "órfã".

O desenho abaixo mostra a chave estrangeira. Cada tarefa aponta para o seu dono; uma tarefa apontando para um usuário que **não existe** não pode entrar:

```
usuarios                          tarefas
+----+------------------+         +----+------------+----------------------+
| id | email            |         | id | usuario_id | titulo               |
+----+------------------+         +----+------------+----------------------+
|  1 | ana@exemplo.com  |  <-----  |  1 |     1      | Estudar              |  ok
+----+------------------+         |  2 |   9999     | Tarefa órfã          |  RECUSAR
                                  +----+------------+----------------------+
```

O banco de teste

Os testes não usam o banco em que você trabalha (`todo_list.db`). No Dia 4, o `conftest.py` passou a definir `DATABASE_URL=sqlite://` — um **banco em memória**, que existe enquanto o teste roda e desaparece depois. Hoje ele passa a ser usado de verdade.

O problema que a ferramenta de hoje resolve

Quando um teste grava no banco, o que ele gravou **fica lá**. O teste seguinte encontra um banco sujo — e passa a depender de quem rodou antes dele:

Situação	Resultado
Os dois testes rodam, nessa ordem	passam
O segundo roda sozinho	**falha**
A ordem muda	**falha**

Um teste que falha dependendo da ordem é o **pior** tipo de teste: ele não aponta um defeito do sistema — ele mente, às vezes. A solução é cada teste começar com um banco **vazio** e terminar limpando o que fez. Hoje você vai fazer o pytest garantir isso sozinho, e no Passo 5 vai ver o problema acontecer de propósito.

O que vamos testar hoje

#	O que o sistema promete	Fixture usada
1	As duas tabelas existem no banco	`banco_limpo` (automática)
2	A tabela `usuarios` tem as colunas esperadas	`banco_limpo`
3	A coluna se chama `senha_hash`, e não `_senha_hash`	—
4	Uma tarefa de um usuário que existe é gravada	sessão aberta **à mão**
5	E-mail duplicado é recusado pelo banco (RN-01)	`sessao` e `hasher`
6	A verificação de chave estrangeira está ligada	`banco_limpo`
7	Uma tarefa órfã é recusada pelo banco (RN-04)	`sessao`

O que você vai aprender hoje

Ferramenta nova	fixture com `yield` (prepara **e** desfaz) e `autouse=True` (aplicada a todos os testes, sem pedir)
Arquivo que nasce	`tests/test_banco.py`
Arquivo que cresce	`tests/conftest.py` — as fixtures `banco_limpo` e `sessao`
A suíte passa a saber	gravar no banco sem contaminar o teste seguinte; verificar o que só o esquema do banco prova
Ainda não sabe	alterar uma variável de ambiente durante um teste — é o Dia 6
Testes ao final	74 (67 de antes + 7 de hoje)
Commits	13, 14 e 15

O procedimento do teste vermelho

Hoje volta a haver defeito. Relembre o procedimento do Dia 2:

1. Leia a saída do pytest até o fim.

2. Confira o teste contra a **especificação** (o manual técnico).

3. Se o teste está certo, encontre a linha na aplicação.

4. Corrija, rode de novo, confira tudo verde.

2. A fixture que prepara e desfaz

Passo 1 — O banco limpo, automático

Arquivo: tests/conftest.py (acrescentar) · Commit 13

A fixture de ontem só **preparava** (`return`). A de hoje prepara, **deixa o teste rodar**, e depois desfaz. Primeiro, acrescente os imports do banco no `conftest.py`, logo depois do `import pytest`:

Código para digitar — os imports do tests/conftest.py

```
import pytest  # noqa: E402
from sqlalchemy.orm import Session  # noqa: E402

from app.database import Base, SessionLocal, criar_tabelas, engine  # noqa: E402
from app.security import HashBcrypt  # noqa: E402
```

E, no **final** do arquivo, depois da `hasher`:

Código para digitar — no final do tests/conftest.py

```
# ================================================================================
# Dia 5 — o banco: limpo antes, limpo depois
# ================================================================================

@pytest.fixture(autouse=True)
def banco_limpo():
    """Garante um banco vazio no início e no fim de cada teste.

    `autouse=True` aplica esta fixture a todos os testes, sem que nenhum precise
    pedi-la. É o que torna o isolamento uma garantia e não uma disciplina: não
    existe teste que "esqueceu" de limpar o banco.

    O que vem antes do `yield` roda antes do teste; o que vem depois, ao final —
    inclusive se o teste falhar.
    """
    Base.metadata.drop_all(bind=engine)
    criar_tabelas()
    yield
    Base.metadata.drop_all(bind=engine)
```

Entendendo o código

`from app.database import Base, ..., engine`	As peças do banco: `engine` é a conexão; `Base.metadata` conhece todas as tabelas; `criar_tabelas()` as cria.
`@pytest.fixture(autouse=True)`	**Ferramenta nova 1.** Com `autouse=True`, a fixture roda para **todo** teste, sem que nenhum a peça pelo nome.
`drop_all(...)` `criar_tabelas()`	**Antes** do teste: apaga qualquer tabela que tenha sobrado e cria todas de novo, vazias.
`yield`	**Ferramenta nova 2.** É aqui que o **teste roda**. A função "pausa" no `yield`, o pytest executa o teste, e depois volta para a linha seguinte.
`drop_all(...)  (depois do yield)`	**Depois** do teste: apaga tudo. Roda **mesmo que o teste tenha falhado**.

O caminho de uma fixture com yield

```
   pytest chama banco_limpo()
      |-- drop_all + criar_tabelas     <- prepara (antes do yield)
      |-- yield  ------------------->  o TESTE roda aqui
      |-- drop_all                     <- desfaz (depois do yield)
```

Execute no terminal

```
pytest
```

Confira no seu print

- `67 passed` — nenhum teste novo, e nenhum quebrou com a fixture aplicada a todos.

Veja a fixture automática em ação

Rode o `--setup-show` num teste de **ontem**, que nunca pediu banco nenhum:

Execute no terminal

```
pytest --setup-show tests/test_dominio.py::test_senha_correta_confere
```

O que deve aparecer (trecho)

```
        SETUP    F banco_limpo
        SETUP    F hasher
        tests/test_dominio.py::test_senha_correta_confere (fixtures used: banco_limpo, hasher) .
        TEARDOWN F hasher
        TEARDOWN F banco_limpo
============================== 1 passed in 0.07s ===============================
```

Confira no seu print

- A `banco_limpo` aparece **sem ter sido pedida**: é o `autouse`.
- Repare na ordem: o que é montado primeiro é desmontado por último — como pratos empilhados.

Feche o commit 13

```
git add tests/conftest.py
git commit -m "Isola cada teste com um banco limpo"
```

3. Os testes do banco

Passo 2 — A estrutura do banco

Arquivo: tests/test_banco.py (novo) · ainda sem commit

Os três primeiros testes olham a **estrutura** que a `banco_limpo` criou: as tabelas e as colunas. Crie `tests/test_banco.py`:

Código para digitar — tests/test_banco.py (novo)

```
"""Testes do comportamento do banco de dados.

Os primeiros testes da suíte que **não** rodam só em memória: eles gravam,
consultam e esperam que o banco recuse o que não deve aceitar. É o nível de
**integração** — a conversa entre o modelo e o banco.

Cada teste recebe um banco vazio, criado pela fixture `banco_limpo` do
`conftest.py` — que nenhum teste pede pelo nome, porque ela é `autouse`.
"""

import pytest
from sqlalchemy import inspect, text
from sqlalchemy.exc import IntegrityError

from app.database import SessionLocal, engine
from app.models import Tarefa, Usuario


# ---------------------------------------------------------- a estrutura do banco

def test_as_duas_tabelas_existem_no_banco():
    """Olha o banco, e não o `metadata`."""
    assert set(inspect(engine).get_table_names()) == {"usuarios", "tarefas"}


def test_a_tabela_usuarios_tem_as_colunas_esperadas():
    colunas = {coluna["name"] for coluna in inspect(engine).get_columns("usuarios")}
    assert colunas == {"id", "nome", "email", "senha_hash", "criado_em"}


def test_a_coluna_do_banco_se_chama_senha_hash_e_nao_senha():
    """O atributo Python é `_senha_hash`; a coluna é `senha_hash`."""
    assert Usuario.__table__.columns["senha_hash"].name == "senha_hash"
```

Entendendo o código

`from sqlalchemy import inspect, text`	`inspect` examina o banco por dentro; `text` permite mandar um comando SQL direto — vai ser usado no Passo 6.
`IntegrityError`	A exceção que o banco levanta quando uma regra do esquema é violada.
`inspect(engine).get_table_names()`	Pergunta **ao banco** quais tabelas existem. Não basta o código "saber" das tabelas: elas precisam estar lá.
`set(...) == {"usuarios", "tarefas"}`	Compara como **conjunto**: a ordem não importa, mas não pode faltar nem sobrar tabela.
`{coluna["name"] for ...}`	Monta o conjunto com o nome de cada coluna da tabela `usuarios`.
`columns["senha_hash"].name`	No Python o atributo é `_senha_hash` (com sublinhado, de "privado"); no banco a coluna é `senha_hash`. O sublinhado não precisa vazar para o esquema.

> Nenhum destes testes pede uma fixture — e mesmo assim cada um encontra o banco criado e vazio. É a `banco_limpo` agindo sozinha.

Execute no terminal

```
pytest
```

Confira no seu print

- `70 passed`.

Passo 3 — Gravando à mão

Arquivo: tests/test_banco.py (acrescentar ao final) · ainda sem commit

Agora um teste que **grava**. Para gravar é preciso abrir uma **sessão** — a conversa com o banco — e fechá-la no fim, aconteça o que acontecer. Este teste faz tudo isso **à mão**:

Código para digitar

```
# ---------------------------------------------------- gravando e consultando

def test_tarefa_com_usuario_existente_e_aceita():
    """Uma tarefa de um usuário que existe é gravada normalmente.

    Este teste abre e fecha a sessão **à mão**, de propósito: é o jeito
    "antes da fixture". Compare com os testes abaixo, que pedem a `sessao`
    pronta e não precisam do `try/finally`.
    """
    sessao = SessionLocal()
    try:
        usuario = Usuario(nome="Ana", email="ana@exemplo.com")
        usuario._senha_hash = "hash-qualquer"
        sessao.add(usuario)
        sessao.commit()

        sessao.add(Tarefa(usuario_id=usuario.id, titulo="Tarefa válida"))
        sessao.commit()

        assert sessao.query(Tarefa).count() == 1
    finally:
        sessao.rollback()
        sessao.close()
```

Entendendo o código

`sessao = SessionLocal()`	Abre uma sessão com o banco.
`try: ... finally:`	Tudo o que está no `try` pode falhar. O `finally` roda **sempre** — com ou sem erro — e é onde a sessão é fechada.
`usuario._senha_hash = "hash-qualquer"`	A coluna da senha é obrigatória no banco. Como este teste não é sobre senha, qualquer texto serve.
`sessao.add(...)` `sessao.commit()`	`add` prepara a gravação; `commit` confirma. Depois do `commit`, o usuário ganha um `id`.
`Tarefa(usuario_id= usuario.id, ...)`	Uma tarefa que aponta para um usuário que **existe**: a chave estrangeira deve aceitar.
`query(Tarefa).count() == 1`	Pergunta ao banco quantas tarefas há: exatamente a que foi gravada.
`rollback()` `close()`	Desfaz o que tiver ficado pendente e fecha a sessão.

> Repare quanto código **não é teste** aqui: abrir, `try`, `finally`, `rollback`, `close`. Imagine isso repetido em todo teste que grava. O próximo passo resolve com a ferramenta de hoje.

Execute no terminal

```
pytest
```

Confira no seu print

- `71 passed`.

Passo 4 — A fixture sessao

Arquivo: tests/conftest.py e tests/test_banco.py · ainda sem commit

O `try/finally` do passo anterior vira uma fixture com `yield`. Acrescente-a no **final** do `conftest.py`, depois da `banco_limpo`:

Código para digitar — no final do tests/conftest.py

```
@pytest.fixture
def sessao() -> Session:
    """Sessão de banco para os testes que precisam gravar direto.

    O `rollback()` antes do `close()` desfaz qualquer transação que o teste
    tenha deixado aberta. Sem ele, um teste que falha no meio de um `commit()`
    deixaria a sessão em estado inconsistente para o próximo.
    """
    sessao = SessionLocal()
    try:
        yield sessao
    finally:
        sessao.rollback()
        sessao.close()
```

Entendendo o código

`@pytest.fixture  (sem autouse)`	Esta **não** é automática: só os testes que gravam direto precisam de uma sessão, e eles a pedem pelo nome.
`sessao = SessionLocal()`	Prepara: abre a sessão — igual ao passo anterior.
`yield sessao`	Desta vez o `yield` **entrega um valor**: o teste recebe a sessão aberta no parâmetro `sessao`.
`try: / finally:`	O mesmo `try/finally` do passo anterior — mas escrito **uma vez só**, aqui, para todos os testes.

Agora um teste que usa a fixture. Acrescente ao final do `test_banco.py`:

Código para digitar — no final do tests/test_banco.py

```
def test_email_duplicado_e_recusado_pelo_banco(sessao, hasher):
    """A RN-01 tem duas camadas de defesa.

    O serviço de cadastro consulta antes de inserir, para devolver uma
    mensagem clara. Mas entre a consulta e a inserção há uma janela em que
    outra requisição pode cadastrar o mesmo e-mail. O `unique` da coluna é o
    que fecha essa janela — e é ele que este teste verifica.
    """
    for _ in range(2):
        usuario = Usuario(nome="Ana", email="ana@exemplo.com")
        usuario.definir_senha("senha123", hasher)
        sessao.add(usuario)

    with pytest.raises(IntegrityError):
        sessao.commit()
```

Entendendo o código

`(sessao, hasher)`	**Duas** fixtures: a `sessao` de hoje e a `hasher` do Dia 4. Pedir várias é só listar os nomes.
`for _ in range(2):`	Repete duas vezes: prepara **dois** usuários com o **mesmo** e-mail. O `_` é o nome para uma variável que não vai ser usada.
`definir_senha("senha123", hasher)`	Desta vez a senha é definida do jeito certo, com a fixture.
`with pytest.raises(IntegrityError):` `sessao.commit()`	Ao confirmar, o banco precisa **recusar**: a coluna `email` é única. O `pytest.raises` do Dia 2, agora contra o banco.

> Compare com o Passo 3: este teste grava no banco e não tem **nenhuma** linha de abrir ou fechar sessão. O isolamento e a limpeza deixaram de ser disciplina de quem escreve e passaram a ser **garantia da ferramenta**.

Execute no terminal

```
pytest
pytest --setup-show tests/test_banco.py::test_email_duplicado_e_recusado_pelo_banco
```

O que deve aparecer (trechos)

```
============================== 72 passed in 0.51s ==============================
...
        SETUP    F banco_limpo
        SETUP    F sessao
        SETUP    F hasher
        tests/test_banco.py::test_email_duplicado_e_recusado_pelo_banco (fixtures used:
banco_limpo, hasher, sessao) .
        TEARDOWN F hasher
        TEARDOWN F sessao
        TEARDOWN F banco_limpo
```

Confira no seu print

- `72 passed`.
- Três fixtures montadas: duas pedidas pelo nome (`sessao`, `hasher`) e uma automática (`banco_limpo`).

Passo 5 — Veja o isolamento quebrar

Arquivo: tests/test_vizinho.py (descartável) e tests/conftest.py (alteração temporária) · sem commit

Lá no cenário, a tabela dizia que um teste sem limpeza "passa junto e falha sozinho". Agora você vai ver acontecer. Crie este arquivo **descartável**:

Código para digitar — tests/test_vizinho.py (será apagado no fim do passo)

```
from app.models import Usuario


def test_a_cria_um_usuario(sessao):
    usuario = Usuario(nome="Ana", email="ana@exemplo.com")
    usuario._senha_hash = "hash-qualquer"
    sessao.add(usuario)
    sessao.commit()


def test_b_o_banco_comeca_vazio(sessao):
    assert sessao.query(Usuario).count() == 0
```

Entendendo o código

`test_a_cria_um_usuario`	Grava um usuário e não afirma nada: ele só existe para deixar "sujeira".
`test_b_o_banco_comeca_vazio`	Espera um banco vazio. Ele está **certo**: todo teste deveria começar assim.

Primeiro, com a limpeza funcionando:

Execute no terminal

```
pytest tests/test_vizinho.py
```

O que deve aparecer (final)

```
============================== 2 passed in 0.14s ===============================
```

Agora desligue a limpeza. No `conftest.py`, troque as **duas** linhas `Base.metadata.drop_all(bind=engine)` da `banco_limpo` por `pass` e rode de novo — o arquivo inteiro, e depois só o teste B:

Execute no terminal

```
pytest tests/test_vizinho.py
pytest tests/test_vizinho.py::test_b_o_banco_comeca_vazio
```

O que deve aparecer (trechos)

```
E       assert 1 == 0
...
======================== 1 failed, 1 passed in 0.13s ========================
...
============================== 1 passed in 0.04s ===============================
```

> É a tabela do cenário acontecendo na sua frente:
>
> • **junto** com o teste A, o teste B **falha**: `assert 1 == 0` — o usuário de A sobrou para B;
>
> • **sozinho**, o teste B **passa**.
>
> O teste B não tem defeito nenhum. Quem tem é a falta de limpeza — e quem fosse investigar procuraria o problema no lugar errado.

> **Desfaça tudo antes de seguir:** volte as duas linhas `drop_all` no `conftest.py` e **apague o arquivo** `tests/test_vizinho.py`. Rode `pytest` e confirme `72 passed`.

4. A chave estrangeira — e o defeito

Passo 6 — A RN-04 no banco

Arquivo: tests/test_banco.py (acrescentar ao final) · Commit 14 — vermelho, de propósito

Os dois últimos testes verificam a chave estrangeira do desenho do cenário: uma tarefa apontando para um usuário que não existe **não pode** entrar no banco.

Código para digitar

```
# ---------------------------------------------- RN-04: a chave estrangeira

def test_verificacao_de_chave_estrangeira_esta_ligada():
    """Confere o PRAGMA diretamente na conexão."""
    with engine.connect() as conexao:
        ligado = conexao.execute(text("PRAGMA foreign_keys")).scalar()
    assert ligado == 1, "a verificação de chave estrangeira do SQLite está desligada"


def test_tarefa_orfa_e_recusada_pelo_banco(sessao):
    """RN-04 no nível do banco, e não apenas no do serviço.

    O serviço já verifica se o usuário existe antes de criar a tarefa. Este
    teste é a segunda linha de defesa: mesmo que alguém grave direto pela
    sessão, passando por cima do serviço, o banco recusa uma tarefa cujo
    dono não existe.
    """
    sessao.add(Tarefa(usuario_id=9999, titulo="Tarefa de usuário inexistente"))
    with pytest.raises(IntegrityError):
        sessao.commit()
```

Entendendo o código

`with engine.connect() as conexao:`	Abre uma conexão com o banco, que se fecha sozinha no fim do `with`.
`text("PRAGMA foreign_keys")`	`PRAGMA` é um comando de configuração do SQLite. Este pergunta: "a verificação de chave estrangeira está ligada?" A resposta é `1` (sim) ou `0` (não).
`.scalar()`	Pega o valor único da resposta.
`assert ligado == 1, "..."`	O texto depois da vírgula é a **mensagem** que o pytest mostra se o `assert` falhar. Ajuda quem lê o relatório.
`Tarefa(usuario_id=9999, ...)`	Uma tarefa cujo dono, o usuário 9999, **não existe** — o banco está vazio.
`with pytest.raises(IntegrityError):`	O banco precisa **recusar** ao confirmar.

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
tests\test_banco.py .....FF                                              [ 27%]
...
_____________ test_verificacao_de_chave_estrangeira_esta_ligada _____________
...
>       assert ligado == 1, "a verificação de chave estrangeira do SQLite está desligada"
E       AssertionError: a verificação de chave estrangeira do SQLite está desligada
E       assert 0 == 1
...
___________________ test_tarefa_orfa_e_recusada_pelo_banco ____________________
...
>       with pytest.raises(IntegrityError):
E       Failed: DID NOT RAISE IntegrityError
...
FAILED tests/test_banco.py::test_verificacao_de_chave_estrangeira_esta_ligada
FAILED tests/test_banco.py::test_tarefa_orfa_e_recusada_pelo_banco - Failed: ...
======================== 2 failed, 72 passed in 0.47s ========================
```

Confira no seu print

- **2 failed, 72 passed** — e os dois que falham são **exatamente** estes.
- Se falhar algum outro, pare: algo além do esperado foi afetado. Confira se você desfez o experimento do Passo 5.

Feche o commit 14 — vermelho, de propósito

```
git add tests/conftest.py tests/test_banco.py
git commit -m "Adiciona os testes do esquema do banco - FALHA"
```

> **Não corrija nada ainda.** Como no Dia 2: o commit do teste vermelho vem **antes** da correção, para que o histórico registre que o defeito existia.

Passo 7 — O diagnóstico

Arquivo: leitura: a saída do pytest, o manual técnico e app/database.py · sem commit

Este passo **não tem código para digitar**: tem perguntas para guiar a investigação.

1. O que as duas saídas dizem?

Lendo as falhas

`assert 0 == 1`	O primeiro teste perguntou ao banco se a verificação está ligada. A resposta foi **0**: desligada.
`DID NOT RAISE IntegrityError`	O segundo gravou uma tarefa de um usuário inexistente e esperava recusa. O banco **aceitou**.

As duas falhas contam a **mesma** história, de dois ângulos: a configuração diz que a verificação está desligada, e o comportamento confirma — a tarefa órfã entrou.

2. O teste está certo?

Leia no manual técnico a seção **Chave estrangeira no SQLite** (`docs/manual_tecnico.md`). Ela explica que o SQLite vem com essa verificação **desligada por padrão** — e que a aplicação é que deveria ligá-la, em cada conexão. Os testes cobram exatamente isso. O problema está na aplicação.

3. Onde está o defeito?

Abra `app/database.py`. Procure a função cujo **nome** descreve exatamente o que deveria estar acontecendo.

> **Pistas**
>
> **Perguntas para guiar a leitura:**
>
> • O que o nome da função promete fazer?
>
> • O que a docstring dela diz?
>
> • Linha a linha: o que o corpo da função **realmente** faz entre abrir e fechar o cursor?
>
> **Uma pista sobre a forma do defeito:** ele não é uma linha **errada**, é uma linha **ausente**. O código que sobrou parece inofensivo — e é justamente por isso que passou despercebido.

4. Por que isso importa?

Pense antes de ler: em produção o banco é PostgreSQL, que **sempre** verifica a chave estrangeira. Com o defeito:

Ambiente	Uma tarefa órfã…
Desenvolvimento (SQLite)	é **aceita** — nada acontece, nenhum erro
Produção (PostgreSQL)	é **recusada** — erro de integridade

> O mesmo código, dois comportamentos. Tudo funciona na máquina de quem programou, e quebra depois de publicado — e quem for investigar não vai achar nada errado na regra de negócio, porque o problema está na **configuração do ambiente**. É a categoria de defeito mais cara de encontrar, e por isso tem um teste próprio.

Passo 8 — A correção

Arquivo: app/database.py (alterado) · Commit 15

Você tem o teste vermelho que justifica alterar `app/`. **A correção é sua** — e é pequena: fazer a função cumprir o que o nome dela promete.

> **Duas orientações:**
>
> • Altere **só** a função que você identificou. Nenhum outro arquivo de `app/` precisa mudar.
>
> • O manual técnico explica **qual** comando do SQLite liga a verificação. O teste do Passo 6 usa o mesmo `PRAGMA` — só que para **perguntar**, e não para **ligar**.

Execute no terminal

```
pytest
```

O que deve aparecer

```
tests\test_autenticacao.py .............                                [ 17%]
tests\test_banco.py .......                                             [ 27%]
tests\test_configuracao.py .....                                        [ 33%]
tests\test_dominio.py ..................................                [ 83%]
tests\test_seguranca.py ............                                    [100%]

============================== 74 passed in 0.42s ==============================
```

Confira no seu print

- `74 passed` — os **dois** testes vermelhos passaram com **uma** correção.
- Nenhum outro teste quebrou.

Feche o commit 15

```
git add app/database.py
git commit -m "Liga a verificacao de chave estrangeira no SQLite"
```

A prova de ida e volta

Execute no terminal

```
git checkout HEAD~1       # o commit do teste, antes da correção
pytest                    # 2 failed, 72 passed
git checkout -            # volta ao presente
pytest                    # 74 passed
```

5. Fechamento do dia

As fixtures do projeto até aqui

Fixture	Onde	Automática?	O que faz
`hasher`	`conftest.py`	não	entrega um bcrypt com custo 4 (Dia 4)
`tokens`	`test_autenticacao.py`	não	entrega um emissor de tokens (Dia 4)
`banco_limpo`	`conftest.py`	**sim**	recria o banco antes e apaga depois de cada teste
`sessao`	`conftest.py`	não	abre uma sessão e a fecha no fim, com `rollback`

Confira o que o dia entregou

Verificação	Esperado
`pytest`	74 passed
Nenhum arquivo `.db` foi criado pela suíte	o banco de teste é em memória
O `test_vizinho.py` foi apagado	ele era uma demonstração
A única alteração em `app/` é a correção	`git diff HEAD~3 --stat` lista só `app/database.py` dentro de `app/`

O histórico do dia

Execute no terminal

```
git log --oneline -3
```

O que deve aparecer (os códigos dos commits serão outros)

```
15  Liga a verificacao de chave estrangeira no SQLite
14  Adiciona os testes do esquema do banco - FALHA
13  Isola cada teste com um banco limpo
```

Se algo der errado

Sintoma	Causa provável	O que fazer
`fixture 'sessao' not found`	A fixture foi escrita no `test_banco.py`, ou o nome não bate	Ela mora no `conftest.py`
`no such table: usuarios`	A `banco_limpo` não rodou antes do teste	Confira o `autouse=True`
Um teste passa sozinho e falha junto	Falta de limpeza entre testes	Confira os dois `drop_all` da `banco_limpo`
Mais de 2 testes falham no Passo 6	O experimento do Passo 5 não foi desfeito	Volte os `drop_all` e apague o `test_vizinho.py`
Continua `2 failed` depois da correção	O comando foi posto fora do caminho que a função percorre	Confira em que ponto da função ele está
Apareceu um `todo_list.db` novo	A `DATABASE_URL` do `conftest.py` não é `sqlite://`	Banco em memória não cria arquivo

Amanhã

> **Dia 6 — controlar o ambiente.** O `conftest.py` define as variáveis de ambiente **uma vez**, para a suíte inteira. Amanhã você vai alterá-las **dentro de um teste** — "e se a `DATABASE_URL` não existir?" — sem sujar os outros testes. A ferramenta é o `monkeypatch`. **E há outro defeito para encontrar** — o mais sutil dos cinco.

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 6 — Controlar o ambiente sem sujá-lo

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	raises	parametrize	fixture	banco	**monkeypatch**	HTTP	sessão	duas contas

> **Hoje:** o `monkeypatch` — alterar variáveis de ambiente dentro de um teste, com tudo desfeito sozinho no final — e os testes da **configuração** da aplicação.
>
> **Ao final:** `pytest` responde **105 passed** (74 de antes + 31 de hoje).
>
> **Defeito a encontrar hoje: SIM** — e ele aparece já no **primeiro** teste do dia. É o mais sutil dos cinco: com ele, o sistema funciona.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

De onde paramos

No Dia 4 o `conftest.py` passou a definir as variáveis de ambiente da suíte — `DATABASE_URL` e `JWT_SECRET` — **uma vez**, para todos os testes. Ótimo para testar o resto do sistema. Mas e quando o **assunto** do teste é justamente a configuração? "O que acontece se a `DATABASE_URL` **não existir**?" Para responder, o teste precisa mudar o ambiente — e só para ele.

Como a aplicação se configura

A aplicação não tem endereço de banco nem segredo escritos no código. Ela lê tudo de **variáveis de ambiente** — no desenvolvimento, vindas do arquivo `.env`; em produção, definidas na plataforma de hospedagem. Quem lê é `Configuracao.do_ambiente()`, em `app/config.py`:

```
   .env  ou  variáveis da hospedagem
          |
          v
   Configuracao.do_ambiente()   --->  url_banco, segredo_jwt, origens, echo_sql, ...
          |
          +--> variável OBRIGATÓRIA faltando?  ---> RuntimeError  (a aplicação não sobe)
```

Leia no manual técnico a seção **Variáveis de ambiente** (`docs/manual_tecnico.md`). A tabela dela é a especificação dos testes de hoje:

Variável	Obrigatória?	Se faltar
`DATABASE_URL`	**sim**	a aplicação **recusa subir**, com uma mensagem que diz onde o `.env` foi procurado
`JWT_SECRET`	**sim**	a aplicação recusa subir, e ensina a gerar um segredo
`ORIGENS_PERMITIDAS`	não	valem as origens de desenvolvimento
`ECHO_SQL`	não	desligado

Por que configuração ausente precisa ser erro

Seria mais "cômodo" a aplicação usar um banco padrão quando a variável falta. O problema é o que acontece no dia em que o `.env` de produção **não for lido**:

Com um padrão silencioso	Com erro na inicialização
a aplicação sobe normalmente, apontando para um banco vazio	a aplicação não sobe
os usuários entram e não encontram os seus dados	a mensagem diz qual variável falta e onde o `.env` foi procurado
o chamado aberto diz "os dados sumiram"	o problema é corrigido antes de alguém usar

A ferramenta: monkeypatch

O `monkeypatch` é uma **fixture que já vem com o pytest** — você a viu na lista do `--fixtures` no Dia 4. O teste a pede pelo nome, como qualquer fixture, e ela oferece métodos para alterar o ambiente:

Método	O que faz
`monkeypatch.setenv(nome, valor)`	define uma variável de ambiente
`monkeypatch.delenv(nome, raising=False)`	remove uma variável; `raising=False` evita erro se ela já não existir

> **A característica que importa:** tudo o que o `monkeypatch` altera é **desfeito sozinho** no final do teste — como a `banco_limpo` de ontem desfaz o banco. No Passo 5 você vai ver o que acontece quando se altera o ambiente **sem** ele.

O que vamos testar hoje

Bloco	Testes	O que verifica
A `DATABASE_URL` é obrigatória	4	ausente, vazia ou só espaços → recusa; a mensagem diz onde está o `.env`
A leitura do ambiente	13	dialeto do banco, origens do CORS, `ECHO_SQL`, imutabilidade
De onde vem o `.env`	3	caminho absoluto, na raiz, e a variável vence o arquivo
O segredo do token	11	ausente, de exemplo, curto, próprio — e o limite de 32 bytes

O que você vai aprender hoje

Ferramenta nova	`monkeypatch` (`setenv`, `delenv`); `pytest.raises(..., match=...)` e `... as erro`; a fixture `tmp_path`
Técnica nova	valor limite — testar as **bordas** de uma regra
Arquivo que cresce	`tests/test_configuracao.py` — de 5 para 36 testes
Ainda não sabe	fazer requisições HTTP — é o Dia 7
Testes ao final	105 (74 de antes + 31 de hoje)
Commits	16, 17, 18 e 19

> **O procedimento do teste vermelho**
>
> Hoje a ordem é diferente: o defeito aparece **logo no primeiro teste**. Você vai encontrá-lo, diagnosticá-lo e corrigi-lo **antes** de escrever o resto da bateria.
>
> Relembre o procedimento: **1.** leia a saída até o fim; **2.** confira o teste contra a especificação; **3.** encontre a linha; **4.** corrija e rode de novo.

2. A DATABASE_URL obrigatória — e o defeito

Passo 1 — Os imports e a seção do dia

Arquivo: tests/test_configuracao.py (alterado) · ainda sem commit

Até hoje o arquivo importava só `normalizar_url`. Troque essa linha pelo import abaixo, que traz a classe da configuração e algumas constantes que os testes vão usar:

Código para digitar — substitua o import de app.config

```
from app.config import (
    CAMINHO_DO_ENV,
    ORIGENS_PADRAO,
    RAIZ_DO_PROJETO,
    SEGREDO_DE_EXEMPLO,
    TAMANHO_MINIMO_DO_SEGREDO,
    Configuracao,
    normalizar_url,
)
```

E, no final do arquivo, abra a seção do dia:

Código para digitar — no final do arquivo

```
# ============================================================================
# Dia 6 — `monkeypatch`: a DATABASE_URL é obrigatória
# ============================================================================
```

Entendendo o código

`from app.config import (`
`    ...`
`)`
	Com muitos nomes, o import fica entre parênteses, um nome por linha. É o mesmo `import`, só mais legível.

`Configuracao`	A classe cujo método `do_ambiente()` lê as variáveis. É o alvo de quase todos os testes de hoje.
`CAMINHO_DO_ENV`	Onde a aplicação procura o `.env`. Um teste vai conferir que esse caminho aparece na mensagem de erro.
`SEGREDO_DE_EXEMPLO`, `TAMANHO_MINIMO_DO_SEGREDO`	Usados no último bloco, o do segredo do token.

Execute no terminal

```
pytest
```

Confira no seu print

- Continua `74 passed`.

Passo 2 — O primeiro monkeypatch

Arquivo: tests/test_configuracao.py (acrescentar ao final) · ainda sem commit

O primeiro teste pergunta: **sem `DATABASE_URL`, a aplicação recusa subir?** A tabela do manual diz que sim.

Código para digitar

```
def test_sem_database_url_a_aplicacao_recusa_subir(monkeypatch):
    """Não existe banco padrão silencioso.

    Cair em SQLite quando a variável falta esconde um problema: quando o
    padrão coincide com o valor esperado, não há como saber se o `.env` foi
    lido ou se a aplicação apenas caiu no padrão.
    """
    monkeypatch.delenv("DATABASE_URL", raising=False)
    with pytest.raises(RuntimeError):
        Configuracao.do_ambiente()
```

Entendendo o código

`def test_...(monkeypatch):`	O teste **pede** a fixture `monkeypatch`, que já vem com o pytest. Você não precisa escrevê-la.
`monkeypatch.delenv( "DATABASE_URL", raising=False)`	**Prepara** o cenário: remove a variável **só durante este teste**. O `raising=False` evita erro caso ela já não exista.
`with pytest.raises( RuntimeError):`	O que a especificação promete: sem a variável, **erro de inicialização**.
`Configuracao.do_ambiente()`	**Executa**: pede à aplicação que leia a configuração, agora sem a `DATABASE_URL`.

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
>       with pytest.raises(RuntimeError):
E       Failed: DID NOT RAISE RuntimeError

tests\test_configuracao.py:70: Failed
========================== short test summary info ==========================
FAILED tests/test_configuracao.py::test_sem_database_url_a_aplicacao_recusa_subir
======================== 1 failed, 74 passed in 0.31s ========================
```

`DID NOT RAISE` — a mensagem que você conhece desde o Dia 2: esperava-se uma recusa, e a aplicação **aceitou**. O primeiro teste escrito com a ferramenta nova já encontrou algo. Antes de investigar, complete a bateria da `DATABASE_URL` no próximo passo: mais testes vermelhos dão mais pistas.

Passo 3 — Vazia também é ausente, e a mensagem

Arquivo: tests/test_configuracao.py (acrescentar ao final) · Commit 16 — vermelho, de propósito

Código para digitar

```
@pytest.mark.parametrize("valor_vazio", ["", "   "])
def test_database_url_vazia_conta_como_ausente(monkeypatch, valor_vazio):
    """Variável declarada mas sem valor é o erro de digitação mais comum no
    `.env`, e precisa falhar igual à variável ausente."""
    monkeypatch.setenv("DATABASE_URL", valor_vazio)
    with pytest.raises(RuntimeError):
        Configuracao.do_ambiente()


def test_a_mensagem_de_erro_diz_onde_o_env_foi_procurado(monkeypatch):
    """O caminho na mensagem é o que resolve o caso mais frequente: o `.env`
    existe, mas em outra pasta.

    `pytest.raises(...) as erro` guarda a exceção para inspecioná-la depois.
    """
    monkeypatch.delenv("DATABASE_URL", raising=False)
    with pytest.raises(RuntimeError, match="DATABASE_URL") as erro:
        Configuracao.do_ambiente()
    assert str(CAMINHO_DO_ENV) in str(erro.value)
```

Entendendo o código

`monkeypatch.setenv( "DATABASE_URL", valor_vazio)`	Desta vez a variável **existe**, mas vazia — `DATABASE_URL=` no `.env`, o erro de digitação mais comum.
`["", "   "]`	Fixture e `parametrize` juntos, como no Dia 4: duas formas de "declarada mas sem valor".
`pytest.raises(RuntimeError, match="DATABASE_URL")`	**Novo:** o `match` exige que a **mensagem** do erro contenha o texto. Não basta errar: tem de dizer qual variável falta.
`... as erro:`	**Novo:** guarda a exceção na variável `erro`, para examiná-la depois do `with`.
`str(CAMINHO_DO_ENV) in str(erro.value)`	`erro.value` é a exceção levantada. O teste exige que a mensagem diga **onde** o `.env` foi procurado — é o que resolve o caso "o arquivo existe, mas em outra pasta".

Execute no terminal

```
pytest
```

O que deve aparecer (o resumo do final)

```
FAILED tests/test_configuracao.py::test_sem_database_url_a_aplicacao_recusa_subir
FAILED tests/test_configuracao.py::test_database_url_vazia_conta_como_ausente[]
FAILED tests/test_configuracao.py::test_database_url_vazia_conta_como_ausente[   ]
FAILED tests/test_configuracao.py::test_a_mensagem_de_erro_diz_onde_o_env_foi_procurado
======================== 4 failed, 74 passed in 0.31s ========================
```

Confira no seu print

- **4 failed, 74 passed** — três funções, quatro casos, porque o `parametrize` conta dois.
- Todos com a mesma mensagem: `DID NOT RAISE RuntimeError`.

Feche o commit 16 — vermelho, de propósito

```
git add tests/test_configuracao.py
git commit -m "Adiciona os testes da DATABASE_URL obrigatoria - FALHA"
```

Passo 4 — O diagnóstico

Arquivo: leitura: a saída do pytest, o manual técnico e app/config.py · sem commit

Este passo **não tem código para digitar**: tem perguntas para guiar a investigação.

1. O que as saídas dizem?

Os quatro testes dizem a mesma coisa de quatro jeitos: sem `DATABASE_URL` — ausente, vazia ou só com espaços —, `Configuracao.do_ambiente()` **não levantou erro nenhum**. A aplicação aceitou subir sem saber qual banco usar.

2. O teste está certo?

Volte à seção **Variáveis de ambiente** do manual técnico: a `DATABASE_URL` é **obrigatória** e, se faltar, a aplicação **recusa subir**. Os testes cobram exatamente isso. O problema está na aplicação.

3. Onde está o defeito?

Abra `app/config.py` e encontre o método `do_ambiente`. Leia **a docstring primeiro** — em especial a seção `Raises` —, e depois o trecho que lê a `DATABASE_URL`.

> **Pistas**
>
> **Perguntas para guiar a leitura:**
>
> • O que a docstring promete que acontece quando uma variável obrigatória não está definida?
>
> • O que o código faz **quando a `DATABASE_URL` está vazia**?
>
> • Logo abaixo há o bloco que lê a `JWT_SECRET`, outra variável obrigatória. Compare os dois: o que um faz que o outro não faz?

4. Antes de corrigir, responda

Qual é o problema de ter um padrão conveniente? Com o defeito, a aplicação sobe, conecta e responde — ninguém abre um chamado. O problema não é o sistema quebrar: é não haver como **diagnosticar**. Quando o `.env` de produção não for lido, a aplicação vai subir apontando para um banco vazio, e o sintoma relatado será "os dados desapareceram" — não "a configuração está errada".

Passo 5 — A correção

Arquivo: app/config.py (alterado) · Commit 17

A correção é sua. Faça o trecho da `DATABASE_URL` cumprir o que a docstring promete.

> **Duas orientações:**
>
> • Altere **só** esse trecho de `app/config.py`. Nenhum outro arquivo precisa mudar.
>
> • Um dos quatro testes verifica a **mensagem**, não só o tipo do erro: ela precisa citar a `DATABASE_URL` **e** o caminho onde o `.env` foi procurado (a constante `CAMINHO_DO_ENV`). Os outros erros do mesmo método são um bom modelo.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
============================== 78 passed in 0.25s ==============================
```

Confira no seu print

- `78 passed` — os quatro vermelhos passaram, e nenhum outro quebrou.
- Se só três passaram, confira a mensagem do erro: ela precisa conter o caminho do `.env`.

Feche o commit 17

```
git add app/config.py
git commit -m "Recusa subir sem DATABASE_URL"
```

Experimento — por que monkeypatch, e não os.environ?

Você poderia remover a variável "na mão", com `del os.environ[...]`. Veja a diferença. Crie este arquivo **descartável**:

Código para digitar — tests/test_vazamento.py (será apagado)

```
import os


def test_1_remove_a_variavel(monkeypatch):
    monkeypatch.delenv("DATABASE_URL")
    assert "DATABASE_URL" not in os.environ


def test_2_a_variavel_continua_la():
    assert "DATABASE_URL" in os.environ
```

Execute no terminal

```
pytest tests/test_vazamento.py
```

O que deve aparecer (final)

```
============================== 2 passed in 0.04s ===============================
```

Agora troque as duas primeiras linhas do teste 1 por estas, sem o `monkeypatch`, e rode de novo:

```
def test_1_remove_a_variavel():
    del os.environ["DATABASE_URL"]
```

Execute no terminal

```
pytest tests/test_vazamento.py
```

O que deve aparecer (trecho)

```
>       assert "DATABASE_URL" in os.environ
E       AssertionError: assert 'DATABASE_URL' in environ({...})
...
FAILED tests/test_vazamento.py::test_2_a_variavel_continua_la - AssertionErro...
========================= 1 failed, 1 passed in 0.10s ==========================
```

O teste 2 **não mudou** e passou a falhar: a remoção feita no teste 1 **vazou** para ele. É o problema de isolamento de ontem, agora com variáveis de ambiente.

Com o `monkeypatch`, a variável volta sozinha no final do teste 1.

Apague o `tests/test_vazamento.py` e confirme `78 passed`.

3. O resto da configuração

Passo 6 — A leitura do ambiente

Arquivo: tests/test_configuracao.py (acrescentar ao final) · ainda sem commit

Com o defeito corrigido, o resto da bateria. Todos os testes seguem o mesmo formato: `monkeypatch` monta o ambiente, `Configuracao.do_ambiente()` lê, e um `assert` confere.

Código para digitar

```
# ------------------------------------------------------ leitura do ambiente

def test_postgres_e_reconhecido_como_banco_em_servidor(monkeypatch):
    monkeypatch.setenv("DATABASE_URL", "postgres://u:s@host:5432/banco")
    configuracao = Configuracao.do_ambiente()
    assert configuracao.dialeto == "postgresql"
    assert configuracao.banco_em_servidor is True
    assert configuracao.banco_em_memoria is False


def test_sqlite_em_memoria_e_reconhecido(monkeypatch):
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    assert Configuracao.do_ambiente().banco_em_memoria is True


def test_sem_variavel_de_ambiente_as_origens_sao_as_de_desenvolvimento(monkeypatch):
    monkeypatch.delenv("ORIGENS_PERMITIDAS", raising=False)
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    assert Configuracao.do_ambiente().origens_permitidas == ORIGENS_PADRAO


def test_origens_permitidas_aceita_lista_separada_por_virgula(monkeypatch):
    monkeypatch.setenv(
        "ORIGENS_PERMITIDAS", "https://a.com , https://b.com ,, https://c.com"
    )
    origens = Configuracao.do_ambiente().origens_permitidas
    assert origens == ("https://a.com", "https://b.com", "https://c.com")


@pytest.mark.parametrize("valor", ["1", "true", "TRUE", "sim"])
def test_echo_sql_ligado(monkeypatch, valor):
    monkeypatch.setenv("ECHO_SQL", valor)
    assert Configuracao.do_ambiente().echo_sql is True


@pytest.mark.parametrize("valor", ["", "0", "false", "nao"])
def test_echo_sql_desligado(monkeypatch, valor):
    monkeypatch.setenv("ECHO_SQL", valor)
    assert Configuracao.do_ambiente().echo_sql is False


def test_configuracao_e_imutavel():
    """Configuração alterada em tempo de execução é defeito difícil de achar."""
    configuracao = Configuracao(url_banco="sqlite://", segredo_jwt="segredo")
    with pytest.raises(Exception):
        configuracao.url_banco = "sqlite:///outro.db"
```

Entendendo o código

`setenv("DATABASE_URL", "postgres://...")`	Simula o ambiente de **produção**. A configuração reconhece o dialeto e que o banco está num servidor.
`"sqlite://"`	O banco em memória dos testes também precisa ser reconhecido como tal.
`ORIGENS_PERMITIDAS`	Quais sites podem chamar a API pelo navegador (o CORS). Sem a variável, valem os endereços de desenvolvimento.
`"https://a.com , https://b.com ,, https://c.com"`	Espaços sobrando e uma vírgula dupla — o jeito como pessoas de verdade escrevem. A leitura limpa tudo.
`== ("https://a.com", ...)`	O resultado é uma **tupla** — uma sequência que não pode ser alterada.
`["1", "true", "TRUE", "sim"]` `["", "0", "false", "nao"]`	Duas **classes de equivalência** do Dia 3: as formas de dizer "ligado" e as de dizer "desligado".
`Configuracao(url_banco=..., segredo_jwt=...)`	Cria a configuração direto, sem ler o ambiente — por isso este teste não precisa de `monkeypatch`.
`pytest.raises(Exception): configuracao.url_banco = ...`	Depois de criada, a configuração **não pode** ser alterada.

> Repare que vários testes **não** definem a `DATABASE_URL`: eles usam a que o `conftest.py` definiu para a suíte. O `monkeypatch` só altera o que o teste pede; o resto do ambiente continua como estava.

Execute no terminal

```
pytest -v tests/test_configuracao.py
```

O que deve aparecer (final)

```
tests/test_configuracao.py::test_echo_sql_ligado[1] PASSED              [ 63%]
tests/test_configuracao.py::test_echo_sql_ligado[true] PASSED           [ 68%]
tests/test_configuracao.py::test_echo_sql_ligado[TRUE] PASSED           [ 72%]
tests/test_configuracao.py::test_echo_sql_ligado[sim] PASSED            [ 77%]
tests/test_configuracao.py::test_echo_sql_desligado[] PASSED            [ 81%]
tests/test_configuracao.py::test_echo_sql_desligado[0] PASSED           [ 86%]
tests/test_configuracao.py::test_echo_sql_desligado[false] PASSED       [ 90%]
tests/test_configuracao.py::test_echo_sql_desligado[nao] PASSED         [ 95%]
tests/test_configuracao.py::test_configuracao_e_imutavel PASSED         [100%]

============================== 22 passed in 0.09s ==============================
```

Confira no seu print

- `22 passed` no `test_configuracao.py`. Rode `pytest` sozinho: **91 passed** na suíte inteira.

Passo 7 — De onde vem o .env

Arquivo: tests/test_configuracao.py (acrescentar ao final) · Commit 18

Três testes sobre **onde** a aplicação procura o `.env` — e sobre uma garantia da qual a própria suíte depende.

Código para digitar

```
# ---------------------------------------------------- de onde vem o `.env`

def test_o_env_e_procurado_na_raiz_do_projeto():
    """A raiz é a pasta que contém `app/` — a mesma do `requirements.txt`."""
    assert CAMINHO_DO_ENV.name == ".env"
    assert CAMINHO_DO_ENV.parent == RAIZ_DO_PROJETO
    assert (RAIZ_DO_PROJETO / "requirements.txt").exists()
    assert (RAIZ_DO_PROJETO / "app").is_dir()


def test_o_caminho_do_env_e_absoluto():
    """É o que torna a leitura independente do diretório de trabalho.

    Com um caminho relativo, rodar `uvicorn` de outra pasta faria o `.env` ser
    ignorado em silêncio.
    """
    assert CAMINHO_DO_ENV.is_absolute()


def test_variavel_do_ambiente_vence_o_arquivo_env(tmp_path, monkeypatch):
    """Garantia da qual a própria suíte depende.

    O `conftest.py` define `DATABASE_URL=sqlite://` antes de importar a
    aplicação. Se o `.env` da máquina sobrescrevesse esse valor, rodar os
    testes apagaria o banco de trabalho do desenvolvedor.

    `tmp_path` é outra fixture embutida do pytest: uma pasta temporária,
    exclusiva deste teste, apagada depois.
    """
    from dotenv import load_dotenv

    arquivo = tmp_path / ".env"
    arquivo.write_text(
        "DATABASE_URL=postgresql+psycopg://do-arquivo/banco", encoding="utf-8"
    )
    monkeypatch.setenv("DATABASE_URL", "sqlite://")

    load_dotenv(arquivo)

    assert Configuracao.do_ambiente().url_banco == "sqlite://"
```

Entendendo o código

`CAMINHO_DO_ENV.name` `CAMINHO_DO_ENV.parent`	O caminho é um objeto `Path`: `.name` é o nome do arquivo, `.parent` a pasta onde ele está.
`RAIZ_DO_PROJETO / "requirements.txt"`	Com `Path`, o `/` **junta caminhos**. Os dois últimos `assert` provam que a "raiz" é mesmo a pasta do projeto.
`.is_absolute()`	Caminho **absoluto** (`C:\...`) funciona de qualquer pasta. Um relativo dependeria de onde o comando foi dado.
`(tmp_path, monkeypatch)`	**Novo:** `tmp_path` é mais uma fixture embutida — uma pasta temporária só deste teste, apagada depois. Duas fixtures do pytest no mesmo teste.
`arquivo = tmp_path / ".env"` `arquivo.write_text(...)`	Cria um `.env` **falso**, dizendo que o banco é um PostgreSQL.
`monkeypatch.setenv("DATABASE_URL", "sqlite://")`	Mas a variável de ambiente diz outra coisa.
`load_dotenv(arquivo)`	Carrega o `.env` falso, do jeito que a aplicação faz.
`url_banco == "sqlite://"`	**Quem vence é a variável de ambiente.** O arquivo não sobrescreve o que já estava definido.

> Por que isso importa para **você**? O `conftest.py` define `DATABASE_URL=sqlite://` para os testes. Se o `.env` da sua máquina vencesse, rodar `pytest` apontaria para o seu banco de trabalho — e a `banco_limpo` o apagaria a cada teste.

Execute no terminal

```
pytest
pytest --setup-show
tests/test_configuracao.py::test_variavel_do_ambiente_vence_o_arquivo_env
```

O que deve aparecer (trechos)

```
============================== 94 passed in 0.31s ==============================
...
SETUP    S tmp_path_factory
        SETUP    F banco_limpo
        SETUP    F tmp_path (fixtures used: tmp_path_factory)
        SETUP    F monkeypatch
...
```

Confira no seu print

- `94 passed`.
- O `--setup-show` mostra a sua `banco_limpo` ao lado das fixtures do próprio pytest. O `S` de `tmp_path_factory` quer dizer **sessão**: ela é montada uma vez para a rodada inteira, e não a cada teste.

Feche o commit 18

```
git add tests/test_configuracao.py
git commit -m "Acrescenta os testes da leitura do ambiente"
```

Passo 8 — O segredo do token e o valor limite

Arquivo: tests/test_configuracao.py (acrescentar ao final) · Commit 19

No Dia 4 você testou o `TokenDeAcesso` recebendo o segredo pelo construtor. Na aplicação, ele vem da variável `JWT_SECRET` — e a configuração **recusa segredos fracos**: no mínimo 32 bytes, exigência da norma do algoritmo de assinatura.

A técnica do dia: valor limite

No Dia 3 você viu a **classe de equivalência**. A técnica complementar é o **valor limite**: os defeitos se concentram nas **bordas** das classes. Para uma regra "mínimo de 32 bytes":

```
   recusado                                  |  aceito
   ... 10 ... 30 31                          |  32 33 ... 50 ...
               ^^                            |  ^^
         o último recusado                   |  o primeiro aceito
```

Testar só 10 e 50 deixaria passar um código que recusasse abaixo de **40**: os dois testes passariam e nada provaria que a fronteira está no lugar certo. Os valores que importam são **31** e **32**.

Código para digitar

```
# ------------------------------------------------ o segredo que assina o token

def test_sem_jwt_secret_a_aplicacao_recusa_subir(monkeypatch):
    """Sem segredo não há como assinar nem verificar sessão alguma."""
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.delenv("JWT_SECRET", raising=False)
    with pytest.raises(RuntimeError, match="JWT_SECRET"):
        Configuracao.do_ambiente()


def test_a_mensagem_de_jwt_secret_ensina_a_gerar_um(monkeypatch):
    """Recusar sem dizer o que fazer transforma o erro em obstáculo."""
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.delenv("JWT_SECRET", raising=False)
    with pytest.raises(RuntimeError) as erro:
        Configuracao.do_ambiente()
    assert "secrets.token_urlsafe" in str(erro.value)


def test_segredo_de_exemplo_e_aceito_com_sqlite(monkeypatch):
    """Em desenvolvimento ele precisa funcionar: é o que faz o projeto rodar."""
    monkeypatch.setenv("DATABASE_URL", "sqlite:///./dev.db")
    monkeypatch.setenv("JWT_SECRET", SEGREDO_DE_EXEMPLO)
    assert Configuracao.do_ambiente().segredo_jwt == SEGREDO_DE_EXEMPLO


def test_segredo_de_exemplo_e_recusado_com_banco_de_servidor(monkeypatch):
    """O valor está publicado no repositório: em produção, é porta aberta."""
    monkeypatch.setenv(
        "DATABASE_URL", "postgresql+psycopg://u:s@host:5432/banco"
    )
    monkeypatch.setenv("JWT_SECRET", SEGREDO_DE_EXEMPLO)
    with pytest.raises(RuntimeError, match="valor de exemplo"):
        Configuracao.do_ambiente()
```

```
def test_segredo_proprio_e_aceito_com_banco_de_servidor(monkeypatch):
    """A recusa é do valor de exemplo, não de usar PostgreSQL."""
    monkeypatch.setenv(
        "DATABASE_URL", "postgresql+psycopg://u:s@host:5432/banco"
    )
    monkeypatch.setenv("JWT_SECRET", "um-segredo-gerado-de-verdade-com-32-bytes")
    assert Configuracao.do_ambiente().banco_em_servidor is True


@pytest.mark.parametrize("curto", ["x", "segredo", "a" * 31])
def test_segredo_curto_e_recusado(monkeypatch, curto):
    """RFC 7518, seção 3.2: chave HMAC-SHA256 precisa de 256 bits (32 bytes).

    `"a" * 31` é o **valor limite**: o último que ainda deve ser recusado.
    """
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.setenv("JWT_SECRET", curto)
    with pytest.raises(RuntimeError, match="minimo e 32|mínimo é 32"):
        Configuracao.do_ambiente()


def test_segredo_com_exatamente_32_bytes_e_aceito(monkeypatch):
    """O outro lado do limite: 32 bytes basta."""
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.setenv("JWT_SECRET", "a" * 32)
    assert Configuracao.do_ambiente().segredo_jwt == "a" * 32


def test_o_segredo_de_exemplo_atende_ao_tamanho_minimo():
    """Se não atendesse, copiar o .env.example deixaria o projeto sem subir."""
    assert len(SEGREDO_DE_EXEMPLO.encode("utf-8")) >= TAMANHO_MINIMO_DO_SEGREDO


def test_tamanho_e_medido_em_bytes_nao_em_caracteres(monkeypatch):
    """Um acento ocupa 2 bytes em UTF-8: 17 caracteres `ç` são 34 bytes.

    Medir em caracteres recusaria um segredo que a RFC aceita.
    """
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.setenv("JWT_SECRET", "ç" * 17)
    assert Configuracao.do_ambiente().segredo_jwt == "ç" * 17
```

Entendendo o código

`match="JWT_SECRET"`	A mensagem precisa dizer **qual** variável falta.
`"secrets.token_urlsafe" in str(erro.value)`	A mensagem precisa **ensinar** a gerar um segredo: recusar sem dizer o que fazer transforma o erro em obstáculo.
`SEGREDO_DE_EXEMPLO`	O segredo que vem no `.env.example`. Com SQLite (desenvolvimento), é aceito…
`"postgresql+psycopg://..." + SEGREDO_DE_EXEMPLO`	…com PostgreSQL (produção), é **recusado**: o valor está publicado no repositório, qualquer um o conhece.
`"um-segredo-gerado-de-verdade-com-32-bytes"`	O contraponto: a recusa é do valor de exemplo, não do PostgreSQL.
`["x", "segredo", "a" * 31]`	Três representantes da classe "curto demais" — e o último, com **31** bytes, é o **valor limite**.
`match="minimo e 32|mínimo é 32"`	A barra `|` quer dizer **ou**: o `match` aceita o texto com ou sem acento.
`"a" * 32`	O **outro lado** da fronteira: 32 bytes é aceito.
`len(SEGREDO_DE_EXEMPLO.encode("utf-8"))`	O próprio exemplo precisa cumprir a regra; senão, copiar o `.env.example` deixaria o projeto sem subir.
`"ç" * 17`	17 **caracteres**, 34 **bytes**. A regra é em bytes — o mesmo cuidado do limite de 72 bytes do bcrypt, no Dia 4.

Execute no terminal

```
pytest -v -k "segredo or jwt or bytes" tests/test_configuracao.py
```

O que deve aparecer

```
tests/test_configuracao.py::test_sem_jwt_secret_a_aplicacao_recusa_subir PASSED [  9%]
tests/test_configuracao.py::test_a_mensagem_de_jwt_secret_ensina_a_gerar_um PASSED [ 18%]
tests/test_configuracao.py::test_segredo_de_exemplo_e_aceito_com_sqlite PASSED [ 27%]
tests/test_configuracao.py::test_segredo_de_exemplo_e_recusado_com_banco_de_servidor PASSED [ 36%]
tests/test_configuracao.py::test_segredo_proprio_e_aceito_com_banco_de_servidor PASSED [ 45%]
tests/test_configuracao.py::test_segredo_curto_e_recusado[x] PASSED      [ 54%]
tests/test_configuracao.py::test_segredo_curto_e_recusado[segredo] PASSED [ 63%]
tests/test_configuracao.py::test_segredo_curto_e_recusado[aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa] PASSED [
72%]
tests/test_configuracao.py::test_segredo_com_exatamente_32_bytes_e_aceito PASSED [ 81%]
tests/test_configuracao.py::test_o_segredo_de_exemplo_atende_ao_tamanho_minimo PASSED [ 90%]
tests/test_configuracao.py::test_tamanho_e_medido_em_bytes_nao_em_caracteres PASSED [100%]

====================== 11 passed, 25 deselected in 0.07s ======================
```

Lendo a saída

`-k "segredo or jwt or bytes"`	**Novo:** o `-k` roda só os testes cujo **nome** contém alguma dessas palavras. Útil para focar num assunto sem rodar a suíte inteira.
`25 deselected`	Os outros 25 testes do arquivo foram deixados de fora pelo filtro — não falharam, só não rodaram.

Experimento — mova a fronteira

Para ver o valor limite trabalhando, troque o `"a" * 31` do `parametrize` por `"a" * 32` — um valor que **não** é curto — e rode `pytest`:

O que deve aparecer (trecho)

```
_______ test_segredo_curto_e_recusado[aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa] _______
...
>       with pytest.raises(RuntimeError, match="minimo e 32|mínimo é 32"):
E       Failed: DID NOT RAISE RuntimeError
...
======================== 1 failed, 104 passed in 0.39s ========================
```

> Com 32 bytes, o segredo é **aceito** — então o teste que esperava recusa falha. Os testes com 31 e 32 juntos **cercam** a fronteira: se alguém mudar o mínimo para 31 ou para 33, um dos dois quebra. **Desfaça o experimento**: volte para `"a" * 31` e confirme `105 passed`.

Execute no terminal

```
pytest
```

Confira no seu print

- `105 passed`.

Feche o commit 19

```
git add tests/test_configuracao.py
git commit -m "Acrescenta os testes do segredo do token"
```

4. Fechamento do dia

O que o monkeypatch trouxe

Recurso	Para quê	Onde você usou
`monkeypatch.delenv`	simular a variável **ausente**	`DATABASE_URL`, `JWT_SECRET`, `ORIGENS_PERMITIDAS`
`monkeypatch.setenv`	simular um valor qualquer	quase todos os testes do dia
`pytest.raises(..., match=)`	exigir o **texto** da mensagem de erro	`DATABASE_URL`, segredo
`pytest.raises(...) as erro`	examinar a exceção depois	mensagem do `.env`, do `JWT_SECRET`
`tmp_path`	uma pasta temporária só do teste	o `.env` falso
`pytest -k`	rodar só os testes de um assunto	os testes do segredo

Confira o que o dia entregou

Verificação	Esperado
`pytest`	105 passed
Nenhum teste usa `os.environ[...] =` ou `del os.environ`	todos alteram o ambiente pelo `monkeypatch`
O `test_vazamento.py` foi apagado	ele era uma demonstração
Os limites aparecem como casos	31 e 32 bytes
A única alteração em `app/` é a correção	`git diff HEAD~4 --stat` lista só `app/config.py` dentro de `app/`

O histórico do dia

Execute no terminal

```
git log --oneline -4
```

O que deve aparecer (os códigos dos commits serão outros)

```
19  Acrescenta os testes do segredo do token
18  Acrescenta os testes da leitura do ambiente
17  Recusa subir sem DATABASE_URL
16  Adiciona os testes da DATABASE_URL obrigatoria - FALHA
```

Se algo der errado

Sintoma	Causa provável	O que fazer
Um teste de configuração contamina outro	A variável foi alterada com `os.environ`	Use o `monkeypatch`, que desfaz sozinho
`KeyError: 'DATABASE_URL'` no `delenv`	Faltou o `raising=False`	A variável pode já não existir
Só 3 dos 4 testes passam depois da correção	A mensagem não cita o caminho do `.env`	Inclua `CAMINHO_DO_ENV` na mensagem
`Regex pattern did not match`	O texto do `match` não está na mensagem do erro	Leia a mensagem que o pytest mostra e compare
`NameError: name 'Configuracao'`	O import do Passo 1 não foi trocado	Substitua o import de `app.config`

Amanhã

> **Dia 7 — o teste que atravessa a aplicação.** Até hoje nenhum teste passou por uma **rota** da API: se alguém apagasse os arquivos de rotas, os 105 continuariam verdes. Amanhã entra o `TestClient`, que faz requisições HTTP de verdade, e o **dublê de teste**. **E o quarto defeito** — um defeito de segurança que não quebra nada.

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 7 — O teste que atravessa a aplicação inteira

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	raises	parametrize	fixture	banco	monkeypatch	**HTTP**	sessão	duas contas

Hoje: o `TestClient` — testes que fazem **requisições HTTP** à API — e o **dublê de teste**, um substituto do hash de senha que deixa a suíte rápida.

Ao final: `pytest` responde **128 passed** (105 de antes + 23 de hoje).

Defeito a encontrar hoje: SIM — um defeito de **segurança** que não quebra nenhuma funcionalidade.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

De onde paramos

Os 105 testes que você escreveu até ontem chamam funções e métodos **diretamente**: `Tarefa(...)`, `TokenDeAcesso.emitir(...)`, `Configuracao.do_ambiente()`. Nenhum deles passa por uma **rota** da API. Se alguém apagasse os arquivos de `app/routers/`, os 105 continuariam verdes — e o front-end não conseguiria fazer nada.

O caminho de uma requisição

Quando o front-end cadastra um usuário, a requisição atravessa todas as camadas da aplicação, e volta como JSON:

```
POST /usuarios  {"nome": ..., "email": ..., "senha": ...}
       |
       v
rota (app/routers)  -->  serviço  -->  repositório  -->  modelo  -->  banco
       |
       v
schema de saída (app/schemas.py)  -->  JSON da resposta  -->  front-end
```

Um teste de API percorre esse caminho inteiro numa chamada só. Ele não substitui os testes de domínio — mede outra coisa:

	Teste de domínio (Dias 1 a 6)	Teste de API (hoje)
O que prova	que a regra está correta	que a regra **chega ao cliente**
Quando falha	diz exatamente qual regra quebrou	diz que algo no caminho quebrou
Custo	microssegundos	milissegundos
Se a rota não existir	continua verde	falha

Os endpoints da API

A API tem sete endpoints, divididos em dois arquivos de rotas: `app/routers/usuarios.py` e `app/routers/tasks.py`. Hoje você testa só os **públicos** — os que não exigem sessão aberta:

Endpoint	Exige token?	Hoje?
`POST /usuarios` — cadastro	não	**sim**
`POST /usuarios/login` — login	não	**sim**
`GET /usuarios/eu` — a própria conta	sim	só que **sem** token ele recusa
`POST`, `GET`, `PATCH`, `DELETE` em `/tasks`	sim	não — é o Dia 8

As ferramentas novas

O TestClient

O `TestClient`, do FastAPI, faz requisições à aplicação **sem** subir servidor, sem abrir porta e sem usar a rede: ele conversa com o objeto `app` dentro do próprio processo do teste. A resposta tem três atributos que você vai usar:

Atributo	O que traz
`resposta.status_code`	o código HTTP — 200, 201, 401, 409, 422
`resposta.json()`	o corpo interpretado como dicionário Python
`resposta.text`	o corpo como **texto cru**, antes de interpretar

Os códigos de status HTTP que vão aparecer nos testes:

Código	Nome	Quando a API devolve
`200`	OK	deu certo (login)
`201`	Created	deu certo **e criou** algo (cadastro)
`401`	Unauthorized	não sei quem você é: credencial errada ou sem token
`404`	Not Found	esse endereço não existe
`409`	Conflict	o pedido está certo, mas o estado atual não permite (e-mail repetido)
`422`	Unprocessable Entity	o corpo da requisição é inválido (e-mail malformado)

O dublê de teste

Cada cadastro e cada login geram um hash de senha com o bcrypt — que é **lento de propósito**: centenas de milissegundos por hash, para encarecer ataques. Uma suíte que cadastra dezenas de usuários pagaria esse custo em cada um, sem testar nada de novo.

A saída é um **dublê**: um objeto substituto que cumpre o mesmo contrato do original — como o dublê de um ator, que entra na cena perigosa no lugar dele. Hoje você vai escrever o `HashFake` e entregá-lo à aplicação pelo `dependency_overrides`, o mecanismo do FastAPI para trocar uma dependência durante o teste.

> O dublê só é possível porque `HashDeSenha` é uma **classe abstrata** — lembra do `test_implementacao_incompleta_nao_pode_ser_instanciada`, do Dia 4? A aplicação depende do **contrato**, não do bcrypt. Qualquer classe que cumpra o contrato pode entrar no lugar.

O que vamos testar hoje

Arquivo	Testes	O que verifica
`tests/test_api.py` (novo)	13	cadastro, login e o que **não** deve existir na API
`tests/test_integracao_frontend.py` (novo)	10	o **formato** que o front-end consome: campos, erros e mensagens

O que você vai aprender hoje

Ferramenta nova	`TestClient`; o dublê de teste (`HashFake`); `app.dependency_overrides`; `filterwarnings` no `pytest.ini`
Técnica nova	fixture que **pede outra** fixture; o registro de defeito com evidência
Arquivos que nascem	`tests/test_api.py` e `tests/test_integracao_frontend.py`
Arquivos que crescem	`tests/conftest.py` e `pytest.ini`
Ainda não sabe	fazer requisições **autenticadas** — é o Dia 8
Testes ao final	128 (105 de antes + 23 de hoje)
Commits	20, 21, 22 e 23

> **O procedimento do teste vermelho**
>
> Hoje o defeito aparece no **terceiro** teste da API. Como no Dia 6, você vai continuar escrevendo a bateria com ele vermelho — os outros testes precisam passar, e cada um que passa **descarta** um suspeito.
>
> E hoje há uma etapa a mais antes da correção: **registrar** o defeito, com evidência.

2. Preparar o terreno: o conftest.py

Passo 1 — Importar a aplicação, e os avisos que aparecem

Arquivo: tests/conftest.py e pytest.ini (alterados) · ainda sem commit

Para fazer requisições, o `conftest.py` precisa conhecer a aplicação inteira. No bloco de imports do topo do arquivo, **acrescente** as três linhas marcadas e troque o import de `app.security`:

Código para digitar — o bloco de imports do conftest.py (sem as setas)

```
import pytest  # noqa: E402
from fastapi.testclient import TestClient  # noqa: E402           <- nova
from sqlalchemy.orm import Session  # noqa: E402

from app.database import Base, SessionLocal, criar_tabelas, engine  # noqa: E402
from app.dependencies import obter_hasher  # noqa: E402           <- nova
from app.main import app  # noqa: E402                            <- nova
from app.security import HashBcrypt, HashDeSenha  # noqa: E402    <- alterada
```

Entendendo o código

`from fastapi.testclient import TestClient`	O cliente HTTP de teste, que vem com o FastAPI. Ele usa a biblioteca `httpx`, que já está no `requirements.txt` desde o Dia 1.
`from app.dependencies import obter_hasher`	A **dependência** que a aplicação chama para conseguir um hasher. É ela que o teste vai substituir.
`from app.main import app`	O objeto da aplicação FastAPI — o mesmo que o `uvicorn` sobe. O `TestClient` conversa com ele.
`HashDeSenha`	A classe abstrata, o **contrato** do hash. O dublê vai herdar dela.
`# noqa: E402`	Avisa ao verificador de estilo que o import fora do topo é proposital: as variáveis de ambiente precisam ser definidas **antes** (Dia 4).

Execute no terminal

```
pytest
```

O que deve aparecer (trecho final)

```
============================== warnings summary ===============================
venv\Lib\site-packages\fastapi\testclient.py:1
  C:\projetos\todo-list-backend\venv\Lib\site-packages\fastapi\testclient.py:1:
  StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
  deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================= 105 passed, 1 warning in 0.52s ========================
```

> Nenhum teste novo foi escrito, e surgiu uma seção **warnings summary**. Um **aviso** (*warning*) não é falha: os 105 passaram. É uma biblioteca avisando que algo vai mudar.
>
> **O número e o texto dos avisos dependem das versões instaladas na sua máquina.** Você pode ver este aviso, outros, ou dezenas deles. O procedimento é o mesmo para todos.

O procedimento: investigar antes de silenciar

O reflexo errado é esconder os avisos. O procedimento é responder a três perguntas, nesta ordem:

Pergunta	Como responder	No aviso acima
1. De onde vem?	o caminho do arquivo, na primeira linha do aviso	`site-packages\fastapi` — uma **biblioteca**, não o seu código
2. O que ele pede?	o texto do aviso	trocar o `httpx` pelo `httpx2`: uma decisão sobre **dependências** do projeto
3. Há algo a corrigir no código?	se a resposta 1 for uma biblioteca, quase sempre não	não: nenhuma linha deste projeto causa o aviso

Quando a resposta é "não há o que corrigir aqui", o aviso pode ser **silenciado — com a justificativa escrita**. O projeto já investigou dois avisos que aparecem com outras versões do FastAPI, do Pydantic e do Python. Acrescente ao **final** do `pytest.ini` o bloco que registra essa decisão:

Código para digitar — no final do pytest.ini

```
# Os dois avisos abaixo vêm das bibliotecas, não deste projeto, e começaram a
# aparecer no Dia 7, quando o conftest.py passou a importar a aplicação inteira
# — a ponto de esconder o que importa na saída. Ficam silenciados aqui, com a
# justificativa registrada.
#
# A regra de conduta é essa: silenciar aviso **só** depois de descobrir de onde
# ele vem e por que não há o que corrigir, e deixar a justificativa escrita.
# Silenciar sem investigar é como apagar o alarme de incêndio sem olhar a casa.
#
# Nota: foi considerado usar `error` para transformar aviso novo em falha, que
# é uma boa prática em projeto de produção. Foi descartado de propósito: em
# laboratório, cada máquina tem um conjunto diferente de plugins do pytest
# instalados, e qualquer aviso de um deles quebraria a suíte do aluno por um
# motivo que não tem relação com o código dele.
filterwarnings =
    # FastAPI 0.115.6 + Pydantic 2.13.3: ao clonar os schemas usados em
    # `response_model`, o FastAPI aplica `Field()` sobre campos de tipo união
    # (por exemplo `prazo: date | None`). O aviso se reproduz com qualquer
    # modelo nessas condições, com ou sem `alias_generator`, e desaparece ao
    # atualizar o FastAPI. Não há o que corrigir no código da aplicação.
    ignore::pydantic.warnings.UnsupportedFieldAttributeWarning
    # Starlette usa `asyncio.iscoroutinefunction`, descontinuado no Python 3.14.
    ignore:'asyncio.iscoroutinefunction' is deprecated:DeprecationWarning
```

Entendendo o código

`filterwarnings =`	A opção do `pytest.ini` que decide o que fazer com cada aviso. Cada linha recuada abaixo dela é uma regra.
`ignore::pydantic.warnings.UnsupportedFieldAttributeWarning`	`ignore` + a **categoria** do aviso: silencia todos os avisos dessa classe.
`ignore:'asyncio.iscorout... ' is deprecated:DeprecationWarning`	`ignore` + o **começo do texto** + a categoria: silencia só esse aviso específico.
`# FastAPI 0.115.6 + ...`	O comentário acima de cada regra é a **justificativa**. Sem ela, ninguém saberia daqui a seis meses se a regra ainda faz sentido.

Execute no terminal

```
pytest
```

Confira no seu print

- Continua `105 passed`.
- Se você via os avisos do Pydantic ou do `asyncio`, eles sumiram.
- O aviso do `httpx2`, se apareceu, **continua** — e tudo bem: ele foi investigado, e a decisão de trocar uma dependência não é do teste. Fica visível, como lembrete.

> **No máximo um aviso, investigado.** É por isso que o projeto **não** transforma aviso em erro: em laboratório, cada máquina tem um conjunto diferente de pacotes, e um aviso de biblioteca quebraria a suíte por um motivo que não tem nada a ver com o seu código.

Passo 2 — O dublê e o cliente HTTP

Arquivo: tests/conftest.py (acrescentar ao final) · ainda sem commit

Código para digitar

```
# ================================================================================
# Dia 7 — HTTP: o dublê e o cliente
# ================================================================================

class HashFake(HashDeSenha):
    """Hash previsível e instantâneo, para uso exclusivo nos testes de API.

    É um **dublê de teste**: um substituto que cumpre o mesmo contrato do objeto
    real e serve a um propósito diferente.

    Existe porque a aplicação usa o bcrypt com custo 12 — centenas de
    milissegundos por hash, de propósito, para encarecer um ataque de força
    bruta. Cada cadastro e cada login pela API pagariam esse custo, sem testar
    nada de novo.

    Só é possível substituir porque `HashDeSenha` é uma classe abstrata, e a
    aplicação depende do contrato, não da implementação.

    O formato `fake$senha` é legível de propósito: quando um teste falha, a
    mensagem mostra exatamente qual senha foi usada. Em produção isso seria uma
    falha grave de segurança — e é por isso que esta classe nunca sai de
    `tests/`. A resistência do algoritmo real continua verificada em
    `test_seguranca.py`.
    """

    def gerar(self, senha: str) -> str:
        return f"fake${senha}"

    def verificar(self, senha: str, hash_armazenado: str) -> bool:
        return hash_armazenado == self.gerar(senha)


@pytest.fixture
def cliente(banco_limpo) -> TestClient:
    """Cliente HTTP **sem** autenticação, com o hash substituído pelo dublê.

    **`dependency_overrides`** é o mecanismo do FastAPI para trocar uma
    dependência durante o teste. A aplicação pede `obter_hasher`; aqui o
    `HashFake` é posto no lugar, e o serviço nem sabe da substituição.

    O `finally` com `clear()` é obrigatório: `app` é um objeto global,
    compartilhado por todos os testes. Uma substituição que não fosse desfeita
    vazaria para o teste seguinte.

    **`TestClient(app)` é usado sem `with` de propósito.** Com `with`, o
    cliente dispara o ciclo de vida da aplicação, que tentaria criar as tabelas
    de novo. Quem cuida disso é a `banco_limpo`, pedida aqui no parâmetro para
    garantir a ordem: banco limpo primeiro, cliente depois.
    """
    app.dependency_overrides[obter_hasher] = HashFake
    try:
        yield TestClient(app)
    finally:
        app.dependency_overrides.clear()
```

Entendendo o código

`class HashFake(HashDeSenha):`	O dublê **herda** do contrato. Por isso precisa ter os dois métodos que o contrato exige: `gerar` e `verificar`.
`return f"fake${senha}"`	O "hash" é só `fake$` + a senha: instantâneo e **legível**. Guarde esse detalhe — ele vai ser importante no Passo 4.
`return hash_armazenado == self.gerar(senha)`	Confere a senha do jeito mais simples: gera de novo e compara.
`def cliente(banco_limpo):`	A fixture **pede** a `banco_limpo` no parâmetro, para garantir a ordem: primeiro o banco limpo, depois o cliente.
`app.dependency_overrides [obter_hasher] = HashFake`	"Quando a aplicação pedir `obter_hasher`, entregue um `HashFake`." O serviço recebe o dublê sem saber da troca.
`try: yield TestClient(app)`	Entrega ao teste um cliente HTTP ligado à aplicação — o mesmo `yield` da `sessao`, do Dia 5.
`finally: app.dependency_overrides .clear()`	Desfaz a troca, **mesmo se o teste falhar**. O `app` é global: sem o `clear()`, o dublê vazaria para o teste seguinte.

> O `HashFake` entra **só** pelo `cliente`, nos testes de API — onde o hash é um **meio**. Nos testes em que o hash é o **assunto** (`test_seguranca.py`), o bcrypt de verdade continua, pela fixture `hasher`. Trocar tudo por dublê deixaria o algoritmo real sem verificação nenhuma.

Execute no terminal

```
pytest
```

PRINT DO TERMINAL · nada quebrou

Cole aqui o print do terminal.

Confira no seu print

- Continua `105 passed` — nenhum teste pede o `cliente` ainda.

Passo 3 — Uma fixture que pede outra

Arquivo: tests/conftest.py (acrescentar ao final) · Commit 20

Vários testes de hoje precisam de um usuário que **já existe** — para testar o login, por exemplo. Duas fixtures resolvem isso, uma em cima da outra:

Código para digitar

```
@pytest.fixture
def sessao_aberta(cliente: TestClient) -> dict:
    """Cadastra um usuário e devolve a resposta do cadastro: token e usuário.

    Uma fixture que **pede outra**: ela recebe o `cliente` pronto.

    O `assert` aqui dentro não é um teste: é uma **pré-condição**. Se o
    cadastro falhar, o erro aparece aqui, com o corpo da resposta na mensagem,
    em vez de virar um `KeyError` incompreensível dentro do teste.
    """
    resposta = cliente.post(
        "/usuarios",
        json={"nome": "Ana Souza", "email": "ana@exemplo.com", "senha": "senha123"},
    )
    assert resposta.status_code == 201, resposta.text
    return resposta.json()


@pytest.fixture
def usuario(sessao_aberta: dict) -> dict:
    """Um usuário já cadastrado: o dicionário `usuario` da resposta.

    A cadeia inteira — `banco_limpo` → `cliente` → `sessao_aberta` →
    `usuario` — é montada pelo pytest na ordem certa. Um teste que pede só
    `usuario` recebe tudo isso de graça.
    """
    return sessao_aberta["usuario"]
```

Entendendo o código

`def sessao_aberta(cliente):`	Uma fixture que **pede outra**: ela recebe o `cliente` pronto, com o dublê já instalado.
`cliente.post("/usuarios", json={...})`	A primeira **requisição HTTP** da trilha: um `POST` para `/usuarios`, com o corpo em JSON. O `json=` converte o dicionário sozinho.
`assert resposta.status_code == 201, resposta.text`	Não é um teste, é uma **pré-condição**. Se o cadastro falhar, a mensagem mostra o corpo da resposta — em vez de um `KeyError` sem sentido no teste.
`return resposta.json()`	Devolve o corpo da resposta como dicionário: o token e os dados do usuário.
`return sessao_aberta["usuario"]`	A fixture `usuario` pega só uma parte do que a `sessao_aberta` devolveu.

```
   o teste pede:   usuario
                      |  precisa de
                   sessao_aberta
                      |  precisa de
                   cliente
                      |  precisa de
                   banco_limpo

   o pytest monta de baixo para cima, e desmonta de cima para baixo
```

Execute no terminal

```
pytest --fixtures
```

O que deve aparecer (trecho)

```
-------------------- fixtures defined from tests.conftest --------------------
cliente -- tests\conftest.py:132
    Cliente HTTP **sem** autenticação, com o hash substituído pelo dublê.

sessao_aberta -- tests\conftest.py:156
    Cadastra um usuário e devolve a resposta do cadastro: token e usuário.

usuario -- tests\conftest.py:174
    Um usuário já cadastrado: o dicionário `usuario` da resposta.

hasher -- tests\conftest.py:48
    O hash de senha **de verdade**, com o custo no mínimo.

banco_limpo -- tests\conftest.py:67
    Garante um banco vazio no início e no fim de cada teste.

sessao -- tests\conftest.py:84
    Sessão de banco para os testes que precisam gravar direto.
```

Confira no seu print

- As seis fixtures do `conftest.py`, cada uma com a **primeira linha da docstring** — a docstring é a documentação que o `--fixtures` mostra.
- Rode `pytest`: continua `105 passed`.

Feche o commit 20

```
git add tests/conftest.py pytest.ini
git commit -m "Adiciona o cliente HTTP e o duble do hash"
```

3. Os testes da API — e o defeito

Passo 4 — As primeiras requisições

Arquivo: tests/test_api.py (novo) · ainda sem commit

Crie o arquivo `tests/test_api.py` com o cabeçalho, um corpo de cadastro reaproveitável e os dois primeiros testes:

Código para digitar — tests/test_api.py

```
"""Testes dos endpoints: a aplicação inteira, atravessada por HTTP.
Os arquivos anteriores chamam funções e métodos diretamente. Este faz
**requisições**: chega pela rota, passa pelo serviço, pelo repositório e pelo
modelo, e volta como JSON. É o nível que prova que as camadas conversam.
| | Teste de domínio | Teste de API |
|---|---|---|
| O que prova | que a regra está certa | que a regra chega ao cliente |
| Quando falha | diz exatamente qual regra | diz que algo no caminho quebrou |
| Custo | microssegundos | milissegundos |
Nenhum dos dois substitui o outro.
"""
from fastapi.testclient import TestClient

#: Corpo de cadastro válido, reaproveitado com `{**USUARIO_VALIDO, "campo": ...}`.
#:
#: É o mesmo papel das fábricas `nova_tarefa` e `novo_usuario` do Dia 1: um
#: ponto único a alterar quando o contrato de entrada mudar.
USUARIO_VALIDO = {
    "nome": "Bruno Lima",
    "email": "bruno@exemplo.com",
    "senha": "senha123",
}
# ============================================================================
# Dia 7 — `TestClient`: os endpoints públicos
# ============================================================================


def test_raiz_responde(cliente: TestClient):
    """O teste mais simples do arquivo, e o que falha primeiro quando a
    aplicação nem sobe. Serve de *smoke test*: se este falha, não vale
    investigar os outros."""
    assert cliente.get("/").status_code == 200



def test_cadastrar_usuario_devolve_201_com_sessao_aberta(cliente: TestClient):
    """O cadastro já devolve o token: quem criou a conta está autenticado.

    Três coisas são verificadas de uma vez, e cada uma é parte do contrato: o
    **código** 201 (recurso criado), o **corpo** com o token, e o **formato**
    dos nomes de campo — `criadoEm`, em camelCase, e não `criado_em`.
    """
    resposta = cliente.post("/usuarios", json=USUARIO_VALIDO)
    assert resposta.status_code == 201
    corpo = resposta.json()
    assert corpo["accessToken"]
    assert corpo["tokenType"] == "bearer"
    assert corpo["usuario"]["nome"] == "Bruno Lima"
    assert "criadoEm" in corpo["usuario"], "o contrato usa camelCase"
```

Entendendo o código

`USUARIO_VALIDO = {...}`	Um corpo de cadastro válido, escrito **uma vez**. Os testes de erro vão reaproveitá-lo trocando um campo só.
`def test_raiz_responde(cliente):`	O *smoke test* (teste de fumaça): o mais simples possível. Se a aplicação nem sobe, é ele que falha primeiro.
`cliente.get("/")` `.status_code == 200`	Uma requisição `GET` à raiz e a conferência do código de status.
`cliente.post("/usuarios",` `json=USUARIO_VALIDO)`	O cadastro, pela API — o mesmo que o front-end vai fazer.
`== 201`	**Created**: o cadastro criou um recurso. O código de status é parte do contrato.
`corpo["accessToken"]`	O `assert` de um texto é verdadeiro se ele **não for vazio**: o token veio.
`"criadoEm" in` `corpo["usuario"]`	O **formato**: no Python o campo é `criado_em`; no JSON, em camelCase, como o front-end espera. A frase depois da vírgula aparece se o `assert` falhar.

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
tests\test_api.py ..                                                    [  1%]
tests\test_autenticacao.py .............                                [ 14%]
...
====================== 107 passed, 1 warning in 0.37s ======================
```

Confira no seu print

- `107 passed` — o `test_api.py` aparece **primeiro** na lista, em ordem alfabética.
- Perceba a velocidade: o cadastro fez um hash de senha, mas com o `HashFake` ele é instantâneo.

Passo 5 — O cadastro não pode expor a senha

Arquivo: tests/test_api.py (acrescentar ao final) · ainda sem commit

Leia no manual técnico a regra **RN-05**, na seção **Regras de negócio**: a senha nunca é armazenada nem **exposta** em texto puro. O teste a seguir verifica a parte do "exposta": nada da senha pode aparecer na resposta.

Código para digitar

```
def test_cadastro_nao_expoe_a_senha(cliente: TestClient):
    """Nem a senha nem o hash dela podem aparecer na resposta (RN-05).

    A verificação é feita sobre `resposta.text` — o JSON como texto, antes de
    ser interpretado — de propósito: assim ela não depende de saber em qual
    campo o vazamento aconteceria.

    As duas grafias são procuradas porque a API traduz nomes na borda: o que no
    Python se chama `senha_hash` chegaria ao cliente como `senhaHash`.
    """
    resposta = cliente.post("/usuarios", json=USUARIO_VALIDO)
    inteiro = resposta.text
    assert "senha123" not in inteiro
    assert "senhaHash" not in inteiro and "senha_hash" not in inteiro
```

Entendendo o código

`inteiro = resposta.text`	O corpo como **texto cru**. Procurar no texto inteiro dispensa saber **em qual campo** o vazamento estaria — o teste pega qualquer um.
`assert "senha123" not in inteiro`	A senha digitada no cadastro não pode aparecer em lugar nenhum da resposta.
`"senhaHash" not in ... and "senha_hash" not in`	Nem o **hash** da senha. As duas grafias porque a API traduz os nomes para camelCase.

> Repare na forma do teste: ele não verifica um **valor**, verifica uma **propriedade** — "nenhuma resposta contém a senha". É o tipo de teste que encontra falhas de segurança.

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
>       assert "senha123" not in inteiro
E       assert 'senha123' not in '{"accessTok...9.516490Z"}}'
E
E         'senha123' is contained here:
E           sh":"fake$senha123","criadoEm":"2026-11-09T13:58:49.516490Z"}}
E         ?           ++++++++

tests\test_api.py:68: AssertionError
========================== short test summary info ==========================
FAILED tests/test_api.py::test_cadastro_nao_expoe_a_senha - assert 'senha123'...
================== 1 failed, 107 passed, 1 warning in 0.44s ==================
```

Lendo a saída

`'senha123' is contained here:`	O pytest mostra **onde**, no texto da resposta, encontrou a senha.
`sh":"fake$senha123"`	O trecho cortado é o final de `"senhaHash":"fake$senha123"`: há um campo chamado **senhaHash** na resposta.
`++++++++`	As cruzes marcam, embaixo, exatamente os caracteres de `senha123`.

> **Por que a senha aparece legível?** Porque o hasher dos testes é o `HashFake`, que devolve `fake$` + a senha. O dublê não só acelerou a suíte: tornou o vazamento **legível**.
>
> Antes de investigar, complete a bateria da API nos próximos passos: os testes que passarem vão descartar suspeitos.

Passo 6 — Os erros do cadastro

Arquivo: tests/test_api.py (acrescentar ao final) · ainda sem commit

Código para digitar

```
def test_cadastrar_com_email_repetido_devolve_409(cliente: TestClient):
    """409 Conflict: o pedido está correto, o estado atual é que não permite."""
    cliente.post("/usuarios", json=USUARIO_VALIDO)
    resposta = cliente.post("/usuarios", json=USUARIO_VALIDO)
    assert resposta.status_code == 409
    assert resposta.json()["detail"] == "E-mail já cadastrado."


def test_email_duplicado_ignora_maiusculas_e_espacos(cliente: TestClient):
    """RN-01 de verdade: o mesmo endereço escrito de outra forma é duplicata."""
    cliente.post("/usuarios", json=USUARIO_VALIDO)
    resposta = cliente.post(
        "/usuarios", json={**USUARIO_VALIDO, "email": "  BRUNO@Exemplo.com "}
    )
    assert resposta.status_code == 409


def test_email_malformado_devolve_422(cliente: TestClient):
    """422 vem do Pydantic, antes de a requisição chegar ao serviço."""
    resposta = cliente.post("/usuarios", json={**USUARIO_VALIDO, "email": "bruno@"})
    assert resposta.status_code == 422


def test_senha_curta_devolve_422(cliente: TestClient):
    resposta = cliente.post("/usuarios", json={**USUARIO_VALIDO, "senha": "123"})
    assert resposta.status_code == 422
```

Entendendo o código

`cliente.post(...)` `cliente.post(...)`	O mesmo cadastro **duas vezes**. A primeira prepara o cenário; a segunda é a que se testa.
`== 409`	**Conflict**: o pedido está certo, o estado do sistema é que não permite.
`resposta.json()["detail"]`	A mensagem de erro, que o front-end vai mostrar na tela.
`{**USUARIO_VALIDO,` `"email": "  BRUNO@..."}`	O `**` **copia** o dicionário inteiro, e o campo escrito depois **substitui** o original. Muda só o e-mail.
`"  BRUNO@Exemplo.com "`	O mesmo e-mail com maiúsculas e espaços: a RN-01 de verdade. Você testou a normalização no domínio (Dia 3); aqui ela chega até a API.
`== 422`	O corpo é inválido. Quem recusa é o **Pydantic**, antes de a requisição chegar ao serviço.

Execute no terminal

```
pytest
```

O que deve aparecer (o resumo do final)

```
FAILED tests/test_api.py::test_cadastro_nao_expoe_a_senha - assert 'senha123'...
================== 1 failed, 111 passed, 1 warning in 0.48s ==================
```

Confira no seu print

- **1 failed, 111 passed** — os quatro testes novos passaram; o vermelho continua sendo só o da senha.
- Conclusão parcial: o cadastro **funciona** — cria, recusa duplicatas, valida o corpo. O problema não é o cadastro em si.

Passo 7 — O login, e o que não deve existir

Arquivo: tests/test_api.py (acrescentar ao final) · Commit 21 — vermelho, de propósito

Os testes de login precisam de um usuário cadastrado **antes** — e para isso pedem a fixture `usuario`, do Passo 3. Em seguida, três testes sobre o que a API **não** pode ter.

Código para digitar

```
def test_login_correto_devolve_200_com_token(cliente: TestClient, usuario: dict):
    """200 e não 201: o login não cria recurso nenhum, só abre sessão.
    O usuário já existe porque o teste pediu a fixture `usuario`, que faz o
    cadastro antes do teste começar.
    """
    resposta = cliente.post(
        "/usuarios/login", json={"email": "ana@exemplo.com", "senha": "senha123"}
    )
    assert resposta.status_code == 200
    corpo = resposta.json()
    assert corpo["usuario"]["id"] == usuario["id"]
    assert corpo["accessToken"]
def test_login_com_senha_errada_devolve_401(cliente: TestClient, usuario: dict):
    resposta = cliente.post(
        "/usuarios/login", json={"email": "ana@exemplo.com", "senha": "errada"}
    )
    assert resposta.status_code == 401
def test_login_com_email_inexistente_devolve_401_com_a_mesma_mensagem(
    cliente: TestClient, usuario: dict
):
    """Mensagem idêntica nos dois casos: não revela quais e-mails existem.

    Repare no formato do teste: ele não verifica *qual* é a mensagem, e sim que
    as duas são **iguais**. É uma propriedade, não um valor.
    """
    inexistente = cliente.post(
        "/usuarios/login", json={"email": "ninguem@exemplo.com", "senha": "senha123"}
    )
    senha_errada = cliente.post(
        "/usuarios/login", json={"email": "ana@exemplo.com", "senha": "errada"}
    )
    assert inexistente.status_code == senha_errada.status_code == 401
    assert inexistente.json()["detail"] == senha_errada.json()["detail"]
def test_nao_existe_mais_busca_de_usuario_por_id(cliente: TestClient):
    """`GET /usuarios/{id}` saiu da API — era enumeração de contas.
    Um teste que verifica a **ausência** de um endpoint impede que a rota volte
    numa refatoração futura "porque o front-end precisava".
    """
    assert cliente.get("/usuarios/1").status_code == 404
    assert cliente.get("/usuarios/9999").status_code == 404
def test_consultar_a_propria_conta_exige_token(cliente: TestClient):
    """401 e não 403: o servidor não sabe quem está pedindo."""
    assert cliente.get("/usuarios/eu").status_code == 401
def test_openapi_expoe_os_sete_endpoints(cliente: TestClient):
    """A API documenta exatamente o que tem — nada a mais.

    Uma rota acrescentada sem intenção, ou uma que devia ter saído e ficou,
    muda este número.
    """
    caminhos = cliente.get("/openapi.json").json()["paths"]
    operacoes = sum(len(metodos) for metodos in caminhos.values())
    assert operacoes == 8, "7 endpoints da API + a rota raiz"
```

Entendendo o código

`(cliente, usuario)`	O teste pede as duas fixtures. O `usuario` traz junto a cadeia inteira — e a Ana Souza, cadastrada pela `sessao_aberta`, já existe quando o teste começa.
`== 200`	**OK**, e não 201: o login não cria nada, só abre uma sessão.
`corpo["usuario"]["id"] == usuario["id"]`	Quem entrou é a **mesma** pessoa que foi cadastrada pela fixture.
`== 401`	**Unauthorized**: a credencial não confere.
`inexistente.json()["detail"] == senha_errada.json()["detail"]`	A mensagem precisa ser **igual** nos dois casos. Se fosse diferente, um atacante descobriria quais e-mails têm conta.
`cliente.get("/usuarios/1") == 404`	Um teste de **ausência**: essa rota foi retirada de propósito, e não pode voltar.
`cliente.get("/usuarios/eu") == 401`	Sem token, a API não sabe quem está pedindo.
`"/openapi.json"`	A documentação automática da API, em JSON. O teste conta as operações: 7 endpoints + a raiz = 8. Uma rota a mais ou a menos muda o número.

Execute no terminal

```
pytest
pytest --setup-show tests/test_api.py::test_login_correto_devolve_200_com_token
```

O que deve aparecer (trechos)

```
FAILED tests/test_api.py::test_cadastro_nao_expoe_a_senha - assert 'senha123'...
================== 1 failed, 117 passed, 1 warning in 0.59s ===================
...
        SETUP    F banco_limpo
        SETUP    F cliente (fixtures used: banco_limpo)
        SETUP    F sessao_aberta (fixtures used: cliente)
        SETUP    F usuario (fixtures used: sessao_aberta)
        tests/test_api.py::test_login_correto_devolve_200_com_token (fixtures used:
        banco_limpo, cliente, sessao_aberta, usuario) .
        TEARDOWN F usuario
        TEARDOWN F sessao_aberta
        TEARDOWN F cliente
        TEARDOWN F banco_limpo
```

Confira no seu print

- **1 failed, 117 passed** — os 13 testes do arquivo, e só o da senha vermelho.
- O `--setup-show` mostra a cadeia do Passo 3 acontecendo: o `SETUP` de baixo para cima, o `TEARDOWN` na ordem inversa.

Feche o commit 21 — vermelho, de propósito

```
git add tests/test_api.py
git commit -m "Adiciona os testes dos endpoints publicos - FALHA"
```

Passo 8 — O diagnóstico, e o registro do defeito

Arquivo: leitura: a saída do pytest, a aplicação rodando, app/schemas.py e app/models.py · sem commit

Este passo **não tem código para digitar**: tem perguntas para guiar a investigação — e, no final, o registro do defeito.

1. O que a saída diz?

A saída resumida corta o texto da resposta. Para ver a resposta **inteira**, rode só o teste que falha, com `-v`:

Execute no terminal

```
pytest -v tests/test_api.py::test_cadastro_nao_expoe_a_senha
```

O que deve aparecer (trecho; no terminal o JSON vem numa linha só)

```
E         'senha123' is contained here:
E           {"accessToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","tokenType":"bearer",
E           "usuario":{"id":1,"nome":"Bruno Lima","email":"bruno@exemplo.com",
E           "senhaHash":"fake$senha123","criadoEm":"2026-11-09T13:58:56.168364Z"}}
...
FAILED tests/test_api.py::test_cadastro_nao_expoe_a_senha - assert 'senha123'...
======================== 1 failed, 1 warning in 0.11s ========================
```

Agora está claro: o objeto `usuario` da resposta tem um campo **`senhaHash`**, com o hash da senha dentro.

2. O teste está certo?

Volte à **RN-05** e ao **Dicionário de dados — usuarios**, na seção **Modelo de dados** do manual técnico: a coluna do hash é "**privado e nunca exposto pela API**". O teste cobra exatamente isso. O problema está na aplicação.

3. Veja com os próprios olhos, na aplicação de verdade

O teste usa o dublê. Será que o vazamento também acontece com o bcrypt de produção? Suba a aplicação:

Execute no terminal

```
uvicorn app.main:app --reload
```

Abra no navegador **http://127.0.0.1:8000/docs** — a documentação interativa da API. Clique em **POST /usuarios**, depois em **Try it out**, preencha o corpo com um e-mail que você ainda não cadastrou e clique em **Execute**:

O corpo da requisição

```
{
  "nome": "Ana",
  "email": "ana@exemplo.com",
  "senha": "senha123"
}
```

O que deve aparecer (o id, o token e o hash serão outros)

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tokenType": "bearer",
  "usuario": {
    "id": 1,
    "nome": "Ana",
    "email": "ana@exemplo.com",
    "senhaHash": "$2b$12$ENBl78Sq9jwYiW5tQjci0.RRjEf6Fjk9NpeeIMiVDSjK/.1zrOoI.",
    "criadoEm": "2026-11-09T13:58:59.069364Z"
  }
}
```

> Com o bcrypt, o campo traz `$2b$12$...` em vez de `fake$senha123`: a senha não aparece legível, e a **primeira** asserção do teste passaria. Quem pegaria o vazamento é a **segunda** — `"senhaHash" not in`. É por isso que as duas existem.
>
> E é por isso que o dublê ajudou: com ele, o vazamento ficou **legível** na saída do teste.
>
> Se aparecer `409`, o e-mail já existe no seu banco de trabalho: use outro. Pare o servidor com **Ctrl+C**.

4. Onde está o defeito?

O JSON da resposta é montado pelos **schemas de saída**. Abra `app/schemas.py` e encontre a classe que define o que é devolvido sobre um usuário. Leia a classe **inteira**, docstring incluída.

> **Pistas**
>
> **Perguntas para guiar a leitura:**
>
> • O que a docstring da classe **promete** sobre a senha e o hash?
>
> • Quais campos a classe **declara**, logo abaixo da docstring?
>
> • O schema lê os atributos do objeto `Usuario` (veja o `from_attributes=True` na classe base). Mas no modelo o hash está guardado em `_senha_hash`, com sublinhado. De onde vem, então, um valor para o campo do schema? Procure em `app/models.py`, na classe `Usuario`.
>
> • Uma docstring não é executada. Um `assert` é.

5. Antes de corrigir: registre

Este é o defeito da trilha que passa pelo **ciclo completo de gestão**, e não só pela correção. Registre-o no lugar que o professor indicar (quadro de tarefas, planilha ou *issue* do repositório), com estes campos:

Campo do registro	O que vai nele
Título	uma frase que diga o problema, não a solução
Severidade	crítica — exposição de credencial
Prioridade	a mais alta: está em toda resposta de cadastro e de login
Passos para reproduzir	o cadastro pela documentação da API, com o corpo usado
Resultado esperado	a resposta não contém a senha nem o hash dela (RN-05)
Resultado obtido	o campo `senhaHash`, com o hash completo
Evidência anexada	o print do `pytest -v` do item 1 e o da documentação da API

> A saída do `pytest -v` é **evidência**: quem recebe o registro roda o mesmo comando e vê o mesmo resultado. É a diferença entre **relatar** e **provar**.

Passo 9 — A correção

Arquivo: app/schemas.py e app/models.py (alterados) · Commit 22

A correção é sua. Faça a resposta da API cumprir o que a docstring do schema promete.

> **Três orientações:**
>
> • São **duas** alterações coordenadas, uma em cada arquivo: a correção desfaz as duas partes do defeito que você encontrou no diagnóstico.
>
> • O hash continua sendo **guardado** no banco: não mexa na coluna `_senha_hash`, nem em `definir_senha` ou `senha_confere`. O problema é **expor**, não armazenar.
>
> • Antes de apagar algo, confira se mais alguém usa: no VS Code, **Ctrl+Shift+F** procura um texto no projeto inteiro.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
====================== 118 passed, 1 warning in 0.46s ======================
```

Confira no seu print

- `118 passed` — o vermelho passou, e nenhum outro quebrou.

- Os testes do `test_seguranca.py` e do `test_dominio.py` continuam verdes: o hash continua sendo gerado e conferido.

> **Apareceram testes em `ERROR`, e não `FAILED`?** `FAILED` quer dizer que o **teste** falhou. `ERROR` quer dizer que algo deu errado **antes** dele — numa fixture. Se a mensagem fala em `ValidationError ... senhaHash ... Field required`, o cadastro da `sessao_aberta` quebrou: a correção foi feita só pela metade.

Feche o commit 22

```
git add app/schemas.py app/models.py
git commit -m "Remove o hash da senha das respostas da API"
```

A conversa que fecha o defeito

> **Nenhuma funcionalidade estava quebrada.** O sistema cadastrava, autenticava e respondia perfeitamente. Um roteiro de teste manual passaria inteiro — porque a tela nunca mostrou esse campo.
>
> **1. Severidade não é o mesmo que impacto funcional.** Este defeito é crítico e não quebra nada.
>
> **2. Teste manual de funcionalidade não encontra falha de segurança.** O defeito não tem sintoma visível. Encontrá-lo exige um teste que afirme uma **propriedade**: "nenhuma resposta contém a senha".

4. O contrato com o front-end

Passo 10 — O formato que a tela consome

Arquivo: tests/test_integracao_frontend.py (novo) · Commit 23

O `test_api.py` verifica se a API **faz** o que promete. Este arquivo verifica outra coisa: se ela **fala** exatamente o que o front-end sabe consumir — os nomes dos campos, o formato dos erros, o texto das mensagens. Um `rename` bem-intencionado de `criadoEm` para `criado_em` passaria por toda a suíte e quebraria a tela.

Código para digitar — tests/test_integracao_frontend.py, primeira parte

```
"""O contrato com o front-end, verificado como teste.

Os outros arquivos verificam se a API faz o que promete. Este verifica outra
coisa: se ela fala **exatamente** o que o front-end em React sabe consumir — o
formato do JSON, o vocabulário dos parâmetros, o texto das mensagens de erro.

Esse contrato não está no código de nenhum dos dois lados. Um `rename`
bem-intencionado de `criadoEm` para `criado_em` passaria por toda a suíte sem
falhar e quebraria a integração só na hora de ligar as duas pontas.

**O front-end ainda não consome esta API** — continua mockado no
`localStorage`. Estes testes travam a parte do contrato que já está pronta, e
cada um aponta para o arquivo do front-end que depende dele.
"""

from fastapi.testclient import TestClient



# ============================================================================
# Dia 7 — o contrato dos endpoints públicos
# ============================================================================

# ---------------------------------------------- campos — `@typedef Usuario`

def test_usuario_tem_exatamente_os_campos_publicos(usuario: dict):
    """O `@typedef Usuario` menos a senha.

    Este teste verifica a RN-05 por um caminho diferente do
    `test_cadastro_nao_expoe_a_senha`: ele não procura a palavra `senhaHash`,
    ele exige que o conjunto de campos seja *exatamente* este. Duas formas de
    verificar a mesma propriedade — e é bom que existam as duas.
    """
    assert set(usuario) == {"id", "nome", "email", "criadoEm"}



def test_sessao_tem_os_campos_que_o_authcontext_vai_consumir(sessao_aberta: dict):
    """O que `AuthContext.login` e `AuthContext.cadastrar` vão receber."""
    assert set(sessao_aberta) == {"accessToken", "tokenType", "usuario"}
    assert sessao_aberta["tokenType"] == "bearer"



def test_cadastro_e_login_continuam_publicos(cliente: TestClient):
    """Os dois únicos endpoints que não podem exigir token.

    Não há como pedir credencial de quem está criando a conta ou tentando
    entrar nela — é o `CadastroForm.jsx` e o `LoginForm.jsx` chamando a API
    antes de existir qualquer sessão.
    """
    cadastro = cliente.post(
        "/usuarios",
        json={"nome": "Carla", "email": "carla@exemplo.com", "senha": "senha123"},
    )
    assert cadastro.status_code == 201

    login = cliente.post(
        "/usuarios/login",
        json={"email": "carla@exemplo.com", "senha": "senha123"},
    )
    assert login.status_code == 200
```

Entendendo o código

`def test_...(usuario):`	O teste pede **só** a fixture `usuario`, sem o `cliente`: ele não faz requisição nenhuma, só examina o que a fixture trouxe.
`set(usuario) == {"id", "nome", "email", "criadoEm"}`	`set(dicionario)` é o conjunto das **chaves**. O `==` exige que sejam **exatamente** essas — nem uma a mais, nem uma a menos.
`set(sessao_aberta) == {"accessToken", ...}`	O mesmo para a resposta inteira do cadastro: o que a tela de login vai receber.
`cadastro = ...` `login = ...`	Cadastro e login sem token nenhum — os dois **precisam** ser públicos: quem está criando a conta ainda não tem sessão.

> O `test_usuario_tem_exatamente_os_campos_publicos` também pegaria o defeito de hoje — por outro caminho. Ele não procura a palavra `senhaHash`: ele exige que o conjunto de campos seja **exato**. Você vai ver isso acontecer no experimento do final do passo.

Agora a segunda parte, no final do mesmo arquivo — o formato dos erros e as mensagens:

Código para digitar — segunda parte, no final do arquivo test_integracao_frontend.py

```
# ------------------------------ formato do erro — `setErro(e.message)` no front

def test_erro_de_validacao_tem_detail_em_texto(cliente: TestClient):
    """`detail` é sempre uma frase, nunca uma lista de objetos.

    Com o formato padrão do FastAPI — `detail` como lista —, a tela exibiria
    `[object Object]`. O `isinstance` trava o **tipo** do campo.
    """
    resposta = cliente.post(
        "/usuarios", json={"nome": "A", "email": "nao-e-email", "senha": "123"}
    )
    assert resposta.status_code == 422
    corpo = resposta.json()
    assert isinstance(corpo["detail"], str)
    assert corpo["detail"].strip()


def test_erro_de_validacao_preserva_os_detalhes_em_erros(cliente: TestClient):
    """A lista original continua disponível, só não ocupa mais o `detail`."""
    resposta = cliente.post("/usuarios", json={"email": "nao-e-email"})
    corpo = resposta.json()
    assert isinstance(corpo["erros"], list)
    assert corpo["erros"]
    assert all("loc" in erro for erro in corpo["erros"])


def test_erro_de_negocio_tem_detail_em_texto(cliente: TestClient, usuario: dict):
    """Erro de domínio e erro de validação têm a mesma forma de `detail`."""
    resposta = cliente.post(
        "/usuarios",
        json={"nome": "Outra", "email": usuario["email"], "senha": "senha123"},
    )
    assert resposta.status_code == 409
    assert resposta.json()["detail"] == "E-mail já cadastrado."
```

```python
def test_credencial_invalida_responde_401(cliente: TestClient, usuario: dict):
    """O front traduz este 401 no `null` que o `verificarCredenciais` devolve."""
    resposta = cliente.post(
        "/usuarios/login", json={"email": usuario["email"], "senha": "errada"}
    )
    assert resposta.status_code == 401


# ------------------------------------ mensagens em pt-BR, prontas para a tela

def test_campo_obrigatorio_em_portugues(cliente: TestClient):
    resposta = cliente.post("/usuarios", json={})
    assert resposta.status_code == 422
    assert "campo obrigatório" in resposta.json()["detail"]


def test_tamanho_minimo_cita_o_numero(cliente: TestClient):
    """A mensagem diz quantos caracteres faltam, e dos **dois** campos."""
    resposta = cliente.post(
        "/usuarios", json={"nome": "A", "email": "ana@exemplo.com", "senha": "123"}
    )
    detalhe = resposta.json()["detail"]
    assert "nome: deve ter no mínimo 2 caracteres." in detalhe
    assert "senha: deve ter no mínimo 6 caracteres." in detalhe


def test_email_invalido_em_portugues(cliente: TestClient):
    resposta = cliente.post(
        "/usuarios", json={"nome": "Ana", "email": "nao-e-email", "senha": "senha123"}
    )
    assert "e-mail inválido" in resposta.json()["detail"]
```

Entendendo o código

`isinstance(corpo["detail"], str)`	Trava o **tipo** do campo: `detail` tem de ser texto. Se fosse uma lista, a tela mostraria `[object Object]`.
`corpo["detail"].strip()`	E não pode ser um texto vazio: `strip()` tira os espaços, e o `assert` exige que sobre alguma coisa.
`isinstance(corpo["erros"], list)`	Os detalhes técnicos continuam disponíveis num campo à parte, `erros`, para quem precisar.
`all("loc" in erro for erro in ...)`	`all(...)` é verdadeiro se **todos** os itens cumprirem a condição: todo erro diz **onde** (`loc`) aconteceu.
`json={"nome": ..., "email": usuario["email"]}`	Um e-mail que já existe, tirado da fixture: o erro de **negócio** (409) tem o mesmo formato do erro de validação.
`"campo obrigatório" in ...["detail"]`	As mensagens chegam em **português**, prontas para a tela — não em inglês, como o Pydantic devolveria.
`"nome: deve ter no mínimo 2 caracteres."`	Dois erros no mesmo corpo, e a mensagem cita **os dois**, com o número de caracteres.

Execute no terminal

```
pytest -v tests/test_integracao_frontend.py
pytest
```

O que deve aparecer (trechos)

```
tests/test_integracao_frontend.py::test_usuario_tem_exatamente_os_campos_publicos PASSED [ 10%]
tests/test_integracao_frontend.py::test_sessao_tem_os_campos_que_o_authcontext_vai_consumir PASSED
[ 20%]
tests/test_integracao_frontend.py::test_cadastro_e_login_continuam_publicos PASSED [ 30%]
tests/test_integracao_frontend.py::test_erro_de_validacao_tem_detail_em_texto PASSED [ 40%]
tests/test_integracao_frontend.py::test_erro_de_validacao_preserva_os_detalhes_em_erros PASSED [
50%]
tests/test_integracao_frontend.py::test_erro_de_negocio_tem_detail_em_texto PASSED [ 60%]
tests/test_integracao_frontend.py::test_credencial_invalida_responde_401 PASSED [ 70%]
tests/test_integracao_frontend.py::test_campo_obrigatorio_em_portugues PASSED [ 80%]
tests/test_integracao_frontend.py::test_tamanho_minimo_cita_o_numero PASSED [ 90%]
tests/test_integracao_frontend.py::test_email_invalido_em_portugues PASSED [100%]

======================== 10 passed, 1 warning in 0.11s ========================
...
====================== 128 passed, 1 warning in 0.60s ========================
```

Confira no seu print

- `10 passed` no arquivo novo e **128 passed** na suíte inteira.

Feche o commit 23

```
git add tests/test_integracao_frontend.py
git commit -m "Adiciona o contrato dos endpoints publicos"
```

Experimento — duas formas de pegar o mesmo defeito

Com os dois arquivos prontos, traga de volta o `app/` de **antes da correção** — o do commit 21 — e veja quem percebe:

Execute no terminal

```
git stash                          # só se houver algo não commitado
git checkout HEAD~2 -- app/        # o app/ do commit 21, com o defeito
pytest                             # 2 failed, 126 passed
git checkout HEAD -- app/          # volta ao app/ corrigido
pytest                             # 128 passed
```

O que deve aparecer no primeiro pytest (trecho)

```
>       assert set(usuario) == {"id", "nome", "email", "criadoEm"}
E       AssertionError: assert {'criadoEm', ..., 'senhaHash'} == {'criadoEm', ... 'id', 'nome'}
E
E         Extra items in the left set:
E         'senhaHash'
...
FAILED tests/test_api.py::test_cadastro_nao_expoe_a_senha - assert 'senha123'...
FAILED tests/test_integracao_frontend.py::test_usuario_tem_exatamente_os_campos_publicos
================== 2 failed, 126 passed, 1 warning in 0.67s ==================
```

Dois testes, dois caminhos, o mesmo defeito. O primeiro procura uma **palavra** proibida no texto da resposta. O segundo exige o **conjunto exato** de campos e aponta o intruso: *Extra items in the left set: 'senhaHash'*.

O segundo pegaria também um vazamento com **outro nome** — `hashDaSenha`, por exemplo — que o primeiro deixaria passar. É bom que existam os dois.

Confirme que o segundo `pytest` deu **128 passed**: o `app/` voltou ao corrigido.

5. Fechamento do dia

O que o HTTP trouxe

Recurso	Para quê	Onde você usou
`TestClient(app)`	fazer requisições sem subir servidor	fixture `cliente`
`cliente.get` / `cliente.post`	chamar um endpoint	todos os testes de API
`resposta.status_code`	conferir o código HTTP	201, 200, 401, 404, 409, 422
`resposta.json()`	ler o corpo como dicionário	token, campos, `detail`
`resposta.text`	procurar em **qualquer** campo	o teste da senha
o dublê `HashFake`	trocar o bcrypt lento por um substituto	fixture `cliente`
`dependency_overrides`	entregar o dublê à aplicação	fixture `cliente`
fixture que pede fixture	montar uma cadeia de preparação	`sessao_aberta`, `usuario`
`filterwarnings`	silenciar aviso investigado, com justificativa	`pytest.ini`

Confira o que o dia entregou

Verificação	Esperado
`pytest`	128 passed
A saída está limpa de avisos de biblioteca	no máximo 1, investigado
O `filterwarnings` tem justificativa escrita	um comentário por linha de `ignore`
`test_seguranca.py` continua com o bcrypt de verdade	o `HashFake` só entra pelo `cliente`
O defeito foi registrado antes de corrigido	registro com evidência anexada
A única alteração em `app/` é a correção	`git diff HEAD~4 --stat` lista só `app/models.py` e `app/schemas.py` dentro de `app/`

O histórico do dia

Execute no terminal

```
git log --oneline -4
```

O que deve aparecer (os códigos dos commits serão outros)

```
23  Adiciona o contrato dos endpoints publicos
22  Remove o hash da senha das respostas da API
21  Adiciona os testes dos endpoints publicos - FALHA
20  Adiciona o cliente HTTP e o duble do hash
```

Se algo der errado

Sintoma	Causa provável	O que fazer
`ModuleNotFoundError: No module named 'httpx'`	O `TestClient` depende do `httpx`	`pip install -r requirements.txt`
Um teste passa sozinho e falha na suíte	O `dependency_overrides` não foi limpo	O `clear()` vai no `finally`
`no such table` em teste de API	O `cliente` foi criado antes do banco	A fixture `cliente` pede `banco_limpo` no parâmetro
`fixture 'usuario' not found`	A fixture foi escrita no arquivo de teste, ou o nome não bate	As fixtures de hoje moram no `conftest.py`
Vários testes em `ERROR` com `ValidationError ... senhaHash Field required`	A correção foi feita só no modelo: o schema ainda declara o campo	São duas alterações coordenadas
Dezenas de avisos na saída	O `filterwarnings` não foi configurado	Investigue a origem **antes** de silenciar

Amanhã

> **Dia 8 — a sessão aberta.** Hoje a API foi testada só pela porta da frente: cadastro e login. Amanhã entra uma fixture que **modifica** outra — o `cliente_logado`, que já envia o token em toda requisição — e com ela os endpoints de **tarefas**. Nenhum defeito: um dia para consolidar o HTTP.

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 8 — A sessão aberta: fixtures encadeadas

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	raises	parametrize	fixture	banco	monkeypatch	HTTP	**sessão**	duas contas

> **Hoje:** uma fixture que **modifica** outra — o `cliente_logado`, que já envia o token em toda requisição — e, com ela, os endpoints de **tarefas**.
>
> **Ao final:** `pytest` responde **162 passed** (128 de antes + 34 de hoje).
>
> **Defeito a encontrar hoje:** nenhum — é o dia com mais testes novos, e nenhuma ferramenta difícil.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

De onde paramos

Ontem a API foi testada só pela **porta da frente**: cadastro, login e o formato das respostas. Os endpoints de **tarefas** — o motivo de a aplicação existir — ficaram de fora, porque todos exigem uma sessão aberta. Um teste que chame `GET /tasks` com o `cliente` de ontem recebe `401` e para por aí.

Como funciona a sessão

O cadastro e o login devolvem um **token de acesso** (`accessToken`). Daí em diante, o front-end envia esse token em **toda** requisição, num cabeçalho HTTP chamado `Authorization`:

```
POST /usuarios  ------------>  {"accessToken": "eyJhbGci...", "usuario": {...}}
                                          |
                                          |  o front-end guarda o token
                                          v
GET /tasks
Authorization: Bearer eyJhbGci...  -->  a API confere a assinatura do token,
                                        busca o usuário no banco, e responde
                                        só com as tarefas DELE
```

A palavra `Bearer` ("portador") é o **esquema**: quem porta este token é quem ele identifica. Você testou o token **por dentro** no Dia 4, chamando o `TokenDeAcesso` diretamente. Hoje ele é testado **pela API**, do jeito que o front-end vai usá-lo.

Os endpoints de tarefas

Estão em `app/routers/tasks.py`. Abra o arquivo e confira: a autenticação é declarada uma vez só, para o grupo inteiro.

Endpoint	O que faz	Sucesso
`POST /tasks`	cria uma tarefa — o dono vem do **token**, não do corpo	`201`
`GET /tasks`	lista as **próprias** tarefas, com filtros (`status`, `prazoAte`...) e ordenação (`ordenarPor`, `ordem`)	`200`
`PATCH /tasks/{id}`	altera **parte** de uma tarefa: só os campos enviados	`200`
`DELETE /tasks/{id}`	remove uma tarefa	`204`
`GET /usuarios/eu`	devolve a própria conta (ontem você testou que ele recusa sem token)	`200`

Dois códigos de status entram hoje na conversa:

Código	Nome	Quando a API devolve
`204`	No Content	deu certo e **não há nada para devolver** (remoção): o corpo vem vazio
`401`	Unauthorized	"**não sei quem você é**": sem token, token inválido, conta que não existe mais
`403`	Forbidden	"**sei quem você é, e você não pode**" — aparece no Dia 9

A ferramenta nova: uma fixture que modifica outra

Todos os testes de tarefa precisam do cabeçalho `Authorization`. Repetir essa linha em cada requisição de cada teste seria ruído — e esconderia o que o teste realmente verifica.

A saída é uma fixture que **recebe** o `cliente` de ontem, **acrescenta** o cabeçalho e o devolve: o `cliente_logado`. Um teste que o pede já começa com a sessão aberta, sem uma linha de preparação.

> E vem junto uma pergunta que vale a pena responder com evidência: se dois parâmetros do teste dependem da mesma fixture, ela roda **duas vezes**? Você vai ver a resposta no terminal, no Passo 2.

O que vamos testar hoje

Hoje nenhum arquivo nasce: **quatro** arquivos de ontem ganham uma seção "Dia 8" no final, e o `conftest.py` ganha duas fixtures.

Arquivo	Testes	O que verifica
`tests/test_api.py`	17	as tarefas: criar, listar, alterar, remover
`tests/test_autenticacao.py`	7	o cabeçalho `Authorization`, pela API
`tests/test_integracao_frontend.py`	4	o contrato da autenticação: o 401 e o CORS
`tests/test_configuracao.py`	6	a validade do token, que vem do ambiente

O que você vai aprender hoje

Ferramenta nova	fixture que **modifica** outra (`cliente_logado`); o **cache** de fixture: uma instância por teste
Técnica nova	montar o estado do teste **pela própria API**; datas **relativas** a hoje
Arquivos que crescem	`conftest.py`, `test_api.py`, `test_autenticacao.py`, `test_integracao_frontend.py` e `test_configuracao.py`
Ainda não sabe	agir como **duas** pessoas ao mesmo tempo — é o Dia 9
Testes ao final	162 (128 de antes + 34 de hoje)
Commits	24, 25, 26 e 27 — nenhum deles toca em `app/`

> **Um dia sem defeito**
>
> Hoje todo teste novo deve passar **na primeira execução**. Isso não quer dizer que eles não servem para nada: eles **travam** o comportamento correto. Se alguém quebrar a listagem ou a remoção daqui a três meses, são eles que avisam.
>
> Se um teste de hoje ficar vermelho, a primeira suspeita é o **seu** teste — confira o código digitado e a seção **Se algo der errado**, no final do roteiro.

2. O cliente logado

Passo 1 — O token e o cliente_logado

Arquivo: tests/conftest.py (acrescentar ao final) · ainda sem commit

Acrescente ao **final** do `conftest.py`, depois da fixture `usuario`:

Código para digitar

```
# ================================================================================
# Dia 8 — a sessão aberta
# ================================================================================


@pytest.fixture
def token(sessao_aberta: dict) -> str:
    """O token de acesso do usuário da fixture `usuario`.

    `usuario` e `token` pedem a **mesma** `sessao_aberta`. O pytest cria cada
    fixture uma única vez por teste, então os dois vêm do mesmo cadastro — o
    token identifica exatamente aquele usuário, e não um segundo parecido.
    """
    return sessao_aberta["accessToken"]


@pytest.fixture
def cliente_logado(cliente: TestClient, token: str) -> TestClient:
    """Cliente HTTP que envia `Authorization: Bearer <token>` em toda chamada.

    O cabeçalho entra nos *headers* padrão do cliente em vez de ser repetido em
    cada requisição: o que cada teste verifica é o comportamento do endpoint,
    não a montagem do cabeçalho — essa é verificada à parte, em
    `test_autenticacao.py`.
    """
    cliente.headers["Authorization"] = f"Bearer {token}"
    return cliente
```

Entendendo o código

`def token(sessao_aberta):`	Pede a `sessao_aberta` de ontem — o cadastro da Ana Souza — e devolve só o token da resposta.
`sessao_aberta["accessToken"]`	O mesmo campo que você verificou ontem no `test_cadastrar_usuario_devolve_201_com_sessao_aberta`.
`def cliente_logado(` `  cliente, token):`	Pede **duas** fixtures: o cliente HTTP e o token. O pytest monta as duas antes.
`cliente.headers` `  ["Authorization"] = ...`	`headers` são os cabeçalhos **padrão** do cliente: tudo o que estiver ali vai junto em **toda** requisição que ele fizer.
`f"Bearer {token}"`	O formato exato que a API espera: a palavra `Bearer`, um espaço, e o token.
`return cliente`	Devolve o **mesmo** objeto que recebeu — agora com o cabeçalho. Não é um cliente novo: é o `cliente`, modificado.

> Repare na última linha da explicação: o `cliente_logado` **é** o `cliente`. Um teste que pedir os dois terá dois nomes para o mesmo objeto — e os dois enviarão o token. Por isso, nos testes em que a **ausência** do token é o cenário, peça só o `cliente`.

Execute no terminal

```
pytest --fixtures
```

O que deve aparecer (trecho)

```
-------------------- fixtures defined from tests.conftest ---------------------
cliente -- tests\conftest.py:132
    Cliente HTTP **sem** autenticação, com o hash substituído pelo dublê.

sessao_aberta -- tests\conftest.py:156
    Cadastra um usuário e devolve a resposta do cadastro: token e usuário.

usuario -- tests\conftest.py:174
    Um usuário já cadastrado: o dicionário `usuario` da resposta.

token -- tests\conftest.py:189
    O token de acesso do usuário da fixture `usuario`.

cliente_logado -- tests\conftest.py:200
    Cliente HTTP que envia `Authorization: Bearer <token>` em toda chamada.
```

Confira no seu print

- As duas fixtures novas aparecem, cada uma com a primeira linha da docstring.
- Rode `pytest`: continua **128 passed** — nenhum teste pede o `cliente_logado` ainda.

A cadeia de fixtures, agora com dois ramos:

```
   banco_limpo --> cliente --> sessao_aberta --+--> usuario
                    |                          |
                    |                          +--> token --+
                    |                                       |
                    +---------------------------------------+--> cliente_logado
```

3. As tarefas, pela API

Passo 2 — A própria conta e a criação

Arquivo: tests/test_api.py (alterar o topo e acrescentar ao final) · ainda sem commit

Os testes de hoje usam datas. No topo do `test_api.py`, **acima** do import do `TestClient`, acrescente:

Código para digitar — o bloco de imports do test_api.py

```
from datetime import date, timedelta

from fastapi.testclient import TestClient
```

E no **final** do arquivo, a seção do Dia 8 com os quatro primeiros testes:

Código para digitar

```
# ============================================================================
# Dia 8 — `cliente_logado`: as tarefas, com a sessão aberta
# ============================================================================
#
# Todos os endpoints de /tasks exigem token, e o dono da tarefa vem do token.
# Por isso estes testes usam `cliente_logado` — um cliente que já envia o
# cabeçalho `Authorization` — e os corpos não trazem `usuarioId`.

def test_consultar_a_propria_conta_devolve_o_usuario_do_token(
    cliente_logado: TestClient, usuario: dict
):
    """A comparação é com o dicionário **inteiro** (`== usuario`): um campo a
    mais ou a menos quebra o teste."""
    resposta = cliente_logado.get("/usuarios/eu")
    assert resposta.status_code == 200
    assert resposta.json() == usuario

def test_criar_tarefa_nasce_pendente(cliente_logado: TestClient, usuario: dict):
    """A RN-02 do Dia 1, agora verificada do outro lado da aplicação.

    Entre o objeto `Tarefa` e esta resposta há uma rota, um schema de entrada,
    um serviço, um repositório e uma gravação no banco.
    """
    resposta = cliente_logado.post(
        "/tasks",
        json={
            "titulo": "Entregar o relatório",
            "prioridade": "alta",
            "prazo": str(date.today() + timedelta(days=30)),
        },
    )
    assert resposta.status_code == 201
    corpo = resposta.json()
    assert corpo["status"] == "pendente"
    assert corpo["concluidoEm"] is None
    assert corpo["usuarioId"] == usuario["id"], "o dono vem do token"

def test_criar_tarefa_sem_token_devolve_401(cliente: TestClient):
    """Usa `cliente`, e não `cliente_logado`: a ausência do token é o cenário."""
    assert cliente.post("/tasks", json={"titulo": "X"}).status_code == 401


def test_criar_tarefa_com_prioridade_invalida_devolve_422(
    cliente_logado: TestClient,
):
    resposta = cliente_logado.post(
        "/tasks", json={"titulo": "X", "prioridade": "altissima"}
    )
    assert resposta.status_code == 422
```

Entendendo o código

`cliente_logado.get( "/usuarios/eu")`	A primeira requisição **autenticada** da trilha. Nenhuma linha monta o cabeçalho: o `cliente_logado` já o envia.
`resposta.json() == usuario`	Compara o dicionário **inteiro**, e não campo por campo. Um campo a mais ou a menos na resposta quebra o teste.
`str(date.today() + timedelta(days=30))`	Um prazo daqui a 30 dias, **contado a partir de hoje**, convertido para texto (`"2026-12-09"`, por exemplo) porque o JSON não tem tipo data.
`corpo["status"] == "pendente"`	A **RN-02** do Dia 1 — toda tarefa nasce pendente — agora verificada pela API.
`corpo["usuarioId"] == usuario["id"]`	O corpo da requisição **não** tem `usuarioId`. O dono vem do token, e o teste confere que é o usuário da fixture.
`(cliente: TestClient)`	O teste do 401 pede o `cliente` **sem** token. Aqui a ausência do token é justamente o cenário.
`"prioridade": "altissima"`	Um valor fora da lista (`baixa`, `media`, `alta`, `urgente`): o Pydantic recusa com 422.

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
tests\test_api.py .................                                    [ 12%]
tests\test_autenticacao.py .............                               [ 22%]
...
======================== 132 passed, 1 warning in 0.66s ========================
```

Confira no seu print

- **132 passed** — 17 pontos na linha do `test_api.py`: os 13 de ontem e os 4 de hoje.

A pergunta: foram feitos dois cadastros?

O `test_criar_tarefa_nasce_pendente` pede `cliente_logado` **e** `usuario`. Os dois dependem da `sessao_aberta`, que faz um cadastro. Se ela rodasse duas vezes, o segundo cadastro da Ana Souza daria `409` — e o `usuarioId` da tarefa nem seria o do `usuario`. Pergunte ao pytest:

Execute no terminal

```
pytest --setup-show tests/test_api.py::test_criar_tarefa_nasce_pendente
```

O que deve aparecer (trecho)

```
        SETUP    F banco_limpo
        SETUP    F cliente (fixtures used: banco_limpo)
        SETUP    F sessao_aberta (fixtures used: cliente)
        SETUP    F token (fixtures used: sessao_aberta)
        SETUP    F cliente_logado (fixtures used: cliente, token)
        SETUP    F usuario (fixtures used: sessao_aberta)
        tests/test_api.py::test_criar_tarefa_nasce_pendente (fixtures used:
        banco_limpo, cliente, cliente_logado, sessao_aberta, token, usuario) .
        TEARDOWN F usuario
        TEARDOWN F cliente_logado
        TEARDOWN F token
        TEARDOWN F sessao_aberta
        TEARDOWN F cliente
        TEARDOWN F banco_limpo
```

Lendo a saída

`SETUP F sessao_aberta`	Aparece **uma vez só**, embora três fixtures dependam dela: `token`, `usuario` e, por meio do `token`, o `cliente_logado`.
`SETUP F cliente`	Também uma vez só. O `cliente_logado` recebe **esse** cliente, e não um novo.
`F`	O escopo da fixture: **F** de *function*, uma instância por **teste**. No teste seguinte, tudo é montado de novo, do zero.

> **O pytest cria cada fixture uma única vez por teste**, e entrega a mesma instância a todos que a pedirem. É esse cache que garante que `usuario` e `token` vêm do **mesmo** cadastro — e por isso o `assert corpo["usuarioId"] == usuario["id"]` funciona.

Experimento — o teste que esquece o token

No primeiro teste de hoje, o `test_consultar_a_propria_conta_devolve_o_usuario_do_token`, troque as **duas** ocorrências de `cliente_logado` por `cliente` — no parâmetro e na requisição — e rode só esse teste:

Execute no terminal

```
pytest --tb=short
tests/test_api.py::test_consultar_a_propria_conta_devolve_o_usuario_do_token
```

O que deve aparecer (trecho)

```
tests\test_api.py:182: in test_consultar_a_propria_conta_devolve_o_usuario_do_token
    assert resposta.status_code == 200
E   assert 401 == 200
E    +  where 401 = <Response [401 Unauthorized]>.status_code
```

Confira no seu print

- A API respondeu **401**: sem o cabeçalho, ela não sabe quem está pedindo.
- Guarde o sintoma — **401 em teste de tarefa** quase sempre é um `cliente` no lugar do `cliente_logado`.
- **Desfaça a troca** e rode `pytest`: volta a **132 passed**.

Passo 3 — A listagem

Arquivo: tests/test_api.py (acrescentar ao final) · ainda sem commit

Código para digitar

```
def test_listar_tarefas_de_usuario_sem_tarefas_devolve_lista_vazia(
    cliente_logado: TestClient,
):
    """Lista vazia é 200, não 404: a coleção existe, só está vazia."""
    resposta = cliente_logado.get("/tasks")
    assert resposta.status_code == 200
    assert resposta.json() == []


def test_listar_filtra_por_status(cliente_logado: TestClient):
    """O estado do teste é montado pela própria API, não pelo banco.

    Para ter uma tarefa concluída, o teste cria duas pela API e conclui uma
    pela API. Nenhuma linha de SQL, nenhuma sessão.
    """
    for titulo in ("A", "B"):
        cliente_logado.post("/tasks", json={"titulo": titulo})
    primeira = cliente_logado.get("/tasks").json()[0]
    cliente_logado.patch(f"/tasks/{primeira['id']}", json={"status": "concluida"})

    pendentes = cliente_logado.get("/tasks?status=pendente").json()
    assert len(pendentes) == 1
    assert pendentes[0]["status"] == "pendente"


def test_listar_ordena_por_prioridade_da_mais_urgente_para_a_menos(
    cliente_logado: TestClient,
):
    """A asserção compara a **lista inteira**, na ordem."""
    for prioridade in ("baixa", "urgente", "media"):
        cliente_logado.post(
            "/tasks", json={"titulo": prioridade, "prioridade": prioridade}
        )
    resposta = cliente_logado.get("/tasks?ordenarPor=prioridade&ordem=asc")
    assert [t["prioridade"] for t in resposta.json()] == [
        "urgente",
        "media",
        "baixa",
    ]

def test_campo_de_ordenacao_invalido_devolve_422(cliente_logado: TestClient):
    """Recusar é melhor que adivinhar: o erro aparece em quem o cometeu."""
    resposta = cliente_logado.get("/tasks?ordenarPor=inventado")
    assert resposta.status_code == 422

def test_filtrar_por_prazo_ate(cliente_logado: TestClient):
    """Datas relativas a hoje, e não fixas no calendário.

    Um teste que quebra pela passagem do tempo é pior do que não ter o teste:
    falha sem que ninguém tenha mexido em nada.
    """
    hoje = date.today()
    cliente_logado.post(
        "/tasks",
        json={"titulo": "Vence logo", "prazo": str(hoje + timedelta(days=1))},
    )
```

```
    cliente_logado.post(
        "/tasks",
        json={"titulo": "Vence tarde", "prazo": str(hoje + timedelta(days=30))},
    )
    resposta = cliente_logado.get(
        f"/tasks?prazoAte={hoje + timedelta(days=7)}"
    )
    assert [t["titulo"] for t in resposta.json()] == ["Vence logo"]
```

Entendendo o código

`resposta.json() == []`	Sem tarefas, a resposta é uma **lista vazia** com 200 — e não 404. A coleção existe; só não tem nada dentro.
`for titulo in ("A", "B"): cliente_logado.post(...)`	Monta o cenário **pela própria API**: duas tarefas criadas como o front-end criaria.
`.json()[0]`	Pega a primeira tarefa da lista — qualquer uma das duas serve — para concluí-la em seguida.
`f"/tasks/{primeira['id']}"`	O endereço da tarefa, montado com o `id` que a própria API devolveu.
`"/tasks?status=pendente"`	Um **parâmetro de consulta** (*query string*): o que vem depois do `?` filtra a lista.
`[t["prioridade"] for t in resposta.json()]`	Extrai só as prioridades, **na ordem** em que vieram, e compara com a lista esperada inteira.
`["urgente", "media", "baixa"]`	Criadas fora de ordem (`baixa`, `urgente`, `media`), precisam voltar da mais urgente para a menos. Em ordem **alfabética** viria `baixa` primeiro.
`"ordenarPor=inventado"`	Um campo que não existe: a API **recusa** com 422 em vez de ignorar o parâmetro em silêncio.
`hoje + timedelta(days=1)`	Datas **relativas**: amanhã, daqui a 30 dias, daqui a 7. O teste dá o mesmo resultado em qualquer dia em que for rodado.

> **Por que não escrever `"2026-12-20"` direto?** Porque essa data vai passar. No dia seguinte a ela, a tarefa que "vence logo" estaria vencida, o filtro daria outro resultado, e o teste ficaria vermelho **sem que ninguém tenha mexido em nada**.
>
> Um teste que quebra pela passagem do tempo ensina a equipe a ignorar testes vermelhos — o pior resultado possível.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
====================== 137 passed, 1 warning in 0.80s ======================
```

Confira no seu print

- **137 passed** — cinco testes novos.

Passo 4 — A alteração

Arquivo: tests/test_api.py (acrescentar ao final) · ainda sem commit

O PATCH altera **parte** de uma tarefa. Os seis testes a seguir verificam o que muda, o que **não** muda e o que é recusado:

Código para digitar

```
def test_concluir_tarefa_preenche_a_data(cliente_logado: TestClient):
    criada = cliente_logado.post("/tasks", json={"titulo": "Concluir"}).json()
    resposta = cliente_logado.patch(
        f"/tasks/{criada['id']}", json={"status": "concluida"}
    )
    assert resposta.status_code == 200
    assert resposta.json()["concluidoEm"] is not None


def test_reabrir_tarefa_limpa_a_data(cliente_logado: TestClient):
    criada = cliente_logado.post("/tasks", json={"titulo": "Reabrir"}).json()
    cliente_logado.patch(f"/tasks/{criada['id']}", json={"status": "concluida"})
    resposta = cliente_logado.patch(
        f"/tasks/{criada['id']}", json={"status": "pendente"}
    )
    assert resposta.json()["concluidoEm"] is None


def test_editar_titulo_de_tarefa_concluida_preserva_a_data(
    cliente_logado: TestClient,
):
    """A RN-03 verificada de ponta a ponta, atravessando todas as camadas.

    É a mesma regra do Dia 2, vista pelo outro lado. Se ela quebrar, compare
    as duas mensagens de falha: a do domínio aponta o método; esta diz apenas
    que duas datas vindas de duas respostas HTTP não são iguais.
    """
    criada = cliente_logado.post("/tasks", json={"titulo": "Original"}).json()
    concluida = cliente_logado.patch(
        f"/tasks/{criada['id']}", json={"status": "concluida"}
    ).json()
    editada = cliente_logado.patch(
        f"/tasks/{criada['id']}", json={"titulo": "Título novo"}
    ).json()
    assert editada["titulo"] == "Título novo"
    assert editada["concluidoEm"] == concluida["concluidoEm"]


def test_apagar_o_prazo_e_diferente_de_nao_informar(cliente_logado: TestClient):
    """No JSON, "não mencionei o prazo" e "quero o prazo vazio" são coisas
    diferentes: a primeira é a ausência da chave, a segunda é `null`."""
    prazo = str(date.today() + timedelta(days=10))
    criada = cliente_logado.post(
        "/tasks", json={"titulo": "Com prazo", "prazo": prazo}
    ).json()
    sem_mencionar = cliente_logado.patch(
        f"/tasks/{criada['id']}", json={"titulo": "Outro título"}
    ).json()
    assert sem_mencionar["prazo"] == prazo
    apagando = cliente_logado.patch(
        f"/tasks/{criada['id']}", json={"prazo": None}
    ).json()
    assert apagando["prazo"] is None
```

```
def test_atualizar_tarefa_inexistente_devolve_404(cliente_logado: TestClient):
    resposta = cliente_logado.patch("/tasks/9999", json={"status": "concluida"})
    assert resposta.status_code == 404


def test_titulo_vazio_no_patch_devolve_422(cliente_logado: TestClient):
    criada = cliente_logado.post("/tasks", json={"titulo": "Original"}).json()
    resposta = cliente_logado.patch(f"/tasks/{criada['id']}", json={"titulo": "   "})
    assert resposta.status_code == 422
```

Entendendo o código

`criada = cliente_logado .post(...).json()`	Cria a tarefa e já guarda o corpo da resposta: é dele que sai o `id` usado no `PATCH`.
`json={"status": "concluida"}`	O corpo do `PATCH` leva **só** o que muda. Os outros campos ficam como estão.
`["concluidoEm"] is not None`	Concluir **preenche** a data de conclusão; reabrir (`"pendente"`) a **limpa**.
`editada["concluidoEm"] == concluida["concluidoEm"]`	A **RN-03** do Dia 2, de ponta a ponta: editar o título de uma tarefa concluída não pode mexer na data de conclusão.
`json={"titulo": "Outro título"}`	Um `PATCH` que **não menciona** o prazo: o prazo tem de continuar igual.
`json={"prazo": None}`	O `None` do Python vira `null` no JSON: "quero o prazo **vazio**". Aí sim o prazo é apagado.
`"/tasks/9999"`	Uma tarefa que não existe: 404.
`{"titulo": "   "}`	Um título só de espaços também é vazio — a mesma regra que você testou no domínio, agora pelo `PATCH`.

> Compare com o Dia 2: lá, a RN-03 foi testada chamando `tarefa.atualizar(...)` diretamente. Aqui, a mesma regra atravessa rota, schema, serviço, repositório e banco. Se ela quebrar, os **dois** testes ficam vermelhos — o de domínio diz **qual método**; este diz só que duas datas não batem. É por isso que existem os dois níveis.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
======================== 143 passed, 1 warning in 0.91s ========================
```

Confira no seu print

- **143 passed** — seis testes novos.

Passo 5 — A remoção

Arquivo: tests/test_api.py (acrescentar ao final) · Commit 24

Código para digitar

```
def test_remover_tarefa_devolve_204_sem_corpo(cliente_logado: TestClient):
    """204 No Content: o corpo precisa estar **vazio**, não só ausente."""
    criada = cliente_logado.post("/tasks", json={"titulo": "Remover"}).json()
    resposta = cliente_logado.delete(f"/tasks/{criada['id']}")
    assert resposta.status_code == 204
    assert resposta.content == b""


def test_remover_a_mesma_tarefa_duas_vezes_devolve_404(cliente_logado: TestClient):
    """A segunda remoção não é sucesso silencioso: a tarefa não existe mais."""
    criada = cliente_logado.post("/tasks", json={"titulo": "Remover"}).json()
    cliente_logado.delete(f"/tasks/{criada['id']}")
    assert cliente_logado.delete(f"/tasks/{criada['id']}").status_code == 404
```

Entendendo o código

`cliente_logado.delete(...)`	O quarto verbo HTTP da trilha: depois de `get`, `post` e `patch`, o `delete`.
`== 204`	**No Content**: deu certo, e não há nada a devolver.
`resposta.content == b""`	`content` é o corpo em **bytes**, e `b""` são zero bytes. O teste exige o corpo **vazio** — e não só o código 204.
`o segundo delete == 404`	Remover o que já foi removido não é sucesso silencioso: a tarefa não existe mais.

> Por que verificar o corpo, se o código já é 204? Porque o código e o corpo são promessas **diferentes**. Uma API que devolvesse 204 com um JSON no corpo passaria num teste que só olha o código — e quebraria clientes que confiam no "sem conteúdo".

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
====================== 145 passed, 1 warning in 0.94s ======================
```

Confira no seu print

- **145 passed** — os 17 testes de tarefa do dia, todos verdes.

Feche o commit 24

```
git add tests/conftest.py tests/test_api.py
git commit -m "Adiciona o cliente logado e os testes das tarefas"
```

Experimento — quanto custa um teste de API

Rode os dois arquivos separados e compare os tempos na última linha:

Execute no terminal

```
pytest -q tests/test_api.py
pytest -q tests/test_dominio.py
```

O que deve aparecer (os tempos na sua máquina serão outros)

```
30 passed, 1 warning in 0.57s
...
37 passed, 1 warning in 0.09s
```

Confira no seu print

- Divida o tempo pelo número de testes: cada teste de API custou **várias vezes** mais do que um de domínio — e isso com o `HashFake` deixando o hash instantâneo.
- É o motivo da divisão de trabalho da suíte: o **domínio** cobre as **variações** de cada regra (dezenas de casos, baratos); a **API** cobre os **caminhos** (um por endpoint e por resposta).

4. O cabeçalho Authorization

Passo 6 — Sem cabeçalho, e cabeçalho mal formado

Arquivo: tests/test_autenticacao.py (alterar o topo e acrescentar ao final) · ainda sem commit

O `test_autenticacao.py` do Dia 4 testou o token **por dentro**, sem HTTP. A seção de hoje testa o cabeçalho **pela API**. Primeiro, os imports: acrescente as três linhas marcadas ao bloco do topo do arquivo.

Código para digitar — o bloco de imports do test_autenticacao.py (sem as setas)

```
from datetime import timedelta

import jwt
import pytest
from fastapi.testclient import TestClient            <- nova
from sqlalchemy.orm import Session                    <- nova

from app.exceptions import NaoAutenticado, SessaoExpirada
from app.models import Usuario                        <- nova
from app.tokens import ALGORITMO, TokenDeAcesso
```

E no **final** do arquivo, os dois primeiros testes:

Código para digitar

```
# ============================================================================
# Dia 8 — o cabeçalho `Authorization`, pela API
# ============================================================================

def test_sem_cabecalho_responde_401(cliente: TestClient):
    """401 é "não sei quem você é"; 403 seria "sei e você não pode".

    O cliente trata 401 limpando o token e mandando entrar de novo, e 403
    mostrando "sem permissão". Trocar os dois faz o aplicativo pedir login a
    quem já está logado, ou o contrário.
    """
    assert cliente.get("/tasks").status_code == 401


@pytest.mark.parametrize(
    "cabecalho",
    [
        "token-sem-esquema",
        "Basic YWxhZGRpbjpvcGVuc2VzYW1l",
        "Bearer",
        "Bearer   ",
    ],
)
def test_cabecalho_mal_formado_responde_401(cliente: TestClient, cabecalho: str):
    """Quatro formas de errar o cabeçalho, todas com a mesma resposta.

    Esqueceu o esquema; usou o esquema errado; mandou o esquema sem token;
    mandou o esquema com espaços. Nenhuma pode virar 500.
    """
    resposta = cliente.get("/tasks", headers={"Authorization": cabecalho})
    assert resposta.status_code == 401
```

Entendendo o código

`(cliente: TestClient)`	Os dois testes usam o `cliente` **sem** token: cada um monta o cabeçalho — certo ou errado — por conta própria.
`== 401`	"Não sei quem você é." O front-end reage a 401 limpando o token e mandando entrar de novo; a 403, mostrando "sem permissão". Trocar um pelo outro confunde o aplicativo.
`@pytest.mark.parametrize( "cabecalho", [...])`	O `parametrize` do Dia 3: um teste escrito uma vez, executado com quatro valores.
`"token-sem-esquema"`	Esqueceu a palavra `Bearer`.
`"Basic YWxhZGRp..."`	Usou **outro** esquema — o `Basic`, de usuário e senha.
`"Bearer"` e `"Bearer   "`	O esquema **sem** o token, e o esquema seguido só de espaços.
`headers={"Authorization": cabecalho}`	Um cabeçalho enviado **só nesta requisição**, diferente do `cliente.headers` do `cliente_logado`, que vale para todas.

> Nenhuma das quatro formas pode virar **500** (erro interno). Um cabeçalho mal formado é um erro de **quem chamou**, e a resposta precisa dizer isso. Um 500 diria que a aplicação quebrou.

Execute no terminal

```
pytest -v -k cabecalho tests/test_autenticacao.py
```

O que deve aparecer (trecho)

```
tests/test_autenticacao.py::test_sem_cabecalho_responde_401 PASSED       [ 20%]
tests/test_autenticacao.py::test_cabecalho_mal_formado_responde_401[token-sem-esquema] PASSED [
40%]
tests/test_autenticacao.py::test_cabecalho_mal_formado_responde_401[Basic YWxhZGRpbjpvcGVuc2VzYW1l]
PASSED [ 60%]
tests/test_autenticacao.py::test_cabecalho_mal_formado_responde_401[Bearer] PASSED [ 80%]
tests/test_autenticacao.py::test_cabecalho_mal_formado_responde_401[Bearer   ] PASSED [100%]

================= 5 passed, 13 deselected, 1 warning in 0.05s =================
```

Confira no seu print

- Cinco `PASSED`: o teste sem cabeçalho e as **quatro** variações do parametrizado, com o valor entre colchetes. O `-k cabecalho` deixou de fora (`deselected`) os 13 testes sem essa palavra no nome.
- Rode `pytest`: **150 passed**.

Passo 7 — A conta apagada, e o botão Authorize

Arquivo: tests/test_autenticacao.py (acrescentar ao final) · Commit 25

Código para digitar

```
def test_token_valido_de_conta_apagada_responde_401(
    cliente: TestClient, sessao_aberta: dict, sessao: Session
):
    """A conta sumiu, o token continua assinado — e a sessão não vale mais.

    É por isso que o usuário é buscado no banco a cada requisição, em vez de
    reconstruído a partir do token.

    É o único teste da suíte que usa ao mesmo tempo o `cliente` (Dia 7) e a
    `sessao` de banco (Dia 5): é preciso apagar pelo banco o que a API não
    oferece endpoint para apagar.
    """
    autorizacao = {"Authorization": f"Bearer {sessao_aberta['accessToken']}"}
    assert cliente.get("/usuarios/eu", headers=autorizacao).status_code == 200

    apagado = sessao.get(Usuario, sessao_aberta["usuario"]["id"])
    sessao.delete(apagado)
    sessao.commit()

    assert cliente.get("/usuarios/eu", headers=autorizacao).status_code == 401


def test_openapi_declara_o_esquema_bearer(cliente: TestClient):
    """É o que faz aparecer o botão **Authorize** no `/docs`."""
    esquema = cliente.get("/openapi.json").json()
    esquemas = esquema["components"].get("securitySchemes", {})
    assert any(
        definicao.get("scheme") == "bearer" for definicao in esquemas.values()
    )
```

Entendendo o código

`(cliente, sessao_aberta, sessao)`	Três fixtures de três dias: o cliente HTTP (Dia 7), o cadastro (Dia 7) e a sessão de **banco** (Dia 5).
`autorizacao = {...}`	O cabeçalho montado à mão, com o token da `sessao_aberta`.
`primeiro assert == 200`	A **pré-condição**: antes de apagar, o token funciona. Sem ela, um 401 no final não provaria nada.
`sessao.get(Usuario, id)`	Busca o usuário **direto no banco**, pela chave primária — como no Dia 5.
`sessao.delete(apagado)` `sessao.commit()`	Apaga a conta e grava. A API não tem endpoint para apagar conta; por isso o teste vai pelo banco.
`último assert == 401`	O **mesmo** token, ainda válido e assinado, agora é recusado: a conta que ele identifica não existe mais.
`["components"] ["securitySchemes"]`	A parte da documentação da API que declara **como** se autenticar.
`any(... == "bearer" ...)`	`any` é verdadeiro se **pelo menos um** esquema declarado for do tipo `bearer`.

> O teste da conta apagada prova uma decisão de projeto: a cada requisição, a API **busca o usuário no banco**, em vez de confiar só no que está escrito no token. Um token continua válido até expirar — dias, às vezes. Sem essa busca, uma conta apagada continuaria agindo até lá.

Execute no terminal

```
pytest
pytest --setup-show
tests/test_autenticacao.py::test_token_valido_de_conta_apagada_responde_401
```

O que deve aparecer (trechos)

```
====================== 152 passed, 1 warning in 1.03s ======================
...
        SETUP    F banco_limpo
        SETUP    F cliente (fixtures used: banco_limpo)
        SETUP    F sessao_aberta (fixtures used: cliente)
        SETUP    F sessao
        tests/test_autenticacao.py::test_token_valido_de_conta_apagada_responde_401
        (fixtures used: banco_limpo, cliente, sessao, sessao_aberta) .
        TEARDOWN F sessao
        TEARDOWN F sessao_aberta
        TEARDOWN F cliente
        TEARDOWN F banco_limpo
```

Confira no seu print

- **152 passed**.
- No `--setup-show`, as fixtures HTTP (`cliente`, `sessao_aberta`) e a de banco (`sessao`) montadas no **mesmo** teste — sobre o mesmo banco limpo.

Feche o commit 25

```
git add tests/test_autenticacao.py
git commit -m "Acrescenta os testes do cabecalho Authorization"
```

Veja com os próprios olhos: o botão Authorize

Suba a aplicação, como no Dia 7, e abra **http://127.0.0.1:8000/docs**:

Execute no terminal

```
uvicorn app.main:app --reload
```

No alto da página, à direita, há um botão **Authorize**, e um cadeado ao lado de cada endpoint de tarefa. É o esquema `bearer` que o teste acabou de verificar. Para usá-lo:

	Na documentação da API
1	Em **POST /usuarios**, cadastre um e-mail novo (*Try it out* → *Execute*) e copie o valor de `accessToken` da resposta, **sem** as aspas. Se vier `409`, o e-mail já existe no seu banco de trabalho: use outro.
2	Clique em **Authorize**, cole o token no campo **Value** e confirme. Não escreva `Bearer`: a página acrescenta sozinha.
3	Em **GET /tasks**, clique em *Try it out* → *Execute*. A resposta é **200** com `[]`. Sem o passo 2, seria **401**. Depois, pare o servidor com **Ctrl+C**.

5. O contrato da autenticação

Passo 8 — O que o front-end precisa da autenticação

Arquivo: tests/test_integracao_frontend.py (acrescentar ao final) · Commit 26

O `test_integracao_frontend.py` trava o que o front-end consome. Hoje ele ganha a parte da autenticação: o 401 de todos os endpoints protegidos, a mensagem do 401, e duas configurações de **CORS** sem as quais o navegador bloqueia a comunicação.

> **Antes de digitar**
>
> **CORS em uma frase:** o front-end roda num endereço (`localhost:5173`) e a API em outro (`localhost:8000`). Por segurança, o **navegador** só deixa um conversar com o outro se a API autorizar, em cabeçalhos da resposta.
>
> Antes de uma requisição com o cabeçalho `Authorization`, o navegador manda sozinho uma requisição de consulta — o *preflight*, com o verbo `OPTIONS` — perguntando "posso enviar este cabeçalho?".

Código para digitar

```
# ============================================================================
# Dia 8 — o contrato da autenticação
# ============================================================================

def test_todos_os_endpoints_de_tarefa_exigem_token(cliente: TestClient):
    """Sem `Authorization`, nenhuma operação de tarefa passa.

    O teste verifica os cinco endpoints de uma vez, comparando a **lista** de
    códigos. Um endpoint novo que esquecesse a autenticação não apareceria
    aqui — apareceria no `test_openapi_expoe_os_sete_endpoints`, do Dia 7, que
    conta as rotas. Os dois juntos fecham o cerco.
    """
    chamadas = [
        cliente.get("/tasks"),
        cliente.post("/tasks", json={"titulo": "X"}),
        cliente.patch("/tasks/1", json={"titulo": "X"}),
        cliente.delete("/tasks/1"),
        cliente.get("/usuarios/eu"),
    ]
    assert [c.status_code for c in chamadas] == [401, 401, 401, 401, 401]


def test_erro_de_autenticacao_tem_detail_em_texto(cliente: TestClient):
    """O 401 também precisa caber em `setErro(e.message)`."""
    resposta = cliente.get("/tasks")
    assert resposta.status_code == 401
    assert isinstance(resposta.json()["detail"], str)
    assert resposta.json()["detail"].strip()


def test_cors_libera_o_cabecalho_authorization(cliente: TestClient):
    """O `preflight` precisa autorizar o cabeçalho que o cliente vai enviar.

    `cliente.options(...)` é a requisição que o navegador faz por conta
    própria, antes da requisição de verdade.
    """
    resposta = cliente.options(
        "/tasks",
        headers={
            "Origin": "http://localhost:5173",
            "Access-Control-Request-Method": "GET",
            "Access-Control-Request-Headers": "authorization,content-type",
        },
    )
    assert resposta.status_code == 200
    permitidos = resposta.headers.get("access-control-allow-headers", "").lower()
    assert "authorization" in permitidos


def test_total_e_exposto_ao_navegador(cliente: TestClient):
    """Sem `expose_headers`, o `fetch` do navegador não lê o `X-Total-Count`.

    O servidor responde certo, o cliente não consegue ler, e os logs do
    servidor não registram nada de anormal. É a categoria de defeito que só um
    teste de contrato encontra.
    """
    resposta = cliente.get("/", headers={"Origin": "http://localhost:5173"})
    expostos = resposta.headers.get("access-control-expose-headers", "")
    assert "X-Total-Count" in expostos
```

Entendendo o código

`chamadas = [...]`	Cinco requisições sem token, uma para cada endpoint protegido, guardadas numa lista.
`[c.status_code for c in chamadas] == [401, ...]`	Compara a **lista** de códigos inteira. Se um falhar, a mensagem mostra **qual posição** destoou.
`isinstance(...["detail"], str)`	O `detail` precisa ser **texto**, para o front-end mostrar com `setErro(e.message)`.
`["detail"].strip()`	E não pode ser texto vazio nem só espaços.
`cliente.options(...)`	O *preflight*: a pergunta que o navegador faz antes da requisição de verdade.
`"Origin": "http://localhost:5173"`	De onde vem a requisição: o endereço do front-end em desenvolvimento.
`"access-control-allow-headers"`	A resposta da API: quais cabeçalhos o navegador pode enviar. O `authorization` precisa estar ali.
`"access-control-expose-headers"`	Quais cabeçalhos da resposta o JavaScript pode **ler**. Sem o `X-Total-Count` ali, o front-end não sabe quantas tarefas existem.

> O último teste protege um tipo de defeito traiçoeiro: o servidor responde certo, o cabeçalho está lá, os logs não mostram nada — e o navegador **esconde** o valor do JavaScript. Ninguém percebe pelo servidor. Só um teste de contrato encontra.

Execute no terminal

```
pytest -v tests/test_integracao_frontend.py
```

O que deve aparecer (trecho final)

```
tests/test_integracao_frontend.py::test_credencial_invalida_responde_401 PASSED [ 50%]
tests/test_integracao_frontend.py::test_campo_obrigatorio_em_portugues PASSED [ 57%]
tests/test_integracao_frontend.py::test_tamanho_minimo_cita_o_numero PASSED [ 64%]
tests/test_integracao_frontend.py::test_email_invalido_em_portugues PASSED [ 71%]
tests/test_integracao_frontend.py::test_todos_os_endpoints_de_tarefa_exigem_token PASSED [ 78%]
tests/test_integracao_frontend.py::test_erro_de_autenticacao_tem_detail_em_texto PASSED [ 85%]
tests/test_integracao_frontend.py::test_cors_libera_o_cabecalho_authorization PASSED [ 92%]
tests/test_integracao_frontend.py::test_total_e_exposto_ao_navegador PASSED [100%]

======================== 14 passed, 1 warning in 0.15s ========================
```

Confira no seu print

- **14 passed** no arquivo: os 10 de ontem e os 4 de hoje, no final da lista.
- Rode `pytest`: **156 passed**.

Feche o commit 26

```
git add tests/test_integracao_frontend.py
git commit -m "Acrescenta o contrato da autenticacao"
```

6. A validade do token

Passo 9 — O monkeypatch, de volta

Arquivo: tests/test_configuracao.py (alterar o topo e acrescentar ao final) · Commit 27

Quanto tempo um token vale? A aplicação lê isso da variável de ambiente `JWT_EXPIRACAO_HORAS` — e, se ela não existir, usa o padrão de **7 dias** (168 horas). É configuração, como as do Dia 6, e se testa do mesmo jeito: com o `monkeypatch`.

Todos os testes de hoje precisam de um segredo válido, só para a configuração conseguir carregar. Logo **abaixo** do bloco de imports do `test_configuracao.py`, acrescente a constante:

Código para digitar — logo abaixo dos imports

```
#: Segredo com tamanho suficiente para passar pela validação de comprimento.
#: Os testes que verificam outra coisa não deveriam falhar por causa dele.
SEGREDO_LONGO = "segredo-de-teste-com-mais-de-trinta-e-dois-bytes"
```

E no **final** do arquivo:

Código para digitar

```
# ============================================================================
# Dia 8 — a validade do token
# ============================================================================

def test_validade_padrao_do_token_e_de_sete_dias(monkeypatch):
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.setenv("JWT_SECRET", SEGREDO_LONGO)
    monkeypatch.delenv("JWT_EXPIRACAO_HORAS", raising=False)
    assert Configuracao.do_ambiente().validade_do_token_em_horas == 168.0


def test_validade_do_token_vem_do_ambiente(monkeypatch):
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.setenv("JWT_SECRET", SEGREDO_LONGO)
    monkeypatch.setenv("JWT_EXPIRACAO_HORAS", "0.5")
    assert Configuracao.do_ambiente().validade_do_token_em_horas == 0.5


@pytest.mark.parametrize("valor_invalido", ["meia hora", "-1", "0"])
def test_validade_do_token_invalida_e_recusada(monkeypatch, valor_invalido):
    """Um token que nasce expirado impediria qualquer login."""
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.setenv("JWT_SECRET", SEGREDO_LONGO)
    monkeypatch.setenv("JWT_EXPIRACAO_HORAS", valor_invalido)
    with pytest.raises(RuntimeError, match="JWT_EXPIRACAO_HORAS"):
        Configuracao.do_ambiente()


def test_validade_vazia_conta_como_nao_declarada(monkeypatch):
    """A string vazia **não** é erro: cai no padrão, como as outras opcionais.

    Fica fora do parametrizado acima de propósito: o resultado esperado é
    outro. Um `if` dentro de um teste parametrizado verificaria duas promessas
    diferentes sob o mesmo nome — e esconderia qual das duas falhou.
    """
    monkeypatch.setenv("DATABASE_URL", "sqlite://")
    monkeypatch.setenv("JWT_SECRET", SEGREDO_LONGO)
    monkeypatch.setenv("JWT_EXPIRACAO_HORAS", "")
    assert Configuracao.do_ambiente().validade_do_token_em_horas == 168.0
```

Entendendo o código

`monkeypatch.setenv( "DATABASE_URL", ...)`	As duas primeiras linhas de cada teste garantem as variáveis **obrigatórias**, para que a configuração carregue e o teste chegue ao que interessa.
`monkeypatch.delenv( "JWT_EXPIRACAO_HORAS", raising=False)`	**Remove** a variável, se existir. O `raising=False` evita erro se ela já não existia.
`== 168.0`	Sem a variável, vale o padrão: 7 dias × 24 horas.
`"0.5"`	Meia hora, vinda do ambiente. Toda variável de ambiente é **texto**; a configuração converte para número.
`["meia hora", "-1", "0"]`	Três jeitos de errar: não é número; é negativo; é zero. Um token que nasce expirado impediria qualquer login.
`pytest.raises(RuntimeError, match="JWT_EXPIRACAO_HORAS")`	O `raises` do Dia 2: o erro precisa acontecer **e** a mensagem precisa citar a variável errada.
`setenv(..., "")`	A variável **declarada vazia**. Não é erro: conta como não declarada, e cai no padrão.

Uma decisão de desenho

Por que a string vazia não entrou no `parametrize` dos inválidos? Porque o resultado esperado é **outro**: os inválidos dão erro; a vazia dá 168.

Para caber no mesmo teste, seria preciso um `if valor == ""` lá dentro — e aí o teste verificaria **duas promessas sob o mesmo nome**. Quando falhasse, o nome não diria qual das duas quebrou.

A regra: um `parametrize` varia a **entrada**, nunca a **promessa**.

Execute no terminal

```
pytest -v -k validade tests/test_configuracao.py
pytest
```

O que deve aparecer (trechos)

```
tests/test_configuracao.py::test_validade_padrao_do_token_e_de_sete_dias PASSED [ 16%]
tests/test_configuracao.py::test_validade_do_token_vem_do_ambiente PASSED [ 33%]
tests/test_configuracao.py::test_validade_do_token_invalida_e_recusada[meia hora] PASSED [ 50%]
tests/test_configuracao.py::test_validade_do_token_invalida_e_recusada[-1] PASSED [ 66%]
tests/test_configuracao.py::test_validade_do_token_invalida_e_recusada[0] PASSED [ 83%]
tests/test_configuracao.py::test_validade_vazia_conta_como_nao_declarada PASSED [100%]

================= 6 passed, 36 deselected, 1 warning in 0.08s =================
...
======================== 162 passed, 1 warning in 1.17s ========================
```

Confira no seu print

- Seis linhas `PASSED` — o parametrizado aparece três vezes, uma por valor.
- **162 passed** na suíte inteira.

Feche o commit 27

```
git add tests/test_configuracao.py
git commit -m "Acrescenta os testes da validade do token"
```

7. Fechamento do dia

O que a sessão aberta trouxe

Recurso	Para quê	Onde você usou
fixture que modifica outra	acrescentar o token ao `cliente`	`cliente_logado`
`cliente.headers[...]`	um cabeçalho em **toda** requisição	`cliente_logado`
`headers={...}` na chamada	um cabeçalho em **uma** requisição	os testes do cabeçalho
o cache de fixture	uma instância por teste, para todos que pedirem	`usuario` e `token` do mesmo cadastro
`cliente.patch` / `cliente.delete`	alterar e remover	os testes de tarefa
`date.today() + timedelta(...)`	datas que não envelhecem	prazos e filtros
`resposta.content == b""`	exigir o corpo vazio	o teste do 204
`cliente.options(...)`	simular o *preflight* do navegador	o teste do CORS
`monkeypatch.delenv`	garantir que uma variável **não** existe	a validade padrão

Confira o que o dia entregou

Verificação	Esperado
`pytest`	162 passed
Nenhum teste de tarefa monta o cabeçalho à mão	todos pedem `cliente_logado`
Nenhuma data fixa no futuro	prazos com `date.today() + timedelta(...)`
O teste do 204 verifica o corpo	`resposta.content == b""`
Nada mudou em `app/`	`git diff HEAD~4 --stat` só lista arquivos de `tests/`

O histórico do dia

Execute no terminal

```
git log --oneline -4
git diff HEAD~4 --stat
```

O que deve aparecer (os códigos dos commits e as contagens podem variar)

```
27  Acrescenta os testes da validade do token
26  Acrescenta o contrato da autenticacao
25  Acrescenta os testes do cabecalho Authorization
24  Adiciona o cliente logado e os testes das tarefas

 tests/conftest.py                |  28 ++++
 tests/test_api.py                | 206 ++++++++++++++++++++++++
 tests/test_autenticacao.py       |  67 ++++++++
 tests/test_configuracao.py       |  45 ++++++
 tests/test_integracao_frontend.py |  61 +++++++
 5 files changed, 407 insertions(+)
```

Confira no seu print

- Só arquivos de `tests/`, e só **inserções**: nada foi apagado nem reescrito — cada arquivo cresceu pelo final.
- **Nenhum arquivo de `app/`**: hoje não houve defeito.

Se algo der errado

Sintoma	Causa provável	O que fazer
`401` em todo teste de tarefa	O teste pediu `cliente` em vez de `cliente_logado`	Só o `cliente_logado` envia o token
Um teste de "sem token" não recebe o `401`	O teste pediu `cliente_logado` — ou os dois, que são o mesmo objeto	A ausência do token é o cenário: peça só o `cliente`
`409` dentro de uma fixture	Uma fixture nova cadastrou o mesmo e-mail da `sessao_aberta`	Cada fixture de conta usa um e-mail próprio
`test_filtrar_por_prazo_ate` falha em alguma data	O prazo foi escrito fixo	`date.today() + timedelta(...)`
`NameError: name 'date' is not defined`	Faltou o import no topo do `test_api.py`	`from datetime import date, timedelta`
`NameError: name 'SEGREDO_LONGO'...`	Faltou a constante no `test_configuracao.py`	Logo abaixo dos imports
`204` com corpo passa no teste	Só o status foi verificado	`assert resposta.content == b""`

Amanhã

> **Dia 9 — duas contas.** Hoje a suíte aprendeu a agir como **uma** pessoa logada. Amanhã ela age como **duas** ao mesmo tempo — e é aí que aparece o quinto defeito, o mais grave dos cinco, que só existe **entre** as contas. Depois, a **cobertura**: o que ela mede, e o que ela não mede.

SUÍTE DE TESTES — TODO LIST API

Roteiro de Implementação

Dia 9 — Duas contas, o contrato e o que a cobertura não mede

Roteiro guiado: o cenário que vamos testar, o código de cada teste explicado passo a passo, e os espaços para você registrar o seu código e o seu terminal.

Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7	Dia 8	Dia 9
assert	raises	parametrize	fixture	banco	monkeypatch	HTTP	sessão	**duas contas**

> **Hoje:** agir como **duas pessoas** no mesmo teste, fechar o contrato com o front-end e medir a suíte com a **cobertura** (`pytest-cov`).
>
> **Ao final:** `pytest` responde **188 passed** (162 de antes + 26 de hoje).
>
> **Defeito a encontrar hoje: SIM** — o mais grave dos cinco, e o último da trilha.

Curso Técnico em Informática — SENAC Minas Gerais
UC 11 — Executar teste e implantação de aplicativos

1. O cenário de hoje

De onde paramos

Ontem a suíte aprendeu a agir como **uma** pessoa logada: o `cliente_logado` cria, lista, altera e remove as tarefas da Ana Souza. Todos os testes passaram. Mas nenhum deles fez a pergunta que importa num sistema com várias contas: **a Ana consegue mexer nas tarefas de outra pessoa?**

O cenário que nenhum teste de ontem cobre

Os ids das tarefas são números **sequenciais**. Quem tem uma conta qualquer sabe que existem as tarefas 1, 2, 3...

```
Bruno cria uma tarefa  ------------>  tarefa 7, dono: Bruno
                                                  |
Ana envia:                                        |
  PATCH /tasks/7                                  |
  Authorization: Bearer <token da Ana>  -----+
  {"titulo": "Invadida"}

Esperado: 403 - "sei quem você é, e esta tarefa não é sua"
          e a tarefa 7 continua exatamente como estava
```

Para testar isso, o teste precisa agir como **duas** pessoas: o Bruno cria, a Ana tenta alterar. O `cliente_logado` não serve — ele tem **um** token fixo.

Código	Significado	No teste de hoje
`401`	"não sei quem você é"	sem token (Dia 8)
`403`	"sei quem você é, e **isto não é seu**"	a Ana mexendo na tarefa do Bruno
`404`	"isto **não existe**"	uma tarefa que ninguém criou

As ferramentas novas

Duas sessões no mesmo teste

Uma fixture nova, `outro_usuario`, cadastra a segunda conta. E nos testes de isolamento o cabeçalho `Authorization` volta a ser montado **à mão**, de propósito: agora ele **muda** de uma requisição para a outra — ora o token da Ana, ora o do Bruno.

A cobertura

O `pytest-cov` é um plugin que observa a suíte rodando e anota **quais linhas do `app/` foram executadas**. No final, mostra por arquivo a porcentagem e as linhas que nenhum teste alcançou. Hoje você vai aprender a usá-lo — e, mais importante, o que ele **não** mede.

O que vamos testar hoje

Arquivo	Testes	O que verifica
`tests/conftest.py`	—	a fixture `outro_usuario`, a segunda conta
`tests/test_autenticacao.py`	5	o **isolamento** entre as contas
`tests/test_api.py`	1	o `usuarioId` do corpo é ignorado
`tests/test_integracao_frontend.py`	20	o contrato das tarefas com o front-end
`requirements.txt`	—	o `pytest-cov`, a dependência da cobertura

O que você vai aprender hoje

Ferramenta nova	duas sessões no mesmo teste (`outro_usuario`); `pytest-cov`
Técnica nova	provar que uma operação recusada **não deixou nada alterado**; ler a cobertura como **localizador de ausências**
Arquivos que crescem	`conftest.py`, `test_autenticacao.py`, `test_api.py`, `test_integracao_frontend.py` e `requirements.txt`
Testes ao final	188 (162 de antes + 26 de hoje)
Commits	28, 29, 30 e 31 — o 31 fecha a trilha

O procedimento do teste vermelho

Hoje o defeito aparece no **segundo** teste de isolamento, e derruba **três** testes de uma vez. O procedimento é o do Dia 7: completar a bateria com ele vermelho, **registrar** o defeito com evidência e só então corrigir.

E o dia termina com o **parecer**: o documento que diz, com evidência, se a aplicação pode ser entregue.

2. A segunda conta

Passo 1 — A fixture outro_usuario

Arquivo: tests/conftest.py (acrescentar ao final) · ainda sem commit

Código para digitar

```
# ================================================================================
# Dia 9 — a segunda conta
# ================================================================================

@pytest.fixture
def outro_usuario(cliente: TestClient) -> dict:
    """Uma segunda conta, com a sua própria sessão.

    Provar que uma conta não alcança as tarefas da outra exige duas contas de
    verdade. Devolve a resposta inteira do cadastro — token e usuário —,
    porque os testes de isolamento precisam dos dois.
    """
    resposta = cliente.post(
        "/usuarios",
        json={"nome": "Bruno Lima", "email": "bruno@exemplo.com", "senha": "senha456"},
    )
    assert resposta.status_code == 201, resposta.text
    return resposta.json()
```

Entendendo o código

`def outro_usuario(cliente):`	Pede o **mesmo** `cliente` da `sessao_aberta` — o cache de fixture do Dia 8. As duas contas são cadastradas no mesmo banco, pelo mesmo cliente.
`"email": "bruno@exemplo.com"`	Um e-mail **diferente** do da Ana. Com o mesmo e-mail, o segundo cadastro daria 409 e a fixture quebraria.
`assert ... == 201, resposta.text`	A pré-condição do Dia 7: se o cadastro falhar, a mensagem mostra o porquê.
`return resposta.json()`	Devolve a resposta **inteira** — o `accessToken` **e** o `usuario` do Bruno. Os testes vão precisar dos dois.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
======================== 162 passed, 1 warning in 1.15s ========================
```

Confira no seu print

- Continua **162 passed** — nenhum teste pede a segunda conta ainda.

3. O isolamento entre contas — e o defeito

Passo 2 — Uma fábrica e a listagem

Arquivo: tests/test_autenticacao.py (acrescentar ao final) · ainda sem commit

Os testes de hoje criam tarefas em nome de pessoas diferentes. Uma função auxiliar — uma **fábrica**, como as do Dia 1 — evita repetir essa preparação. E o primeiro teste confere a listagem:

Código para digitar

```
# ================================================================================
# Dia 9 — isolamento entre contas: duas sessões ao mesmo tempo
# ================================================================================

def tarefa_de(cliente: TestClient, sessao: dict, titulo: str) -> dict:
    """Cria uma tarefa em nome do dono da sessão informada.

    É uma fábrica, como as do Dia 1, e existe por um motivo específico: estes
    testes precisam agir como **duas** pessoas diferentes, o que o
    `cliente_logado` — que tem um token fixo — não permite.
    """
    resposta = cliente.post(
        "/tasks",
        json={"titulo": titulo},
        headers={"Authorization": f"Bearer {sessao['accessToken']}"},
    )
    assert resposta.status_code == 201, resposta.text
    return resposta.json()


def test_a_listagem_so_traz_as_tarefas_do_dono_do_token(
    cliente: TestClient, sessao_aberta: dict, outro_usuario: dict
):
    """Não existe parâmetro para pedir "as tarefas de alguém".

    O `X-Total-Count` é conferido junto porque um total que contasse as
    tarefas de todos vazaria informação mesmo com a lista correta.
    """
    tarefa_de(cliente, sessao_aberta, "Minha")
    tarefa_de(cliente, outro_usuario, "Dele")

    minhas = cliente.get(
        "/tasks",
        headers={"Authorization": f"Bearer {sessao_aberta['accessToken']}"},
    )
    assert [t["titulo"] for t in minhas.json()] == ["Minha"]
    assert minhas.headers["X-Total-Count"] == "1"
```

Entendendo o código

`def tarefa_de(cliente, sessao, titulo):`	Não começa com `test_`: o pytest não a executa como teste. É uma função comum, chamada pelos testes.
`sessao: dict`	A resposta de um cadastro — a `sessao_aberta` da Ana ou o `outro_usuario` do Bruno. A tarefa é criada em nome de **quem** for passado.
`headers={"Authorization": f"Bearer {sessao[...]}"}`	O cabeçalho só **desta** requisição, com o token da sessão informada.
`(cliente, sessao_aberta, outro_usuario)`	O teste pede o `cliente` **sem** token e as duas contas. Ele decide, a cada requisição, quem está pedindo.
`tarefa_de(..., sessao_aberta, "Minha")`	Uma tarefa da Ana e, na linha de baixo, uma do Bruno.
`== ["Minha"]`	A Ana lista e vê **só** a dela.
`headers["X-Total-Count"] == "1"`	E o total também conta só a dela. Um total "2" contaria que o Bruno tem tarefas — um vazamento, mesmo com a lista certa.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
====================== 163 passed, 1 warning in 1.10s ======================
```

Confira no seu print

- **163 passed** — a listagem está isolada: cada conta vê só as próprias tarefas.

Passo 3 — A Ana tenta alterar a tarefa do Bruno

Arquivo: tests/test_autenticacao.py (acrescentar ao final) · ainda sem commit

Código para digitar

```
def test_nao_se_altera_a_tarefa_de_outra_conta(
    cliente: TestClient, sessao_aberta: dict, outro_usuario: dict
):
    """Os ids são sequenciais: quem tem uma conta qualquer sabe que existe a
    tarefa 1, a 2, a 3. Saber o id não pode bastar para editar.

    A mensagem é conferida porque ela precisa dizer o que aconteceu sem
    revelar nada sobre a tarefa alheia.
    """
    alheia = tarefa_de(cliente, outro_usuario, "Tarefa do Bruno")

    resposta = cliente.patch(
        f"/tasks/{alheia['id']}",
        json={"titulo": "Invadida"},
        headers={"Authorization": f"Bearer {sessao_aberta['accessToken']}"},
    )
    assert resposta.status_code == 403
    assert resposta.json()["detail"] == "Esta tarefa pertence a outro usuário."
```

Entendendo o código

`alheia = tarefa_de(cliente, outro_usuario, ...)`	O **Bruno** cria a tarefa. `alheia` guarda a resposta — o `id` da tarefa está ali.
`f"/tasks/{alheia['id']}"`	A Ana sabe o id. Na vida real, bastaria chutar números sequenciais.
`sessao_aberta['accessToken']`	Mas o token é o da **Ana**. É ela quem pede a alteração.
`== 403`	A resposta esperada: "sei quem você é, e isto não é seu".
`["detail"] == "Esta tarefa pertence a outro usuário."`	A mensagem diz o que aconteceu **sem** revelar nada da tarefa alheia — nem o título, nem o dono.

Execute no terminal

```
pytest
```

O que deve aparecer (trecho)

```
        resposta = cliente.patch(
            f"/tasks/{alheia['id']}",
            json={"titulo": "Invadida"},
            headers={"Authorization": f"Bearer {sessao_aberta['accessToken']}"},
        )
>       assert resposta.status_code == 403
E       assert 200 == 403
E        +  where 200 = <Response [200 OK]>.status_code

tests\test_autenticacao.py:269: AssertionError
========================== short test summary info ==========================
FAILED tests/test_autenticacao.py::test_nao_se_altera_a_tarefa_de_outra_conta
================== 1 failed, 163 passed, 1 warning in 1.23s ==================
```

Lendo a saída

`assert 200 == 403`	A API respondeu **200 OK**. A requisição que editava a tarefa de outra pessoa não foi recusada: foi **atendida**.
`<Response [200 OK]>`	O pytest mostra de onde veio o 200: o `status_code` da resposta.
`sessao_aberta = {...}` `outro_usuario = {...}`	No topo da falha, o pytest lista os valores das fixtures: `'id': 1` para a Ana, `'id': 2` para o Bruno. São mesmo duas contas diferentes.

> Pare um instante e leia de novo: **com o próprio token, a Ana alterou a tarefa do Bruno.** Qualquer pessoa com uma conta pode editar as tarefas de todas as outras, só trocando um número no endereço.
>
> Mesmo assim, **não corra para corrigir.** Complete a bateria nos próximos passos: os testes que falharem e os que passarem vão dizer até onde o defeito vai.

Passo 4 — Apagar, o estado depois, e o que não existe

Arquivo: tests/test_autenticacao.py (acrescentar ao final) · ainda sem commit

Código para digitar

```
def test_nao_se_apaga_a_tarefa_de_outra_conta(
    cliente: TestClient, sessao_aberta: dict, outro_usuario: dict
):
    """A mesma verificação no `DELETE`, e ela precisa ser testada à parte.

    `PATCH` e `DELETE` são dois caminhos no código. Um deles podia ter a
    verificação e o outro não.
    """
    alheia = tarefa_de(cliente, outro_usuario, "Tarefa do Bruno")

    resposta = cliente.delete(
        f"/tasks/{alheia['id']}",
        headers={"Authorization": f"Bearer {sessao_aberta['accessToken']}"},
    )
    assert resposta.status_code == 403


def test_a_tarefa_recusada_continua_intacta(
    cliente: TestClient, sessao_aberta: dict, outro_usuario: dict
):
    """Recusar não é o bastante: é preciso que nada tenha mudado.

    Um 403 devolvido *depois* de gravar a alteração seria **pior do que não
    ter verificação nenhuma**, porque pareceria seguro. A resposta é uma
    evidência do que aconteceu, não a prova. A prova é olhar o estado depois.
    """
    dele = {"Authorization": f"Bearer {outro_usuario['accessToken']}"}
    alheia = tarefa_de(cliente, outro_usuario, "Original")

    cliente.patch(
        f"/tasks/{alheia['id']}",
        json={"titulo": "Invadida"},
        headers={"Authorization": f"Bearer {sessao_aberta['accessToken']}"},
    )

    depois = cliente.get("/tasks", headers=dele).json()
    assert [t["titulo"] for t in depois] == ["Original"]


def test_tarefa_inexistente_responde_404_e_nao_403(
    cliente_logado: TestClient,
):
    """404 é "não existe"; 403 é "existe e não é sua".

    Responder 403 confirma que a tarefa **existe**, o que é um vazamento
    pequeno mas real. Este projeto escolheu a clareza para quem integra, e a
    decisão está registrada no manual técnico.
    """
    assert cliente_logado.delete("/tasks/9999").status_code == 404
```

Entendendo o código

`cliente.delete(...)` `    == 403`	A mesma tentativa, agora para **apagar**. É um caminho diferente no código, e por isso tem teste próprio.
`dele = {"Authorization":` `    ...outro_usuario...}`	O cabeçalho do **Bruno**, guardado para ser usado no final.
`cliente.patch(...)` `(sem guardar a resposta)`	A Ana tenta alterar — e o teste **ignora** a resposta. Nem guarda numa variável.
`depois = cliente.get(` `    "/tasks", headers=dele)`	O **Bruno** lista as próprias tarefas, depois da tentativa.
`== ["Original"]`	A prova: o título continua o que o Bruno escreveu. Esse teste olha o **estado**, não a resposta.
`cliente_logado.delete(` `    "/tasks/9999")`	Uma tarefa que ninguém criou: **404**, e não 403. "Não existe" e "não é sua" são casos diferentes.

> **Por que um teste que ignora a resposta?** Imagine uma correção mal feita que **grava** a alteração e só depois responde 403. O teste do Passo 3 passaria — a resposta é 403 — e a tarefa do Bruno estaria alterada.
>
> Um 403 devolvido depois de gravar seria **pior do que nenhuma verificação**, porque pareceria seguro. A resposta é uma evidência do que aconteceu, não a prova. **A prova é o estado.**

Execute no terminal

```
pytest
pytest tests/test_autenticacao.py::test_a_tarefa_recusada_continua_intacta
```

O que deve aparecer (trechos)

```
FAILED tests/test_autenticacao.py::test_nao_se_altera_a_tarefa_de_outra_conta
FAILED tests/test_autenticacao.py::test_nao_se_apaga_a_tarefa_de_outra_conta
FAILED tests/test_autenticacao.py::test_a_tarefa_recusada_continua_intacta - ...
================== 3 failed, 164 passed, 1 warning in 1.28s ==================
...
        depois = cliente.get("/tasks", headers=dele).json()
>       assert [t["titulo"] for t in depois] == ["Original"]
E       AssertionError: assert ['Invadida'] == ['Original']
E
E         At index 0 diff: 'Invadida' != 'Original'
```

Confira no seu print

- **3 failed, 164 passed.** O `DELETE` também foi atendido (`assert 204 == 403`): a Ana apagou a tarefa do Bruno.
- `['Invadida'] == ['Original']` — é esta a linha que você mostra para quem precisa entender a gravidade: a tarefa do Bruno **foi alterada** pela Ana.
- O 404 da tarefa inexistente **passou**: a API sabe dizer "não existe". O que falta é dizer "não é sua".

Passo 5 — O dono vem do token, não do corpo

Arquivo: tests/test_api.py (acrescentar ao final) · Commit 28 — vermelho, de propósito

Uma última tentativa de invasão, pelo outro lado: criar uma tarefa **mandando o id de outra pessoa** no corpo da requisição.

Código para digitar

```
# ================================================================================
# Dia 9 — duas contas
# ================================================================================

def test_usuario_id_no_corpo_e_ignorado(
    cliente_logado: TestClient, usuario: dict, outro_usuario: dict
):
    """Mandar `usuarioId` de outra pessoa não cria tarefa para ela.

    É um **teste de regressão**: não verifica uma regra nova, e sim que uma
    falha já corrigida — o dono vir do corpo da requisição — não volta.
    """
    resposta = cliente_logado.post(
        "/tasks",
        json={"titulo": "Tentativa", "usuarioId": outro_usuario["usuario"]["id"]},
    )
    assert resposta.status_code == 201
    assert resposta.json()["usuarioId"] == usuario["id"]
```

Entendendo o código

`(cliente_logado, usuario, outro_usuario)`	A Ana logada, os dados dela, e a conta do Bruno — só para saber o id dele.
`"usuarioId": outro_usuario ["usuario"]["id"]`	A Ana manda, no corpo, o id do **Bruno** como dono da tarefa.
`== 201`	A tarefa é criada normalmente: o campo a mais não é erro.
`["usuarioId"] == usuario["id"]`	Mas o dono é a **Ana**, a dona do token. O `usuarioId` do corpo foi ignorado.

> É um **teste de regressão**: ele não procura um defeito novo, e sim garante que um defeito antigo — numa versão anterior da API, o dono vinha do corpo — não volte numa refatoração futura.

Execute no terminal

```
pytest
```

O que deve aparecer (o resumo do final)

```
FAILED tests/test_autenticacao.py::test_nao_se_altera_a_tarefa_de_outra_conta
FAILED tests/test_autenticacao.py::test_nao_se_apaga_a_tarefa_de_outra_conta
FAILED tests/test_autenticacao.py::test_a_tarefa_recusada_continua_intacta - ...
================== 3 failed, 165 passed, 1 warning in 1.31s ===================
```

Confira no seu print

- **3 failed, 165 passed** — o teste novo passou: criar tarefa está protegido. O problema está só em **alterar** e **apagar** a tarefa de outra pessoa.

Feche o commit 28 — vermelho, de propósito

```
git add tests/conftest.py tests/test_autenticacao.py tests/test_api.py
git commit -m "Adiciona os testes de isolamento entre contas - FALHA"
```

Passo 6 — O diagnóstico, e o registro do defeito

Arquivo: leitura: a saída do pytest, o manual técnico e app/services.py · sem commit

Este passo **não tem código para digitar**: tem perguntas para guiar a investigação — e, no final, o registro do defeito.

1. O que a saída diz?

Rode o arquivo de autenticação inteiro com `-v`. Esta saída é a **evidência** do registro:

Execute no terminal

```
pytest -v tests/test_autenticacao.py
```

O que deve aparecer (trecho)

```
tests/test_autenticacao.py::test_openapi_declara_o_esquema_bearer PASSED [ 80%]
tests/test_autenticacao.py::test_a_listagem_so_traz_as_tarefas_do_dono_do_token PASSED [ 84%]
tests/test_autenticacao.py::test_nao_se_altera_a_tarefa_de_outra_conta FAILED [ 88%]
tests/test_autenticacao.py::test_nao_se_apaga_a_tarefa_de_outra_conta FAILED [ 92%]
tests/test_autenticacao.py::test_a_tarefa_recusada_continua_intacta FAILED [ 96%]
tests/test_autenticacao.py::test_tarefa_inexistente_responde_404_e_nao_403 PASSED [100%]
...
=================== 3 failed, 22 passed, 1 warning in 0.39s ====================
```

O que passou	O que falhou	O que isso diz
a listagem		ler é seguro: a listagem já filtra pelo dono
	alterar e apagar	os dois caminhos que recebem um **id** de tarefa
o 404		a API encontra a tarefa — só não confere **de quem** ela é

2. O teste está certo?

Abra o manual técnico na seção **9. Segurança**. Na tabela "O que está implementado", a linha **Autorização por dono**: "toda tarefa é verificada contra o usuário do token". E logo abaixo, em **401 e 403 são coisas diferentes**: 403 é "token válido pedindo a tarefa de outra conta". O teste cobra exatamente o que está documentado. O problema está na aplicação.

3. Isto tem nome

IDOR — *Insecure Direct Object Reference* (referência direta insegura a objeto). O OWASP, a referência mundial em segurança de aplicações web, classifica como **A01: Broken Access Control** — a **primeira** da lista das dez falhas mais críticas.

A mecânica é sempre a mesma: a aplicação recebe um identificador do cliente, encontra o recurso e **não verifica se aquele cliente pode acessar aquele recurso**.

4. Onde está o defeito?

Quem decide se uma tarefa pode ser alterada ou removida é o **serviço**. Abra `app/services.py`, encontre a classe `TarefaServico` e leia o método que **busca** uma tarefa para alterar ou remover — inteiro, docstring incluída.

Pistas

Perguntas para guiar a leitura:

• Que exceções a docstring **promete** levantar? Quantas o código realmente levanta?

• O método recebe um parâmetro **obrigatório** além do id da tarefa. Em que linha do corpo do método ele é **usado**?

• Procure `AcessoNaoAutorizado` no projeto inteiro (**Ctrl+Shift+F**). Ela é **importada** no `services.py`, **documentada** nas docstrings e **traduzida para 403** no `app/main.py`. Em que linha ela é **levantada**?

• Em `app/models.py`, a classe `Tarefa` tem um método que responde à pergunta "esta tarefa é deste usuário?". Quem o chama?

É a segunda vez na trilha que a docstring promete o que o código não faz — no Dia 7, o schema dizia não declarar o hash e declarava. **Tornar um parâmetro obrigatório impede esquecer de passá-lo. Não impede esquecer de usá-lo.**

5. Antes de corrigir: registre

Como no Dia 7, o defeito passa pelo ciclo completo de gestão. Registre-o no lugar que o professor indicar, com estes campos:

Campo do registro	O que vai nele
Título	uma frase que diga o problema, não a solução
Severidade	crítica — quebra de autorização entre contas
Prioridade	máxima: qualquer usuário autenticado explora, sem ferramenta nenhuma
Classificação	OWASP A01 — Broken Access Control (IDOR)
Passos para reproduzir	duas contas; a tarefa de uma é editada com o token da outra
Resultado esperado	403, e a tarefa **inalterada**
Resultado obtido	200, e a tarefa com o título trocado; 204 no `DELETE`, e a tarefa apagada
Evidência anexada	o print do `pytest -v tests/test_autenticacao.py`

A conversa sobre os dois defeitos de segurança

	Defeito do Dia 7	Defeito de hoje
O que expunha	o hash da senha	as tarefas de todas as outras contas
Precisa de ação do atacante?	não — vazava sozinho	sim, mas trivial: trocar um número
Quebrava alguma funcionalidade?	não	não
Teste manual encontraria?	não	**talvez** — se o roteiro previsse duas contas

> Um roteiro de teste que use **uma** conta nunca encontra esta falha. Ela não está em nenhuma tela: está na **relação entre duas sessões**. É por isso que "testar com dois usuários" precisa estar escrito no plano de testes.

Passo 7 — A correção

Arquivo: app/services.py (alterado) · Commit 29

A correção é sua. Faça o método cumprir o que a própria docstring promete.

> **Três orientações:**
>
> • Corrija **no método de busca**, e não nas rotas nem em `atualizar` e `remover` separadamente. Os dois passam por ele: uma correção ali fecha os dois caminhos.
>
> • Tudo o que você precisa **já existe**: a exceção, o método da `Tarefa` que responde de quem ela é, e a tradução para 403. Nenhum arquivo além do `services.py` muda.
>
> • A ordem importa: primeiro "a tarefa existe?" (404), depois "é deste usuário?" (403). O teste do 404 vigia isso.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
====================== 168 passed, 1 warning in 1.19s ======================
```

Confira no seu print

- **168 passed** — os três vermelhos passaram com **uma** correção, e nenhum outro quebrou.

> **Sobrou `1 failed`, no `test_nao_se_apaga_a_tarefa_de_outra_conta`?** A verificação foi posta em `atualizar`, e não no método de busca: o `PATCH` ficou protegido, o `DELETE` não.
>
> É exatamente por isso que os dois têm testes separados: eles **provam** que a correção cobriu os dois caminhos.

Feche o commit 29

```
git add app/services.py
git commit -m "Verifica o dono da tarefa antes de alterar ou remover"
```

4. O contrato das tarefas

Os 20 testes desta seção não verificam nenhuma regra de negócio nova. Verificam que a API fala **exatamente** o vocabulário que o front-end sabe consumir: os campos, as datas, os nomes dos filtros, a ordem, as mensagens. Cada grupo aponta para o arquivo do front-end que depende dele.

Passo 8 — Os campos e as datas

Arquivo: tests/test_integracao_frontend.py (alterar o topo e acrescentar ao final) · ainda sem commit

No topo do arquivo, **acima** do import do `TestClient`, acrescente:

Código para digitar — o bloco de imports do test_integracao_frontend.py

```
from datetime import date, timedelta

from fastapi.testclient import TestClient
```

E no **final** do arquivo:

Código para digitar

```
# ============================================================================
# Dia 9 — o contrato das tarefas
# ============================================================================

def criar_tarefa(cliente: TestClient, **campos) -> dict:
    """Cria uma tarefa pela API e devolve o JSON da resposta.

    Não recebe o dono: ele vem do token do cliente informado.
    """
    corpo = {"titulo": "Tarefa"}
    corpo.update(campos)
    resposta = cliente.post("/tasks", json=corpo)
    assert resposta.status_code == 201, resposta.text
    return resposta.json()


# ------------------------------------------------- campos — `@typedef Task`

def test_tarefa_tem_exatamente_os_campos_do_typedef(cliente_logado: TestClient):
    """Nem a mais nem a menos: um campo faltando quebra a tela, e um campo
    extra indica que o domínio vazou para a borda."""
    tarefa = criar_tarefa(cliente_logado)
    assert set(tarefa) == {
        "id",
        "usuarioId",
        "titulo",
        "status",
        "prioridade",
        "prazo",
        "criadoEm",
        "atualizadoEm",
        "concluidoEm",
    }


# ------------------------------------------- datas — `new Date(...)` no front

def test_datas_saem_em_utc_declarado(cliente_logado: TestClient, usuario: dict):
    """Toda data e hora termina em `Z`.

    Sem o sufixo, `new Date(...)` lê o horário como **local do navegador**, e
    um servidor em UTC e um navegador em Brasília discordariam em três horas.
    """
    tarefa = criar_tarefa(cliente_logado)
    concluida = cliente_logado.patch(
        f"/tasks/{tarefa['id']}", json={"status": "concluida"}
    ).json()

    assert tarefa["criadoEm"].endswith("Z")
    assert tarefa["atualizadoEm"].endswith("Z")
    assert concluida["concluidoEm"].endswith("Z")
    assert usuario["criadoEm"].endswith("Z")


def test_prazo_continua_sendo_so_a_data(cliente_logado: TestClient):
    """`prazo` é `YYYY-MM-DD`, sem hora e sem fuso."""
    tarefa = criar_tarefa(cliente_logado, prazo="2026-03-15")
    assert tarefa["prazo"] == "2026-03-15"
```

Entendendo o código

`def criar_tarefa(cliente, **campos):`	Outra fábrica. O `**campos` recebe **quantos** argumentos nomeados vierem — `prazo=...`, `prioridade=...` — num dicionário.
`corpo = {"titulo": "Tarefa"}` `corpo.update(campos)`	Começa com um título padrão e sobrescreve com o que o teste passar. O teste só escreve o que importa para ele.
`set(tarefa) == {...}`	O **conjunto exato** de campos, como no teste do usuário do Dia 7. `@typedef Task` é a definição da tarefa no código do front-end.
`.endswith("Z")`	O `Z` no final de uma data e hora quer dizer **UTC**. Sem ele, o navegador interpretaria o horário como local.
`prazo="2026-03-15"`	O prazo é só a **data**, sem hora: volta exatamente como foi. Aqui a data fixa não é problema, porque nenhum filtro depende de hoje.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
====================== 171 passed, 1 warning in 1.32s ======================
```

Confira no seu print

- **171 passed** — três testes novos.

Passo 9 — O vocabulário dos filtros e a ordem natural

Arquivo: tests/test_integracao_frontend.py (acrescentar ao final) · ainda sem commit

Código para digitar

```
# ------------------------------ vocabulário dos filtros — TaskFilters.jsx

def test_ordenar_por_recente_e_aceito(cliente_logado: TestClient):
    """`useTasks.js` chama de `recente` o que a API chama de `criadoEm`."""
    criar_tarefa(cliente_logado, titulo="Primeira")
    criar_tarefa(cliente_logado, titulo="Segunda")

    resposta = cliente_logado.get("/tasks", params={"ordenarPor": "recente"})
    assert resposta.status_code == 200
    assert [t["titulo"] for t in resposta.json()] == ["Segunda", "Primeira"]


def test_sentinela_todas_vale_como_sem_filtro(cliente_logado: TestClient):
    """Os seletores do front começam em `todas` e mandam esse valor."""
    criar_tarefa(cliente_logado, titulo="Pendente")
    concluida = criar_tarefa(cliente_logado, titulo="Concluida")
    cliente_logado.patch(f"/tasks/{concluida['id']}", json={"status": "concluida"})

    resposta = cliente_logado.get(
        "/tasks", params={"status": "todas", "prioridade": "todas"}
    )
    assert resposta.status_code == 200
    assert len(resposta.json()) == 2


def test_filtro_enviado_vazio_vale_como_sem_filtro(cliente_logado: TestClient):
    """`?status=` é o que um formulário HTML produz com o campo em branco."""
    criar_tarefa(cliente_logado)
    resposta = cliente_logado.get("/tasks?status=&prioridade=")
    assert resposta.status_code == 200
    assert len(resposta.json()) == 1


def test_valor_de_filtro_invalido_continua_sendo_recusado(
    cliente_logado: TestClient,
):
    """Aceitar a sentinela não é aceitar qualquer coisa: `inventado` não passa."""
    resposta = cliente_logado.get("/tasks", params={"status": "inventado"})
    assert resposta.status_code == 422


# ---------------------------- sentido natural da ordenação — `ordenar()` no front

def test_ordem_natural_de_prazo_e_crescente(cliente_logado: TestClient):
    """Sem `ordem`, prazo ordena do mais próximo para o mais distante."""
    hoje = date.today()
    criar_tarefa(
        cliente_logado, titulo="Distante", prazo=str(hoje + timedelta(days=30))
    )
    criar_tarefa(
        cliente_logado, titulo="Proxima", prazo=str(hoje + timedelta(days=1))
    )

    resposta = cliente_logado.get("/tasks", params={"ordenarPor": "prazo"})
    assert [t["titulo"] for t in resposta.json()] == ["Proxima", "Distante"]
```

```
def test_tarefa_sem_prazo_vai_para_o_fim_nos_dois_sentidos(
    cliente_logado: TestClient,
):
    """O SQLite trata `NULL` como o menor valor; o PostgreSQL, não.

    Sem `NULLS LAST` explícito, este teste passaria em desenvolvimento e
    falharia em produção. Por isso o laço roda os **dois** sentidos.
    """
    hoje = date.today()
    criar_tarefa(cliente_logado, titulo="Com prazo", prazo=str(hoje))
    criar_tarefa(cliente_logado, titulo="Sem prazo", prazo=None)

    for sentido in ("asc", "desc"):
        resposta = cliente_logado.get(
            "/tasks", params={"ordenarPor": "prazo", "ordem": sentido}
        )
        titulos = [t["titulo"] for t in resposta.json()]
        assert titulos[-1] == "Sem prazo", f"ordem={sentido} devolveu {titulos}"


def test_ordem_natural_de_prioridade_comeca_na_urgente(cliente_logado: TestClient):
    """Sem `ordem`, prioridade vai da mais urgente para a menos."""
    for prioridade in ("baixa", "urgente", "media"):
        criar_tarefa(cliente_logado, titulo=prioridade, prioridade=prioridade)

    resposta = cliente_logado.get("/tasks", params={"ordenarPor": "prioridade"})
    assert [t["prioridade"] for t in resposta.json()] == ["urgente", "media", "baixa"]


def test_ordem_natural_de_criado_em_comeca_na_mais_recente(
    cliente_logado: TestClient,
):
    """Cada campo de ordenação tem um sentido natural **diferente** — é a
    tabela `_ORDENS_NATURAIS` do `app/enums.py` que registra isso."""
    criar_tarefa(cliente_logado, titulo="Antiga")
    criar_tarefa(cliente_logado, titulo="Nova")

    resposta = cliente_logado.get("/tasks")
    assert [t["titulo"] for t in resposta.json()] == ["Nova", "Antiga"]
```

Entendendo o código

`params={"ordenarPor": "recente"}`	Outra forma de mandar parâmetros de consulta: o `params=` monta o `?ordenarPor=recente` sozinho — e cuida de caracteres especiais.
`"recente"`	O nome que o **front-end** usa. A API chama de `criadoEm`, mas aceita os dois.
`"todas"`	A **sentinela**: o valor que os seletores do front mandam quando nada está filtrado. Vale como "sem filtro".
`"/tasks?status=&prioridade="`	Parâmetros **vazios**, como um formulário em branco produz. Também valem como "sem filtro".
`"inventado" -> 422`	Mas aceitar a sentinela não é aceitar qualquer coisa.
`for sentido in ("asc", "desc"):`	Um laço **dentro** do teste: a mesma verificação nos dois sentidos de ordenação.
`titulos[-1] == "Sem prazo"`	`[-1]` é o **último** item da lista. A tarefa sem prazo vai para o fim nos dois sentidos.
`f"ordem={sentido} devolveu {titulos}"`	A mensagem diz **em qual volta** do laço o teste falhou, e o que veio.

> O teste da tarefa sem prazo mostra o que só este nível alcança: o SQLite trata o `NULL` como o menor valor; o PostgreSQL, de produção, o posiciona de outra forma. Sem o teste, a ordem passaria em desenvolvimento e mudaria em produção.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
====================== 179 passed, 1 warning in 1.50s ======================
```

Confira no seu print

- **179 passed** — oito testes novos.

Passo 10 — A busca e o total

Arquivo: tests/test_integracao_frontend.py (acrescentar ao final) · ainda sem commit

Código para digitar

```
# ------------------------------------------------------------ busca por título

def test_busca_filtra_pelo_titulo_ignorando_maiusculas(cliente_logado: TestClient):
    """O front busca com `titulo.toLowerCase().includes(termo)`."""
    criar_tarefa(cliente_logado, titulo="Revisar proposta do cliente")
    criar_tarefa(cliente_logado, titulo="Organizar planilha")

    resposta = cliente_logado.get("/tasks", params={"busca": "PROPOSTA"})
    assert [t["titulo"] for t in resposta.json()] == ["Revisar proposta do cliente"]


def test_busca_trata_curingas_do_sql_como_texto(cliente_logado: TestClient):
    """Procurar por `100%` não pode devolver tudo.

    `%` é o curinga do `LIKE`. É o mesmo raciocínio da injeção de SQL, numa
    versão sem malícia: um caractere digitado como texto sendo interpretado
    como comando.
    """
    criar_tarefa(cliente_logado, titulo="Pagar 100% da fatura")
    criar_tarefa(cliente_logado, titulo="Revisar proposta")

    resposta = cliente_logado.get("/tasks", params={"busca": "100%"})
    assert [t["titulo"] for t in resposta.json()] == ["Pagar 100% da fatura"]

    resposta = cliente_logado.get("/tasks", params={"busca": "%"})
    assert [t["titulo"] for t in resposta.json()] == ["Pagar 100% da fatura"]


def test_busca_vazia_nao_filtra(cliente_logado: TestClient):
    criar_tarefa(cliente_logado)
    resposta = cliente_logado.get("/tasks", params={"busca": "   "})
    assert len(resposta.json()) == 1


# ------------------------------------ total de registros — DashboardPage.jsx

def test_total_vem_no_cabecalho_e_ignora_o_limite(cliente_logado: TestClient):
    """`X-Total-Count` diz quantas existem, não quantas vieram."""
    for n in range(3):
        criar_tarefa(cliente_logado, titulo=f"Tarefa {n}")

    resposta = cliente_logado.get("/tasks", params={"limit": 2})
    assert len(resposta.json()) == 2
    assert resposta.headers["X-Total-Count"] == "3"


def test_total_respeita_os_filtros(cliente_logado: TestClient):
    """Um total correto sobre o conjunto errado é tão ruim quanto um errado."""
    criar_tarefa(cliente_logado, titulo="Urgente", prioridade="urgente")
    criar_tarefa(cliente_logado, titulo="Baixa", prioridade="baixa")

    resposta = cliente_logado.get("/tasks", params={"prioridade": "urgente"})
    assert resposta.headers["X-Total-Count"] == "1"
```

Entendendo o código

`{"busca": "PROPOSTA"}`	Busca em maiúsculas encontra o título em minúsculas, como o front-end já fazia.
`{"busca": "100%"}`	No SQL, `%` quer dizer "qualquer coisa". Se a API não o tratar como texto, buscar `100%` devolveria a lista inteira.
`{"busca": "%"}`	O caso extremo: buscar só o `%` precisa encontrar só o título que **contém** um `%`.
`{"busca": "   "}`	Uma busca só de espaços não filtra nada.
`{"limit": 2}`	Pede só **duas** tarefas, de três existentes.
`headers["X-Total-Count"] == "3"`	O total conta **todas** as que atendem ao filtro, não só as que vieram. É assim que o front sabe que há mais páginas.
`{"prioridade": "urgente"} -> "1"`	E o total respeita o filtro: só uma é urgente.

Execute no terminal

```
pytest
```

O que deve aparecer (final)

```
====================== 184 passed, 1 warning in 1.63s ======================
```

Confira no seu print

- **184 passed** — cinco testes novos.

Passo 11 — As mensagens de erro e o fluxo completo

Arquivo: tests/test_integracao_frontend.py (acrescentar ao final) · Commit 30

Código para digitar

```
# ------------------------------------------- mensagens de erro das tarefas

def test_valor_de_enum_invalido_lista_os_aceitos(cliente_logado: TestClient):
    """Quem recebeu o erro precisa saber o que mandar em vez disso."""
    resposta = cliente_logado.post(
        "/tasks", json={"titulo": "X", "prioridade": "altissima"}
    )
    assert resposta.json()["detail"] == (
        "prioridade: valor inválido. Aceitos: baixa, media, alta ou urgente."
    )


def test_data_malformada_informa_o_formato(cliente_logado: TestClient):
    """O erro mais comum de integração: data em formato brasileiro."""
    resposta = cliente_logado.post(
        "/tasks", json={"titulo": "X", "prazo": "15/03/2026"}
    )
    assert resposta.json()["detail"] == (
        "prazo: deve ser uma data no formato AAAA-MM-DD."
    )


def test_mensagem_usa_o_nome_publico_do_campo(cliente_logado: TestClient):
    """O cliente envia `prazoAte`; a mensagem não pode falar de `prazo_ate`."""
    resposta = cliente_logado.get("/tasks?prazoAte=15/03/2026")
    detalhe = resposta.json()["detail"]
    assert detalhe.startswith("prazoAte:"), detalhe
    assert "prazo_ate" not in detalhe
# ------------------------------------------- o fluxo que o AuthContext executa

def test_fluxo_completo_que_o_front_vai_executar(cliente: TestClient):
    """Cadastrar, usar o token, reabrir o app e recuperar a sessão.

    O único teste da suíte que percorre um **cenário de uso** de ponta a
    ponta. Vale um por suíte: é caro e, quando falha, não diz onde — mas é o
    único que prova que as peças se encaixam na ordem em que alguém vai usá-las.
    """
    sessao = cliente.post(
        "/usuarios",
        json={"nome": "Davi", "email": "davi@exemplo.com", "senha": "senha123"},
    ).json()
    autorizacao = {"Authorization": f"Bearer {sessao['accessToken']}"}

    criada = cliente.post(
        "/tasks", json={"titulo": "Primeira tarefa"}, headers=autorizacao
    )
    assert criada.status_code == 201

    conta = cliente.get("/usuarios/eu", headers=autorizacao)
    assert conta.status_code == 200
    assert conta.json() == sessao["usuario"]

    minhas = cliente.get("/tasks", headers=autorizacao)
    assert [t["titulo"] for t in minhas.json()] == ["Primeira tarefa"]
```

Entendendo o código

`"Aceitos: baixa, media, alta ou urgente."`	A mensagem do erro **lista** os valores aceitos: quem errou sabe o que mandar.
`"prazo": "15/03/2026"`	A data no formato brasileiro — o erro de integração mais comum. A mensagem informa o formato certo.
`detalhe.startswith("prazoAte:")`	A mensagem usa o nome **público** do campo, o que o cliente enviou — e nunca o nome interno do Python.
`(cliente: TestClient)`	O fluxo completo usa o `cliente` sem fixture de conta: ele mesmo cadastra o Davi, do zero.
`cadastrar -> criar -> /usuarios/eu -> listar`	A sequência que o front-end vai executar: cadastrar, criar a primeira tarefa, "reabrir o app" recuperando a conta pelo token, e listar.
`conta.json() == sessao["usuario"]`	Ao "reabrir o app", a conta recuperada pelo token é **exatamente** a do cadastro.

> Um teste de fluxo completo por suíte é a medida certa. Ele é caro e, quando falha, não diz **onde** — mas é o único que prova que as peças se encaixam na ordem em que alguém vai usá-las.

Execute no terminal

```
pytest
pytest -v -k "fluxo or curingas or sem_prazo" tests/test_integracao_frontend.py
```

O que deve aparecer (trechos)

```
====================== 188 passed, 1 warning in 1.74s ======================
...
tests/test_integracao_frontend.py::test_tarefa_sem_prazo_vai_para_o_fim_nos_dois_sentidos PASSED [
33%]
tests/test_integracao_frontend.py::test_busca_trata_curingas_do_sql_como_texto PASSED [ 66%]
tests/test_integracao_frontend.py::test_fluxo_completo_que_o_front_vai_executar PASSED [100%]

================= 3 passed, 31 deselected, 1 warning in 0.18s =================
```

Confira no seu print

- **188 passed** — a suíte completa da trilha.
- O `-k` com `or` escolhe os testes que tenham **qualquer uma** das palavras no nome: os três mais interessantes do contrato.

Feche o commit 30

```
git add tests/test_integracao_frontend.py
git commit -m "Adiciona o contrato das tarefas"
```

5. A cobertura: o que mede e o que não mede

Passo 12 — Medir a suíte

Arquivo: requirements.txt (acrescentar ao final) · Commit 31

Uma ferramenta que a suíte usa precisa estar declarada, para que quem clonar o projeto meça a mesma coisa. Acrescente ao **final** do `requirements.txt`:

Código para digitar — no final do requirements.txt

```
# Medição de cobertura, acrescentada no Dia 9.
#
# Não entrou junto com as duas de cima de propósito: uma dependência nova é
# uma decisão, e esta tem data e motivo. Declará-la aqui é o que faz a medição
# ser reproduzível — quem clonar o projeto mede a mesma coisa, em vez de
# depender do que estava instalado na máquina de quem rodou.
pytest-cov
```

Execute no terminal

```
pip install -r requirements.txt
pytest --cov=app --cov-report=term-missing
```

Entendendo o comando

`--cov=app`	Liga a medição e diz **o que** medir: só o código da pasta `app/`.
`--cov-report=term-missing`	Mostra o relatório no terminal, com a coluna das linhas **não executadas**.

O que deve aparecer (trecho final)

```
Name                       Stmts   Miss  Cover   Missing
----------------------------------------------------------
app\__init__.py                0      0   100%
app\config.py                 63      0   100%
app\database.py               34      1    97%   44
app\dependencies.py           29      1    97%   45
app\enums.py                  41      1    98%   133
app\exceptions.py             27      0   100%
app\main.py                   34      3    91%   46-47, 95
app\models.py                100      2    98%   216, 269
app\repositories.py           78      0   100%
app\routers\__init__.py        0      0   100%
app\routers\tasks.py          32      0   100%
app\routers\usuarios.py       21      0   100%
app\schemas.py                43      0   100%
app\security.py               22      0   100%
app\services.py               71      2    97%   126, 181
app\tempo.py                  21      2    90%   65, 76
app\tokens.py                 27      0   100%
app\validacao.py              28      1    96%   74
----------------------------------------------------------
TOTAL                        671     13    98%
======================== 188 passed, 1 warning in 2.92s ========================
```

Lendo a tabela

`Stmts`	*Statements*: quantas instruções executáveis o arquivo tem. Comentários e docstrings não contam.
`Miss`	Quantas **nenhum** teste executou.
`Cover`	A porcentagem executada.
`Missing`	**Quais** linhas faltaram. É a coluna mais útil da tabela.

> **98%, com 13 linhas faltando.** O número parece o fim da conversa. É o começo: a cobertura diz **onde** a suíte não passou — é um **localizador de ausências**. O que ela **não** diz, você vai ver agora.

Experimento — a armadilha da cobertura

Crie um arquivo **descartável**, `tests/test_armadilha.py`, com o conteúdo abaixo. Ele não fica no projeto: depois do experimento, você vai apagá-lo.

Código para digitar — tests/test_armadilha.py (descartável)

```
# tests/test_armadilha.py  —  descartável, não fica no projeto
from app.models import Tarefa


def test_que_nao_verifica_nada():
    tarefa = Tarefa(usuario_id=1, titulo="Qualquer coisa")
    tarefa.reabrir()
    tarefa.concluir()
    tarefa.reabrir()
    repr(tarefa)
```

Repare: **nenhum** `assert`. O teste chama métodos e não confere nada. Rode a cobertura de novo:

Execute no terminal

```
pytest --cov=app --cov-report=term-missing
```

O que deve aparecer (trechos)

```
app\models.py              100      0   100%
...
TOTAL                      671     11    98%
======================== 189 passed, 1 warning in 3.05s ========================
```

	`app/models.py`	Suíte
Antes	98% — faltam as linhas 216 e 269	188 passed
Com a armadilha	**100%**	189 passed

> **O arquivo que guarda as regras de negócio chegou a 100% com um teste que não verifica nada.** Se a RN-03 fosse quebrada amanhã, esse teste continuaria verde.
>
> **Cobertura mede o que foi executado, não o que foi verificado.** Por isso ela nunca pode ser **meta**: vira meta, e se cumpre escrevendo testes sem `assert`.

Confira no seu print

- **Apague** o `tests/test_armadilha.py` e rode `pytest`: volta a **188 passed**.
- Rode `git status`: o arquivo **não** pode aparecer.

Experimento — esquecer o alvo

Rode `pytest --cov --cov-report=term`, **sem** o `=app`. A tabela passa a listar também os arquivos de `tests/`, todos perto de 100% — afinal, a suíte executa a si mesma. O total sobe para **99%**, sem nenhum teste novo. Sem o alvo, o número fica maior e significa menos.

As 13 linhas que faltam, uma por uma

As linhas não cobertas **não** são uma lista de tarefas. Abra cada uma no VS Code (no terminal, **Ctrl+clique** no nome do arquivo ajuda) e confira a classificação:

Linha	O que é	Classificação
`database.py:44`	opções de *pool* de conexão	**inalcançável no teste** — só com banco de servidor
`dependencies.py:45`	`return HashBcrypt()`	**coberta por outro caminho** — o dublê a substitui; o algoritmo é testado em `test_seguranca.py`
`enums.py:133`	valor de filtro que não é texto	**inalcançável por HTTP** — parâmetro de URL sempre chega como texto
`main.py:46-47`	`criar_tabelas()` no ciclo de vida	**inalcançável por desenho** — o `TestClient` sem `with`, decisão do Dia 7
`main.py:95`	resposta 400 para exceção sem mapeamento	**defesa em profundidade** — todas as exceções estão mapeadas
`models.py:216`	a guarda de `reabrir` numa tarefa já pendente	**lacuna real**
`models.py:269`	o `__repr__` da `Tarefa`	**código de depuração** — o do `Usuario` é testado porque poderia vazar a senha
`services.py:126`	`UsuarioNaoEncontrado` ao criar tarefa	**defesa em profundidade** — o usuário vem de um token já validado
`services.py:181`	o `PATCH` alterando só a prioridade	**lacuna real**
`tempo.py:65, 76`	ramos de fuso que o SQLite não produz	**inalcançável no teste** — apareceriam com PostgreSQL
`validacao.py:74`	erro do Pydantic sem caminho de campo	**defesa em profundidade**

Duas lacunas reais em 671 linhas. Elas vão para o parecer como recomendação:

1. `concluir` tem teste para "concluir duas vezes" (o do Dia 2); `reabrir` não tem o equivalente. A regra é simétrica, a verificação não.

2. O `PATCH` é testado alterando título, status e prazo — mas nunca **só** a prioridade.

Desafio opcional. Escreva os dois testes que fecham as lacunas — um no `test_dominio.py`, outro no `test_api.py` — **com** `assert`: o que deveria continuar igual depois de reabrir uma tarefa pendente? O que a resposta do `PATCH` precisa mostrar?

Com eles, a suíte lê **190 passed**, e as linhas 216 e 181 saem da coluna *Missing*. O `models.py` e o `services.py` ficam em 99% — o que sobra é o `__repr__` e a defesa em profundidade, que você já classificou. É o mesmo número do `test_armadilha.py`, agora **verificando**: a diferença entre os dois é o assunto inteiro do dia.

Feche o commit 31

```
git status                          # o test_armadilha.py NÃO pode aparecer
git add requirements.txt
git commit -m "Adiciona o pytest-cov como dependencia de desenvolvimento"
```

6. O critério de saída e o parecer

Um plano de testes declara, **antes** de começar, o que significa "pronto". O entregável final da trilha é conferir a execução real contra esse critério:

Critério declarado	Verificação	Resultado
100% dos testes planejados executados	`pytest -v`	188 de 188
Nenhum teste falhando	a última linha da saída	188 passed
Nenhum defeito crítico aberto	os registros dos Dias 7 e 9	ambos resolvidos, com teste de confirmação
Todas as regras de negócio com teste	RN-01 a RN-05	cobertas, várias em dois níveis
Cobertura	`pytest --cov=app`	98%, com as 13 linhas classificadas
Recomendações registradas	o parecer	2 lacunas reais, com o teste sugerido

O parecer

Escreva o parecer no formato que o professor indicar. Ele tem **três partes**:

Parte	O que vai nela
1. O que foi verificado	a tabela acima, com o comando e a **data** da execução
2. Os defeitos encontrados e tratados	os cinco, com severidade, forma de descoberta e o commit da correção
3. As recomendações	as duas lacunas, mais o que a trilha mostrou: cobertura não deve ser meta, e o plano de testes precisa exigir cenários com duas contas

Dia	Defeito	Como foi encontrado
2	RN-03: a data de conclusão sobrescrita	teste vermelho
5	integridade referencial desligada no SQLite	teste vermelho
6	configuração caindo em padrão silencioso	teste vermelho
7	o hash da senha em toda resposta	teste vermelho
9	IDOR no `PATCH` e no `DELETE`	teste vermelho

> Repare na última coluna. **Nenhum dos cinco defeitos foi encontrado lendo código.** Todos apareceram como teste vermelho. Um parecer que só diz "188 testes passando" não é um parecer: é uma captura de tela.

A verificação final da trilha

Execute no terminal

```
git rev-list --count HEAD
git diff $(git rev-list --max-parents=0 HEAD) --stat -- app/
```

Entendendo os comandos

`git rev-list --count HEAD`	Conta os commits do projeto: o commit 0, do ponto de partida, mais os 31 da trilha.
`git rev-list --max-parents=0 HEAD`	Encontra o **primeiro** commit — o único sem pai. O `$(...)` põe o resultado dentro do outro comando.
`git diff ... --stat -- app/`	Tudo o que mudou em `app/` desde o ponto de partida.

O que deve aparecer (as contagens de linhas dependem das suas correções)

```
32

 app/config.py   | 8 +++++++-
 app/database.py | 1 +
 app/models.py   | 7 ++-----
 app/schemas.py  | 1 -
 app/services.py | 2 ++
 5 files changed, 12 insertions(+), 7 deletions(-)
```

Confira no seu print

- **32** commits: o 0 mais os 31.
- **Exatamente cinco** arquivos de `app/` — um por defeito. Cada alteração tem um teste vermelho no commit anterior. É isso, e não a palavra de ninguém, que prova que a aplicação só foi tocada com evidência na mão.

7. Fechamento da trilha

O que o último dia trouxe

Recurso	Para quê	Onde você usou
duas sessões no mesmo teste	agir como duas pessoas	`outro_usuario` e `tarefa_de`
olhar o **estado** depois	provar que nada mudou	a tarefa recusada continua intacta
`params={...}`	parâmetros de consulta sem montar a URL	o contrato das tarefas
`**campos`	uma fábrica com campos opcionais	`criar_tarefa`
`pytest --cov=app`	localizar o que nenhum teste executou	a cobertura
`--cov-report=term-missing`	ver **quais** linhas faltaram	a classificação das 13

Confira o que o dia entregou

Verificação	Esperado
`pytest`	188 passed
O `test_armadilha.py` foi **apagado**	não aparece no `git status`
As 13 linhas não cobertas estão classificadas	com as 2 lacunas reais no parecer
Os cinco defeitos estão corrigidos	os dos Dias 7 e 9 com registro formal
Nenhuma alteração em `app/` sem teste vermelho antes	o `git log` mostra o par teste → correção, cinco vezes

Se algo der errado

Sintoma	Causa provável	O que fazer
`409` na fixture `outro_usuario`	Ela usou o mesmo e-mail da `sessao_aberta`	Cada conta tem o seu e-mail
Os testes de isolamento passam com o defeito presente	Foi usado o `cliente_logado` nos dois lados	O token da Ana e o do Bruno precisam ser diferentes
`1 failed` no teste do `DELETE` depois da correção	A verificação foi posta só em `atualizar`	Ela vai no método de busca, que serve aos dois
O teste do 404 falha com `AttributeError: 'NoneType' ... 'pertence_a'`	A verificação do dono veio **antes** da de existência: a tarefa inexistente é `None`	Primeiro "existe?", depois "é seu?"
A cobertura lista os arquivos de `tests/`	Faltou o `=app` no `--cov`	`pytest --cov=app`
`No module named pytest_cov` ou opção `--cov` desconhecida	O `pytest-cov` não foi instalado	`pip install -r requirements.txt`
Cobertura em 100% sem testes novos	O `test_armadilha.py` ficou no projeto	Era a demonstração — apague-o

O que você construiu em nove dias

Dia	Ferramenta	Testes	Defeito encontrado
1	`assert`	9	—
2	`pytest.raises`	22	RN-03: a data de conclusão sobrescrita
3	`parametrize`	37	—
4	`fixture` e `conftest.py`	67	—
5	`yield` e `autouse`	74	integridade referencial desligada no SQLite
6	`monkeypatch`	105	configuração caindo em padrão silencioso
7	`TestClient` e o dublê	128	o hash da senha em toda resposta
8	fixtures encadeadas	162	—
9	duas sessões e `pytest-cov`	**188**	IDOR no `PATCH` e no `DELETE`

> **A frase para levar.** Uma suíte de testes não prova que o sistema está correto. Prova que as afirmações que você soube fazer sobre ele continuam verdadeiras. Toda a habilidade está em saber **quais afirmações fazer** — e é isso, e não a sintaxe do pytest, que você treinou aqui.